CUDA · TRITON · PYTORCH · FULL SOURCE

GPU Programming 面试：12 题完整代码与中文答案

CUDA C++ · Triton · PyTorch · 数学推导 · 优化问答 · 测试与实验

阅读说明： 每道题先给讲解和追问答案，再列出本题核心函数的完整代码。章节中的代码由工程源码自动提取，没有省略函数体；公共依赖、完整模块、测试与工具在文末逐文件全文收录。直接运行请使用代码包中的工程，避免只复制一段函数后遗漏 import、常量或公共校验函数。

目录

1. 怎样使用这套材料
2. 验证记录与适用范围
3. 各后端共享的工程约定
4. CUDA C++：从能写对，到能解释性能
5. Triton 实现、逐题解释与面试追问
6. PyTorch 层：从形状、数值到训练与 KV cache
7. torch.compile 与自定义算子接入
8. 第 1 题｜Vector Add 与 Bias + ReLU 融合
9. 第 2 题｜Row Sum 与 Row Max 归约
10. 第 3 题｜数值稳定的行 Softmax
11. 第 4 题｜Naive 与 Tiled 物理转置
12. 第 5 题｜Naive 与 Tiled GEMM
13. 第 6 题｜Roofline：工作量、流量与瓶颈模型
14. 第 7 题｜Linear、QKV、Shape 与 Stride
15. 第 8 题｜LayerNorm 与 RMSNorm：前向、反向与 GPU 实现
16. 第 9 题｜Attention、MHA、Causal Mask 与 KV Cache
17. 第 10 题｜torch.compile：Graph Break、Guard 与稳态性能
18. 第 11 题｜自定义 CUDA / Triton 算子：Fake、Autograd 与编译
19. 第 12 题｜性能实验、诊断与项目答辩
20. 参考资料与来源边界
21. 完整源码附录

怎样使用这套材料

这套材料对应此前设计的 12 道核心练习，目的是准备 GPU programming / ML systems 面试。它是原创练习和参考答案，不代表 Together AI 公布了这 12 道原题，也不能用它推断所有面试轮次都固定不变。

你需要同时训练三种能力：从公式写出正确实现；把工作分配给线程、warp、block 或 Triton program；用实验判断优化是否有效。只记住最后一版代码会漏掉面试中最常见的追问：为什么这样索引、为什么这里需要同步、为什么另一种形状会变慢。

一、先分清材料中的四个层次

层次	代码所在位置	用途	是否训练可用
PyTorch 公式实现	gpu_prep/torch_layers.py	Linear、Norm、Attention、MHA、KV cache	常规 forward 支持 autograd；cache 增量推理有明确限制
CUDA C++ 教学 kernel	gpu_prep/csrc/kernels.cu	看清线程、共享内存、同步和尾块处理	原始入口仅 forward
Triton 教学 kernel	gpu_prep/triton_ops.py	看清 block 表达、mask、归约和矩阵 tile	原始入口仅 forward
PyTorch 自定义算子接入	gpu_prep/custom_ops.py	注册算子、fake、autograd 与 compile 接入	Bias+ReLU 示例提供 backward

GPU 的基础入口刻意限制输入布局和 dtype，使你能先审查核心算法。CUDA C++ 使用 contiguous FP32；Triton 使用 contiguous FP32、FP16、BF16，并要求 NVIDIA compute capability 至少 8.0。它们没有悄悄替你搬设备、转 dtype 或复制 noncontiguous 输入。需要复制时，由调用者显式写 contiguous()，这样性能分析才不会把布局转换藏起来。

这些实现可以作为练习基线，不能仅凭“实现了 tiled”“用了 Triton”就认定优于 cuBLAS、PyTorch 或生产级 FlashAttention。源码中给定的 tile 是便于推导的起点，没有在当前无 GPU 环境中被实测为最佳配置。

二、运行顺序

先进入解压后的项目根目录。建议创建独立虚拟环境，随后按照 PyTorch 官方安装选择器安装与你的 NVIDIA 驱动、GPU 和 CUDA 运行时兼容的 PyTorch。不要在已有 CUDA 版 PyTorch 的环境中为了照抄命令而把它换成 CPU 版。

```
python -m venv .venv
source .venv/bin/activate
# 先安装与你机器匹配的 PyTorch，再安装本项目。
python -m pip install -e ".[test,cuda]"
# Linux / NVIDIA 上安装与 PyTorch、GPU 兼容的 Triton：
python -m pip install -e ".[triton]"

python -m gpu_prep.check_env
python -m pytest -q
python examples/torch_layers_demo.py
python examples/compile_demo.py --help
python -m gpu_prep.roofline
```

有真实 NVIDIA GPU 后再运行：

```
python -m gpu_prep.check_env --require-gpu
python examples/run_gpu.py --backend all --op all
python -m pytest tests/test_cuda_ops.py tests/test_triton_ops.py -q
python -m gpu_prep.benchmark --op matmul --shape 128,256,384 --backends torch,cuda,triton --output
results/matmul_fp32.json
```

CUDA 扩展第一次调用会编译。GPU PyTorch wheel 提供运行所需的相关库，但不能据此假定你的机器已经有 nvcc；CUDA C++ 路径还需要完整的编译工具链、可用 C++ 编译器和 Ninja。Triton 路径不需要本项目的 CUDA 扩展编译。请保留第一次编译的错误信息，按缺失依赖修复；不要通过吞掉异常后换成 PyTorch 实现来假装自定义 kernel 已经运行。

FP16/BF16 对比只选支持的后端：

```
python -m gpu_prep.benchmark --op matmul --shape 256,512,768 --dtype float16 --backends torch,triton
```

形状参数的含义见每道题：GEMM 的 shape 顺序为 M,N,K，Attention 的顺序为 B,H,L,S,D。性能脚本只接受正维度；零维度与空张量属于正确性测试，避免把“没有实际工作”混入吞吐量结果。

三、每题按这个顺序准备

1. 不看答案，用 30 秒写出输入、输出、shape、dtype、layout 和数学公式。
2. 先写 PyTorch reference，至少准备一个很小、可以手工核对的例子。
3. 写直接实现；对 non-multiple、很短维度、很长维度和零尺寸约定做检查。
4. 解释每个线程或 program 负责哪一部分输出，以及它需要读哪些输入。
5. 确认同步与掩码，再讨论共享内存、寄存器、向量化、融合和 Tensor Core。
6. 先验证精度，再运行统一 benchmark。记录硬件、版本、形状、dtype 和计算精度策略。
7. 用“观察—假设—改动—验证—结果—适用范围”的顺序讲优化。

题目中的“答案示例”可以练口述，但不要把示例假设当成你真实做过的实验。没有测量时应说“我的假设是……，我会看……指标来验证”，而不是说某个优化必然快两倍。

四、全局正确性约定

4.1 容差不等于随意放宽

FP32 的归约顺序、FMA 与不同实现的 exp 近似会影响最后几位。比较浮点结果一般使用绝对误差和相对误差的组合，而不是要求逐 bit 相同。接近零时相对误差会失真，因此不能只报 max_relative_error。矩阵元素很多时，还应检查是否存在少数离群错误，而不是只看平均误差。

低精度输入的内部累加通常用 FP32，输出再转回输入 dtype。FP16/BF16 的输入量化误差仍然存在，FP32 累加不能恢复输入中已经丢失的有效位。基准对照还必须统一 FP32/TF32、累加精度以及算子语义。

4.2 尾块与空输入是两类问题

N=1003、BLOCK=256 时，最后一个 program 的一部分 lane 越界，这是尾块，应该用 load/store mask。N=0 时根本没有有效工作，通常由 host wrapper 提前返回。对于 row_max、softmax 这类沿空维度没有本题定义输出的操作，wrapper 明确拒绝 N=0。

GPU 行归约和 Norm 允许行数为零，但要求被归约的最后一维大于零。GEMM 的 K=0 有清楚的数学意义：非空输出为零；M=0 或 N=0 则返回空输出。Attention 的 GPU 练习要求 S>0，PyTorch 公式实现对 S=0 有单独定义。完整行为由源码的检查和 tests 共同给出。

4.3 Mask 与 NaN

普通 softmax 的数学定义不包含一整行负无穷的归一化：减去最大值时会出现负无穷减负无穷。普通 softmax 练习约定有限输入；Attention 单独处理 boolean mask，并把全遮蔽行定义成零输出。不能把两个合同混为一谈。

同样，测试有限随机数不能证明 NaN/Inf 的传播与框架完全一致。CUDA 的 bias_relu 专门保留 NaN，而其他基础 kernel 不承诺所有特殊值兼容。代码不会为每次调用扫描整个张量来检查有限性，那本身会增加一次完整读和同步；输入约定由调用者负责。

4.4 Forward、backward 与 compile 是三个检查点

一个 forward kernel 返回正确数值，不等于它能训练；注册了 autograd，不等于导数公式正确；能被 torch.compile 捕获，也不等于 Inductor 已经把它和邻近算子融合。第 11 题分别演示 forward reference、gradcheck、fake、opcheck 和 compile 检查，避免用一个“测试通过”覆盖几个不同结论。

五、源码阅读地图

题号	要先读的函数/文件	要背熟的核心句
1	vector_add、bias_relu	先确认连续访存，再判断融合节省了什么
2	row_sum、row_max	归约的 identity 和同步范围必须正确
3	softmax	减最大值保稳定，FP32 做统计与归一化
4	transpose_naive、transpose_tiled	物理转置需要同时改善读和写的访问模式
5	matmul_naive、matmul_tiled	Tile 的价值来自输入复用，代价是资源占用
6	roofline.py	用一致单位算 FLOP/byte，再判断模型里的瓶颈
7	ManualLinear、split_qkv_heads	Shape 正确还要检查 stride 与是否复制
8	ManualLayerNorm、ManualRMSNorm	统计维度、方差分母、eps 位置和精度要一致
9	Attention、MHA、KVCache	Prefill 和 decode 的 query 绝对位置不同
10	compile_demo.py	分开 graph break、guard/recompile、首次编译和稳态执行
11	custom_ops.py	schema、fake、autograd、compile 各有责任
12	benchmark.py	先定义测什么，再对正确且可比的实现计时

验证记录与适用范围

记录日期：2026-10-10。完整原始记录在 results 目录。下表区分数学/注册检查、编译检查和真实 GPU 运行；这些结论不能互相替代。

已完成的实际检查

检查	结果	能支持的结论
最终 pytest 全套收集与执行	**119 passed、330 skipped、0 failures、0 errors；共 449 项**	CPU 公式、梯度、参数合同、注册与 capture 的对应测试通过；GPU 项明确跳过
PyTorch 层与 custom-op 测试子集	**78 passed、2 skipped**	Linear/Norm/attention/cache，fake、opcheck、gradcheck、CPU fullgraph 等检查
Triton host 参数检查	**17 passed**	EPS 的 FP32 可表示范围与类型合同，不需 GPU
Roofline/benchmark 参考与统计检查	**24 passed**	数学单位、流量模型、reference 语义和统计定义
Triton 自带 interpreter	**34/34 FP32 检查通过**	原 kernel 的高层算术、索引和 mask；不等于 GPU 执行
Triton 离线编译	**48/48 特化成功生成 PTX 和 cubin**	三种 dtype、多种 shape/mask/head dimension 的编译路径成立，目标 sm_80
完整 CUDA kernels.cu 离线编译	**NVCC 13.0.48 编译成功**	12 个 kernel entry 经 ptxas，生成 411,864-byte sm_80 object；未链接
CUDA bindings.cpp host 语法	**g++ -fsyntax-only 通过**	与本次 PyTorch headers 对应的绑定声明和 C++ 语法检查
torch_layers_demo.py	**执行成功**	手写层参考一致、QKV 布局输出、prefill + decode 等价示例
compile_demo.py，CPU / backend=eager	**执行成功**	预期 graph break、where/cond 捕获、两种注册路径与梯度检查

330 个 skipped 的组成是：CUDA C++ GPU 测试 87 项、Triton GPU 测试 241 项、custom-op 的 GPU 接入测试 2 项。它们已经写好并被测试框架收集；“跳过”不代表数值通过。没有使用 CPU 时间代替 GPU 性能。

环境

项目	本次验证环境
Python	3.12.14
PyTorch	2.5.1+cpu
Triton	3.1.0
pytest	9.1.1

项目	本次验证环境
主环境 NumPy	2.3.5
Interpreter 独立进程 NumPy	1.26.4
NVIDIA GPU	当前环境没有可用设备
离线 CUDA compiler	官方 NVIDIA pip 组件 NVCC 13.0.48
离线编译目标	sm_80 / compute capability 8.0

NVCC 通过独立安装路径调用，没有加入主环境 PATH，因此 check_env 中 nvcc 字段可能为空；离线编译的版本、命令与结果另有完整记录。当前 CPU wheel 没有 CUDA runtime，不能据此检查扩展实际链接或在 GPU 上运行。

这组版本用于记录实际做过的检查，不是对所有 GPU 的安装推荐。新硬件应选择支持它的 PyTorch、CUDA toolkit、驱动和 Triton，重新运行实际 GPU 测试。

测试重点

CPU 检查覆盖 Linear 的参数注册和梯度、QKV 形状/stride、Norm 的前向/反向及 FP32/FP64 统计策略、autocast、不同 normalized_shape、causal offset、全遮蔽行的输出与梯度、空 attention key 域的约定，以及 cache 的单 token/多 token chunk 等价。

两种 custom_op 同时检查 schema/fake/autograd/compile 的集成合同与 double 数值梯度。新增的定向测试检查 inactive ReLU 如何阻断上游 Inf/NaN：实际 backward 使用 where，避免用乘法产生 0×Inf。Norm 还拒绝在 FP32 accumulator 中舍入为 0 或 Inf 的 eps，同时保留 FP64 中合法的极小 eps。

GPU 测试文件覆盖尾块、非整除尺寸、空张量、K=0、Norm、attention offset 和 mask、非默认 stream 等。CUDA 文件额外包含需要两块 GPU 的 device guard 检查。实际运行时需要相应硬件条件。

Interpreter 的边界与复现

本次直接调用原 Triton kernel，由 Triton 自带 interpreter 在 CPU 上执行。没有改写 kernel 为另一套模拟实现，也没有让公开 CUDA-only wrapper 自动退回 CPU。

Triton 3.1.0 在本次 NumPy 2.3.5 环境里，最小独立 load/store probe 就不能正确读写。将 NumPy 1.26.4 安装到独立 target 目录，并只在解释器进程内设置 PYTHONPATH 后，该 probe 与全部 34 项数学检查通过。工具保留这项 preflight，遇到不兼容环境会报告 blocked，不会把环境错误当成练习通过。

需要复现时，建议创建单独环境并安装兼容版本，例如 CPU PyTorch、Triton 3.1.0、NumPy 1.26.4，再执行：

```
TRITON_INTERPRET=1 python tools/check_triton_interpreter.py
```

该检查不覆盖 GPU lane/layout lowering、真实并行同步、竞态、寄存器/shared-memory 资源限制、FP16/BF16 运行或性能。正式 GPU benchmark 和 demo 会拒绝在 TRITON_INTERPRET=1 下运行 Triton 路径，以防计时对象被改变。

离线编译的边界与复现

Triton 检查入口：

```
python tools/compile_triton_offline.py --arch 80 --output results/triton_offline_compile.json
```

该工具使用的 ASTSource/GPUTarget 接口是在 Triton 3.1.0 上验证的内部编译接口，新版本可能需要调整。它验证 AST→IR→PTX→cubin 的编译路径，没有分配 GPU tensor，也没有运行 device code。

CUDA 离线检查使用完整 kernels.cu；具体编译参数保存于 results/cuda_offline_compile_command.json。CPU Torch headers 不包含 CUDA 生成宏文件，因此仅在离线命令中使用头文件支持的 C10_CUDA_NO_CMAKE_CONFIGURE_FILE 开关。NVIDIA pip compiler 的 NVVM 工具目录还做了一处本地路径修正，详细记录于 results/cuda_offline_environment.txt。交付的 CUDA 源码与正常 JIT loader 没有加入这些环境修正。

尚未由本环境验证的项目

- CUDA 扩展在用户实际 CUDA/PyTorch 组合上的完整链接。
- NVIDIA GPU 上 CUDA 和 Triton 的数值结果、真实线程同步、stream 与设备切换。
- GPU FP16/BF16 的运行数值与实际 Tensor Core 路径。
- GPU Inductor 编译后的前后向运行。
- GPU 延迟、吞吐、显存和任何优化 speedup。

教材中的 roofline 图和时间数字明确属于假设硬件上的模型计算。compile demo 日志中的 CPU 计时用于展示“首次/稳态”测量方法；没有把它们写成 GPU 优化结论。

用户机器上的下一组检查

```
python -m gpu_prep.check_env --require-gpu
python -m pytest tests/test_cuda_ops.py tests/test_triton_ops.py tests/test_custom_ops.py -q
python examples/run_gpu.py --backend all --op all
python examples/compile_demo.py --device cuda --backend inductor --include-cond
python -m gpu_prep.benchmark --op matmul --shape 128,256,384 --backends torch,cuda,triton --output
results/my_gpu_matmul.json
```

准备好 GPU 和编译工具链后，这些命令会真正调用相应实现。某个 GPU-only case 被跳过时，应查看其 skip 原因，而不是只看进程退出码。

各后端共享的工程约定

CUDA C++：从能写对，到能解释性能

本章对应练习 1、2、3、4、5、8、9。代码是本教材编写的教学实现；目标是让线程映射、边界、同步和数据流可以逐行解释。它没有承诺比 PyTorch、cuBLAS 或成熟 attention 库更快。

共同约定与运行方式

完整源码位于 `gpu_prep/csrc/ops.h`、`gpu_prep/csrc/bindings.cpp`、`gpu_prep/csrc/kernels.cu`；Python 入口为 `gpu_prep/cuda_ops.py`。`import gpu_prep.cuda_ops` 不编译，第一次实际调用或显式 `load_extension()` 才进行 JIT 构建。

```
import torch
from gpu_prep import cuda_ops

cuda_ops.load_extension()  # 先构建，别把构建时间算进性能数字。
x = torch.randn(257, device="cuda", dtype=torch.float32)
y = torch.randn_like(x)
torch.testing.assert_close(cuda_ops.vector_add(x, y), x + y)
```

所有 CUDA 入口要求输入在同一 NVIDIA GPU 上、`torch.float32`、连续存储。CUDA 部分只有 forward；启用梯度记录且输入需要梯度时明确报错。推理时可以在 `torch.no_grad()` 中调用。这个限制避免让一个缺少 backward 的实现产生看似可训练的错误结果；它不是说 PyTorch 自定义 CUDA 算子不能支持训练。

输入数值的主要约定是有限 FP32 值，并要求计算中间量不发生溢出。host 不扫描张量内容，否则会增加 GPU 操作和同步。`row_max` 使用 `fmaxf`，不承诺与 PyTorch 的 NaN 传播一致；softmax 和 attention 也没有为输入 NaN、正无穷或整行负无穷定义额外协议。融合 `bias_relu` 特别保留了加法结果的 NaN，便于与对应 PyTorch 表达式对照。

API	输入	输出	空维与其他限制
`vector_add(x, y)`	相同 shape	相同 shape	允许零元素，也允许标量
`bias_relu(x, bias)`	`[M,N]`、`[N]`	`[M,N]`	M/N 可为 0
`row_sum` / `row_max`	`[M,N]`	`[M]`	N > 0；M 可为 0
`softmax`	`[M,N]`	`[M,N]`	N > 0；M 可为 0
两种 `transpose`	`[M,N]`	连续的 `[N,M]`	M/N 可为 0
两种 `matmul`	`[M,K]`、`[K,N]`	`[M,N]`	K=0 时返回零；M/N=0 时为空
`layer_norm`	`[...,D]`、weight/bias `[D]`	与 x 同形状	D > 0；前导维可为 0；eps 正且能以 FP32 表示
`rms_norm`	`[...,D]`、weight `[D]`	与 x 同形状	同上
`attention`	q `[B,H,L,D]`，k/v `[B,H,S,D]`	`[B,H,L,D]`	1 ⩽ S ⩽ 4096；D > 0；L/B/H 可为 0；query_start ⩾ 0

Host 代码的六个关键点

1. **先检查 shape、dtype、device、contiguous。** 内核按连续 FP32 数据解释指针，不能把调用者传错的 layout 当成正确输入。
2. **用 `CUDAGuard` 切换到输入的设备。** 用户当前设备可能不是 x 所在设备；函数离开后 guard 恢复原来的设备状态。
3. **在 PyTorch 当前 stream 上 launch。** 输入的生成与后续使用都可能发生在非默认 stream；擅自使用默认 stream 会破坏依赖顺序。
4. **先处理空张量，避免零 block launch。** GEMM 的 K=0 是空求和，因此返回 0；它不同于未初始化的 `empty`。
5. **指针偏移与维度用 `int64_t`。** `blockIdx.x * blockDim.x` 在相乘前就转换为 64 位，避免先发生 32 位溢出再转换。
6. **launch 后检查 CUDA 错误。** `C10_CUDA_KERNEL_LAUNCH_CHECK()` 不等于等待 kernel 执行结束；异步执行中的越界错误可能在后续同步才暴露。常规算子内不加入 `cudaDeviceSynchronize()`，否则会改变上层的并发与计时行为。

这些 API 的设备与 stream 语义可参考 PyTorch CUDA Support；JIT extension 的构建入口参考 PyTorch C++ Extensions。

怎样证明“没有漏算，也没有重复写”

元素级 kernel 使用 `first = blockIdx.x * blockDim.x + threadIdx.x` 和 `step = gridDim.x * blockDim.x`。每个线程处理 `first, first+step, first+2*step, ...`，直到超出元素个数。

对于任意合法下标 i，整数除法唯一地确定 `i % step` 对应的初始线程和 `i // step` 对应的循环次数。因此每个合法元素恰好被一个线程写一次。grid 的 block 数可以封顶，不会因此遗漏更大的输入。行级和 tile 级 kernel 使用相同原理在 rows 或 tiles 上循环。

Triton 实现、逐题解释与面试追问

本章对应源码 `gpu_prep/triton_ops.py` 和验证脚本 `tests/test_triton_ops.py`。源码是便于逐行解释的教学实现；固定 tile 配置让不同版本容易比较。这里不预设某个配置在你的 GPU 上最快，也不把没有测量的数据写成加速比。

先理解：CUDA 的线程视角，与 Triton 的 tile 视角

在 CUDA C++ 中，你通常先计算一个线程的全局编号，再决定这个线程读写哪些元素。在本章的 Triton 中，你先描述一个 program 处理的逻辑张量块，再由编译器把这个块分配给线程、warp 和寄存器。在这些常规 NVIDIA kernels 中，program 通常对应一个 CTA／thread block；但是 `tl.arange(0, 256)` 表示 256 个逻辑元素，不是“启动 256 个 CUDA 线程”。

例如 `BLOCK=256, num_warps=4`：一个 program 描述 256 个元素，配置 4 个 warp；具体哪些元素交给哪些 lane，是编译器布局的一部分。不要机械地把 CUDA `blockDim.x` 替换成 Triton 的 `BLOCK`。

代码构件	你应该怎样解释
`tl.program_id(0)`	当前 program 在 launch grid 中的编号
`tl.arange(0, BLOCK)`	一个编译期已知长度的逻辑索引向量
`row[:, None]`、`col[None, :]`	通过广播构造二维索引，而不是用 Python 循环逐元素处理
`tl.load(ptr, mask, other=...)`	只访问 mask 为真的地址；被屏蔽的位置使用指定占位值
`tl.store(ptr, value, mask)`	控制实际写入的地址，避免尾块越界
`tl.sum`、`tl.max`	program 内的归约，编译器生成所需线程间通信
`tl.constexpr`	参数是编译期常量，可用于张量形状、分支特化等
`num_warps`	program 使用的 warp 数，影响并行归约与资源分配
`num_stages`	流水线调度配置；效果与 kernel、后端、硬件有关

本实现把 M/N/K 等尺寸作为 `tl.constexpr`。尺寸变化可能触发新的编译特化，第一次调用时间不能直接作为稳态 kernel latency。实际部署可以进一步减少不必要的特化，并把编译成本单独统计。

所有接口共同遵守的范围

- NVIDIA CUDA、compute capability 至少为 8.0；不在这个模块中适配 ROCm、CPU 或旧架构。当前 Triton 官方仓库的 Compatibility 也将 NVIDIA 支持范围列为 8.0+。Triton 官方仓库
- 支持 FP32／FP16／BF16。同一个操作中的浮点输入具有相同 dtype 和 device，均为 contiguous。
- 所有浮点输出保持输入 dtype。归约、Norm 统计量、矩乘累加器、attention 在线状态使用 FP32。输出转回 FP16 仍可能溢出，内部 FP32 累加并不等于输出自动变成 FP32。
- 普通数值测试针对有限输入，不承诺完整复刻各框架对 NaN、Inf、溢出和 signed zero 的全部行为。独立 softmax 不支持全 `-inf` 行；attention 对全布尔 mask 行有专门约定。
- 单 program 处理一行的 sum／max／softmax／Norm 限制最后一维在 1–8192，目的是控制教学实现的资源范围；8192 不是 Triton 的理论最大行宽。
- vector、bias+ReLU、transpose 的零元素输出直接返回。row sum／max／softmax 要求 N>0，M 可以为 0。Norm 的归一化维度必须大于 0，前导维可以为 0。GEMM 的 K=0 返回零矩阵；M=0 或 N=0 返回空矩阵。attention 要求 S>0，L=0

可以返回空输出。

- 索引范围限制为不超过 `2**31-1` 个元素；这是当前实现的显式限制。所有逻辑二维 grid 展平成一维 launch，避免额外依赖 CUDA grid 的 y 维上限。
- 这些 raw Triton kernels 只实现 forward。grad mode 开启、任何浮点输入 `requires_grad=True` 时明确报错；在 `torch.no_grad()` 下允许推理调用。若要训练，必须另写并注册正确的 backward。
- wrapper 用 `torch.cuda.device(input.device)` 选择输入设备，退出时恢复原来的 current device；没有强制全设备同步。

`tl.dot(..., input_precision="ieee")` 的目的，是明确 FP32 点积不默默使用 TF32。它不保证整个 softmax／Norm／attention 与 PyTorch 逐 bit 相同：归约顺序、`exp`、`rsqrt`、FMA 等仍会带来舍入差异。API 口径见 Triton `dot`。

PyTorch 层：从形状、数值到训练与 KV cache

本章的实现位于 `gpu_prep/torch_layers.py`；演示位于 `examples/torch_layers_demo.py`；有独立参考实现、梯度和缓存等价性检查的测试位于 `tests/test_torch_layers.py`。这些是可读、可验证的教学实现。它们使用 PyTorch 已有算子组成计算图，所以可在 CPU 上检查数学与 autograd，在 GPU 上执行；它们不会自动等同于一个融合 CUDA kernel。

---

torch.compile 与自定义算子接入

本章使用 `examples/compile_demo.py` 和 `gpu_prep/custom_ops.py`，采用在 PyTorch 2.5 已有的 `torch.library.custom_op`、`register_fake`、`register_autograd`、`opcheck` 接口。提供 CUDA C++ 与 Triton 两条 GPU 接入路径；两者都有 CPU 数学参考，以便没有 GPU 时检查注册和梯度。CPU fallback 不代表 GPU kernel 已经通过运行验证。

第 1 题｜Vector Add 与 Bias + ReLU 融合

完整源文件： gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 1 的 CUDA 实现：Vector Add 与 Bias + ReLU

题目与代码入口

先写 `z[i] = x[i] + y[i]`，再扩展成 `out[m,n] = max(x[m,n] + bias[n], 0)`。对应 `vector_add_kernel`、`bias_relu_kernel` 及其 host wrapper。

用下面的例子检查自己是否真的理解索引：N=257，block=256。初次 launch 有两个 block，第二个 block 的第 0 个线程处理 i=256，其余线程因为 `i < N` 为假而不写内存。这些线程可以直接结束，因为该 kernel 没有任何 block 内同步。

Bias 版本先把 `[M,N]` 展平为 `numel=M*N` 个元素。展平下标 i 对应列 `i % N`，因此使用 `bias[i % N]`；行号不需要显式算出。N=0 时 host 已提前返回，kernel 内不会发生除以零或模零。

数据流、复杂度与正确性

每个线程只读自己的输入位置、只写自己的输出位置，不与其他线程共享中间结果，因此没有原子操作或 `__syncthreads()`。相邻线程对 x/y/out 访问相邻 FP32 元素，适合合并访存。元素总工作量为 O(N)，每线程额外存储 O(1)。Bias + ReLU 为 O(MN)。

向量加法的理想有效数据量是 `12N Byte`：两次 FP32 读取和一次 FP32 写入。每元素只有 1 次浮点加法，算术强度约为 `1/12 FLOP/Byte`。这个字节数是算法层面的有效流量模型，缓存、写策略和事务粒度会影响实际 HBM 流量。

Bias + ReLU 若拆成两个 kernel，需要把中间 `x+bias` 写回并再次读取。融合版省去这份中间张量的读写，并少一次 launch。bias 能在缓存中被重复使用，但不能把“所有 bias 访问都命中缓存”当成无需验证的事实。

可以直接复述的追问答案

问：为什么不需要同步?

答：每个输出只依赖同下标的输入，线程之间没有生产者与消费者关系，也没有共享写入。加 barrier 不会补充正确性，只会增加等待。是否需要同步取决于数据依赖，不能因为是 CUDA 就习惯性加同步。

问：block size 取 256 是最优吗?

答：它是方便理解和比较的起点，不是对所有 GPU、长度和融合表达式的最优结论。需要比较若干 block size，并观察寄存器、可同时驻留的 blocks、launch 开销和有效带宽。简单大向量通常很容易获得足够并行度；小向量可能主要受固定启动成本影响。

问：怎样继续优化?

答：先排除首次构建，并明确测量范围。若测纯 kernel，要预分配输出并说明这一点；若测本教材的 wrapper，则保留它内部必要的输出分配，让 baseline 使用相同口径，并区分 GPU event 时间与 host wall 时间。随后检查是否接近合理的带宽水平，再考虑减少 launch、与相邻算子融合、一次处理多个元素、在满足对齐和尾部处理条件下做向量化 load/store。`float4` 不是无条件加速开关：起始地址、长度、stride 都要满足要求，尾部仍需正确处理。

问：如果 x 是转置出来的非连续 tensor 呢?

答：这份接口明确拒绝。扩展方法有两种：上层显式 `.contiguous()`，或者把 stride 作为参数，根据坐标计算真实地址。前者代码简单但可能发生额外拷贝；后者支持面广，但地址计算和访存模式更复杂。性能比较时必须把所选方案的必要转换成本纳入目标范围。

问：FP16/BF16 版本怎么写？

答：把 load/store 类型模板化；加载后按所需数值约定转换到 float 运算，再转换回输出类型。带宽需求会下降，但转换、对齐和实际指令路径需要测量。纯加法也可以研究 half2 等打包运算，不过这与保持 FP32 中间结果是不同的精度和实现选择，必须明确比较标准。

问：原地写 out=x 可以吗？

答：对于同形状逐元素加法，某些别名关系在数学上可以安全实现；但要逐一分析输入与输出是否有重叠、broadcast 是否导致一个位置被多处读取。本教材始终新分配输出，便于建立无别名的基础契约。若增加 `out=`，要同时设计别名检查与 autograd 语义。

练习顺序

从空文件写基础版本，测试 N=0、1、31、32、33、255、256、257、1000003；再写 bias 广播；最后解释融合省下的具体读写。代码与讲解都完成后，再比较性能。

第 1 题的 Triton 版本：elementwise 与融合 bias+ReLU

对应实现与调用

- `vector_add(x, y)` → `_vector_add_kernel`
- `bias_relu(x, bias)` → `_bias_relu_kernel`

调用示例：

```
import torch
from gpu_prep import triton_ops

x = torch.randn(257, device="cuda", dtype=torch.float32)
y = torch.randn_like(x)
z = triton_ops.vector_add(x, y)

matrix = torch.randn(5, 513, device="cuda")
bias = torch.randn(513, device="cuda")
activated = triton_ops.bias_relu(matrix, bias)
```

为什么索引是这样写的？

令一个 program 负责 256 个逻辑位置：

$$i = 256 \times \text{program_id} + \{0, 1, \cdots, 255\}.$$

长度为 257 时，grid 大小为 `ceil(257/256)=2`。第 0 个 program 处理 0–255；第 1 个 program 的逻辑范围是 256–511，但只有 `i=256` 的 mask 为真。所有 load 和 store 都使用 `i<N`，因此尾部无效位置不会产生实际内存访问。

注意，“构造了一个超出数组范围的逻辑偏移”与“真的从那个地址读取”是两件事。掩码必须放在 `tl.load`／`tl.store` 上，不能先无条件读取，再在数值结果上 `where` 掉越界元素。`tl.load` 的 mask 与占位值语义见 官方 API。

对 `X[M,N]+bias[N]`，先把连续矩阵看成长度 MN 的数组。位置 `i` 的列编号为 `i % N`，因此读取 `Bias+i%N` 就实现了列广播，不需要先创建 M 份 bias。N=0 时 wrapper 在 launch 前直接返回，避免 kernel 中出现模 0。

精度与复杂度

加载的半精度数据先转 FP32，完成加法／ReLU 后按输出指针 dtype 转回。对有限输入，这个计算约定可用 `(x.float()+y.float()).to(x.dtype)` 或 `(x.float()+bias.float()).clamp_min(0).to(x.dtype)` 作为参考。

向量加法总计算量 O(N)，输出空间 O(N)。FP32 情况下，最简单的数据流是读 x、读 y、写 z，约 12N Byte；每元素只有一次加法。大输入往往首先受访存约束，小输入则可能主要受 launch 与调度开销影响。

bias+ReLU 融合避免了中间加法结果的一次写入和一次读取。若中间结果是 FP32、共有 MN 个元素，这两次逻辑流量合计为 8MN Byte；实际 HBM 节省还取决于缓存和执行方式，不能直接把这个数字等同于实测带宽下降。

面试追问与可直接使用的回答

问 1：为什么不用 `tl.arange(0,N)`？

答：这里需要编译器支持的静态块形状，并且不希望一个 program 无限制地包含整个输入。我选固定 256 个元素，使用多个 programs 覆盖 N；行归约则会把行宽向上补到 2 的幂。真实有效范围仍由 mask 决定。这样既处理非整除长度，也控制单个 program 的资源使用。

问 2：`BLOCK=256`、`num_warps=4` 为什么不是矛盾的?

答：BLOCK 是逻辑元素数，num_warps 决定执行这个 program 的 warp 数；一个 lane 可以处理多个逻辑元素。Triton 编译器负责映射。不能认为 256 个逻辑元素就必须对应 256 个硬件线程。

问 3：下一步怎么优化?

答：先区分规模。大数组先检查实际访存吞吐、合并访问与对齐，再比较 BLOCK 和 num_warps；小数组先看能否与相邻算子融合，减少 launch。对于 bias+ReLU，我优先保留融合，并检查是否存在隐式 `.contiguous()`、dtype cast 或额外分配掩盖收益。只有 profiler 和基准显示有必要时，才继续调整布局或更复杂的调度。

问 4：把 BLOCK 提到 4096 一定更好吗?

答：不一定。每个 program 处理更多元素会减少 program 数，也可能增加寄存器占用、减少可同时驻留的 programs；小 N 还会产生更多无效逻辑元素。需要覆盖代表性 N 测试，而不是只选一个大输入。

问 5：为什么拒绝 noncontiguous 输入，不自动复制?

答：当前地址公式利用了连续布局这一合同。自动复制会隐藏数据搬运成本，使我误把“算子加复制”的性能当成算子性能。教学版本明确拒绝；若产品要求支持一般 stride，我会把 strides 纳入地址计算，并把不同布局单独测试。也可以显式复制，但必须把复制的成本放到正确的计时范围里。

这题怎样练到能现场写?

先从空文件写 `idx`、`valid`、两次 load、一次 store；使用 N=257 验证第二个 program 的尾部。再改成二维 bias 广播。最后遮住代码，用 90 秒解释为什么不用同步，以及融合到底少了哪些数据搬运。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/csrc/kernels.cu

```
__global__ void vector_add_kernel(const float* x, const float* y, float* out,
                                  int64_t n) {
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < n; i += step) {
```

```
    out[i] = x[i] + y[i];
  }
}

__global__ void bias_relu_kernel(const float* x, const float* bias, float* out,
                                 int64_t numel, int64_t width) {
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < numel; i += step) {
    const float z = x[i] + bias[i % width];
    // 正常有限值下等价于 ReLU；z 是 NaN 时本分支仍保留 NaN。
    out[i] = z < 0.0f ? 0.0f : z;
  }
}

at::Tensor vector_add_cuda(const at::Tensor& x, const at::Tensor& y) {
  check_tensor(x, "x");
  check_tensor(y, "y");
  check_same_device(x, y, "y");
  TORCH_CHECK(x.sizes() == y.sizes(), "x and y must have identical shapes");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  vector_add_kernel<<<blocks_for(x.numel(), kThreads), kThreads, 0, stream>>>(
      x.data_ptr<float>(), y.data_ptr<float>(), out.data_ptr<float>(), x.numel());
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor bias_relu_cuda(const at::Tensor& x, const at::Tensor& bias) {
  check_matrix(x, "x");
  check_tensor(bias, "bias");
  check_same_device(x, bias, "bias");
  TORCH_CHECK(bias.dim() == 1 && bias.numel() == x.size(1),
              "bias must have shape [N] for x[M, N]");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;  // 也避免在 width=0 时取模。
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  bias_relu_kernel<<<blocks_for(x.numel(), kThreads), kThreads, 0, stream>>>(
      x.data_ptr<float>(), bias.data_ptr<float>(), out.data_ptr<float>(),
      x.numel(), x.size(1));
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}
```

gpu_prep/triton_ops.py

```
@triton.jit
def _vector_add_kernel(X, Y, Z, N: tl.constexpr, BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    valid = idx < N
    x = tl.load(X + idx, valid, other=0).to(tl.float32)
    y = tl.load(Y + idx, valid, other=0).to(tl.float32)
    tl.store(Z + idx, x + y, valid)


def vector_add(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """z = x + y for contiguous one-dimensional tensors of identical shape."""
    _check_tensor(x, "x", ndim=1)
    _check_peer(x, y, "y", ndim=1)
    if x.shape != y.shape:
        raise ValueError("x and y must have identical shapes")
    out = torch.empty_like(x)
```

```
    if x.numel() == 0:
        return out
    with torch.cuda.device(x.device):
        _vector_add_kernel[(triton.cdiv(x.numel(), 256),)](
            x, y, out, x.numel(), BLOCK=256, num_warps=4,
        )
    return out


@triton.jit
def _bias_relu_kernel(X, Bias, Y, TOTAL: tl.constexpr, N: tl.constexpr, BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    valid = idx < TOTAL
    x = tl.load(X + idx, valid, other=0).to(tl.float32)
    b = tl.load(Bias + idx % N, valid, other=0).to(tl.float32)
    tl.store(Y + idx, tl.maximum(x + b, 0.0), valid)


def bias_relu(x: torch.Tensor, bias: torch.Tensor) -> torch.Tensor:
    """y[m,n] = max(x[m,n] + bias[n], 0); no materialized broadcast tensor."""
    _check_tensor(x, "x", ndim=2)
    _check_peer(x, bias, "bias", ndim=1)
    if bias.shape[0] != x.shape[1]:
        raise ValueError("bias must have shape [x.shape[1]]")
    out = torch.empty_like(x)
    if x.numel() == 0:  # Also prevents taking a remainder by N=0 inside the kernel.
        return out
    with torch.cuda.device(x.device):
        _bias_relu_kernel[(triton.cdiv(x.numel(), 256),)](
            x, bias, out, x.numel(), x.shape[1], BLOCK=256, num_warps=4,
        )
    return out
```

第 2 题｜Row Sum 与 Row Max 归约

完整源文件： gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 2 的 CUDA 实现：Row Sum 与 Row Max

线程映射，逐步拆开

一个 CTA/block 负责一行，固定 256 个线程。线程 t 读取列 `t, t+256, t+512, ...`，在寄存器里维护一个局部结果。所有列被这些互不相交的序列覆盖。

以 N=513 为例：线程 0 处理 0、256、512；线程 1 处理 1、257；线程 255 处理 255、511。这里完全不要求 N 是 2 的幂。要求为 2 的幂的是本例进行 shared-memory 树形归约的线程数。

局部计算结束后，每个线程把值放入 `scratch[tid]`。随后依次做 stride=128、64、32、16、8、4、2、1 的合并。第一步线程 0 合并 0 和 128，第二步合并先前结果与 64 的结果，最终 `scratch[0]` 包含整行结果。

Sum 的无效线程初值是 0；max 的无效线程初值是负无穷。把 max 初值设成 0 会使全负数输入出错。

同步为什么正好在这些位置

1. 所有线程写 scratch 后同步，防止读取尚未被写入的局部值。
2. 每一层合并后同步，防止下一层过早读取上一层的结果。
3. 把 `scratch[0]` 复制到每个线程的局部变量后，再同步一次，才允许下一次 reduction 覆盖 scratch。

第三点很容易忽略：这个 helper 在 softmax、norm 中会被连续调用，同一个 block 也可能继续处理下一行。如果某个线程已经覆盖 scratch，而其他线程还没读走旧结果，就存在 race。NVIDIA 对 block 协作和 barrier 的基本语义有说明：Writing SIMT Kernels。

复杂度与带宽模型

总输入工作为 O(MN)，输出 O(M)。固定线程数 B=256 时，一行的计算路径大致为 O(ceil(N/B)+log B)，不能把这个路径长度直接等同于整块 GPU 的总运行时间。额外 shared memory 是每 CTA `256×4=1024 Byte`。理想有效流量约为 `4MN+4M Byte`。

可以直接复述的追问答案

问：为什么 N 不是 2 的幂也正确?

答：原始 N 个元素先被分给固定 256 个线程，每个线程产生一个局部值。树形归约处理的是这 256 个局部值。没有元素的线程使用 sum/max 的单位元，因此不影响结果。N 是否为 2 的幂与后面这棵固定大小的归约树无关。

问：为什么不让所有线程对一个结果做 atomicAdd?

答：大量线程更新同一个地址会产生争用。先在寄存器和 block 内合并，能够显著减少需要全局合并的次数。单行只用一个 block 时根本不需要全局原子操作；如果宽度太大要用多个 blocks，就需要第二阶段 reduction，或者对每个 block 的局部结果做原子更新。选择需要依据行数、行长、开销和确定性要求。

问：为什么与 torch.sum 的结果不逐 bit 一样?

答：浮点加法通常不满足结合律。不同 reduction tree 改变了求和顺序，FMA 或累加精度也会改变舍入。先用 FP64 reference 检查误差，并给出合理 atol/rtol；不能把所有差异当作 bug，也不能用很宽的容差掩盖索引错误。max 对普通有限值是取同一组元素的最大值，通常可以做精确比较。

问：warp shuffle 版本怎样优化这棵树?

答：每个 warp 先在寄存器之间用 shuffle 合并；每个 warp 的 lane 0 将一个结果写入 shared memory；第一个 warp 再合并这些 warp 结果。256 个线程只有 8 个 warp，shared-memory 通信量与 block barrier 数都会减少。必须确保 `_sync` mask 所指的 lanes 确实参与相同的 shuffle；尾部元素应该用单位元填充，而不是让被 mask 声明参与的线程提前退出。

对固定 256 线程、一维 block，可把 sum helper 改写为下面的进阶版本。它返回每个线程都可读取的结果，并保留 shared-memory 复用前的同步；不要未经修改拿来支持任意 block 大小。它是独立的替换示例，默认 `kernels.cu` 仍使用上面的 shared-memory 树；替换后需要重新编译并运行 correctness tests。

```
__device__ float warp_sum_full(float value) {
    for (int offset = 16; offset > 0; offset /= 2)
        value += __shfl_down_sync(0xffffffffu, value, offset);
    return value;
}

__device__ float block_sum_256(float value) {
    __shared__ float warp_totals[8];
    const int tid = threadIdx.x;
    const int lane = tid & 31;
    const int warp = tid >> 5;
    value = warp_sum_full(value);  // 每个 warp 的所有 32 lanes 都参与。
    if (lane == 0) warp_totals[warp] = value;
    __syncthreads();

    if (warp == 0) {
        float total = lane < 8 ? warp_totals[lane] : 0.0f;
        total = warp_sum_full(total);
        if (lane == 0) warp_totals[0] = total;
    }
    __syncthreads();
    const float result = warp_totals[0];
    __syncthreads();
    return result;
}
```

问：什么时候 one-block-per-row 不合适?

答：当行数很少而单行特别长时，总 block 数不足以使用整块 GPU，可以把一行分成多个 chunks，再做第二阶段归约。相反，当每行只有几个元素而行数很多时，256 线程处理一行会浪费，可以让一个 warp 甚至更小线程组处理一行，一个 block 处理多行。两种情况需要不同映射，不存在只调大 block size 就解决所有尺寸的规则。

问：怎么扩展低精度?

答：输入可以是 FP16/BF16，但建议局部和及 block sum 用 FP32，输出最后再转换。累加精度和输出 dtype 是两个独立选择。若要求非常高的求和精度，还可以研究 FP64 或补偿求和，但会改变吞吐和寄存器成本。

第 2 题的 Triton 版本：逐行 sum 与 max

对应实现与思路

`row_sum(x)` 和 `row_max(x)` 共用 `_row_reduce_kernel`，通过编译期 `IS_MAX` 选择归约。输入 X[M,N]；每个 program 负责一行，输出 Y[M]。

例如 N=33，会选择 BLOCK=64。load 得到 64 个逻辑数，其中后 31 个是占位值：sum 使用 0，max 使用负无穷。随后在 program 内执行 `tl.sum(values, axis=0)` 或 `tl.max(values, axis=0)`。无效元素不改变归约结果。

如果 max 的无效位置错误填 0，一行全为负数时就会返回 0；这是一个非常有价值的最小反例。只测 `rand()` 生成的正数可能发现不了这个错误。

总工作量 O(MN)，输出 O(M)。program 内的并行归约有树形合并过程，但每个线程可能处理多个数，因此不要把整道题的总工作量写成 O(log N)。对于一般输入，所有 MN 个元素都需要读取。

精度与形状范围

sum 在 FP32 中累加，然后转回输入 dtype。max 也用 FP32 中间表示；FP16/BF16 有限值能精确表示为 FP32。结果与 `torch.sum` 的误差仍可能来自不同归约顺序；浮点加法不满足结合律。

本版本要求 1≤N≤8192，M=0 允许空输出。N>8192 时直接报错，不悄悄生成一个资源占用未知的超大 program。要支持任意大 N，应该明确增加分块和第二阶段归约。

面试追问与回答

问 1：为什么 sum 填 0，max 填负无穷?

答：它们分别是这两个运算的单位元：任何数加 0 不变，任何有限数与负无穷取 max 不变。mask 只保证不越界；占位值仍会参与后续计算，所以必须按照归约运算选择。

问 2：Triton 没有显式 `__syncthreads()`，归约如何正确?

答：我使用 program 内的 `tl.sum` / `tl.max`，编译器根据数据布局生成所需的 lane、warp 间通信与同步。这里不同 programs 分别处理独立的行，彼此没有数据依赖。若我要跨 program 合并，就必须增加显式的算法阶段，不能假设 program 之间有隐式全局 barrier。

问 3：N 很大应该怎么改?

答：第一阶段将每一行分成若干块，每个 program 输出一个 partial sum 或 partial max；第二阶段归约这些 partials。它多一个临时数组和 kernel launch，但减少单个 program 的资源压力，也可让一行由多个 programs 并行处理。sum 的合并仍然存在浮点顺序差异，需要按目标精度验证。

问 4：可以用 atomicAdd 吗?

答：可以作为一种实现选择，但需要先正确初始化输出，还要考虑竞争和浮点合并顺序。直接把所有块的结果原子加到同一行，可能使结果的末位随调度变化。对于这份便于解释和验证的代码，我先选两阶段归约来扩展大行，不默认 atomic 更快。

问 5：N=8、M 很大时，一行一个 program 合适吗?

答：可能浪费执行资源。此时可以让一个 program 处理多行，使每个 warp 或子组处理一行，减少 program 调度开销。相反，M 很小、N 很大时，需要增加一行内部的并行度。因此 M 和 N 都会影响布局选择，不能只按总元素数选择配置。

问 6：如何判断误差容忍度不是在掩盖 bug?

答：先用可精确验证的输入：全 1、全负数、单个非零、N=1；再用随机输入与更高精度参考比较。对 sum 同时看绝对误差和相对误差，因为真实结果接近 0 时相对误差会夸大。若错误集中在 33、513 这类尾块尺寸，优先怀疑 mask；若误差随长度平滑增长，再分析舍入和归约顺序。

学习 `tl.sum` 与行归约映射时，可对照 官方 Fused Softmax 教程，但本题应先独立完成 sum／max，避免把归约逻辑藏在完整 softmax 里。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/csrc/kernels.cu

```
template <bool IsMax>
__device__ float block_reduce(float value, float* scratch) {
  const int tid = threadIdx.x;
  scratch[tid] = value;
  __syncthreads();
  for (int stride = kThreads / 2; stride > 0; stride /= 2) {
    if (tid < stride) {
      scratch[tid] = IsMax ? fmaxf(scratch[tid], scratch[tid + stride])
                           : scratch[tid] + scratch[tid + stride];
    }
    __syncthreads();
  }
  const float result = scratch[0];
  __syncthreads();
  return result;
}

template <bool IsMax>
__global__ void row_reduce_kernel(const float* x, float* out,
                                  int64_t rows, int64_t width) {
  __shared__ float scratch[kThreads];
  for (int64_t row = blockIdx.x; row < rows; row += gridDim.x) {
    float local = IsMax ? -CUDART_INF_F : 0.0f;
    // 相邻线程读相邻列；width 不需要是 2 的幂。
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x) {
      const float value = x[row * width + col];
      local = IsMax ? fmaxf(local, value) : local + value;
    }
    const float reduced = block_reduce<IsMax>(local, scratch);
    if (threadIdx.x == 0) out[row] = reduced;
  }
}

template <bool IsMax>
at::Tensor row_reduce_impl(const at::Tensor& x) {
  check_matrix(x, "x");
  TORCH_CHECK(x.size(1) > 0, "reduction width N must be positive");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty({x.size(0)}, x.options().requires_grad(false));
  if (x.size(0) == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  row_reduce_kernel<IsMax><<<blocks_for(x.size(0)), kThreads, 0, stream>>>(
      x.data_ptr<float>(), out.data_ptr<float>(), x.size(0), x.size(1));
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor row_sum_cuda(const at::Tensor& x) { return row_reduce_impl<false>(x); }

at::Tensor row_max_cuda(const at::Tensor& x) { return row_reduce_impl<true>(x); }
```

gpu_prep/triton_ops.py

```
@triton.jit
def _row_reduce_kernel(X, Y, N: tl.constexpr, IS_MAX: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    if IS_MAX:
        values = tl.load(X + row * N + col, col < N, other=-float("inf")).to(tl.float32)
        result = tl.max(values, axis=0)
    else:
        values = tl.load(X + row * N + col, col < N, other=0).to(tl.float32)
```

```
        result = tl.sum(values, axis=0)
    tl.store(Y + row, result)


def _row_reduce(x: torch.Tensor, is_max: bool) -> torch.Tensor:
    m, n = _row_shape(x)
    out = torch.empty((m,), dtype=x.dtype, device=x.device)
    if m == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        _row_reduce_kernel[(m,)](x, out, n, is_max, BLOCK=block, num_warps=warps)
    return out


def row_sum(x: torch.Tensor) -> torch.Tensor:
    """Sum each row in float32 and cast the result back to x.dtype; N must be > 0."""
    return _row_reduce(x, False)


def row_max(x: torch.Tensor) -> torch.Tensor:
    """Maximum of each nonempty row; output dtype equals x.dtype."""
    return _row_reduce(x, True)
```

第 3 题｜数值稳定的行 Softmax

完整源文件： gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 3 的 CUDA 实现：数值稳定的行 Softmax

三步计算

对一行 x，计算最大值 m、分母 l，再归一化：

$$m = \max_i x_i, \qquad l = \sum_i e^{x_i - m}, \qquad p_i = e^{x_i - m} / l.$$

`softmax_kernel` 用一个 block 负责一行，依次调用 block max、block sum，然后每线程写自己的输出位置。与 reduction 相同，线程按 256 的步长读取列。

本实现不把整行保存在 shared memory 中，因此 N 可以超过 4096。它为清晰和通用宽度做了一个性能取舍：x 读取三遍，指数计算两遍。源码中的三段循环分别对应 max、exp sum、最终输出。

减最大值为什么不会改变答案

对分子分母同时乘以 `exp(-m)`，比值不变。对于有限输入，所有 `x_i-m ≤ 0`，指数不会因为巨大正数而溢出，且至少有一个最大元素对应 `exp(0)=1`，分母因此大于等于 1。很小的项仍可能下溢到 0，这不同于直接 exp 大正数导致无穷再相除产生 NaN。

复杂度与教学版成本

总工作 O(MN)，每 CTA shared memory 1024 Byte。按源码的显式全局访问计数，约有 3 次输入读、1 次输出写，即 `16MN Byte`，但缓存会改变最终 HBM 流量。单次输入读加输出写的理想最低模型则为 `8MN Byte`。这两个模型回答的是不同问题，不能把后者直接作为这份实现的真实 DRAM 字节数。

可以直接复述的追问答案

问：为什么叫稳定，而不是完全无数值误差？

答：减去最大值主要解决 exp 的正向溢出。有限精度减法、指数近似、求和顺序和输出舍入仍会造成误差。稳定不等于精确实数运算，更不代表自动处理所有 NaN、无穷或全 masked 行。

问：怎样减少三次读取和两次 exp？

答：若行宽和资源允许，可以将加载的行片段保存在寄存器中，或者把 exponentials 存到 shared memory，在算出分母之后复用。寄存器过多会限制驻留甚至 spill；shared memory 也会限制并行度。短行、普通行、超长行应当分别选择实现，不能单凭“少读一次”就断言端到端更快。

问：online softmax 怎么更新？

答：维护当前最大值 m 和相对 m 缩放的指数和 l。读到新块，其最大值为 b，令 `m_new=max(m,b)`，更新

$$l_{new} = e^{m - m_{new}} l + \sum_{x \in \text{新块}} e^{x - m_{new}}.$$

旧的 l 必须随最大值改变重新缩放。只更新 max 而不调整旧分母是错误的。对于只输出 softmax 本身的算子，得到全局统计量后通常还需要再访问输入以产生所有输出；online 统计并不自动让所有输出一次读完。

问：mask 怎么加?

答：将不可见位置的分数视为负无穷，使其 exp 变为零。必须额外定义整行都不可见时的行为：此时 `max=-inf`，直接 `-inf-(-inf)` 会出现 NaN，可以明确返回全零或按上层契约报错。本 CUDA softmax 入口只接受普通有限输入，没有任意 mask 参数；不要声称它已经覆盖该扩展。

问：它一定 memory-bound 吗?

答：需要结合 shape 和实现判断。softmax 的指数、归约和同步也有成本；短输入可能 launch-bound，小行数可能并行度不足。先根据模型估计流量和计算，再用实际计时及 profiler 检查瓶颈，不能仅凭算子名字判断。

问：backward 是什么?

答：设前向输出为 p，上游梯度为 g，则每行 `dx = p * (g - sum(g*p))`。因此 backward 也包含行 reduction，可以复用类似的映射。是否保存 p、重新计算 p、融合后续操作，取决于显存和算力的权衡。本 CUDA 教学接口未实现 backward。

问：FP16/BF16 要注意什么?

答：通常先转换到 FP32 做 max、exp、sum 和归一化，再转换回输出 dtype。低精度输入已经发生的量化误差不能靠 FP32 累加恢复，但能避免进一步用低精度统计造成额外损失。

第 3 题的 Triton 版本：稳定 softmax

对应实现与公式

`softmax(x)` 调用 `_softmax_kernel`。仍是一行一个 program，先加载一整行的逻辑块，再依次执行 max、减 max、exp、sum、除法和写回：

$$m = \max_j x_j, \quad e_j = \exp(x_j - m), \quad Z = \sum_j e_j, \quad p_j = e_j / Z.$$

减去同一个 m 不改变 softmax，因为分子和分母同时乘了 `exp(-m)`。对于有限输入，所有有效指数的自变量都不大于 0，最大元素的指数为 1，这样可以避免直接计算 `exp(1000)` 一类溢出。

尾部加载为负无穷，因此它们不改变 max，指数变成 0，不改变 denominator。虽然它们已经在数值上变为 0，最终 store 仍然需要 mask，不能写入不属于输出的地址。

这里把行留在 program 的逻辑张量中，完成所有步骤后一次写回，避免把减法、exp、中间和等结果分别写到全局内存。总工作量是 O(MN)，输出空间 O(MN)，并没有创建同样规模的多个全局中间张量。

与 attention 中 masked softmax 的区别

独立 `softmax` 要求 N>0 且采用有限输入合同。一个全 `-inf` 行会出现 `-inf-(-inf)`，普通稳定 softmax 公式并不能自动解决这个问题。不要把“加了减 max”说成“所有输入都不会出现 NaN”。

本模块的 `attention` 额外定义：布尔 mask 全为假的 query 行输出零，并在在线递推中避免非法差值。两个接口的合同不同，详见第 9 题。

面试追问与回答

问 1：为什么不能先求 exp 再统一缩放?

答：如果先求 `exp(1000)`，溢出已经发生，后续除法无法恢复丢失的信息。减 max 必须发生在 exp 之前。这个变换既保留数学结果，又让指数输入处在更安全的范围内。

问 2：减 max 是不是导致最小值都下溢?

答：很小的概率确实可能下溢到 0，但这些项相对最大项本来就极小；关键是最大项保持为 1、分母非零。是否能接受这些小项的损失由精度需求决定。若任务要求保留很小的 log 概率，可能应直接计算 log-softmax，避免先产生概率再取 log。

问 3：代码为什么把 FP16 输入转成 FP32?

答：max 后的 exp 与 denominator 累加采用 FP32，可以扩大统计量范围并降低累加误差，尤其是行较长时。最后概率才转回输出 dtype。这不消除输入量化误差，也不意味着 output 具有 FP32 精度。

问 4：如果一行太长，放不进当前 program 呢?

答：我可以先对每个块求局部 max 和局部归一化分母，然后用可合并的在线 softmax 状态得到整行的全局 max 与分母，最后重新读取各块并输出概率。若下游马上计算加权和，就可以把加权和也纳入在线状态，避免单独物化概率矩阵；attention 就使用这种办法。

问 5：这个版本一定比 PyTorch 快吗?

答：不能保证。PyTorch 可能已使用高度优化的融合 kernel；单个 program 的寄存器使用、行宽、行数和 dtype 都影响性能。我的优势首先是数据流清楚、没有全局中间张量，是否更快要在目标形状和相同计时口径下测量。

问 6：`tl.exp` 与 PyTorch 结果为什么不是完全相同?

答：实现的数学函数、归约顺序和舍入路径可能不同。我会用有限输入验证归一化和逐元素误差，并选择与 dtype、行宽相称的容忍度。不能只因为结果总和接近 1 就认为每个概率都算对了；还应覆盖大幅度 logits、常数行、长度 1 和尾块。

官方教程：Fused Softmax。在线分母的原始论文：Online normalizer calculation for softmax。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/csrc/kernels.cu

```
__global__ void softmax_kernel(const float* x, float* out,
                               int64_t rows, int64_t width) {
  __shared__ float scratch[kThreads];
  for (int64_t row = blockIdx.x; row < rows; row += gridDim.x) {
    const int64_t base = row * width;
    float local_max = -CUDART_INF_F;
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
      local_max = fmaxf(local_max, x[base + col]);
    const float row_max = block_reduce<true>(local_max, scratch);

    float local_sum = 0.0f;
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
      local_sum += expf(x[base + col] - row_max);
    const float sum = block_reduce<false>(local_sum, scratch);

    // 为了让实现适用于任意 width，不把整行放进 shared memory。
    // 代价是再次读取 x 和再次计算 exp；优化版可在寄存器/共享内存中复用。
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
      out[base + col] = expf(x[base + col] - row_max) / sum;
  }
}

at::Tensor softmax_cuda(const at::Tensor& x) {
  check_matrix(x, "x");
  TORCH_CHECK(x.size(1) > 0, "softmax width N must be positive");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.size(0) == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
```

```
  softmax_kernel<<<blocks_for(x.size(0)), kThreads, 0, stream>>>(
      x.data_ptr<float>(), out.data_ptr<float>(), x.size(0), x.size(1));
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}
```

gpu_prep/triton_ops.py

```
@triton.jit
def _softmax_kernel(X, Y, N: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    values = tl.load(X + row * N + col, col < N, other=-float("inf")).to(tl.float32)
    shifted = values - tl.max(values, axis=0)
    numerator = tl.exp(shifted)
    result = numerator / tl.sum(numerator, axis=0)
    tl.store(Y + row * N + col, result, col < N)


def softmax(x: torch.Tensor) -> torch.Tensor:
    """Stable row softmax for finite inputs; 1 <= N <= MAX_ROW_WIDTH.

    This standalone operation has no mask argument. A row consisting only of
    -inf is outside its finite-input contract. attention() separately defines
    its boolean-mask all-masked-row behavior as a zero output.
    """
    m, n = _row_shape(x)
    out = torch.empty_like(x)
    if m == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        _softmax_kernel[(m,)](x, out, n, BLOCK=block, num_warps=warps)
    return out
```

第 4 题｜Naive 与 Tiled 物理转置

完整源文件： gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 4 的 CUDA 实现：朴素转置与 Tiled 转置

先区分两种“转置”

本题实际搬运数据，输出是连续存储的 `[N,M]`。PyTorch 的 `x.t()` 经常只是一个共享底层存储的 view，因此公平的正确性和拷贝基准应考虑 `x.t().contiguous()`；不能把只改 metadata 的调用时间与真正的数据搬运混在一起。

朴素版本对输入连续下标 i 计算 `row=i/N`、`col=i%N`，再写 `out[col*M+row]`。相邻线程读相邻 x，但写到 out 时相隔 M 个 float，导致写入不利于合并。

Shared tile 如何同时改善读与写

Tiled 版本一个 block 是 `dim3(32,8)`，共有 256 线程，处理 32×32 个元素。每线程沿 y 方向循环 4 次，合计覆盖 1024 个元素。

读取阶段：同一个 warp 的 x 从 0 到 31，读取输入同一行的 32 个连续元素，存到 `tile[local_row][local_col]`。

写入阶段：交换 tile 的行列角色，让同一个 warp 写输出同一行的连续元素；值从 `tile[local_col][local_row]` 取出。shared memory 在这一步承担了线程间重排数据的作用。

这不是只把两个下标交换就结束。全局输出 tile 的坐标也必须交换：输入 tile `(tile_row,tile_col)` 对应输出 tile `(tile_col,tile_row)`。尾部 tile 的读写都带边界检查，因此 M/N 不需要被 32 整除。

为什么用 `tile[32][33]`

在经典 32-bank、FP32 每元素 4 Byte 的分析模型下，`tile[r][c]` 的 bank 可用线性元素下标对 32 取模来理解。若每行长度 32，同一列的 32 个元素都落到同一个 bank；行长变成 33 后，同列不同 r 的 bank 会分散。额外的一列用于改变地址步长，不是输出数据的一部分。

这个具体转置示例和 bank 解释可对照 NVIDIA 的 An Efficient Matrix Transpose in CUDA C/C++。不要把这个 FP32 示例的 padding 结论不加分析地套用到任意 dtype、向量访问或 Tensor Core layout。

同步和正确性

第一次 barrier 位于 tile 全部写完之后，因为读取转置 tile 的线程可能不是写入该元素的线程。第二次 barrier 在输出阶段结束之后，防止 grid-stride tile 循环下一轮提前覆盖仍在读取的 tile。边界线程仍要经过这两个 barrier，不能看到越界就提前 return。

总工作 O(MN)，每个有效元素读一次写一次，理想有效流量 `8MN Byte`；每 CTA 的 shared memory 为 `32×33×4=4224 Byte`。

可以直接复述的追问答案

问：shared memory 不是增加了一次中间读写吗，为什么可能更快?

答：它增加的是片上重排，换来全局读写更好的合并。如果朴素转置每个 warp 触发很多分散的全局事务，减少这些昂贵事务的收益可能超过片上操作与同步成本。小矩阵时收益未必明显，所以要按规模测量。

问：padding 是否增加 global memory 流量?

答：本例 padding 只存在于 shared tile，没有对应的额外输入或输出列。它消耗一些额外 shared memory，但不会要求从输入读取不存在的第 33 列。

问：M=1003、N=769 时怎么保证不越界？

答：tile 数使用向上取整。读时检查原输入 row<M、col<N，越界槽填零；写时检查转置后的 out_row<N、out_col<M。只有合法输出才写，而且它索引到的 tile 槽对应一个合法输入。所有线程仍按相同顺序同步。

问：为什么不用 32×32=1024 个线程？

答：一个 block 的线程数和它处理的数据元素数不必相同。256 个线程通过每线程 4 次加载完成同一个 tile。线程数、每线程工作、寄存器和调度开销需要一起考虑；最大线程数并不自动带来最大带宽。

问：in-place transpose 呢？

答：它是另一个问题。方阵需要协调对角 tile 内转置与两个非对角 tile 的交换，避免覆盖尚未读取的数据；非方阵还涉及不同的置换循环。当前实现新分配输出，不包含这些同步与别名条件。

问：能否彻底消除转置？

答：如果下游算子能够直接读取该 stride，或通过 layout-aware kernel 在加载时完成需要的重排，就可能避免单独的物理转置。但也可能让下游访存更差。要比较整个消费链，而不只是这一个转置函数。

第 4 题的 Triton 版本：矩阵转置

两个完整版本分别说明什么？

`transpose_naive(x)`：把输入 X[M,N] 展平，`row=i//N, col=i%N`，读取 `X+i`，写入 `Y+col*M+row`。输入是连续读取，但逻辑相邻元素的输出地址间隔 M，适合用来讨论跨步写入。

`transpose_tiled(x)`：每个 program 处理一个 32×32 输入块。输入索引为 `row[:,None]*N+col[None,:]`；使用 `tl.trans(values)` 将逻辑 tensor 的轴交换，再构造 `[输出行,输出列]` 顺序的地址 `col[:,None]*M+row[None,:]`。

M=31、N=33 时，既有列方向尾块，也有行方向尾块。load 和 store 分别根据自己的二维坐标施加 mask。输出是新分配的 contiguous `[N,M]` 矩阵，这里做的是物理转置，不是仅仅返回一个具有交换 stride 的 view。

不要把这个 Triton 版本描述成手写 shared memory

Triton 的逻辑 tile 和 CUDA 的 `__shared__ float tile[32][33]` 不是同一层抽象。这里没有手工声明 shared memory，也没有显式指定 33 的 padding。`tl.trans` 描述轴交换，lane layout 和所需转换由编译器决定。

因此，讲性能时应说“这个版本提供二维分块的数据流，接下来检查编译器生成的布局和访存”，不能仅凭看到 `tl.trans` 就声称读和写一定同时达到最优合并访问，更不能凭源码断言一定不存在 bank conflict。`tl.trans` 官方语义

复杂度与指标

工作量 O(MN)，新输出空间 O(MN)。转置几乎没有有意义的浮点运算量，用 TFLOP/s 比较通常没有帮助。若每元素占 s Byte，一个直接的有效带宽指标为：

$$BW_{\text{effective}} = \frac{2MN\,s}{t}.$$

分子代表一次逻辑读取和一次逻辑写入；它不等于 profiler 测到的所有内存事务。非合并访问、缓存和写入策略都可能造成二者差异。

面试追问与回答

问 1：为什么不能直接返回 `x.T`？

答：`x.T` 通常只是改变 shape／stride 的 view，不一定搬动元素。这道题要求产生连续布局的转置结果，要与 `x.T.contiguous()` 这一类实际物化操作比较。拿零拷贝 view 与复制 kernel 计时，会比较两个不同的任务。

问 2：只把索引换成二维，就一定解决访存问题吗?

答：不一定。二维逻辑块让编译器有机会选择适合读取和写入的 layout，但物理 lane 到元素的映射才决定访存合并程度。我会检查生成代码／布局转换，再结合 memory throughput 与事务指标判断。正确性是由索引和 mask 保证的，性能还需要验证。

问 3：CUDA 里常见的 32×33 padding 在 Triton 里怎么理解?

答：CUDA 中 padding 是程序员直接控制 shared-memory 行跨度、改变 bank 映射的一种手段。标准 Triton 里主要通过 tile 形状、访问模式和编译器布局完成同类优化，不能把一个 logical tensor 的 shape 改成 32×33 就认为效果等价。是否需要更底层的布局控制，要看编译结果和瓶颈。

问 4：怎么继续优化?

答：先比较 16×16、32×32 等配置以及 num_warps，覆盖方阵和长条矩阵。若末端消费者接受转置 stride，最好评估是否能把转置融合进消费者的地址计算，从整个流程中消除一次物化；这比单独加速一次复制可能更有价值。

问 5：为什么测试不能只用 1024×1024?

答：这个尺寸不会检验二维尾块，而且方阵下某些错误索引更难发现。使用 31×33、1003×769，可以同时检查 M/N 没写反、输出行跨度正确、load/store 掩码都正确。转置只是搬运有限值，正常情况下应逐元素完全一致，不需要宽松的数值误差。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/csrc/kernels.cu

```
__global__ void transpose_naive_kernel(const float* x, float* out,
                                       int64_t rows, int64_t cols) {
  const int64_t n = rows * cols;
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < n; i += step) {
    const int64_t row = i / cols;
    const int64_t col = i % cols;
    out[col * rows + row] = x[i];
  }
}

__global__ void transpose_tiled_kernel(const float* x, float* out,
                                       int64_t rows, int64_t cols,
                                       int64_t tile_cols, int64_t tiles) {
  // 针对经典 FP32 32-bank 模型加一列 padding。
  __shared__ float tile[kTransposeTile][kTransposeTile + 1];
  for (int64_t tile_id = blockIdx.x; tile_id < tiles; tile_id += gridDim.x) {
    const int64_t tile_row = tile_id / tile_cols;
    const int64_t tile_col = tile_id % tile_cols;
    const int64_t in_col = tile_col * kTransposeTile + threadIdx.x;
    for (int j = 0; j < kTransposeTile; j += kTransposeRows) {
      const int r = threadIdx.y + j;
      const int64_t in_row = tile_row * kTransposeTile + r;
      tile[r][threadIdx.x] = (in_row < rows && in_col < cols)
                             ? x[in_row * cols + in_col] : 0.0f;
    }
    __syncthreads();  // 写 tile 的线程与读转置 tile 的线程不同。

    const int64_t out_col = tile_row * kTransposeTile + threadIdx.x;
    for (int j = 0; j < kTransposeTile; j += kTransposeRows) {
```

```
      const int r = threadIdx.y + j;
      const int64_t out_row = tile_col * kTransposeTile + r;
      if (out_row < cols && out_col < rows)
        out[out_row * rows + out_col] = tile[threadIdx.x][r];
    }
    __syncthreads();  // 防止下一 tile 提前覆盖仍在被读取的 shared memory。
  }
}

at::Tensor transpose_impl(const at::Tensor& x, bool tiled) {
  check_matrix(x, "x");
  const c10::cuda::CUDAGuard guard(x.device());
  const int64_t m = x.size(0), n = x.size(1);
  auto out = at::empty({n, m}, x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  if (tiled) {
    const int64_t tile_cols = ceil_div(n, kTransposeTile);
    const int64_t tiles = checked_product(ceil_div(m, kTransposeTile),
                                          tile_cols, "transpose tiles");
    transpose_tiled_kernel<<<blocks_for(tiles),
                             dim3(kTransposeTile, kTransposeRows), 0, stream>>>(
        x.data_ptr<float>(), out.data_ptr<float>(), m, n, tile_cols, tiles);
  } else {
    transpose_naive_kernel<<<blocks_for(x.numel(), kThreads), kThreads, 0, stream>>>(
        x.data_ptr<float>(), out.data_ptr<float>(), m, n);
  }
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor transpose_naive_cuda(const at::Tensor& x) { return transpose_impl(x, false); }

at::Tensor transpose_tiled_cuda(const at::Tensor& x) { return transpose_impl(x, true); }
```

gpu_prep/triton_ops.py

```
@triton.jit
def _transpose_naive_kernel(X, Y, M: tl.constexpr, N: tl.constexpr, BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    row, col = idx // N, idx % N
    value = tl.load(X + idx, idx < M * N, other=0)
    tl.store(Y + col * M + row, value, idx < M * N)


@triton.jit
def _transpose_tiled_kernel(X, Y, M: tl.constexpr, N: tl.constexpr, TILE: tl.constexpr):
    pid = tl.program_id(0)
    tiles_n = tl.cdiv(N, TILE)
    row = (pid // tiles_n) * TILE + tl.arange(0, TILE)
    col = (pid % tiles_n) * TILE + tl.arange(0, TILE)
    values = tl.load(
        X + row[:, None] * N + col[None, :],
        (row[:, None] < M) & (col[None, :] < N), other=0,
    )
    # Express the output tile in [output-row, output-column] order.
    # Triton, rather than Python code, chooses lane layout and any conversions.
    tl.store(
        Y + col[:, None] * M + row[None, :], tl.trans(values),
        (col[:, None] < N) & (row[None, :] < M),
    )


def _transpose(x: torch.Tensor, tiled: bool) -> torch.Tensor:
    _check_tensor(x, "x", ndim=2)
```

```
    m, n = x.shape
    out = torch.empty((n, m), dtype=x.dtype, device=x.device)
    if x.numel() == 0:
        return out
    with torch.cuda.device(x.device):
        if tiled:
            _transpose_tiled_kernel[(triton.cdiv(m, 32) * triton.cdiv(n, 32),)](
                x, out, m, n, TILE=32, num_warps=4,
            )
        else:
            _transpose_naive_kernel[(triton.cdiv(x.numel(), 256),)](
                x, out, m, n, BLOCK=256, num_warps=4,
            )
    return out


def transpose_naive(x: torch.Tensor) -> torch.Tensor:
    """Materialize x.T using flattened input indices; educational strided stores."""
    return _transpose(x, False)


def transpose_tiled(x: torch.Tensor) -> torch.Tensor:
    """Materialize x.T with 2D tiles; this is not an explicit CUDA shared-memory tile."""
    return _transpose(x, True)
```

第 5 题｜Naive 与 Tiled GEMM

完整源文件： gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 5 的 CUDA 实现：FP32 GEMM 基线与 Shared-memory Tiling

朴素实现到底怎样算一个元素

对于 `C[M,N] = A[M,K] @ B[K,N]`，令一个线程负责 C[row,col]。该线程循环 kk，从 A 的第 row 行和 B 的第 col 列取数，累加 `A[row,kk]*B[kk,col]`。源码使用 `fmaf`，表示乘加合并执行并按相应规则舍入。

线程的行列由展平输出 i 得到：`row=i/N`、`col=i%N`。相邻线程通常负责相邻输出列，因此它们在同一 kk 上读取 B 相邻列；而 A[row,kk] 在多个线程之间存在明显复用机会。朴素代码没有主动将这些跨线程复用保存在 shared memory。

16×16 tile：分工与完整计算路径

一个 block 是 `dim3(16,16)`，每线程仍负责一个 C 元素。它共同加载一个 A 的 16×16 tile 和一个 B 的 16×16 tile。加载后同步，每个线程在 shared memory 上做 16 次乘加；再次同步，再加载 K 方向的下一组 tile。

以 M=127、N=65、K=93 为例：输出需要 8×5 个 tile，K 方向需要 6 个阶段。最后一个 K 阶段只有 13 个有效位置，其余 shared 槽填零。输出边界之外的线程不写 C，但仍参与 tile load、zero-fill 和所有 barrier。

每一个 C[row,col] 的累加器跨越所有 K stages 保存在寄存器中，最后只写一次。每个 stage 中 A 的一个元素被多个输出列复用，B 的一个元素被多个输出行复用。

为什么 K 循环有两个 barrier

第一次确保本阶段的 A/B tile 已被所有线程加载完成，才能开始消费。第二次确保本阶段 shared tile 已被所有线程读完，才能让快线程开始覆盖下一阶段的数据。第二次并非与第一次重复，它保护的是相反方向的依赖。

FLOPs、数据量与 tile 大小

数学运算量约为 `2MNK FLOPs`，输出 O(MN)。忽略尾块、缓存和输出写回，对边长 T 的一个 K-stage：输入加载 $2T^2$ 个 float，执行约 $2T^3$ FLOPs，因此这一层 global-to-shared 复用的算术强度约为 `T/4 FLOP/Byte`。

这个结果并不是整个 GEMM 的理想最低流量模型。更高层的 cache 还可能在不同 CTA 之间复用数据；输出写回、尾块、事务粒度也需要考虑。全部输入各读一次、输出写一次的理想模型为

$$I_{ideal} = \frac{2MNK}{4(MK + KN + MN)}.$$

两种算式对应不同层次的假设，面试时应先说清层次，再用数字。

可以直接复述的追问答案

问：tiled 版本一定比 naive 快吗?

答：它改善了明确的数据复用，但增加了 shared-memory 操作和 barrier。非常小的矩阵可能没有足够工作来摊薄额外开销；形状不匹配会增加尾块浪费。两者都可能受 launch、并行度和缓存影响，必须实测。本教材提供二者是为了比较优化机制，不是预先宣称所有尺寸上的速度关系。

问：tile 越大越好吗?

答：更大 tile 增强局部复用，也可能增加每 CTA 的 shared memory 和寄存器占用，减少同时驻留的 blocks，增加尾部无效计算。若一线程一个输出，还会碰到最大线程数限制。应根据 GPU 资源、M/N/K 和实际 profiler 指标选择，不能只看算术强度。

问：register tiling 怎么继续提升?

答：让每线程维护一个小的输出子块，而不是一个标量。例如对同一个从 shared memory 读出的 A 元素，与多个 B 元素相乘，更新多个寄存器累加器。这样在 shared-to-register 层也形成复用，减少每 FLOP 的 shared-memory 读取。代价是更多寄存器、更多索引和更复杂的线程布局，需检查是否 spill。

问：Tensor Core 版本与当前版本的区别?

答：当前是普通 FP32 SIMT FMA，没有 WMMA/MMA/Tensor Core 指令路径。Tensor Core 路径通常让一个 warp 或协作线程组以特定 fragment/layout 执行矩阵块运算，需要满足数据类型、形状、对齐与布局等要求。不是把 FP32 指针改成 half 指针就完成了 Tensor Core GEMM。要分别验证允许的输入精度、累加精度和最终误差。

问：double buffering / async copy 有什么作用?

答：当当前 tile 在计算时，提前搬运下一 tile，把数据传输与计算重叠。双缓冲需要两套缓冲或其他正确的流水线存储管理，并有阶段间的到达与等待规则。只有搬运和计算足够大、硬件及资源允许重叠时才可能获益；同步错了会读取未完成的数据或提前覆盖缓冲。

问：为什么 decode 时的 skinny GEMM 可能不适合大方块 tile?

答：小 M 意味着输出行很少，大 tile 会产生许多无效行，CTA 数也可能不足。权重读流量不一定被足够多的 token 复用。可以考虑适合 skinny shape 的 tile、分 K 的并行化或针对该形状的 GEMV/GEMM 路径。Split-K 会增加部分和合并成本，浮点求和顺序也可能改变，应一起比较。

问：K=0 为什么返回全零?

答：每个 C 元素是一个空和，按约定等于加法单位元 0。代码在 host 用 zeros 返回，不进入 kernel。返回未初始化 empty 会违反算子的数学语义；M/N=0 则只需要返回正确 shape 的空输出。

问：怎样公平地与 cuBLAS 比较?

答：先明确 shape、layout、dtype、允许的内部精度、输出容差和是否包含分配／数据转换。预热并排除首次构建，使用可靠的 GPU 时间测量。若参考路径使用 TF32 或不同精度的 Tensor Core，不能把它与严格 FP32 的结果当作完全相同计算契约而不说明。性能和数值正确性都应单独报告。

正确性检查最少应包含什么

至少检查 (1,1,1)、非整除的 (127,65,93)、完整 tile、K=0、M=0 和 N=0。用 FP64 reference 可以避免测试受全局 TF32 设置影响。测试容差要同时考虑绝对与相对误差，特别是期望值接近 0 的输出。

第 5 题的 Triton 版本：从 naive GEMM 到 tiled GEMM

朴素版本：先把地址写对

`matmul_naive(a,b)` 的输入是 A[M,K]、B[K,N]，输出 C[M,N]。每个 program 负责一段扁平输出位置：

$$row = i//N, \quad col = i \bmod N, \quad C_{row,col} = \sum_{k=0}^{K-1} A_{row,k} B_{k,col}.$$

代码沿 K 顺序循环，A 地址为 `row*K+k`，B 地址为 `k*N+col`，使用 FP32 FMA 累加。它没有手工管理输入块的复用，适合当作正确性基线和面试第一版。

代码层面能看到重复加载，不代表每次都会落到 HBM：warp 广播、L1/L2 缓存与编译器优化会影响真实流量。因此描述时应说“缺少显式的 tile 复用结构”，不要把静态 load 条数直接写成实测 HBM 次数。

分块版本：一个 program 计算一块 C

`matmul_tiled(a,b)` 使用 `BM=BN=BK=32`。每个 program 对应一个 C 的 `[BM,BN]` 块；循环遍历 K：

1. 加载 `[BM,BK]` 的 A 块。
2. 加载 `[BK,BN]` 的 B 块。
3. `tl.dot(a_tile,b_tile,acc, input_precision="ieee")`。
4. 继续下一个 K 块，最后一次性写回 C。

在一个 K 块中，一个 A 元素可以影响 BN 个输出，一个 B 元素可以影响 BM 个输出，增加了读取数据后的计算复用。A 的边界 mask 同时检查 `row<M` 与 `k<K`；B 同时检查 `k<K` 与 `col<N`；不足的 K 部分补 0，不影响数学结果。

本代码没有使用 `%M` ／ `%N` 回绕边界，而是直接使用显式 load mask，方便解释所有地址的有效范围。M=0 或 N=0 在 wrapper 中返回空输出；K=0 返回初始化为零的输出，符合空内积定义。

FP32、TF32、Tensor Core，口径必须分清

官方 `tl.dot` 文档说明，FP32 输入默认精度可能是 TF32；本版本显式使用 `input_precision="ieee"`。FP16/BF16 输入则保持其输入精度、在 FP32 中累加，最后将 C 转回输入 dtype。

不要说“用了 `tl.dot`，所以 FP32 一定使用 Tensor Core 并且更快”。严格 FP32 点积与 TF32 路径有不同的性能和精度；这里首先选择容易说明的精度合同。若做性能扩展，可增加一个显式参数比较 IEEE、TF32 等模式，但必须分别报告误差与吞吐，保持 PyTorch／cuBLAS baseline 的精度设置一致。`tl.dot` 精度参数

为什么 tile 改变算术强度?

只考虑一次 K 分块的 A/B 读取、暂不计算输出写入和缓存，一个 tile 做约 `2*BM*BN*BK` FLOPs，加载约 `s*(BM*BK+BK*BN)` Byte：

$$I_{\text{tile}} \approx \frac{2BM\,BN}{s(BM+BN)}.$$

在这个简化模型里，扩大 BM/BN 可以增加数据复用，但也扩大输出累加器；BK 在分式里约掉，不意味着 BK 对实际速度没有作用，它仍影响循环次数、流水线、寄存器／shared memory 使用和尾块浪费。

完整 GEMM 的总工作量 O(MNK)，输出空间 O(MN)。tile 内还需要 A、B 和 accumulator 的片上工作集；这个资源占用不应忽略。

面试追问与回答

问 1：分块为什么能加速?

答：同样一次读取的 A/B 元素能够参与多个输出的计算，增加数据复用；`tl.dot` 也让编译器选择适合矩阵运算的指令和布局。收益取决于 tile 是否合适、精度路径、问题规模和资源使用，不能保证任意形状都变快。

问 2：BM、BN 越大越好吗?

答：越大可以增加复用，但 accumulator 有 BM×BN 个元素，寄存器压力会增长；资源占用过高可能减少同时驻留的 programs，甚至产生 spill。M/N 较小时，大 tile 还会做大量 masked 的无效工作。因此我会先根据矩阵长宽和硬件限制选一小组候选配置，再测量。

问 3：为什么 BK 在你刚才的算术强度公式中消失了?

答：因为这个简化式只比较一个 K tile 的计算量与 A/B 输入量，两者都正比于 BK。实际 kernel 还受循环、流水线、片上存储、指令形状和尾块影响，BK 仍然重要。算术强度解释的是一部分瓶颈，不是完整的性能预测器。

问 4：small-M GEMM 怎么优化?

答：先检查是否只有少量输出 tiles，导致 GPU 并行度不足，以及 N/K 数据搬运是否占主导。可以选择更小的 BM、适合该形状的 BN，必要时考虑 split-K 来增加并行任务。split-K 会引入 partial outputs 和归约，必须比较新增开销；不是看到 M 小就一律打开。

问 5：split-K 为什么需要额外注意正确性?

答：多个 programs 计算同一个 C 块的不同 K 分段，必须在最后合并。如果用 atomicAdd，要先清零输出并处理累加顺序；如果用临时数组，需要第二阶段归约。每个分段独立写完整 C 会覆盖别人的结果。精度也可能因合并顺序改变。

问 6：如何让 A/B tiles 更容易命中 L2?

答：可以调整 program 顺序，让相近时间执行的输出 tiles 复用相同的 A 或 B 区域，例如分组遍历多个 M tiles 后再移动 N。官方矩乘教程有 grouped ordering 的例子。但具体是否受益取决于 L2 容量、并发执行和形状，本版本先保留简单、易核对的一维行优先顺序。官方 Matmul 教程

问 7：怎样做一个公平的性能比较?

答：相同 M/N/K、dtype、布局、输出精度；明确是否允许 TF32；预热并排除 JIT；正确处理异步执行；不要把某一方的数据拷贝混进 kernel 时间却排除另一方的拷贝。先检验输出误差，再报告 latency、由 `2MNK/t` 得到的吞吐，以及代表性形状范围。

建议的优化实验顺序

先比较 naive 与固定 tile，确认差异来自明确的数据复用；随后只改变一个因素，例如 BM/BN/BK；再比较 num_warps、num_stages；最后加入分组遍历、epilogue fusion 或针对 small-M 的算法。每一步保留正确性验证和有收益／无收益的形状，面试时才能解释“为什么这样改”。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/csrc/kernels.cu

```
__global__ void matmul_naive_kernel(const float* a, const float* b, float* out,
                                    int64_t m, int64_t n, int64_t k) {
  const int64_t total = m * n;
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < total; i += step) {
    const int64_t row = i / n;
    const int64_t col = i % n;
    float acc = 0.0f;
    for (int64_t kk = 0; kk < k; ++kk)
      acc = fmaf(a[row * k + kk], b[kk * n + col], acc);
    out[i] = acc;
  }
}

__global__ void matmul_tiled_kernel(const float* a, const float* b, float* out,
                                    int64_t m, int64_t n, int64_t k,
                                    int64_t tile_cols, int64_t tiles) {
  __shared__ float as[kMatmulTile][kMatmulTile];
  __shared__ float bs[kMatmulTile][kMatmulTile];
  const int tx = threadIdx.x;
  const int ty = threadIdx.y;
```

```
  const int64_t stages = k / kMatmulTile + (k % kMatmulTile != 0);

  for (int64_t tile_id = blockIdx.x; tile_id < tiles; tile_id += gridDim.x) {
    const int64_t row = (tile_id / tile_cols) * kMatmulTile + ty;
    const int64_t col = (tile_id % tile_cols) * kMatmulTile + tx;
    float acc = 0.0f;
    for (int64_t stage = 0; stage < stages; ++stage) {
      const int64_t a_col = stage * kMatmulTile + tx;
      const int64_t b_row = stage * kMatmulTile + ty;
      // 尾块显式填零；不能让越界线程提前退出而跳过同步。
      as[ty][tx] = (row < m && a_col < k) ? a[row * k + a_col] : 0.0f;
      bs[ty][tx] = (b_row < k && col < n) ? b[b_row * n + col] : 0.0f;
      __syncthreads();
      #pragma unroll
      for (int kk = 0; kk < kMatmulTile; ++kk)
        acc = fmaf(as[ty][kk], bs[kk][tx], acc);
      __syncthreads();  // 所有乘加完成后，才允许覆盖 as/bs。
    }
    if (row < m && col < n) out[row * n + col] = acc;
  }
}

at::Tensor matmul_impl(const at::Tensor& a, const at::Tensor& b, bool tiled) {
  check_matrix(a, "a");
  check_matrix(b, "b");
  check_same_device(a, b, "b");
  TORCH_CHECK(a.size(1) == b.size(0), "matmul inner dimensions must match");
  const c10::cuda::CUDAGuard guard(a.device());
  const int64_t m = a.size(0), k = a.size(1), n = b.size(1);
  const int64_t total = checked_product(m, n, "matmul output");
  // 空的求和有单位元 0；不能把未初始化的 empty 当作 K=0 的答案。
  if (k == 0) return at::zeros({m, n}, a.options().requires_grad(false));
  auto out = at::empty({m, n}, a.options().requires_grad(false));
  if (total == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(a.get_device()).stream();
  if (tiled) {
    const int64_t tile_cols = ceil_div(n, kMatmulTile);
    const int64_t tiles = checked_product(ceil_div(m, kMatmulTile),
                                          tile_cols, "matmul tiles");
    matmul_tiled_kernel<<<blocks_for(tiles), dim3(kMatmulTile, kMatmulTile),
                          0, stream>>>(a.data_ptr<float>(), b.data_ptr<float>(),
                                       out.data_ptr<float>(), m, n, k,
                                       tile_cols, tiles);
  } else {
    matmul_naive_kernel<<<blocks_for(total, kThreads), kThreads, 0, stream>>>(
        a.data_ptr<float>(), b.data_ptr<float>(), out.data_ptr<float>(), m, n, k);
  }
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor matmul_naive_cuda(const at::Tensor& a, const at::Tensor& b) {
  return matmul_impl(a, b, false);
}

at::Tensor matmul_tiled_cuda(const at::Tensor& a, const at::Tensor& b) {
  return matmul_impl(a, b, true);
}
```

gpu_prep/triton_ops.py

```
@triton.jit
def _matmul_naive_kernel(A, B, C, M: tl.constexpr, N: tl.constexpr, K: tl.constexpr,
                         BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
```

```
    row, col = idx // N, idx % N
    valid = idx < M * N
    acc = tl.full((BLOCK,), 0, tl.float32)
    # A sequential K loop makes the repeated global loads easy to see.
    for k in range(0, K):
        a = tl.load(A + row * K + k, valid, other=0).to(tl.float32)
        b = tl.load(B + k * N + col, valid, other=0).to(tl.float32)
        acc = tl.fma(a, b, acc)
    tl.store(C + idx, acc, valid)


@triton.jit
def _matmul_tiled_kernel(A, B, C, M: tl.constexpr, N: tl.constexpr, K: tl.constexpr,
                         BM: tl.constexpr, BN: tl.constexpr, BK: tl.constexpr):
    pid = tl.program_id(0)
    tiles_n = tl.cdiv(N, BN)
    row = (pid // tiles_n) * BM + tl.arange(0, BM)
    col = (pid % tiles_n) * BN + tl.arange(0, BN)
    inner = tl.arange(0, BK)
    acc = tl.full((BM, BN), 0, tl.float32)
    for tile in range(0, tl.cdiv(K, BK)):
        k = tile * BK + inner
        a = tl.load(A + row[:, None] * K + k[None, :],
                    (row[:, None] < M) & (k[None, :] < K), other=0)
        b = tl.load(B + k[:, None] * N + col[None, :],
                    (k[:, None] < K) & (col[None, :] < N), other=0)
        # For f32 inputs, do not silently compare TF32 math with a strict f32 baseline.
        acc = tl.dot(a, b, acc, input_precision="ieee")
    tl.store(C + row[:, None] * N + col[None, :], acc,
             (row[:, None] < M) & (col[None, :] < N))


def _matmul(a: torch.Tensor, b: torch.Tensor, tiled: bool) -> torch.Tensor:
    _check_tensor(a, "a", ndim=2)
    _check_peer(a, b, "b", ndim=2)
    m, k = a.shape
    kb, n = b.shape
    if k != kb:
        raise ValueError("a.shape[1] must equal b.shape[0]")
    if m * n > MAX_TENSOR_ELEMENTS:
        raise ValueError("output exceeds this exercise's i32 indexing limit")
    out = torch.empty((m, n), dtype=a.dtype, device=a.device)
    if m == 0 or n == 0:
        return out
    if k == 0:
        return out.zero_()
    with torch.cuda.device(a.device):
        if tiled:
            _matmul_tiled_kernel[(triton.cdiv(m, 32) * triton.cdiv(n, 32),)](
                a, b, out, m, n, k, BM=32, BN=32, BK=32,
                num_warps=4, num_stages=2,
            )
        else:
            _matmul_naive_kernel[(triton.cdiv(m * n, 128),)](
                a, b, out, m, n, k, BLOCK=128, num_warps=4,
            )
    return out


def matmul_naive(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """A[M,K] @ B[K,N]; sequential K loop per output, FP32 accumulation."""
    return _matmul(a, b, False)
```

```python
def matmul_tiled(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """A[M,K] @ B[K,N]; fixed 32x32x32 tiles, FP32 accumulation, f32 IEEE math."""
    return _matmul(a, b, True)
```

第 6 题｜Roofline：工作量、流量与瓶颈模型

完整源文件： gpu_prep/roofline.py。

第 6 题：Roofline 计算、解释与答案

题目与完整实现

给定硬件峰值算力 P、可持续内存带宽 B，以及程序的浮点运算量 F 和数据搬运量 Q，估计算术强度、性能上界和执行时间下界。为 vector add、square GEMM 和 skinny GEMM 分别计算，并解释为什么同样叫 GEMM，瓶颈可能不同。

完整代码是 gpu_prep/roofline.py。它提供 Workload、vector_add_workload、gemm_workload、roofline_estimate 和 amdahl_speedup；既能直接调用，也能作为命令行运行。对应 tests/test_roofline.py 验证单位、数值、空维度约定和非法参数。

这道题以计算与分析为主，不需要额外写一个 CUDA 或 Triton 的 roofline kernel。输入性能模型的是前面实际 kernel 的工作量和测量数据，模型本身用 Python 即可清楚表达。

1. 公式要从单位解释

算术强度 AI = F / Q，单位 FLOP/byte。

计算所需时间下界 t_compute = F / P。

内存所需时间下界 t_memory = Q / B。

理想模型中的时间下界 t_min = max(t_compute, t_memory)。

对应的性能上界 perf_max = min(P, B × AI)。

转折点 AI_ridge = P / B。AI 比它低时，模型的带宽屋顶更低；AI 比它高时，模型的算力屋顶更低。

这里用十进制单位：1 TFLOP/s = 10^12 FLOP/s，1 TB/s = 10^12 byte/s，1 GB/s = 10^9 byte/s。GiB 则是 2^30 byte，不能在数字计算时把 GB 与 GiB 混用。一次 fused multiply-add 做一次乘法和一次加法，按通常性能报告约定计 2 FLOPs。

“下界”与“预测”必须区分。这个模型没有直接包含 launch latency、调度、访存延迟、依赖链、寄存器 spill、额外 tensor layout conversion、各层缓存流量和同步。算出 5 微秒，不代表真实实现就会在 5 微秒完成。

2. 手算例一：FP32 vector add

设 z=x+y，有 N 个元素。每个元素做 1 次加法；理想情况下读 x、读 y、写 z，各 4 byte，因此 F=N，Q=12N，AI=1/12≈0.0833 FLOP/byte。

假设一块“教学用虚构 GPU”有 P=100 TFLOP/s、B=2 TB/s。它不是对任何真实型号的规格声明。

此时 ridge=100/2=50 FLOP/byte，vector add 明显落在低 AI 一侧。模型里的性能上界是 2×(1/12)=0.1667 TFLOP/s，而不是 100 TFLOP/s。N=1,000,000 时，数据量为 12,000,000 byte，带宽时间下界为 6 微秒；计算时间下界只有 0.01 微秒。

可复述答案： “Vector add 的每个输出只做一次加法，却要读两个数、写一个数。按 FP32 理想流量算，AI 只有 1/12，所以我优先检查访存连续性、有效带宽和 launch 开销。对大数组，共享内存没有重复使用的数据可供缓存，加入 shared memory 很可能增加操作和同步；对很小数组，即使 AI 很低，实际也可能主要受 launch latency 影响。”

3. 手算例二：Square GEMM

A 是 M×K，B 是 K×N，C=AB 为 M×N，beta=0，不需要读取旧 C。按通常约定，F≈2MNK。算法层面的理想最小流量 Q=s(MK+KN+MN)，其中 s 为一个元素的 byte 数。

对于 FP32、M=N=K=1024：

- F=2,147,483,648 FLOPs。
- Q=12,582,912 byte。
- AI≈170.67 FLOP/byte。
- 在上面的虚构 GPU 上，t_compute≈21.475 微秒，t_memory≈6.291 微秒。
- 因而模型的计算屋顶更低，时间下界约 21.475 微秒。

但这个流量模型假设输入得到了理想复用。朴素实现每个线程独立循环 K，会产生大量重复访问。L1/L2 可能缓解部分 DRAM 流量，却不会消除每条 load 指令的执行成本或自动保证所有复用理想。不能仅凭算法 AI 很高就宣称一个没有测量的 naive kernel 已经 compute-bound。

可复述答案：“我会先区分算法的最小流量与该实现实际产生的流量。GEMM 的算法 AI 可以很高，但 naive 实现未必达到。Tiling 的作用是让加载到 shared memory 或寄存器的数据服务多个 FMA，从而降低实际搬运开销并提高执行效率；它没有改变 C=AB 的数学 FLOP 数。”

4. 手算例三：Skinny GEMM

取 M=1、N=K=4096，FP32：

- F=33,554,432 FLOPs。
- Q=67,141,632 byte。
- AI≈0.49976 FLOP/byte。
- 在虚构 GPU 上，计算时间约 0.336 微秒，带宽时间约 33.571 微秒。
- 模型里的带宽屋顶更低。

直觉是 B 很大，M=1 时 B 的每个元素只服务很少的输出行，跨行复用不足。即使使用相同 dtype、同一块 GPU，“大 square GEMM”与“single-token decode 中的线性层”也可能表现出很不同的资源利用。实际模型还要考虑 batch、权重是否被缓存、量化、并发请求和算子融合。

可复述答案：“我不会直接把大方阵 GEMM 的最佳 tile 搬到 M=1 的场景。M 很小时，输出 tile 的浪费和并行度不足可能突出。我会按实际 M/N/K 分桶，比较较小 M tile、适合 GEMV 的路径，以及必要时 split-K 的收益和归约成本。”

5. 实测结果怎样接入

假设某次真实测量得到 t 秒，则 achieved_FLOPs=F/t。若使用逻辑流量 Q，则 Q/t 只能称为 logical effective bandwidth。要声称实际 HBM 吞吐率，需要对应层级的 profiler 计数器；缓存命中可能让逻辑有效带宽超过硬件标称 HBM 带宽，这本身不说明计时一定错误。

还必须选对 P：FP32 CUDA Core 峰值、TF32 Tensor Core 峰值、FP16/BF16 Tensor Core 峰值，以及是否假设结构化稀疏，分别是不同的屋顶。拿 FP32 SIMT kernel 去除以一个稀疏 FP16 峰值，得到的利用率没有可解释性。

硬件峰值是规格上限，可持续带宽通常低于标称带宽。做实用诊断时，可以同时展示规格 roofline 与在类似条件下测得的 sustained roofline，但需要标明各自来源。Nsight Compute 的 roofline 还可按内存层级分析，不能把 L1、L2、HBM 的 byte 计数混为同一个 Q。

6. 高频追问与完整回答

问：AI 低就一定是 bandwidth-bound 吗?

“在简化 roofline 模型里，低 AI 意味着带宽上界更低；但真实 kernel 可能远没有饱和带宽，而是受 launch latency、访存延迟、并行度不足或指令依赖限制。我会结合数据规模和 profiler 看实际 DRAM 吞吐率、active warps、stall 原因与 kernel 时间，避免只用公式下最终结论。”

问：为什么提高 occupancy 后反而变慢?

“Occupancy 描述并发驻留资源的一部分，不是目标函数。为了提高它而减小寄存器，可能引入 spill；减小 tile 可能降低数据复用；过多 block 也可能改变缓存行为。我要验证的是延迟或吞吐量是否变好，以及这个变化是不是在相同语义和精度下获得的。”

问：Shared memory 一定能增加 AI 吗?

“共享内存可以减少重复的高层内存访问，但不是凭空增加数学运算。对有复用的 GEMM tile 很有价值；对读一次就写一次的 vector add，通常没有该复用。分析时我会说清 AI 的 Q 是算法理想流量、DRAM 实测流量还是 shared-memory 流量。”

问：一个实现的 TFLOP/s 很低，怎样决定优化什么?

“先定位规模和正确的计算屋顶，再看是否达到了对应内存层级的带宽。若 kernel 很短，我会先看 launch/调度成本；若带宽高且 AI 低，优先减少 bytes 或融合；若带宽和算力都低，检查并行度、访存模式、依赖、同步、layout conversion 与资源占用。最后只改一个主要变量，重新验证。”

问：Transpose 的 FLOP/s 有意义吗?

“物理转置主要搬运数据，没有有意义的大规模浮点算术。我会报告延迟和基于读写量计算的 GB/s，并用 profiler 判断真实内存行为，而不会为了套表强行给它一个计算利用率。”

问：Softmax 能不能简单按几个 FLOPs 算?

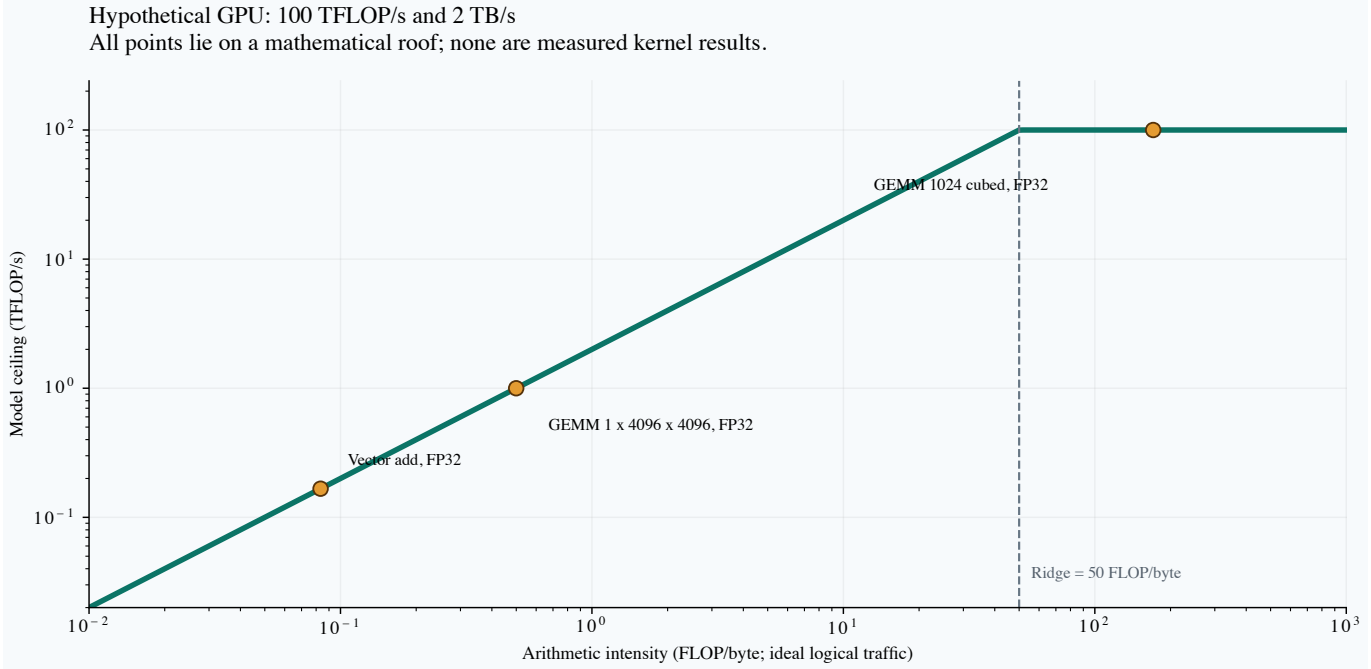
“可以建立一个粗模型，但 exp、除法、归约、比较与普通 FMA 的吞吐特性不同，只给一个 FLOP 数容易隐藏重要信息。我会先分析读写次数、归约结构和行宽，再结合具体指令与 profiler。源码的通用 benchmark 因而不为 softmax 伪造精确的 FLOP/s。”

问：量化为什么可能让 decode 更快?

“如果权重读取是主要成本，减小权重 byte 数可以提高有效 AI，降低传输量。但要同时计入反量化、scale 读取、重排、额外 kernel 与精度约束。是否更快必须按 batch、shape、缓存条件和量化实现测量，不能只按位宽比例预测速度。”

Roofline 模型图

下图使用假想的 100 TFLOP/s、2 TB/s；圆点表示数学屋顶上的位置，不是性能测量。



本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/roofline.py

```python
@dataclass(frozen=True)
class Workload:
    name: str
    flops: int
    ideal_bytes: int
    assumptions: str

    @property
    def arithmetic_intensity(self) -> float:
        return self.flops / self.ideal_bytes if self.ideal_bytes else 0.0


def vector_add_workload(n: int, element_bytes: int = 4) -> Workload:
    _dims(n)
    _element_bytes(element_bytes)
    return Workload(
        "vector_add", n, 3 * n * element_bytes,
        "Read each of two inputs once, write one output once; ignore caching and launch latency.",
    )


def gemm_workload(m: int, n: int, k: int, element_bytes: int = 4) -> Workload:
    _dims(m, n, k)
    _element_bytes(element_bytes)
    # If the output is empty, a well-written wrapper skips the operation.
    if m == 0 or n == 0:
        return Workload("gemm", 0, 0, "Empty output: skip the launch.")
    return Workload(
        "gemm", 2 * m * n * k, element_bytes * (m * k + k * n + m * n),
        "Conventional 2MNK FLOP count, beta=0; load each A/B element once, store each C once. "
        "This ideal reuse is not guaranteed by a real kernel.",
    )
```

```python
def roofline_estimate(workload: Workload, peak_tflops: float, bandwidth_tbps: float) -> dict:
    if not math.isfinite(peak_tflops) or peak_tflops <= 0:
        raise ValueError("peak_tflops must be finite and positive.")
    if not math.isfinite(bandwidth_tbps) or bandwidth_tbps <= 0:
        raise ValueError("bandwidth_tbps must be finite and positive.")
    peak = peak_tflops * 1e12
    bandwidth = bandwidth_tbps * 1e12
    compute_seconds = workload.flops / peak
    memory_seconds = workload.ideal_bytes / bandwidth
    lower_bound = max(compute_seconds, memory_seconds)
    ceiling = workload.flops / lower_bound / 1e12 if lower_bound else 0.0
    return {
        **asdict(workload),
        "arithmetic_intensity_flop_per_byte": workload.arithmetic_intensity,
        "ridge_point_flop_per_byte": peak / bandwidth,
        "hypothetical_peak_tflops": peak_tflops,
        "hypothetical_bandwidth_tbps": bandwidth_tbps,
        "ideal_compute_time_ms": compute_seconds * 1e3,
        "ideal_memory_time_ms": memory_seconds * 1e3,
        "ideal_time_lower_bound_ms": lower_bound * 1e3,
        "roofline_tflops": ceiling,
        "model_limiting_resource": (
            "empty" if workload.ideal_bytes == 0 and workload.flops == 0
            else "memory" if memory_seconds >= compute_seconds else "compute"
        ),
        "measured": False,
    }


def amdahl_speedup(fraction: float, local_speedup: float) -> float:
    """Serial critical-path model, with no new overhead and fixed workload."""
    if not 0.0 <= fraction <= 1.0:
        raise ValueError("fraction must lie in [0, 1].")
    if not math.isfinite(local_speedup) or local_speedup <= 0:
        raise ValueError("local_speedup must be finite and positive.")
    return 1.0 / ((1.0 - fraction) + fraction / local_speedup)
```

第 7 题｜Linear、QKV、Shape 与 Stride

完整源文件： gpu_prep/torch_layers.py。

第 7 题：手写 Linear，正确拆分 Q／K／V

题目与边界

实现 `ManualLinear(in_features, out_features, bias=True)`，不能在实现内部调用 `nn.Linear` 或 `F.linear`。输入形状可以是 `[D_in]`、`[B,D_in]`、`[B,T,D_in]`，统一得到末维为 `D_out` 的输出。使用真正的可训练参数，保留自动求导。外层 batch 或 sequence 维可以为零；本实现要求特征维为正。

第二部分：给定一次合并投影产生的 `qkv[B,T,3D]`，把它拆成三个 `[B,H,T,d]` 张量，满足 `D=H*d`。明确约定投影末维依次排列全部 Q 特征、全部 K 特征、全部 V 特征，不要在没有说明布局的情况下任意 reshape。

先把矩阵式讲清楚

对一个输入向量，输出第 j 项是：

$$y_j = \sum_{i=0}^{D_{in}-1} x_i W_{ji} + b_j.$$

因此 `weight` 的形状选择为 `[D_out,D_in]`，输入的最后一维与 `weight.T` 的倒数第二维相乘：

```
out = x @ self.weight.transpose(0, 1)
if self.bias is not None:
    out = out + self.bias
```

`[B,T,D_in] @ [D_in,D_out]` 得到 `[B,T,D_out]`。`bias[D_out]` 从右侧对齐维度，相当于对所有 batch、所有 token 加同一组偏置。没有必要先把输入手工改成二维再逐行循环。

代码把 `weight` 和 `bias` 声明为 `nn.Parameter`，并挂在 `nn.Module` 属性上。这样优化器可以通过 `model.parameters()` 找到它们。普通 Tensor 属性不会因为“看起来像权重”自动完成参数注册。PyTorch Parameter 文档说明了这一注册行为。

一个贯穿全题的形状例子

取 `B=2，T=5，D=8，H=2，d=4`：

步骤	形状	这一步到底改变了什么
输入 X	`[2,5,8]`	2 个序列，每个 5 个 token，每 token 8 个特征
合并投影	`[2,5,24]`	每个 token 产生 Q、K、V 各 8 个特征
`reshape(B,T,3,H,d)`	`[2,5,3,2,4]`	将末维显式拆成“QKV 类型、head、head 内特征”
`permute(2,0,3,1,4)`	`[3,2,2,5,4]`	把 QKV 类型移动到最前，把 head 放到 sequence 前面
`unbind(0)`	3 个 `[2,2,5,4]`	分别选出 Q、K、V

不能直接把 `[B,T,3D]` 改成 `[B,H,T,3d]` 然后声称已完成多头拆分。这通常改变了维度解释，并未按照正确的 token/head 分组完成重排。一个很好的测试办法是使用递增整数，让每个 Q/K/V 区段的值可辨认，而不只检查最终 shape。

`view`、`reshape`、`permute`、`contiguous` 为什么容易出错?

Tensor 的“形状”不是全部信息。访问元素时，还要用 stride 决定每个维度前进一步，需要在底层存储中移动多少个元素。

例如连续的 `[2,3]` 张量，其 stride 是 `(3,1)`；转置后变成 `[3,2]`，stride 是 `(1,3)`。转置通常只改变索引规则，并没有把数据按新行序复制一遍。此时想按新形状展平，可能无法继续只靠一个视图表达。

操作	语义	是否保证不复制
`permute` / `transpose`	重排维度及对应 stride	返回视图；不承诺结果连续
`view`	在当前存储布局允许的前提下重新解释 shape	不复制；布局不兼容就报错
`reshape`	得到指定 shape	尽量返回视图，必要时复制
`contiguous`	得到指定连续布局	已经满足时可以直接返回；否则复制

这里的代码在合并 attention heads 时使用：

```
attended.transpose(1, 2).reshape(B, T, D)
```

它先把 `[B,H,T,d]` 改成 `[B,T,H,d]`，再把 `H,d` 合并。`reshape` 让语义正确；性能分析时还需要确认它是否产生了复制。不要把“写起来没有报错”解释成“没有额外访存”。官方 Tensor.view 文档给出了按 shape/stride 判断能否合并维度的条件；Tensor Views解释了视图共享存储。

还有两个常见混淆：`clone()` 是数据复制，但并不会自动切断 autograd；`detach()` 切断梯度关系，但通常仍共享存储。为了得到一份独立的测试输入，测试里使用 `x.detach().clone().requires_grad_()`。

Linear 的反向传播，怎样口头推导?

把所有前导维暂时合成 `R` 行，写成 `Y=XW^T+b`。设上游梯度为 `G=dL/dY`，则：

$$\frac{\partial L}{\partial X} = GW, \qquad \frac{\partial L}{\partial W} = G^T X, \qquad \frac{\partial L}{\partial b} = \sum_{r=0}^{R-1} G_{r,:}.$$

可以这样解释偏置梯度：“同一个 bias 元素被广播给所有样本位置。每个位置对它的贡献都要累加，所以沿所有广播出来的前导维求和，最后只保留 `D_out`。”

代码没有另写 backward，因为 `matmul`、转置与加法已经有 autograd 规则。考试要求手写层，不一定要求手写 `autograd.Function`；先确认要求，再选择最小而完整的实现。

面试追问：可直接复述的回答

问：为什么不用普通的 `torch.randn` 作为 `self.weight`？

答：“计算本身可以用普通 Tensor，但如果它只是 Module 的普通属性，优化器通过 `parameters()` 不一定能发现它。我的实现用 `nn.Parameter` 明确登记训练状态，权重初始化在初始化阶段完成，forward 不重新生成权重。”

问：你的代码能处理不同 batch size 吗?

答：“可以。矩阵乘法只要求最后一维与输入特征数匹配，前面的维度代表独立位置。我不把 B 和 T 写死，并且测试了单向量、多维输入和外层为空的情况。特征维为零不在本实现的约定内，会明确报错。”

问：为什么合并 QKV 投影可能更快?

答：“三个线性映射数学上仍独立。合并后可以将三次较小的 GEMM 变成一次更大的 GEMM，减少调度，并可能改善矩阵尺寸和输入复用。但还要考虑拆分后的布局是否导致复制，不能只根据调用次数保证加速。”

问：调用 `model.eval()` 是否会停止计算梯度?

答：“不会。`eval()` 改变依赖训练状态的层行为，例如 dropout；是否记录梯度要用 `no_grad()`、`inference_mode()` 或相应的 requires_grad 设置。这两个维度应分别处理。”

问：非连续输入是否必须先 `.contiguous()`?

答：“纯 PyTorch 运算通常能处理 stride，因此这个参考实现不先强制复制。自写 CUDA 如果只按连续下标访问，就必须声明只接受连续输入，或者显式处理 stride。是否复制应当作为接口和性能成本的一部分说明。”

验收标准

实现能够与 `F.linear` 对比前向和输入/参数梯度；QKV 数值位置而非仅 shape 正确；能够解释 bias 梯度和 reshape 复制；导入模块不会创建 GPU 资源或编译 CUDA。对应测试已经写在 `test_torch_layers.py` 中。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/torch_layers.py

```python
class ManualLinear(nn.Module):
    """y = x @ weight.T + bias, with genuinely registered Parameters.

    Input shape is (..., in_features); arbitrary leading dimensions, including
    empty ones, are supported.  Feature dimensions themselves must be positive.
    """

    def __init__(self, in_features: int, out_features: int, bias: bool = True,
                 device=None, dtype=None) -> None:
        super().__init__()
        if in_features <= 0 or out_features <= 0:
            raise ValueError("in_features and out_features must be positive")
        self.in_features = in_features
        self.out_features = out_features
        factory = {"device": device, "dtype": dtype}
        self.weight = nn.Parameter(torch.empty(out_features, in_features, **factory))
        if bias:
            self.bias = nn.Parameter(torch.empty(out_features, **factory))
        else:
            self.register_parameter("bias", None)
        self.reset_parameters()

    def reset_parameters(self) -> None:
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        if self.bias is not None:
            bound = 1 / math.sqrt(self.in_features)
            nn.init.uniform_(self.bias, -bound, bound)

    def forward(self, x: Tensor) -> Tensor:
        _check_float(x, "x")
        if x.ndim == 0 or x.shape[-1] != self.in_features:
            raise ValueError(f"last input dimension must be {self.in_features}")
        out = x @ self.weight.transpose(0, 1)
        return out if self.bias is None else out + self.bias
```

```python
def split_qkv_heads(qkv: Tensor, num_heads: int) -> tuple[Tensor, Tensor, Tensor]:
    """[B,T,3*D] -> three [B,H,T,D/H] tensors.

    Projection output layout is [all Q features, all K features, all V features]
    for each token. Returned tensors are views when the input layout permits;
    no contiguity promise is made after permute.
    """
    _check_float(qkv, "qkv")
    if qkv.ndim != 3 or num_heads <= 0:
        raise ValueError("qkv must be [B,T,3*D] and num_heads must be positive")
    if qkv.shape[-1] == 0 or qkv.shape[-1] % (3 * num_heads) != 0:
        raise ValueError("qkv last dimension must be a positive multiple of 3*num_heads")
    batch, length, packed_dim = qkv.shape
    head_dim = packed_dim // (3 * num_heads)
    # [B,T,3,H,d] -> [3,B,H,T,d]; unbind selects Q, K and V.
    packed = qkv.reshape(batch, length, 3, num_heads, head_dim)
    q, k, v = packed.permute(2, 0, 3, 1, 4).unbind(0)
    return q, k, v
```

第 8 题｜LayerNorm 与 RMSNorm：前向、反向与 GPU 实现

完整源文件： gpu_prep/torch_layers.py；gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 8 的 CUDA 实现：LayerNorm 与 RMSNorm

Shape 与公式

本 CUDA 版本只沿最后一个维度 D 做归一化。`[B,T,D]` 在连续内存中看成 `B*T` 行、每行 D 列；输入也可以只有 `[D]` 一维，此时只有一行。

LayerNorm：

$$\mu = \frac{1}{D}\sum_i x_i, \quad \sigma^2 = \frac{1}{D}\sum_i (x_i-\mu)^2, \quad y_i = (x_i-\mu)(\sigma^2+\epsilon)^{-1/2} w_i + b_i.$$

RMSNorm：

$$r = \left(\frac{1}{D}\sum_i x_i^2 + \epsilon\right)^{-1/2}, \quad y_i = x_i r w_i.$$

权重和 bias 是 `[D]`，对所有行共享。代码显式要求 eps 为正，且转换成 float 后仍是有限正数，避免极小的 Python double eps 转换为 0 后悄悄改变约定。

逐步实现与数值选择

`norm_kernel<false>` 先 reduce 得到 mean，再重新读取 x 求中心化平方和，最后重新读取 x 做归一化和 affine。它采用两遍中心方差，而不是 `E[x²]−E[x]²`；后者对大均值、小方差的数据容易发生两个相近大数相减的消减问题。

`norm_kernel<true>` 编译掉 mean 分支，将 mean 视为 0，直接归约 x^2，再做缩放，没有 bias。两个版本都用 FP32 累加；这不能消除所有误差，但避免了输入低精度时继续低精度累加的额外问题。

PyTorch LayerNorm 的参考语义使用分母 D 的方差，而非无偏估计的 D−1；可以核对 torch.nn.LayerNorm。

数据流和复杂度

每 CTA 负责一行，block reduction 与前面相同。LayerNorm 读取 x 三遍，RMSNorm 两遍，再各写输出一次；weight/bias 跨行复用的缓存效果取决于具体运行。总工作 O(元素数)，每 CTA 的 scratch 是 1024 Byte。D 很小时一个 CTA 一行可能浪费；D 很大、行数很少时又可能并行度不够。

可以直接复述的追问答案

问：LayerNorm 方差为什么不用 D−1?

答：归一化层按这一行完整的 D 个特征计算其归一化统计量，公式和 PyTorch 参考语义使用 D。这里不是估计未观察总体方差的统计问题。实现时若调用默认采用不同 correction 的方差 API，就可能产生系统性差异，D=1 还会暴露除以零问题。

问：RMSNorm 是把 LayerNorm 的 mean 固定为 0 吗?

答：公式上它不计算或减去样本 mean，使用 x 的均方根归一化；因此可复用类似的 sum-of-squares kernel，但两者不是对任意输入等价的变换。给所有输入加一个常数时，LayerNorm 的中心化部分会消除这个平移，RMSNorm 一般不会。

问：eps 放在 sqrt 里面还是外面？

答：这里的约定是 `rsqrt(statistic + eps)`，即在开平方之前加。`1/(sqrt(statistic)+eps)` 是另一个函数，在 statistic 很小时差异尤其明显。对齐参考结果时必须同时对齐 eps 值和它的位置。

问：如何进一步改进方差算法？

答：可以研究 Welford 状态 `(count, mean, M2)` 的单遍更新与并行合并。合并两组 A/B 时，令 delta=meanB−meanA，n=nA+nB，得到 `mean=meanA+delta*nB/n`，`M2=M2A+M2B+delta²*nA*nB/n`，最后 variance=M2/n。这样可以减少读输入的遍数并改善数值表现，但单个归约元素的状态和运算更多，不能只凭遍数就判定更快。

问：低精度版本如何做？

答：x、weight、bias 可使用 FP16/BF16 存储，加载后转 FP32，统计量和归一化中间值用 FP32，最后按约定写回输入 dtype。特别要检查 eps 不会在低精度中先被量化成 0；也要明确 affine 是在 FP32 中完成还是先截断，二者会影响误差。

问：LayerNorm backward 怎么写？

答：令 `xhat=(x−mean)*r`，`r=rsqrt(var+eps)`，`g=dy*w`，则每行

$$dx = \frac{r}{D}\left(Dg - \Sigma g - xhat\,\Sigma(g\,xhat)\right).$$

输入梯度需要两个行归约。参数梯度为跨行求和 `dw=sum_rows(dy*xhat)`、`db=sum_rows(dy)`，归约方向与输入梯度不同。实际 kernel 设计要分别考虑这两种数据流，不能因为 forward 是一行一个 block 就直接照搬全部 backward。

问：RMSNorm backward 呢？

答：令 `g=dy*w`、`r=rsqrt(mean(x²)+eps)`，则

$$dx = rg - xr^3\,\mathrm{mean}(gx), \qquad dw = \sum_{rows} dy\,x\,r.$$

这里没有 LayerNorm 的减均值项，也没有 bias 梯度。教材的 CUDA forward-only 接口没有实现上述 backward；这些公式是扩展方向，不能误称现有扩展已经支持训练。

必测边界

D=1 时 LayerNorm 的标准化部分应为 0，因此输出等于 bias；RMSNorm 则一般不为 0。再测 D=33、513、常量行、大均值但小方差、前导维为 0、不同 eps。不能用宽容差把方差分母或 eps 位置写错的问题掩盖掉。

第 8 题的 Triton 扩展：LayerNorm 与 RMSNorm

公式、接口与数据布局

`layer_norm(x,weight,bias,eps)` 和 `rms_norm(x,weight,eps)` 都沿最后一个维度 N 归一化。X 可以是 `[N]`、`[M,N]`、`[B,T,N]` 等，只要 contiguous；前导维展平成 rows。weight／bias 为 `[N]`，与 x 同 dtype 和 device。

LayerNorm：

$$\mu = \frac{1}{N}\sum_j x_j, \quad \sigma^2 = \frac{1}{N}\sum_j (x_j-\mu)^2, \quad y_j = (x_j-\mu)(\sigma^2+\epsilon)^{-1/2}\gamma_j + \beta_j.$$

RMSNorm：

$$r = \left(\frac{1}{N}\sum_j x_j^2 + \epsilon\right)^{-1/2}, \quad y_j = x_j r \gamma_j.$$

本版本 epsilon 默认 `1e-5`，要求它在转换成 FP32 后仍为正有限数。仅检查 Python float 正有限还不够：`1e-300` 会在 FP32 中变成 0，`1e300` 会溢出。wrapper 用 host 端的 IEEE FP32 pack/unpack 检查并转换，不读取 GPU 数据；相应 CPU 参数测试在 `tests/test_triton_host.py`。比较框架实现时也应显式传同样的 epsilon，不能依赖双方可能不同的默认值。

为什么 LayerNorm 的 mask 要特别小心?

N=33、BLOCK=64 时，load 对无效位置填 0。均值用 `sum(x)/N`，这样没有问题。但是随后如果对全部 64 个位置直接计算 `(x-mean)^2`，后 31 个占位 0 会变成 `mean^2`，污染方差。

因此源码专门写成：

```
centered = tl.where(valid, x - mean, 0.0)
variance = tl.sum(centered * centered, axis=0) / N
```

分母仍是实际 N，不是 BLOCK。这个 bug 在平均值接近 0 的随机数据上可能不明显，因此测试还使用 `100 + randn(3,33)`，让错误的 padding 贡献变得显著。

两遍统计，与两次读 HBM，是两回事

本 LayerNorm 先求均值，再求中心化平方和，属于两遍统计。但 x 在 program 内的逻辑块中被复用，源码只进行一次 X 的 global load；不是要求把整个 X 从 HBM 再读一遍。实际寄存器压力与 spill 要看编译器和硬件。

与 `E[x²]-E[x]²` 相比，直接归约 `(x-mean)²` 避免把两个可能很接近的大数相减。在均值很大而方差很小时，这个差分更容易损失有效位。

如果要跨很多块归并统计，可以考虑 Welford。对两个统计组 `(n_A,mu_A,M2_A)`、`(n_B,mu_B,M2_B)`，令：

$$\delta = \mu_B - \mu_A, \quad n = n_A + n_B,$$

$$\mu = \mu_A + \delta\frac{n_B}{n}, \qquad M2 = M2_A + M2_B + \delta^2\frac{n_A n_B}{n}.$$

其中 M2 是相对本组均值的平方偏差和，最终方差为 M2/n。这种状态可分块合并，但比单一 sum 状态更复杂，有额外算术和通信。对当前中等行宽、一次加载整行的教育实现，先用两遍中心化统计更容易写对和解释；是否换 Welford，需要由稳定性需求、并行布局与性能证据决定。

面试追问与回答

问 1：LayerNorm 的方差分母是 N 还是 N−1?

答：这里是 N，即总体／biased variance；它是归一化 layer 所使用的定义。若用 `torch.var` 写参考，要明确选择 `correction=0` 或适用版本中的 `unbiased=False`，不能套用估计样本方差的 N−1。

问 2：epsilon 为什么放在平方根内部?

答：题目定义是除以 `sqrt(variance+eps)`。`sqrt(variance)+eps` 是另一个函数，尤其在方差接近 0 时差异很大。RMSNorm 同理，必须与参考实现保持相同位置与数值。

问 3：RMSNorm 为什么少了一个步骤?

答：它不减均值，直接使用均方根归一化。因此只需要平方和归约；LayerNorm 还要估计均值并中心化。这是两个不同的变换，不能只因为 RMSNorm 更容易融合，就在没有模型依据时随意替换。

问 4：FP32 统计就一定数值稳定了吗?

答：它可以降低半精度累加误差，但不能解决所有问题。输入已经量化的信息无法恢复，极端值平方仍可能溢出，最后转回低精度也会舍入。我会明确有限输入的数值范围，使用常数行、较大均值加小扰动、不同 N 和 dtype 测试；必要时再考虑更稳定的算法。

问 5：这个 kernel 可以直接训练吗?

答：不可以。raw Triton forward 不会自动生成正确的 autograd 链，本模块在 grad mode 下对 requires_grad 输入报错。需要另写 backward，并把梯度规则注册到 PyTorch，再验证输入、weight、bias 梯度。关闭这个检查并不会让 backward 自动正确。

问 6：如果面试官继续问输入梯度，可以怎样回答?

答：设 `g=dY*gamma`，LayerNorm 的 `u=(x-mu)*r`、`r=1/sqrt(var+eps)`，则一行的输入梯度可以写为：

$$dX = r[g - \mathrm{mean}(g) - u\,\mathrm{mean}(gu)].$$

RMSNorm 则为 `u=x*r`：

$$dX = r[g - u\,\mathrm{mean}(gu)].$$

weight 梯度需要沿所有前导维归约，LayerNorm 的 bias 梯度是对应 dY 的归约。除了公式，还应考虑前向保存什么统计量、反向是否重算、多个 programs 如何合并参数梯度；这些属于后续实现，当前 Triton 文件没有伪装成已经支持。

问 7：怎样进一步加速 Norm?

答：先按 N 与 rows 调整一 program 多行或一行多 warps；接着看寄存器、spill 和占用。若模型结构允许，可以融合 residual add、bias 或后续逐元素操作，减少中间读写。融合越多，资源占用和编译复杂度也会增加，因此仍要检查端到端收益。

官方实现与 forward／backward 思路可参考 Triton Layer Normalization 教程。

第 8 题：LayerNorm 与 RMSNorm，包括完整反向推导

题目与实现位置

本题的类为 `ManualLayerNorm` 和 `ManualRMSNorm`。要求沿 `normalized_shape` 指定的末尾维度做归一化，显式使用 `eps=1e-5`，支持可训练缩放；LayerNorm 可加偏置。实现内部不调用现成的 norm 层。

FP16/BF16 输入的统计量在 FP32 中计算，最终恢复输入 dtype；FP64 输入保留 FP64，供数值梯度检查使用。不能无条件 `.float()`，否则 double 输入也被降成 FP32，有限差分的精度基础会丢失。特征归约维必须为正，外层没有样本的张量可以自然返回空结果。

epsilon 还必须能用实际累加 dtype 表示为正有限值。例如 Python 中 `1e-300` 为正，在 FP32 中却会舍入为零，零输入就可能重新产生除零问题；在 FP64 中这个值仍然合法。两种 Norm 在 forward 按累加 dtype 检查这一点，使用与 FP32 转换等价的纯 host 边界比较，不创建 GPU 标量或调用 `.item()`。它没有把“可表示”错误地收紧为“必须是正规数”；正的可表示 subnormal 仍可通过。

两个公式放在一起看

把一次归一化的所有特征展成长度 D 的向量，LayerNorm 是：

$$\mu = \frac{1}{D}\sum_i x_i, \quad v = \frac{1}{D}\sum_i (x_i - \mu)^2, \quad r = (v + \epsilon)^{-1/2},$$

$$z_i = (x_i - \mu) r, \qquad y_i = \gamma_i z_i + \beta_i.$$

RMSNorm 是：

$$m_2 = \frac{1}{D}\sum_i x_i^2, \quad r = (m_2 + \epsilon)^{-1/2}, \quad z_i = x_i r, \quad y_i = \gamma_i z_i.$$

区别不只是少写一行代码：LayerNorm 先消除均值，再控制尺度；RMSNorm 用均方根控制尺度，不要求输出均值为零。这两种模型组件不能在未经验证时互换。PyTorch 的 LayerNorm采用分母 D 对应的总体方差，即 `correction=0`；RMSNorm的默认 epsilon 规则可能不同，因此参考比较时要显式传相同的 `eps`。

用一个小向量算一次

设 `x=[1,2,3]`、缩放全为 1、偏置为 0，暂时忽略很小的 epsilon：

- LayerNorm：均值 2，方差 `2/3`，输出约 `[-1.225,0,1.225]`。
- RMSNorm：均方为 `14/3`，输出约 `[0.463,0.926,1.389]`。

后一个输出并不以零为中心，这是公式的预期行为。加入 epsilon 后，未经过 affine 的输出方差或均方也不严格等于 1。例如 LayerNorm 的归一化方差为 `v/(v+eps)`。测试若机械断言方差“精确等于 1”，会把正确实现误判为错误。

为什么减均值后再求平方？

可以把方差写成 `E[x^2]-E[x]^2`，但当 x 的均值很大、波动很小时，这是两个相近大数相减，可能损失精度。参考实现使用两遍思路：先求均值，再求偏差的平方均值，容易解释和验证。

进一步优化时可以考虑 Welford 形式，它把统计量表示成样本数、均值和偏差平方和，支持稳定地组合部分结果。面试时不能只说“Welford 更好”，应指出它在并行归约中的组合能力，以及指令数、数值稳定性和数据读取次数之间的取舍。

`normalized_shape=(3,4)` 到底是什么意思？

输入 `[B,3,4]`，指定 `(3,4)` 表示每个样本的 12 个元素共同计算一组统计量，gamma/beta 的形状也为 `[3,4]`。指定 `4` 则表示每个 `[4]` 行分别归一化，一共有 `B*3` 组统计量。

代码使用末尾的多个轴归约，并保留 `keepdim=True`。保留长度为 1 的维度后，均值或 reciprocal standard deviation 可以按原位置广播，避免减法误对齐。其数学仍可按“每行 D 个特征”推导，D 是归一化维度大小的乘积。

LayerNorm 反向：从中间量推到可实现的公式

设 `g_i=dL/dy_i`，定义经过缩放参数后的梯度：

$$a_i = g_i \gamma_i.$$

前向归一化是 `z_i=(x_i-mu)r`。反向有三条依赖：x 直接影响自己的偏差，x 影响均值，x 也通过方差影响所有元素使用的 r。合并这些贡献，可以得到：

$$\frac{\partial L}{\partial x_i} = r\left[a_i - \text{mean}(a) - z_i\,\text{mean}(az)\right].$$

这个式子包含两次行内归约：`mean(a)` 和 `mean(a*z)`。不能把后两项省掉，只写 `dx=dy*gamma*r`，因为均值和方差也依赖 x。

推导的中间步骤是：

1. `dz_i = a_i`。
2. `dr = sum_i a_i * (x_i-mu)`。
3. 因为 `r=(v+eps)^(-1/2)`，所以 `dv=-0.5*r^3*dr`。
4. `v=mean((x-mu)^2)` 对每个偏差贡献 `2*(x_i-mu)/D`。

5. 再合并 `mu=mean(x)` 带来的所有路径，得到上面的紧凑公式。

参数梯度是对所有外层样本位置求和：

$$\frac{\partial L}{\partial \gamma_i} = \sum_{rows} g_i z_i, \qquad \frac{\partial L}{\partial \beta_i} = \sum_{rows} g_i.$$

注意它们的归约方向与输入梯度不同：`dx` 需要一行内部的统计量；`d_gamma/d_beta` 需要跨行累积。真正写 GPU backward 时，这种差别会影响并行划分、临时缓冲和是否需要两阶段归约。

RMSNorm 反向：少了哪一项？

仍令 `a_i=g_i*gamma_i`、`r=(mean(x^2)+eps)^(-1/2)`、`z_i=x_i*r`。因为没有均值中心化，不再需要 `-mean(a)`：

$$\frac{\partial L}{\partial x_i} = ra_i - r^3 x_i \,\mathrm{mean}(ax) = r\left[a_i - z_i \,\mathrm{mean}(az)\right].$$

参数梯度仍是：

$$\frac{\partial L}{\partial \gamma_i} = \sum_{rows} g_i z_i.$$

这也是一个实用的记忆方法：先理解“均值中心化”引入了什么依赖，再理解 RMSNorm 为什么没有对应项。不要把两个 backward 公式混着背。

如何验证梯度，而不被 ReLU 或低精度干扰？

本题本身在正 epsilon 下是光滑的，适合对很小的 double 输入做 `gradcheck`。测试使用 `torch.func.functional_call` 把 gamma/beta 也作为显式输入，因此可以检查输入和参数的数学导数。普通的前向比较则使用 PyTorch norm API 作为独立参考。

有限差分大致使用 `[f(x+h)-f(x-h)]/(2h)`。h 太大会有截断误差，太小会有舍入误差，所以先在 FP64、小尺寸下检查公式，再在目标 GPU dtype 下检查允许误差。PyTorch 的 gradcheck 文档明确提醒，默认容差针对 double，且不可导点可能使数值与解析梯度不一致。

优化思路应如何回答？

问：为什么值得融合 norm？

答：“参考写法包含求均值、减均值、平方、归约、rsqrt、scale/bias 等多个逻辑操作。Eager 可能为中间结果分配张量并启动多个 kernel。融合可以把一行的数据与中间统计量尽量留在寄存器或共享存储，减少 HBM 读写和启动次数。具体是否只读一遍，取决于一行能否驻留以及实现方式。”

问：每行一个 block 总是最好吗？

答：“这是便于理解的起点。行很短时，一个 block 可能资源利用不足，可以一块处理多行；行很长时，寄存器需求和归约长度增加，可能需要分段归约或多阶段计算。行数很少时还要考虑并行度不足。我会按行长度和行数选择不同方案，而不是只调一个 block size。”

问：能用原子操作累加 gamma 的梯度吗？

答：“可以作为简单版本，因为多行会写同一 gamma 位置。但要考虑竞争、累加顺序和可重复性。另一种方案是先让不同 block 写各自的部分和，再做第二次归约。哪种更合适，要结合行数、特征数和精度要求测量。”

问：RMSNorm 是否一定比 LayerNorm 快？

答：“它减少了中心化相关工作，算法上更简单。但实际速度还受是否已经融合、内存带宽、launch 和形状影响。不能从少一次 mean 就直接承诺端到端加速，更不能不经模型验证就替换归一化类型。”

问：为什么把 epsilon 放到平方根里面?

答：“这是我们定义和参考实现的公式：`rsqrt(statistic+eps)`。改成 `1/(sqrt(statistic)+eps)` 是另一个函数，尤其在统计量很小时前向和梯度都不同。epsilon 也不应只在一端默认，让比较的两边采用不同数值。”

问：能不能把所有张量都转成 FP32 保证稳定?

答：“低精度输入的统计量升为 FP32 是合理起点，但不应无条件把 double 降成 FP32。另一方面，FP32 也不是对所有极端输入都不会溢出。还需要声明支持的数值范围，并单独检查输出误差和模型效果。”

本题的完成标准

你应能在没有参考代码时写出两种 forward；知道方差分母与 epsilon 位置；能画出归约轴；能推导并解释两种 `dx` 公式；能说出 GPU norm 的并行与融合选择。代码中的 autograd 与测试用于验证你的推导，不能代替你解释它。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/torch_layers.py

```
class ManualLayerNorm(nn.Module):
    """LayerNorm with population variance, explicit epsilon and autograd.

    Statistics reduce all normalized_shape axes. Empty leading dimensions are
    supported; an empty reduction domain is rejected at construction time.
    """

    def __init__(self, normalized_shape: int | Sequence[int], eps: float = 1e-5,
                 elementwise_affine: bool = True, bias: bool = True,
                 device=None, dtype=None) -> None:
        super().__init__()
        self.normalized_shape = _normalized_shape(normalized_shape)
        if not math.isfinite(eps) or eps <= 0:
            raise ValueError("eps must be finite and positive")
        self.eps = eps
        if elementwise_affine:
            self.weight = nn.Parameter(torch.ones(self.normalized_shape, device=device, dtype=dtype))
            if bias:
                self.bias = nn.Parameter(torch.zeros(self.normalized_shape, device=device, dtype=dtype))
            else:
                self.register_parameter("bias", None)
        else:
            self.register_parameter("weight", None)
            self.register_parameter("bias", None)

    def forward(self, x: Tensor) -> Tensor:
        _check_normalized_input(x, self.normalized_shape)
        acc = _accumulator(x)
        _check_accumulator_eps(self.eps, acc.dtype)
        axes = tuple(range(acc.ndim - len(self.normalized_shape), acc.ndim))
        mean = acc.mean(dim=axes, keepdim=True)
        centered = acc - mean
        # mean of squared deviations is correction=0, not sample variance.
        variance = centered.square().mean(dim=axes, keepdim=True)
        out = centered * torch.rsqrt(variance + self.eps)
        if self.weight is not None:
            out = out * self.weight.to(dtype=acc.dtype)
        if self.bias is not None:
            out = out + self.bias.to(dtype=acc.dtype)
        return out.to(dtype=x.dtype)
```

```python
class ManualRMSNorm(nn.Module):
    """RMSNorm without mean subtraction; epsilon is deliberately explicit."""

    def __init__(self, normalized_shape: int | Sequence[int], eps: float = 1e-5,
                 elementwise_affine: bool = True, device=None, dtype=None) -> None:
        super().__init__()
        self.normalized_shape = _normalized_shape(normalized_shape)
        if not math.isfinite(eps) or eps <= 0:
            raise ValueError("eps must be finite and positive")
        self.eps = eps
        if elementwise_affine:
            self.weight = nn.Parameter(torch.ones(self.normalized_shape, device=device, dtype=dtype))
        else:
            self.register_parameter("weight", None)

    def forward(self, x: Tensor) -> Tensor:
        _check_normalized_input(x, self.normalized_shape)
        acc = _accumulator(x)
        _check_accumulator_eps(self.eps, acc.dtype)
        axes = tuple(range(acc.ndim - len(self.normalized_shape), acc.ndim))
        out = acc * torch.rsqrt(acc.square().mean(dim=axes, keepdim=True) + self.eps)
        if self.weight is not None:
            out = out * self.weight.to(dtype=acc.dtype)
        return out.to(dtype=x.dtype)
```

gpu_prep/csrc/kernels.cu

```cpp
template <bool IsRms>
__global__ void norm_kernel(const float* x, const float* weight,
                            const float* bias, float* out,
                            int64_t rows, int64_t width, float eps) {
  __shared__ float scratch[kThreads];
  for (int64_t row = blockIdx.x; row < rows; row += gridDim.x) {
    const int64_t base = row * width;
    float mean = 0.0f;
    if constexpr (!IsRms) {
      float local_sum = 0.0f;
      for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
        local_sum += x[base + col];
      mean = block_reduce<false>(local_sum, scratch) / static_cast<float>(width);
    }
    float local_sq = 0.0f;
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x) {
      const float centered = x[base + col] - mean;
      local_sq = fmaf(centered, centered, local_sq);
    }
    const float variance = block_reduce<false>(local_sq, scratch)
                           / static_cast<float>(width);
    const float inv_std = rsqrtf(variance + eps);
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x) {
      const float normalized = (x[base + col] - mean) * inv_std;
      const float affine = normalized * weight[col];
      if constexpr (IsRms) out[base + col] = affine;
      else out[base + col] = affine + bias[col];
    }
  }
}

at::Tensor layer_norm_cuda(const at::Tensor& x, const at::Tensor& weight,
                           const at::Tensor& bias, double eps) {
  check_norm_weight(x, weight);
  check_tensor(bias, "bias");
  check_same_device(x, bias, "bias");
  TORCH_CHECK(bias.dim() == 1 && bias.numel() == x.size(-1),
```

```
              "bias must have shape [x.size(-1)]");
  const float e = check_eps(eps);
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const int64_t width = x.size(-1), rows = x.numel() / width;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  norm_kernel<false><<<blocks_for(rows), kThreads, 0, stream>>>(
      x.data_ptr<float>(), weight.data_ptr<float>(), bias.data_ptr<float>(),
      out.data_ptr<float>(), rows, width, e);
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor rms_norm_cuda(const at::Tensor& x, const at::Tensor& weight, double eps) {
  check_norm_weight(x, weight);
  const float e = check_eps(eps);
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const int64_t width = x.size(-1), rows = x.numel() / width;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  norm_kernel<true><<<blocks_for(rows), kThreads, 0, stream>>>(
      x.data_ptr<float>(), weight.data_ptr<float>(), nullptr, out.data_ptr<float>(),
      rows, width, e);
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}
```

gpu_prep/triton_ops.py

```
@triton.jit
def _norm_kernel(X, W, Bias, Y, N: tl.constexpr, EPS: tl.constexpr,
                 IS_RMS: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    valid = col < N
    x = tl.load(X + row * N + col, valid, other=0).to(tl.float32)
    w = tl.load(W + col, valid, other=0).to(tl.float32)
    if IS_RMS:
        second_moment = tl.sum(x * x, axis=0) / N
        normalized = x * tl.rsqrt(second_moment + EPS)
        out = normalized * w
    else:
        mean = tl.sum(x, axis=0) / N
        # Padding must contribute zero, not (0 - mean)^2, to the variance.
        centered = tl.where(valid, x - mean, 0.0)
        variance = tl.sum(centered * centered, axis=0) / N
        normalized = centered * tl.rsqrt(variance + EPS)
        b = tl.load(Bias + col, valid, other=0).to(tl.float32)
        out = normalized * w + b
    tl.store(Y + row * N + col, out, valid)


def _norm_shape(x: torch.Tensor, weight: torch.Tensor, eps: float) -> tuple[int, int, float]:
    _check_tensor(x, "x")
    if x.ndim < 1:
        raise ValueError("x must have at least one dimension")
    n = x.shape[-1]
    if n == 0 or n > MAX_ROW_WIDTH:
        raise ValueError(f"normalized dimension must be in [1, {MAX_ROW_WIDTH}]")
    _check_peer(x, weight, "weight", ndim=1)
    if weight.shape[0] != n:
        raise ValueError("weight must have shape [x.shape[-1]]")
    return x.numel() // n, n, _check_eps(eps)
```

```
def layer_norm(x: torch.Tensor, weight: torch.Tensor, bias: torch.Tensor,
               eps: float = 1e-5) -> torch.Tensor:
    """Last-dimension LayerNorm; population variance (divide by N), affine=True."""
    rows, n, eps = _norm_shape(x, weight, eps)
    _check_peer(x, bias, "bias", ndim=1)
    if bias.shape != weight.shape:
        raise ValueError("bias must have the same shape as weight")
    out = torch.empty_like(x)
    if rows == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        _norm_kernel[(rows,)](
            x, weight, bias, out, n, eps, False, BLOCK=block, num_warps=warps,
        )
    return out


def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5) -> torch.Tensor:
    """Last-dimension RMSNorm with learnable weight, no bias, explicit eps."""
    rows, n, eps = _norm_shape(x, weight, eps)
    out = torch.empty_like(x)
    if rows == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        # Bias is unused in the IS_RMS specialization, so weight is a harmless placeholder.
        _norm_kernel[(rows,)](
            x, weight, weight, out, n, eps, True, BLOCK=block, num_warps=warps,
        )
    return out
```

第 9 题｜Attention、MHA、Causal Mask 与 KV Cache

完整源文件： gpu_prep/torch_layers.py；gpu_prep/csrc/kernels.cu；gpu_prep/triton_ops.py。

练习 9 的 CUDA 实现：朴素稳定 Attention 与 Causal Offset

接口语义先说明白

输入 q `[B,H,L,D]`，k/v `[B,H,S,D]`。B 是 batch，H 是 head 数，L 是 query 长度，S 是 key/value 长度，D 是 head dimension。它计算每个 head 内

$$P = \mathrm{softmax}(QK^T/\sqrt{D} + mask), \quad O = PV.$$

当前接口的 q/k/v head 数和 D 必须一致，不包含 QKV 投影、GQA、dropout 或任意 padding mask。输出 shape 与 q 相同。

`causal=True` 时，第 i 个 query 的绝对位置为 `query_start+i`，只能看到 `key_index <= query_start+i`。长度为 S 的完整 prefill 通常 L=S、query_start=0；缓存已有前缀后处理最后一个 token，通常 L=1、query_start=S−1。

特别注意：如果 decode 传 L=1、S=100，却错误沿用 query_start=0，这个 query 只会看到第 0 个 key。它不会因为 L=1 就自动知道自己是第 100 个 token。本接口将这个位置显式传入，避免非方阵 causal 对齐的歧义。

一个 block 如何算一条 query

先把 `[B,H,L]` 展平为 query 行号 qr。`bh=qr/L` 定位 batch/head，`qi=qr%L` 定位该 head 的 query。

1. 256 线程按 key 下标分工。每线程负责若干 keys，对每个 key 在 D 维串行累加 dot product，再乘 `1/sqrt(D)`。不可见位置的 score 设为负无穷。
2. 把所有 S 个 scores 写到动态 shared memory，并对各线程局部 max 做 block max，得到这一 query 的全局最大值。
3. 原地将 shared scores 替换成 `exp(score-max)`，再归约得到分母。
4. 改为按输出 d 分工。每个线程负责若干输出维度，遍历 keys，计算归一化权重乘 V 的和。
5. 输出完成后同步，下一条 query 才可以覆盖 shared scores。

S 最大为 4096，因此 scores 至多 `4096×4=16384 Byte`，另有 1024 Byte reduction scratch。该限制是本实现选择的教学范围，不是 attention 算子的数学限制。

正确性与稳定性

每条 query 独立计算完整的 key 分数，稳定 softmax 与前面相同。query_start≥0、S≥1 确保 causal 模式至少 key 0 可见，不会在这份接口中产生整行全 masked 的情况；有限输入和不溢出的 dot product 仍是数值约定。

mask 只改变不可见位置的指数权重为 0，因此它们不会贡献到输出。测试使用显式绝对位置 mask 的 PyTorch FP64 oracle，避免把另一个 API 的默认 causal 对齐约定误当成本接口的答案。

复杂度，以及它为什么不是高性能 Attention

总算术工作量为 O(BHLSD)。本实现没有把整个 `[B,H,L,S]` score 矩阵放到全局显存中，而是每条 query 临时保存 S 个分数；但这并不自动使它成为 FlashAttention。

主要限制很具体：QK 阶段中，同一 warp 的线程负责不同 key，固定 d 时访问 K 的地址间隔为 D，访存合并不理想；dot product 在 D 上由一个线程串行完成；相邻 queries 之间没有主动共享 K/V tiles；PV 阶段若 D 很小，只有少量线程参与输出计算。这些问题比笼统地说“可以进一步优化”更值得说明。

可以直接复述的追问答案

问：为什么除以 sqrt(D)?

答：在 q/k 各维近似独立且具有相近方差的粗略假设下，D 项点积的方差随 D 增长，典型尺度随 sqrt(D) 增长。这个缩放用于控制分数尺度，避免 softmax 随维数变大而过于尖锐。真实模型不必严格满足这些独立假设，但这是常用缩放的直观动机。

问：causal mask 是在 softmax 前还是后加?

答：在 softmax 前把不可见 score 排除，保证分母只归一化可见位置。若先对所有位置 softmax，再把未来概率置零，却不重新归一化，剩余权重和通常小于 1，输出就变了。

问：怎样验证 KV cache 的 decode 正确?

答：先做完整序列的 causal attention，再用最后一条 q 和完整 k/v 调用单 token 路径，显式传 query_start=S−1，比较两者最后位置输出。进一步可逐 token 增长缓存，比较每一步与全序列对应位置。测试必须使用非零、非均匀数据，否则位置 mask 错误也可能被巧合掩盖。

问：不写全局 score 矩阵，为什么还不是 FlashAttention?

答：减少全局中间矩阵只是其中一个特征。高效分块 attention 还要围绕 Q/K/V tiles 做数据复用和矩阵乘法，使用稳定的 online softmax 状态在 key blocks 之间累积输出，并针对线程布局和硬件执行路径优化。这个 CUDA 版本是每 query 独立的串行 dot-product 基线，没有这些完整机制。

问：online softmax 怎样同时累积 attention 输出?

答：除了最大值 m 和分母 l，还维护未归一化输出向量 a。对新 key block，设分数块为 s，令

$$m' = \max(m, \max s), \quad \alpha = e^{m-m'}, \quad p = e^{s-m'},$$

$$l' = \alpha l + \Sigma p, \qquad a' = \alpha a + pV_{block}.$$

最终输出 `a/l`。当最大值变大，旧分母和旧输出累加器必须同时按 alpha 重缩放；只改 l 不改 a 会得到错误输出。初始 m=−∞、l=0、a=0，处理全 masked 块时还需避免出现 `−∞−(−∞)` 的 NaN。

问：prefill 与 decode 的优化关注点为何不同?

答：prefill 有较多 queries，K/V 可以在 query tile 内重复使用，矩阵计算更容易形成较高复用；单 token decode 的 L 很小，通常要为较少输出读取较长缓存，读取 K/V、并行度与 launch 开销更加突出。具体是否带宽受限取决于 batch、head、cache 长度、精度和硬件，不能只用阶段名称下结论。

问：attention backward 的基本公式是什么?

答：设 G 是输出梯度，前向概率矩阵为 P，则

$$dV = P^T G, \quad dP = GV^T,$$

$$dS = P \odot (dP - \text{rowsum}(dP \odot P)),$$

$$dQ = dSK/\sqrt{D}, \quad dK = dS^T Q/\sqrt{D}.$$

mask 位置不应贡献梯度。训练实现要决定保存 P 还是保存少量统计量后重算，以及如何组织 dK/dV 的跨 query 累加。本 CUDA 接口只有 forward，公式用于准备追问。

问：改为 FP16/BF16 应该注意什么?

答：点积累加、softmax 统计量和输出累加可保持 FP32，输入与最终输出采用低精度存储；若改为 Tensor Core 矩阵路径，还需要新的 tile 和 fragment layout。测试应覆盖长序列、较尖锐的分数、不同 D 以及 decode offset，而不只比较一个随机方阵。

最小使用例子

```
q = torch.randn(1, 2, 17, 13, device="cuda")
k = torch.randn_like(q)
v = torch.randn_like(q)
full = cuda_ops.attention(q, k, v, causal=True)

last_q = q[:, :, -1:, :].contiguous()
last = cuda_ops.attention(last_q, k, v, causal=True, query_start=16)
torch.testing.assert_close(last, full[:, :, -1:, :], atol=2e-6, rtol=2e-6)
```

第 9 题的 Triton 扩展：分块 online-softmax attention forward

这是一个完整的 attention 核心前向实现。Q/K/V 的线性投影与多头拆分属于前面的 PyTorch MHA 练习；本 kernel 接收已经拆成 heads 的 Q/K/V，重点解释如何避免物化 `[L,S]` 的分数和概率矩阵。

接口与精度合同

```
out = triton_ops.attention(
    q, k, v,                    # q[B,H,L,D], k/v[B,H,S,D]
    causal=True,
    query_start=0,
    attn_mask=None,              # 可选 bool [L,S] 或精确 [B,H,L,S]
)
```

- S>0，D 仅支持 16、32、64、128；B/H/L 可以使输出为空。
- Q/K/V 同 dtype 和 device、contiguous；输出 dtype 与 Q 相同。
- 布尔 mask 的 True 表示“可见”，与 causal 条件取 AND。支持 `[L,S]` 在所有 B/H 间共享，或精确 `[B,H,L,S]`；不做其他隐式广播。
- 不支持 additive mask、dropout、GQA、任意 stride、用户自定义 scale 或 backward；scale 固定为 `1/sqrt(D)`。
- q/k 点积使用 FP32 累加；FP32 输入明确 `input_precision="ieee"`。概率、中间最大值、分母、输出累加器均保留 FP32。
- P×V 也使用 FP32 点积，没有把概率偷偷转成 FP16/BF16 来获取吞吐。这是便于分析精度的教学选择，可能明显慢于生产 attention kernel。
- 全 mask 行的输出约定为零。普通 softmax 在空集合上没有自然归一化分布，这里是接口主动定义的行为，不能说它是普通公式自动得出的结果。

先解释为什么不能直接“每块各算一个 softmax”

对单个 query，目标为：

$$O = \frac{\sum_j e^{s_j} V_j}{\sum_j e^{s_j}}, \qquad s_j = \frac{QK_j^\top}{\sqrt{D}}.$$

如果每个 key 块单独归一化，它们的分母不同。把每个块的输出直接相加或平均，会改变不同块的权重。正确方法是保留能跨块合并的分母和加权分子，同时处理数值缩放。

每个 query 的三个状态

处理过一个 key 前缀后，保存：

$$m = \max_{j \in \text{seen}} s_j,$$

$$l = \sum_{j \in \text{seen}} e^{s_j - m},$$

$$a = \sum_{j \in \text{seen}} e^{s_j - m} V_j.$$

m 和 l 是每个 query 的标量；a 是长度 D 的向量。注意 a 是尚未除以分母的加权和，不是已经归一化的输出。初始状态为 `m=-inf, l=0, a=0`。

看见一个新 key tile，怎样更新?

新块的分数为 `s_new`，令：

$$m' = \max(m, \max(s_{new})), \qquad \alpha = e^{m - m'}.$$

旧分子和旧分母原来相对于旧最大值 m 缩放，现在要统一到新最大值 m'，所以都乘同一个 alpha：

$$P_{new} = e^{s_{new} - m'},$$

$$l' = \alpha l + \sum P_{new},$$

$$a' = \alpha a + P_{new} V_{new}.$$

所有 key 块处理完后才计算 `O=a/l`。新旧块都采用同一个指数基准，因此合并前后的数学定义一致。这个缩放推导比背住三行代码更重要：只更新 denominator、忘了更新 numerator，会得到错误的加权结果。

用两个元素手算一遍

设两个 score 为 `[0, log(2)]`，对应标量 value 为 `[2,5]`，每块只有一个 key。

第一块后：`m=0, l=1, a=2`。

第二块使最大值变成 `log(2)`，因此 `alpha=1/2`，新块指数为 1：

$$l' = \tfrac{1}{2} \times 1 + 1 = 1.5, \qquad a' = \tfrac{1}{2} \times 2 + 5 = 6.$$

输出为 `6/1.5=4`，与直接计算 `(1*2+2*5)/(1+2)=4` 相同。这一例子适合面试时解释为什么旧状态必须一起缩放。

源码的数据块怎样对应这三个状态?

固定 BM=16、BN=32。一个 program 处理同一 batch/head 下的 16 个 queries，并沿 S 方向顺序扫描 keys：

逻辑 tensor	shape	作用
`q`	`[BM,D]`	一次加载当前 query tile，跨 key tiles 复用
`k`	`[D,BN]`	构造成转置形式，便于 Q×K
`scores`	`[BM,BN]`	当前 tile 的 logits，不写全局中间数组
`v`	`[BN,D]`	当前 values
`running_max`	`[BM]`	每个 query 的 m

逻辑 tensor	shape	作用
`denominator`	`[BM]`	每个 query 的 l
`numerator`	`[BM,D]`	每个 query 的 a

整体 launch 的 program 编号同时编码 query block 和 batch/head：先用 `% q_blocks` 找 query block，再用 `// q_blocks` 找扁平 B/H。这样无须使用可能超过上限的二维 grid y 维。

空块、全 mask、尾块：如何避免 NaN?

如果尚未见过任何可见 key，`running_max=-inf`；当前块也全被屏蔽时，`new_max=-inf`。直接计算 `running_max-new_max` 就会产生 `-inf-(-inf)`。

代码仅在指数计算时建立安全参考值：

```
safe_max = tl.where(new_max == -float("inf"), 0.0, new_max)
old_scale = tl.exp(running_max - safe_max)
probabilities = tl.exp(scores - safe_max[:, None])
```

此时 old_scale=0、所有 probabilities=0，l 和 a 仍为 0。真正的 `running_max` 继续保存 `new_max=-inf`；**不能为了避免 NaN，把状态本身永久改成 0**。否则第一次遇到极小但有限的 logits 时，它们会相对错误的最大值 0 计算指数，可能全部下溢。

如果以前已经有可见 key，而当前块完全不可见，那么 new_max 仍等于旧 max，old_scale=1，新 probabilities=0，旧状态完整保留。

循环结束后，对 denominator=0 的行，用 1 作为安全除数。由于 numerator 仍为 0，输出明确是 0。尾部无效 query 也采用这套安全计算，但最终 store mask 为假，不写越界位置。

测试特意覆盖三种不同情形：整行永远无可见 key；第一个 key tile 全 mask、第二个 tile 才出现可见 key；第一个 tile 有可见 key、后续 tiles 全 mask。它们分别检查初始化、从空状态进入非空状态、非空状态在空块中保持不变。

causal offset：prefill 与 decode 不能混淆

key 位置是 `j=0..S-1`；query 的绝对位置定义为 `query_start+i`。可见条件是：

$$j \leqslant query_start + i.$$

完整 prefill 常用 L=S、query_start=0。缓存已经包含 S 个位置、只计算最后一个 token 的 query 时，L=1，通常应传 `query_start=S-1`。如果还传 0，唯一 query 只能看到第 0 个 key，虽然输出 shape 正确，语义却错了。

对一组新 token，query_start 必须对应它们在完整序列中的实际起点；不要无条件猜成 S−L，尤其在 cross-attention 或特殊缓存布局中。测试会比较完整 causal attention 最后一行与单 token decode 的结果。

计算量、存储量与当前版本的真实限制

QK 和 PV 两次矩乘的主项约为 `4*B*H*L*S*D` FLOPs，softmax 另有 O(BHLS) 的标量工作。算法没有把精确 dense attention 的主计算复杂度从平方降成线性；它主要改变中间数据存储和搬运。

当前 kernel 不分配完整 logits／probability 矩阵；工作 tile 的大小由 BM、BN、D 决定，Q/K/V/O 的全局存储随这些输入输出规模增长。如果你自己传入 `[B,H,L,S]` 的布尔 mask，这个 mask 本身仍然是 O(BHLS) 存储，不能再不加限定地称整个调用的所有内存为线性。

当前 causal 版本仍扫描所有 key tiles，并对不可见位置施加 mask；它没有跳过完全在未来的块。因此“有 causal mask”不等于代码自动节省了一半点积计算。可以根据 query block 的最大绝对位置缩短 key 循环，再对对角块进行逐元素 mask。

面试追问与回答

问 1：这是不是 FlashAttention?

答：它实现了同类的分块、在线归一化思想，用来解释如何不物化概率矩阵；我会称它为教育版 online-softmax attention forward。它没有完整生产实现的 backward、硬件流水线、精细调度和多种 shape／dtype 优化，不能用名字代替功能和性能验证。

问 2：为什么 denominator 与 numerator 都要乘 alpha?

答：它们原来都相对于旧 max 定义。新的 max 改变了指数的公共缩放基准，所以二者必须同时变换。只缩放其中一个，会改变分子与分母之间的比例，也就改变最终概率加权和。

问 3：为什么不每处理一块就除一次 l?

答：可以设计保存归一化输出的等价递推，但更新公式必须相应改变。本代码保存未归一化的 a，到最后只除一次，这样更容易核验。不能拿“保存已归一化 output”的某份实现公式，混进保存未归一化 numerator 的代码里。

问 4：为什么 P×V 不把 P 转 half?

答：生产实现常常需要在精度与吞吐之间做选择。本教育版把 P 保留 FP32，用于更直接地对照在线公式，避免无意引入一次概率量化。若要改用 FP16/BF16 点积，我会明确新增误差来源，比较不同 logits 范围、head dimension 和序列长度下的误差，再报告性能收益。

问 5：如果序列很长，是否还会有数值误差?

答：会。online softmax 避免直接指数溢出和保存整张矩阵，但仍有有限精度的点积、exp、分母和加权和累加。较小项可能下溢，累加顺序与普通 dense 实现也不同。我会用小尺寸 double precision 参考验证公式，再在长序列上检查误差随长度的变化。

问 6：为什么 decode 可能需要不同优化?

答：L=1 时，一个 batch/head 只有很少 query programs，沿 S 的循环很长，GPU 的并行度可能不足。可考虑 split-KV：多个 programs 处理不同 key 范围，各自输出 `(m,l,a)`，再用同样的状态合并公式归约。它增加临时状态和归并成本，但能增加并行度；是否有收益取决于 batch/head 数、S、D 和硬件。

问 7：怎样合并两个 split-KV 结果?

答：设两组是 `(m1,l1,a1)`、`(m2,l2,a2)`，令 `m=max(m1,m2)`，则：

$$l = e^{m_1-m}l_1 + e^{m_2-m}l_2, \qquad a = e^{m_1-m}a_1 + e^{m_2-m}a_2.$$

最后返回 a/l；两组都为空时仍需要处理 `-inf-(-inf)`。这说明在线状态不仅能顺序更新，也能在并行分块后合并。

问 8：怎么支持 GQA?

答：需要区分 query head 数 Hq 与 KV head 数 Hkv，要求二者有合法的分组关系，再把每个 query head 映射到共享的 KV head；不能只把当前三个相同的 B/H 偏移照搬过去。还应关注共享 KV 是否在执行中真正复用、重复读取多少。当前版本明确拒绝不同 head 数，避免假装支持。

问 9：怎么验证 causal mask 没写反?

答：可以改变未来 token 的 K/V，确认较早位置输出不变；比较完整序列与分段／decode 的结果；再用 L≠S 和非零 query_start 检查 offset。仅测试方阵 L=S、query_start=0，无法覆盖缓存场景。

问 10：下一步优化的顺序是什么?

答：先确定主要 workload 是 prefill 还是 decode，并确认 dtype／精度目标；再调 BM/BN、num_warps 与流水线，查看寄存器和片上资源；对 causal prefill 跳过完全不可见块，对 decode 评估 split-KV；最后再考虑更低精度点积、布局和硬件专用能力。每步都保留全 mask、尾块和 offset 测试，因为性能修改不能改变这些语义。

原始思想与完整实现参考：Online normalizer calculation for softmax、FlashAttention 论文、Triton 官方 Fused Attention。本章递推解释与代码是教学重写，不承诺复现论文或官方教程的加速数字。

第 9 题：手写多头 causal attention，并加入可靠的 KV cache

接口先讲清楚，后面才不会混淆 mask

底层函数 `scaled_dot_product_attention` 使用：

参数	形状／约定
q	`[B,H,L,d]`，L 是本次 query 数
k	`[B,H,S,d]`，S 是可访问 key 数
v	`[B,H,S,dv]`，value 的特征维可以与 d 不同
attn_mask	布尔张量，可广播到 `[B,H,L,S]`，`True` 表示允许访问
is_causal	为真时再与 causal 条件相交
query_start	本次 query 第 0 项的绝对位置；key 位置从 0 开始
dropout_p / training	仅在 training 为真且概率非零时应用 attention dropout

同批次、同 head 数的 q/k/v 必须具有相同 dtype 和 device。本实现不隐式广播 batch/head，也没有把 GQA 藏进接口。padding 常见原始形状是 `[B,S]`，必须显式变成 `padding[:,None,None,:]`，否则一个二维 mask 会从最右侧对齐，可能误把 batch 当 query 维。

低精度 q/k/v 的 attention 参考计算转到 FP32；矩阵乘法处还显式关闭外层 autocast，避免“已经 `.float()`，又被 AMP 自动降回低精度”的情况。最后输出转回 q 的 dtype，返回的 probabilities 保留累加 dtype。这里的 FP32 tensor 规则与 GPU 是否采用 TF32 等底层精度选项仍是不同问题，性能对比时应固定相关设置。

PyTorch SDPA 也使用“布尔 True 允许参加 attention”的约定，但 `nn.MultiheadAttention` 的 `key_padding_mask` 使用相反语义；非方阵 `is_causal=True` 的默认对齐也未必等于你想要的 decode 偏移。因此测试使用显式 mask 对照，而不依赖默认行为。官方 SDPA 文档

完整 forward 从哪几步组成?

`MultiHeadSelfAttention` 的 forward 有四步：

1. `X[B,T,D]` 经合并 Linear 得到 `[B,T,3D]`。
2. 拆出 Q/K/V，形状均为 `[B,H,T,d]`。
3. 计算每个 head 的 attention，得到 `[B,H,T,d]`。
4. 转回 `[B,T,H,d]`，合并 heads 为 `[B,T,D]`，再做输出投影。

对一个 head：

$$A = QK^{\top}/\sqrt{d}, \quad P = \mathrm{softmax}(A + mask), \quad O = PV.$$

`QK^T` 的输出 `[L,S]` 中，第 i 行是 query i 对全部 keys 的分数，所以 softmax 必须沿最后一维 S。把 softmax 放在 L 维会变成“不同 query 争夺一个 key”的分配，语义已经改变。

缩放 `1/sqrt(d)` 的常见解释是假设 Q/K 分量独立且具有单位量级方差，其点积方差随 d 增长，除以 sqrt(d) 可控制 logits 尺度，减轻 softmax 过早饱和。这是设计动机，不是对所有实际训练分布的严格统计保证。

数值稳定：先减最大值，再处理“整行不可访问”

对至少有一个有效且有限 logit 的行，先求有效位置最大值 m，再计算 `exp(score-m)`。最大的指数为 1，其他有效指数不超过 1，避免直接对很大的正分数取指数。

但如果整行全部被屏蔽，直接填成 `-inf` 后求 softmax，会出现 `-inf-(-inf)`。不能先制造 NaN，再仅用 `nan_to_num` 修饰输出；前向看起来变成零，并不保证反向没有经过含 NaN 的中间量。

本实现把全 masked 行的 softmax 输入先改成安全的全零行，然后把被屏蔽位置的指数强制设为零。分母为零时用 1 代替，明确得到全零 probabilities 和全零输出。该路径对有限输入保留有限零梯度。

“全 masked 输出为零”这里指 attention 核心输出。完整 MHA 如果输出 Linear 带非零 bias，那么 `out_proj(0)` 会等于 bias；这不是 mask 失败。若业务要求 padding query 的最终整行也为零，应在输出投影后另乘 query-valid mask。

本 PyTorch 参考层还明确支持 `S=0`：用空维矩阵乘法得到零输出，并保留对 q/k/v 的零梯度连接。包内 GPU 教学 attention 的支持域较窄，会拒绝空 key 维；不要把不同实现的边界约定混为一谈。

causal offset：KV cache 面试里最值得主动解释的细节

整序列计算时，query i 和 key j 都从位置 0 开始，条件是 `j<=i`。缓存已有 p 个 token、本次又输入 L 个 token 时，本次 query i 的绝对位置是 `p+i`，正确条件是：

$$j \leqslant p + i.$$

例如已缓存 3 个 token，再输入 2 个 token，新 queries 位于 3、4，总 keys 位于 0…4：

本次 query	绝对位置	可访问 keys
第 0 个	3	0、1、2、3
第 1 个	4	0、1、2、3、4

如果错误地使用一个 `[2,5]` 的普通左上角下三角矩阵，第 0 行只能看到 key 0，第 1 行只能看到 keys 0、1，几乎整个历史都被屏蔽。对于单 token decode，`L=1`，这种错误尤其隐蔽：程序完全能运行，输出却只看到了开头的位置。

代码在追加 cache 前记录 `query_start=cache.length`；追加新 K/V 后，再用这个旧长度生成 causal 条件。记录长度的前后顺序必须正确。

KVCache 的实现和复杂度

`KVCache` 一次分配 `[B,H,T_max,d]` 的 key 和 value 存储，并维护当前有效长度。追加时只把新的 K/V 写入 `[length:length+new_T]`，随后返回有效前缀视图。

完整预分配存储是连续的，但 `cache.get()` 返回的前缀切片在未填满容量时通常不是连续 Tensor：相邻 head 的 stride 仍按 `T_max*d` 跨越，逻辑上的有效长度却只有 `length`。PyTorch reference matmul 可以处理这个 stride；接入本包要求连续输入的 GPU attention 时，需要选择支持 stride 的 kernel 或合适的缓存/读取布局。不要每个 token 都盲目把整个有效前缀 `.contiguous()`，否则会重新引入随历史增长的整段复制，抵消预分配的主要收益。

与每步 `torch.cat([old_k,new_k],dim=-2)` 相比，预分配避免为每个 token 分配并复制全部历史缓存。对于一次单 token 追加，新增写入量约为 `O(B*H*d)`；但 **attention 本身仍要读取历史 keys/values**，其单步成本仍随历史长度增长。不能因此把整个 decode 宣称为 O(1)。

代码选择一个清晰的教学边界：同一 batch 的序列共享一个长度；一个 cache 属于一个 attention layer 和一个独立会话批次。它不实现不同请求长度、分页、回收、prefix sharing、RoPE 或并发调度。真实模型如果使用 RoPE，应保证缓存中的 K 已按正确绝对位置处理；本练习是没有位置编码的单层 attention 等价性验证。

缓存路径会修改内部状态，因此只支持推理。`forward_step` 要求同时满足：

```
model.eval()
with torch.inference_mode():
    y = model.forward_step(x, cache)
```

`eval()` 关闭本层训练 dropout；inference/no-grad 关闭梯度记录。代码不默默吞掉训练梯度：条件不满足时主动抛错。超过容量也先报错再写入，避免验证失败后留下半截状态。

如果在外层开启 autocast，cache 的 dtype 应匹配 QKV 投影后实际得到的 K/V dtype，而不能只看原始 x 的 dtype。预分配也不自动保证适合编译或 CUDA Graph：本教学 cache 用 Python 整数维护 length，切片形状随长度增长；这些状态变化可能触发 guard 或新的捕获。真实部署应另外设计长度表达、固定容量 mask、分桶和图复用策略，并验证性能。

怎样证明 prefill + decode 与整序列一致?

测试采用固定权重、同一输入和关闭 dropout 的模型：

1. 一次性计算完整 causal attention，保存全部位置的输出。
2. 重新建立空 cache，先喂前 3 个 token。
3. 继续追加一个 token，再追加一个多 token chunk，最后追加剩余 token。
4. 把各次新输出拼回原顺序，与第 1 步逐元素比较。

加入多 token chunk 很关键：只测试逐 token decode，有时发现不了 chunk 内未来 token 泄漏。测试还修改了输入中未来 token 的内容，验证过去位置的输出保持不变。

有些优化会改变浮点累加顺序，所以“等价”通常按明确的 dtype 与容差判断，不要求所有 GPU 路径按 bit 完全一致。这里小尺寸 FP64 参考测试采用较严格容差；低精度 GPU kernel 应另设符合精度的阈值。

Attention 反向的完整骨架

忽略 dropout，令上游梯度为 `G=dL/dO`。因为 `O=PV`：

$$dV = P^T G, \qquad dP = GV^T.$$

对一行 softmax，有：

$$\frac{\partial p_i}{\partial a_j} = p_i(\delta_{ij} - p_j),$$

所以不必显式构造 S×S Jacobian，向量形式为：

$$dA = P \odot (dP - \mathrm{sum}(dP \odot P,\ dim = -1,\ keepdim = True)).$$

最后 `A=QK^T/sqrt(d)`：

$$dQ = dAK/\sqrt{d}, \qquad dK = dA^T Q/\sqrt{d}.$$

被屏蔽位置的概率与分数梯度均为零。若训练时加入 dropout，必须把 dropout 的实际 mask 与缩放应用到相应 backward，不能直接复用“没有 dropout”的梯度公式。

本参考实现依靠 autograd 计算这些链式导数；测试既与 SDPA 的前向/梯度比较，也对部分屏蔽与全屏蔽行做 double gradcheck。你应能把这些矩阵形状写出来，而不只是背四个公式。

优化方向：从普通 attention 到 FlashAttention

普通实现显式生成 `[B,H,T,T]` 分数和概率。单个这样的 FP32 张量就占 `4*B*H*T*T` 字节，实际峰值还包括多个中间结果和训练保存状态。将 Q/K/V 改为半精度，并不能让这个参考代码自动摆脱 T^2 级中间存储。

分块 attention 的核心是按 K/V 块更新每行 softmax 的统计量和未归一化输出，不把全部分数写回 HBM。若已有统计量 `(m,l,U)`，新块分数为 S，则可以更新：

$$m' = \max(m, \max S), \qquad a = \exp(m - m'),$$

$$\tilde{P} = \exp(S - m'), \quad l' = al + \Sigma \tilde{P}, \quad U' = aU + \tilde{P} V_{block}.$$

最终 `O=U/l`。这里旧的分母和输出都要乘同一个 rescale 系数 alpha；只更新最大值、不重新缩放旧累积量会出错。处理起始状态或一个块没有任何有效 key 时，还必须避免 `-inf-(-inf)`。这与本题全 masked 行的处理属于同一类数值边界。

面试回答可以是：“FlashAttention 的重要改进是减少 attention 中间结果的 HBM 读写，并安排分块与在线 softmax；精确 attention 的乘加规模不会因此普遍变成线性。对于不同 sequence 长度、head dim、prefill/decode，还需要不同并行和调度策略。”参考 FlashAttention 项目与 Triton fused attention 教程。

MHA、MQA、GQA 和 KV cache 的关系

MHA 中每个 query head 都有对应 K/V head，即 `H_q=H_kv`；MQA 使用一个 K/V head；GQA 让一组 query heads 共享一个 K/V head。若每个 head 维度为 d，每个元素 s 字节，每层缓存约为：

$$\text{KV bytes} = 2BTH_{kv}ds.$$

因此减少 K/V head 数可降低缓存容量和读写量。实际支持 GQA 还要明确 q head 到 kv head 的映射；在演示中 `repeat_interleave` 可以表达数学，但若物化重复 K/V，会抵消一部分内存收益。这个包的 MHA 参考层没有隐式扩展 GQA，面试扩展时应新增明确接口并验证。

面试追问：可直接复述的回答

问：为什么 prefill 和 decode 可能需要不同 kernel?

答：“Prefill 通常同时处理多个 query，QK 和 PV 有较大的矩阵维度，数据复用与 Tensor Core 利用有较多空间。单 token decode 的 query 长度很小，却需要访问长历史 K/V；此时缓存读取、并行划分和低 batch 下的利用率更突出。是否受带宽或算力限制，需要结合实际形状与测量判断。”

问：你的 cache 省掉了哪些计算?

答：“它保存已处理 token 的 K 和 V，后续只对新 token 做 Q/K/V 投影，并用新的 Q 查询全部有效缓存。不再重新计算历史 K/V，但仍然需要把新 Q 与历史 K 做匹配，再用权重读 V。”

问：用了 causal mask，为什么还要检查 cache offset?

答：“causal 是绝对位置关系。缓存输入里 query 长度和 key 长度往往不同，本地 query 下标不能直接当绝对位置。我记录旧 cache 长度 p，让 query i 只访问 `j<=p+i`，并用 prefill+chunk+decode 对完整结果做验证。”

问：能不能直接用 `softmax(masked_fill(-inf))`?

答：“如果保证每行至少有一个有效有限分数，这样可以。如果接口支持全 masked 行，我会在 softmax 之前建立安全分支，并明确返回零，保证 backward 也没有 NaN。我不会只把最后输出的 NaN 改掉就认为问题解决。”

问：返回 attention weights 会有什么影响?

答：“完整 weights 本身是 L×S 规模。如果业务必须拿到它们，就不能同时宣称完全避免存储这张矩阵。生产推理若只需要 attention 输出，应让接口允许不返回权重；本参考代码主要服务可读性和检查，仍显式构建概率。”

问：这个手写版本能替代生产中的 SDPA 吗?

答：“它是透明的数学与边界参考，有训练支持和明确测试。生产选择还要比较融合程度、内存峰值、硬件指令、shape 覆盖和维护成本。我会用它验证新的 CUDA/Triton kernel，再测实际 workload，而不是因为代码能跑就替换已优化的实现。”

本题的完成标准

能手写完整 QKV→attention→output projection；解释每一步 shape；正确区分布尔 mask 语义；处理全 masked 和非方阵因果关系；理解并验证 cache 等价性；能推导 softmax/attention 梯度，并解释融合、GQA 与推理阶段的性能取舍。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/torch_layers.py

```python
def scaled_dot_product_attention(
    q: Tensor,
    k: Tensor,
    v: Tensor,
    attn_mask: Tensor | None = None,
    is_causal: bool = False,
    query_start: int = 0,
    dropout_p: float = 0.0,
    training: bool = False,
    return_weights: bool = False,
) -> Tensor | tuple[Tensor, Tensor]:
    """Explicit, differentiable attention with safe all-masked rows.

    q: [B,H,L,d]; k: [B,H,S,d]; v: [B,H,S,dv]. Batch/head broadcasting and GQA
    are intentionally not implicit. All three inputs share dtype and device.

    A BOOLEAN mask is broadcastable to [B,H,L,S], with True = allowed. For a
    [B,S] key padding mask, explicitly pass mask[:,None,None,:]. Causality means
    key_position <= query_start + local_query_position. Keys start at position 0.
    All-masked rows, and the S=0 case, produce zero pre-projection outputs with
    finite zero gradients for finite inputs/logits. The explicit S=0 convention
    differs from the restricted CUDA/Triton teaching kernels in this package.

    Attention probabilities (if requested) use the accumulator dtype. Returned
    output uses q.dtype. Optional dropout is applied only when training=True.
    """
    for name, tensor in (("q", q), ("k", k), ("v", v)):
        _check_float(tensor, name)
        if tensor.ndim != 4:
            raise ValueError(f"{name} must have four dimensions [B,H,T,d]")
    if q.device != k.device or q.device != v.device:
        raise ValueError("q, k and v must be on the same device")
    if q.dtype != k.dtype or q.dtype != v.dtype:
        raise TypeError("q, k and v must have the same dtype")
    if q.shape[:2] != k.shape[:2] or q.shape[:2] != v.shape[:2]:
        raise ValueError("q, k and v must have matching batch and head dimensions")
    if q.shape[-1] <= 0 or v.shape[-1] <= 0 or q.shape[-1] != k.shape[-1]:
        raise ValueError("q/k head dimensions must match and head dimensions must be positive")
    if k.shape[-2] != v.shape[-2]:
        raise ValueError("k and v sequence lengths must match")
    if not isinstance(query_start, int) or query_start < 0:
```

```
        raise ValueError("query_start must be a nonnegative integer")
    if not is_causal and query_start != 0:
        raise ValueError("query_start is meaningful only with is_causal=True")
    if not 0.0 <= dropout_p < 1.0:
        raise ValueError("dropout_p must lie in [0,1)")
    if attn_mask is not None:
        if attn_mask.dtype != torch.bool:
            raise TypeError("attn_mask must be boolean, with True meaning allowed")
        if attn_mask.device != q.device:
            raise ValueError("attn_mask must be on the same device as q")
    q_acc, k_acc, v_acc = _accumulator(q), _accumulator(k), _accumulator(v)
    # Explicit float32 operands alone are not sufficient inside an outer AMP
    # region: matmul can be autocast back to half/bfloat16. Disable autocast for
    # these reference contractions so our advertised accumulator policy holds.
    with torch.autocast(device_type=q.device.type, enabled=False):
        scores = (q_acc @ k_acc.transpose(-2, -1)) * (q.shape[-1] ** -0.5)
    query_len, key_len = q.shape[-2], k.shape[-2]
    allowed = torch.ones((query_len, key_len), dtype=torch.bool, device=q.device)
    if is_causal:
        query_positions = query_start + torch.arange(query_len, device=q.device)
        key_positions = torch.arange(key_len, device=q.device)
        allowed = key_positions[None, :] <= query_positions[:, None]
    if attn_mask is not None:
        try:
            user_allowed = torch.broadcast_to(attn_mask, scores.shape)
        except RuntimeError as error:
            raise ValueError("attn_mask is not broadcastable to [B,H,L,S]") from error
        allowed = allowed & user_allowed
    if key_len == 0:
        # Matrix products on the empty contraction domain give zero outputs and
        # preserve the gradient connection to all three (possibly empty) inputs.
        with torch.autocast(device_type=q.device.type, enabled=False):
            out = (scores @ v_acc).to(dtype=q.dtype)
        return (out, scores) if return_weights else out
    masked_scores = scores.masked_fill(~allowed, float("-inf"))
    has_allowed_key = allowed.any(dim=-1, keepdim=True)
    # Never run softmax on an all -inf row. Replacing NaNs after softmax is too
    # late: its backward can still contain NaN. Make the input safe beforehand.
    safe_scores = torch.where(has_allowed_key, masked_scores, torch.zeros_like(masked_scores))
    row_max = safe_scores.amax(dim=-1, keepdim=True)
    exponentials = torch.exp(safe_scores - row_max)
    exponentials = torch.where(allowed, exponentials, torch.zeros_like(exponentials))
    # A finite nonempty row has an exponent equal to 1. An empty masked row has
    # denominator 0; clamping to 1 gives the explicitly chosen all-zero result.
    probabilities = exponentials / exponentials.sum(dim=-1, keepdim=True).clamp_min(1.0)
    if training and dropout_p:
        probabilities = F.dropout(probabilities, p=dropout_p, training=True)
    with torch.autocast(device_type=q.device.type, enabled=False):
        out = (probabilities @ v_acc).to(dtype=q.dtype)
    return (out, probabilities) if return_weights else out


class KVCache:
    """Preallocated, contiguous K/V storage for one attention layer and batch.

    This teaching cache has a single shared sequence length for the whole batch.
    It is inference-only, append-only, and not thread-safe. No paging, eviction,
    RoPE, prefix sharing or per-request ragged lengths are hidden inside it.
    Allocate one cache per layer and independent active batch/session.
    """

    def __init__(self, batch_size: int, num_heads: int, max_seq_len: int,
                 head_dim: int, device, dtype: torch.dtype) -> None:
        dimensions = (batch_size, num_heads, max_seq_len, head_dim)
        if any(not isinstance(d, int) for d in dimensions):
            raise TypeError("cache dimensions must be integers")
        if batch_size < 0 or max_seq_len < 0 or num_heads <= 0 or head_dim <= 0:
```

```
            raise ValueError("invalid cache dimensions")
        if dtype not in _FLOAT_DTYPES:
            raise TypeError("cache dtype must be floating point")
        shape = (batch_size, num_heads, max_seq_len, head_dim)
        self.key = torch.empty(shape, device=device, dtype=dtype)
        self.value = torch.empty(shape, device=device, dtype=dtype)
        self.length = 0
        self.batch_size = batch_size
        self.num_heads = num_heads
        self.max_seq_len = max_seq_len
        self.head_dim = head_dim
        self.device = self.key.device
        self.dtype = dtype

    def reset(self) -> None:
        # Old storage is not zeroed; get() and append() expose only the active
        # prefix. External code must not treat unused capacity as valid tokens.
        self.length = 0

    def get(self) -> tuple[Tensor, Tensor]:
        return self.key[:, :, : self.length], self.value[:, :, : self.length]

    def append(self, key: Tensor, value: Tensor) -> tuple[Tensor, Tensor]:
        if torch.is_grad_enabled():
            raise RuntimeError("KVCache is inference-only; use torch.no_grad() or torch.inference_mode()")
        if key.ndim != 4 or value.shape != key.shape:
            raise ValueError("new key/value must have matching [B,H,T,d] shapes")
        if key.shape[:2] != (self.batch_size, self.num_heads) or key.shape[-1] != self.head_dim:
            raise ValueError("new key/value shape does not match this cache")
        if key.device != self.device or value.device != self.device:
            raise ValueError("new key/value device does not match this cache")
        if key.dtype != self.dtype or value.dtype != self.dtype:
            raise TypeError("new key/value dtype does not match this cache")
        end = self.length + key.shape[-2]
        if end > self.max_seq_len:
            raise ValueError(f"cache capacity exceeded: {end} > {self.max_seq_len}")
        # All metadata/capacity checks happen before either mutation.
        self.key[:, :, self.length : end].copy_(key)
        self.value[:, :, self.length : end].copy_(value)
        self.length = end
        return self.get()


class MultiHeadSelfAttention(nn.Module):
    """Packed QKV + manual attention + output projection, batch-first."""

    def __init__(self, embed_dim: int, num_heads: int, bias: bool = True,
                 dropout: float = 0.0, device=None, dtype=None) -> None:
        super().__init__()
        if embed_dim <= 0 or num_heads <= 0 or embed_dim % num_heads:
            raise ValueError("positive embed_dim must be divisible by positive num_heads")
        if not 0.0 <= dropout < 1.0:
            raise ValueError("dropout must lie in [0,1)")
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.dropout = dropout
        self.qkv = ManualLinear(embed_dim, 3 * embed_dim, bias=bias, device=device, dtype=dtype)
        self.out_proj = ManualLinear(embed_dim, embed_dim, bias=bias, device=device, dtype=dtype)

    def _project(self, x: Tensor) -> tuple[Tensor, Tensor, Tensor]:
        if x.ndim != 3 or x.shape[-1] != self.embed_dim:
            raise ValueError(f"x must be [B,T,{self.embed_dim}]")
        return split_qkv_heads(self.qkv(x), self.num_heads)

    def _merge(self, x: Tensor) -> Tensor:
        batch, _, length, _ = x.shape
```

```
        return self.out_proj(x.transpose(1, 2).reshape(batch, length, self.embed_dim))

    def forward(self, x: Tensor, attn_mask: Tensor | None = None,
                is_causal: bool = True, return_weights: bool = False
                ) -> Tensor | tuple[Tensor, Tensor]:
        q, k, v = self._project(x)
        attended, weights = scaled_dot_product_attention(
            q, k, v, attn_mask=attn_mask, is_causal=is_causal,
            dropout_p=self.dropout, training=self.training, return_weights=True,
        )
        out = self._merge(attended)
        # An all-masked row is zero BEFORE out_proj; a nonzero output bias may
        # make this module's final row nonzero. This is intentional and explicit.
        return (out, weights) if return_weights else out

    def forward_step(self, x: Tensor, cache: KVCache) -> Tensor:
        """Append a prefill chunk or new decode token(s), then attend to cache.

        Requires eval() AND a no_grad/inference_mode context. When a chunk starts
        after p cached tokens, query i is at absolute position p+i, not i.
        """
        if self.training or torch.is_grad_enabled():
            raise RuntimeError("forward_step requires model.eval() and torch.no_grad()/inference_mode()")
        if not isinstance(cache, KVCache):
            raise TypeError("cache must be a KVCache")
        if cache.num_heads != self.num_heads or cache.head_dim != self.head_dim:
            raise ValueError("cache head layout does not match this attention module")
        q, new_k, new_v = self._project(x)
        query_start = cache.length
        all_k, all_v = cache.append(new_k, new_v)
        attended = scaled_dot_product_attention(
            q, all_k, all_v, is_causal=True, query_start=query_start,
            dropout_p=0.0, training=False,
        )
        return self._merge(attended)
```

gpu_prep/csrc/kernels.cu

```
__global__ void attention_kernel(const float* q, const float* k, const float* v,
                                 float* out, int64_t query_rows,
                                 int64_t query_len, int64_t key_len,
                                 int64_t head_dim, bool causal,
                                 int64_t query_start) {
  extern __shared__ float scores[];
  __shared__ float scratch[kThreads];
  const float scale = rsqrtf(static_cast<float>(head_dim));
  for (int64_t qr = blockIdx.x; qr < query_rows; qr += gridDim.x) {
    const int64_t bh = qr / query_len;
    const int64_t qi = qr % query_len;
    const int64_t q_base = qr * head_dim;
    const int64_t kv_base = bh * key_len * head_dim;
    float local_max = -CUDART_INF_F;
    for (int64_t key = threadIdx.x; key < key_len; key += blockDim.x) {
      float score = -CUDART_INF_F;
      if (!causal || key <= query_start + qi) {
        float dot = 0.0f;
        for (int64_t d = 0; d < head_dim; ++d)
          dot = fmaf(q[q_base + d], k[kv_base + key * head_dim + d], dot);
        score = dot * scale;
      }
      scores[key] = score;
      local_max = fmaxf(local_max, score);
    }
    const float maximum = block_reduce<true>(local_max, scratch);
```

```
    float local_sum = 0.0f;
    for (int64_t key = threadIdx.x; key < key_len; key += blockDim.x) {
      const float value = expf(scores[key] - maximum);
      scores[key] = value;
      local_sum += value;
    }
    const float inv_sum = 1.0f / block_reduce<false>(local_sum, scratch);

    for (int64_t d = threadIdx.x; d < head_dim; d += blockDim.x) {
      float acc = 0.0f;
      for (int64_t key = 0; key < key_len; ++key)
        acc = fmaf(scores[key] * inv_sum,
                   v[kv_base + key * head_dim + d], acc);
      out[q_base + d] = acc;
    }
    __syncthreads();  // 下一 query 不能覆盖其他线程仍在读取的 scores。
  }
}

at::Tensor attention_cuda(const at::Tensor& q, const at::Tensor& k,
                          const at::Tensor& v, bool causal, int64_t query_start) {
  check_tensor(q, "q");
  check_tensor(k, "k");
  check_tensor(v, "v");
  check_same_device(q, k, "k");
  check_same_device(q, v, "v");
  TORCH_CHECK(q.dim() == 4 && k.dim() == 4 && v.dim() == 4,
              "attention expects q[B,H,L,D], k/v[B,H,S,D]");
  TORCH_CHECK(q.size(0) == k.size(0) && q.size(1) == k.size(1)
                  && q.size(3) == k.size(3) && k.sizes() == v.sizes(),
              "q/k/v batch, head count and head dimension must agree; k/v shapes match");
  TORCH_CHECK(q.size(3) > 0, "head dimension D must be positive");
  TORCH_CHECK(k.size(2) > 0 && k.size(2) <= kMaxAttentionKeys,
              "teaching attention requires 1 <= key length S <= 4096");
  TORCH_CHECK(query_start >= 0, "query_start must be nonnegative");
  TORCH_CHECK(q.size(2) == 0 ||
                  query_start <= std::numeric_limits<int64_t>::max() - (q.size(2) - 1),
              "query_start + query index overflows int64");
  const c10::cuda::CUDAGuard guard(q.device());
  auto out = at::empty(q.sizes(), q.options().requires_grad(false));
  if (q.numel() == 0) return out;
  const int64_t query_rows = q.numel() / q.size(3);
  const auto stream = c10::cuda::getCurrentCUDAStream(q.get_device()).stream();
  const size_t shared_bytes = static_cast<size_t>(k.size(2)) * sizeof(float);
  attention_kernel<<<blocks_for(query_rows), kThreads, shared_bytes, stream>>>(
      q.data_ptr<float>(), k.data_ptr<float>(), v.data_ptr<float>(),
      out.data_ptr<float>(), query_rows, q.size(2), k.size(2), q.size(3),
      causal, query_start);
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}
```

gpu_prep/triton_ops.py

```
@triton.jit
def _attention_kernel(Q, K, V, Mask, O, L: tl.constexpr, S: tl.constexpr,
                      D: tl.constexpr, SCALE: tl.constexpr,
                      CAUSAL: tl.constexpr, QUERY_START: tl.constexpr,
                      HAS_MASK: tl.constexpr, MASK_BH_STRIDE: tl.constexpr,
                      BM: tl.constexpr, BN: tl.constexpr):
    pid = tl.program_id(0)
    q_blocks = tl.cdiv(L, BM)
    q_block = pid % q_blocks
    bh = pid // q_blocks
    r = q_block * BM + tl.arange(0, BM)
```

```
    d = tl.arange(0, D)
    q = tl.load(Q + bh * L * D + r[:, None] * D + d[None, :],
                r[:, None] < L, other=0)

    # Three online-softmax states for each query: maximum, denominator, numerator.
    running_max = tl.full((BM,), -float("inf"), tl.float32)
    denominator = tl.full((BM,), 0.0, tl.float32)
    numerator = tl.full((BM, D), 0.0, tl.float32)

    for tile in range(0, tl.cdiv(S, BN)):
        c = tile * BN + tl.arange(0, BN)
        k = tl.load(K + bh * S * D + c[None, :] * D + d[:, None],
                    c[None, :] < S, other=0)
        v = tl.load(V + bh * S * D + c[:, None] * D + d[None, :],
                    c[:, None] < S, other=0).to(tl.float32)
        scores = tl.dot(q, k, input_precision="ieee") * SCALE
        visible = (r[:, None] < L) & (c[None, :] < S)
        if CAUSAL:
            visible = visible & (c[None, :] <= QUERY_START + r[:, None])
        if HAS_MASK:
            keep = tl.load(Mask + bh * MASK_BH_STRIDE + r[:, None] * S + c[None, :],
                           (r[:, None] < L) & (c[None, :] < S), other=0)
            visible = visible & keep
        scores = tl.where(visible, scores, -float("inf"))
        new_max = tl.maximum(running_max, tl.max(scores, axis=1))

        # Before any key has been visible, new_max is -inf. Replace it only in
        # exponent calculations so we never evaluate -inf - (-inf).
        safe_max = tl.where(new_max == -float("inf"), 0.0, new_max)
        old_scale = tl.exp(running_max - safe_max)
        probabilities = tl.exp(scores - safe_max[:, None])
        denominator = denominator * old_scale + tl.sum(probabilities, axis=1)
        numerator = numerator * old_scale[:, None]
        # Keep the probabilities in FP32. This deliberately sacrifices some
        # Tensor Core throughput to avoid a hidden FP16/BF16 probability cast.
        numerator = tl.dot(probabilities, v, numerator, input_precision="ieee")
        running_max = new_max

    safe_denominator = tl.where(denominator > 0.0, denominator, 1.0)
    output = numerator / safe_denominator[:, None]
    tl.store(O + bh * L * D + r[:, None] * D + d[None, :], output, r[:, None] < L)


def attention(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor,
              causal: bool = False, query_start: int = 0, *,
              attn_mask: torch.Tensor | None = None) -> torch.Tensor:
    """Tiled online-softmax attention forward, without materializing [L,S].

    q: [B,H,L,D], k/v: [B,H,S,D], S>0, D in {16,32,64,128}.
    Keys have positions 0..S-1. For causal=True, query i has absolute position
    query_start+i and sees key j exactly when j <= query_start+i.
    query_start is explicit: decode with a length-S cache usually uses S-1.

    Optional bool attn_mask is [L,S] shared across B/H or exactly [B,H,L,S].
    True means visible; it is ANDed with the causal mask. All-masked rows return
    zeros, by this exercise's explicit convention. No additive mask, dropout,
    GQA, custom scale, backward, or arbitrary-stride inputs are implemented.
    """
    _check_tensor(q, "q", ndim=4)
    _check_peer(q, k, "k", ndim=4)
    _check_peer(q, v, "v", ndim=4)
    b, h, length, d = q.shape
    if k.shape != v.shape or k.shape[:2] != (b, h) or k.shape[-1] != d:
        raise ValueError("expected q[B,H,L,D] and matching k/v[B,H,S,D]")
    s = k.shape[2]
    if s <= 0:
        raise ValueError("attention key/value length S must be positive")
```

```
    if d not in ATTENTION_HEAD_DIMS:
        raise ValueError(f"head dimension D must be one of {ATTENTION_HEAD_DIMS}")
    if not isinstance(causal, bool):
        raise TypeError("causal must be bool")
    if isinstance(query_start, bool) or not isinstance(query_start, int):
        raise TypeError("query_start must be a nonnegative Python int")
    if query_start < 0 or query_start + length > MAX_TENSOR_ELEMENTS:
        raise ValueError("query positions must be nonnegative and fit i32")

    mask_stride = 0
    if attn_mask is not None:
        if not isinstance(attn_mask, torch.Tensor):
            raise TypeError("attn_mask must be a bool torch.Tensor")
        if attn_mask.device != q.device or attn_mask.dtype != torch.bool:
            raise ValueError("attn_mask must be bool and on q.device")
        if not attn_mask.is_contiguous():
            raise ValueError("attn_mask must be contiguous")
        if attn_mask.numel() > MAX_TENSOR_ELEMENTS:
            raise ValueError("attn_mask exceeds this exercise's i32 indexing limit")
        if attn_mask.shape == (length, s):
            mask_stride = 0
        elif attn_mask.shape == (b, h, length, s):
            mask_stride = length * s
        else:
            raise ValueError("attn_mask must have shape [L,S] or exactly [B,H,L,S]")

    out = torch.empty_like(q)
    if q.numel() == 0:
        return out
    mask_pointer = q if attn_mask is None else attn_mask
    with torch.cuda.device(q.device):
        _attention_kernel[(triton.cdiv(length, 16) * b * h,)](
            q, k, v, mask_pointer, out, length, s, d, 1.0 / math.sqrt(d),
            causal, query_start, attn_mask is not None, mask_stride,
            BM=16, BN=32, num_warps=4, num_stages=1,
        )
    return out
```

第 10 题｜torch.compile：Graph Break、Guard 与稳态性能

完整源文件： examples/compile_demo.py。

第 10 题：解释 graph break，修复数据依赖分支，并公平评估编译

原始题目

分析这段函数，说明调用 `torch.compile` 后可能发生什么：

```
def bad_python_branch(x):
    if x.sum().item() > 0:
        return x.sin()
    return x.cos()
```

然后实现能处理正、负输入的可捕获版本；分别测量首次调用和稳态；解释一次 `fullgraph=True` 成功能证明什么。

本包的 CPU 检查命令是：

```
python -m examples.compile_demo --device cpu --backend eager --include-cond
```

GPU 上实际使用 Inductor 的实验命令是：

```
python -m examples.compile_demo --device cuda --backend inductor --include-cond
```

后者需要匹配的 PyTorch/CUDA/Triton 与可用 GPU；如果也选 CUDA C++ custom op，还需要相应 extension 编译工具链。当前生成环境完成的是 CPU capture/数学检查，GPU 速度需在你的机器上实际测量。

把编译链路说到足够准确

`torch.compile` 是用户入口。TorchDynamo 分析 Python 执行，抽取可捕获的 tensor 运算为 FX 图，并生成保证图适用的条件检查，也就是 guards。后端收到图之后再决定如何实现计算。

默认后端 Inductor 进行图级转换、调度和代码生成；在 GPU 路径中会利用 Triton 或已有库调用等实现。训练场景还涉及 AOTAutograd 对前向、反向及其保存状态的处理。只做推理、没有 backward 的实验，不应叙述成“实际跑过了训练反向编译”。官方 torch.compiler 介绍与 编译 FAQ区分了这些部分。

`backend="eager"` 则让捕获后的图仍以 eager 运算执行。它特别适合先分离问题：“Dynamo 是否能表示这段代码？”如果能，再检查后端代码生成和数值。它的计时不能拿来证明 Inductor 优化，也不能根据 CPU 上通过了测试就声称产生了正确的 GPU kernel。

原始函数为什么容易 graph break?

`x.sum()` 是 tensor 计算；`.item()` 把结果变成 Python 标量；`if` 又用这个数据值决定走哪条 Python 分支。该决定取决于本次输入内容，不能简单把第一组样本选中的分支永久固定。

默认允许 graph break 时，编译器可能捕获前一段运算、回到 Python 执行分支，再尝试捕获后续片段。使用 `fullgraph=True` 时，这类无法捕获的部分通常会直接暴露为错误，便于诊断。更改 scalar capture 配置有时可以把 `.item()` 本身纳入图，但不等于自动解决所有数据依赖 Python 控制流。Common Graph Breaks

如果 x 在 GPU 上，读取其值以驱动 CPU Python 逻辑还可能要求等待 GPU 结果。这与编译问题相关，但概念上不同：一个是图能否捕获，一个是 host/device 执行依赖是否造成同步。

不要把所有 `if` 都删掉。按 Tensor shape 或固定配置分支，可能通过符号形状或 guard 特化处理；按实际 tensor 数据分支，则需要合适的图内控制流或算法改写。应分析分支条件依赖什么。

修复方案一：`torch.where`

```
def tensor_where(x):
    return torch.where(x.sum() > 0, x.sin(), x.cos())
```

条件保留为标量布尔 Tensor，并广播到输出形状。对这个例子，正输入选择 sin，非正输入选择 cos，与原函数语义一致。测试必须覆盖两个方向；只用全正输入通过，不足以说明对另一个分支也正确。

重要细节：Python 会先求值 `x.sin()` 和 `x.cos()`，再把两份结果交给 `where`。所以它是选择结果，不是“必然只执行被选中的表达式”。如果某个分支包含非常昂贵的计算、不可定义的数值操作或副作用，不能不分析就机械改成 `where`。

例如写 `torch.where(x != 0, 1/x, 0)` 时，不能据此认为 `1/x` 在零位置从未被求值；涉及梯度和异常数值时仍需单独验证。

修复方案二：`torch.cond`

```
def tensor_cond(x):
    return torch.cond(
        x.sum() > 0,
        lambda t: t.sin(),
        lambda t: t.cos(),
        (x,),
    )
```

这是把条件执行明确表示为可捕获的结构化操作。分支要遵循该 API 的限制，例如接收约定一致的 operands，并返回匹配 shape/dtype 等元数据的结果；不能任意修改输入和全局状态。具体支持范围随 PyTorch 版本演进。本包的简单 sin/cos 例子已在 CPU PyTorch 2.5.1 的 capture 路径检查；不借此推断复杂分支或所有训练情况都支持。官方 Control Flow - Cond

回答方案选择时可以说：“计算轻、两边都定义良好时，where 简单；需要保留真实条件计算时，优先考虑 cond，并验证目标版本的限制。若必须执行外部 Python 逻辑，也可以明确接受局部 eager 区域，判断它是否处在性能关键路径。”

graph break、重新编译、CUDA Graph 分别是什么?

概念	发生的层次	一个例子	应检查什么
Graph break	捕获 Python/tensor 程序时分段	数据值驱动的 Python if	break 原因、片段数量、是否处于热点循环
Recompilation	已有图的适用条件不满足	shape、stride、dtype、配置等变化触发 guard 失效	guard 日志、输入分布、缓存变体数量
CUDA Graph capture/replay	GPU 工作提交和依赖重放	重复运行较稳定的 GPU 工作序列	地址/生命周期、捕获限制、额外内存、实际 host launch 开销

一个程序可以没有 graph break，却因为输入 shape 不断变化而反复编译。另一个程序也可能有几个可接受的图段，但稳态执行仍有明显加速。`torch.compile` 与 CUDA Graph 不是同一个开关；某些编译模式可能配合使用它们，但应分别解释它们解决的问题。

`fullgraph=True` 保证的是这次路径可被捕获为单个图，或明确报错。它不保证输出只由一个 GPU kernel 计算，也不保证无重新编译，更不保证性能必然提高。fullgraph 官方说明

实际排查顺序

先运行 eager，确认错误不是普通 shape/device/算法错误。然后缩小为可复现函数，用 `fullgraph=True` 暴露第一个 break。必要时打开日志：

```
TORCH_LOGS="graph_breaks,recompiles" python -m examples.compile_demo --device cpu --backend eager
```

解决一个 break 后再次检查；但不要把“零 break”作为所有任务的唯一优化目标。如果剩余 Python 区域不在热点，继续改写可能增加复杂度而没有收益。进入后端问题时，分别判断是编译失败、数值差异还是实际变慢，三者需要不同证据。

首次调用与稳态：为什么不能混成一个数字？

创建 `compiled=torch.compile(fn)` 通常只是建立包装，许多工作会在第一次实际调用时发生，包括图捕获、编译、autotuning、缓存初始化等。第一次执行有较大开销，并不直接代表之后每次运行的开销。

`compile_demo.py` 单独报告首次 compiled 调用时间，再 warm-up，最后报告多次稳态 wall time 中位数。在 CUDA 情况下计时前后同步，避免只测到 CPU 发出命令就结束；这是一种端到端调用计时，包含相应 host 和同步开销。专门测 kernel 时间时，还应使用 CUDA Events，并清楚说明是否包含分配和传输。

这里的“首次调用”不自动等同于完全冷启动：运行时可能已有 GPU context，磁盘可能已有编译缓存。若要报告服务冷启动，应明确是否包含进程启动、模型加载、context、编译缓存和首个请求。所有这些状态都会改变测量含义。

比较时固定：同一模型、输入、dtype、训练/推理模式、数值精度选项、shape 分布、计时范围与硬件负载。不能让基线包含数据搬运、编译和分配，而优化版本只测已驻留的 kernel；也不能在 warm-up 后只测一个特别适合优化版本的 shape。

“编译值得吗”还可以算一个回本点

设额外编译成本为 C，eager 单次延迟为 `t_e`，compiled 稳态为 `t_c`，且 `t_e>t_c`。需要大约：

$$N > \frac{C}{t_e - t_c}$$

次调用才能在累计时间上抵消编译开销。这个式子假设调用形状能够复用同一编译结果；频繁重新编译或在线请求对首个响应极敏感时，结论会改变。回答时要把工作负载和服务目标放进去，而不只是宣称“steady state 更快”。

面试追问：可直接复述的回答

问：`torch.compile` 的优化主要来自哪里？

答：“一部分来自看到多步运算后做融合、减少中间读写与调度；另一部分来自对具体形状、布局和后端实现进行选择或专门化。收益取决于工作负载。如果已经是大型优化 GEMM 主导，或被 Python 数据依赖频繁打断，改善可能有限。”

问：为什么会重新编译？我不是已经 compile 过了吗？

答：“编译结果有适用条件。新输入的形状、布局、类型或相关 Python 状态不满足 guard 时，已有版本不能直接复用。我要先看触发的是哪些条件，再决定使用动态形状、按常见长度分桶、移动可变 Python 状态，还是接受少量专门化版本。”

问：用了 dynamic shapes 就不会重编译了吗？

答：“不能这样保证。动态形状扩大了一些维度的适用范围，但其他 guard、结构变化和不支持的路径仍可能触发重新编译。过度动态化还可能损失专门化优化。应针对实际请求分布做选择。”

问：CPU `backend='eager'` 通过了，下一步是什么？

答：“它说明这个输入范围内的 Dynamo 捕获和参考数学能成立。接下来检查实际目标后端的编译、前向与反向数值，再测目标 GPU 的 latency、吞吐和显存。不能把前端验证当作 GPU 性能结果。”

问：看到第一次很慢，是不是直接放弃？

答：“先拆分首次调用和稳态，检查是否有重复编译。离线重复训练和长时间运行服务可能摊薄一次性成本；短生命周期任务或首请求敏感服务则要把启动成本计入决策。是否预热或预先编译，要基于真实使用方式。”

本题的完成标准

能解释给定函数为何难以捕获，写出 where 和 cond 两种等价例子并说明取舍；清楚区分 graph break、guard 重编译、CUDA Graph；知道 eager backend 验证范围；能够拿出首次调用与稳态分开、同条件比较的测量方案。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

examples/compile_demo.py

```
def bad_python_branch(x: torch.Tensor) -> torch.Tensor:
    # .item() transfers a data value to Python; the if depends on that value.
    if x.sum().item() > 0:
        return x.sin()
    return x.cos()


def tensor_where(x: torch.Tensor) -> torch.Tensor:
    # Semantically simple and easy to capture. Both sin/cos expressions are
    # evaluated before where selects their results; it is not lazy branching.
    return torch.where(x.sum() > 0, x.sin(), x.cos())


def tensor_cond(x: torch.Tensor) -> torch.Tensor:
    # Structured control flow: branch outputs need matching metadata and the
    # branches must meet torch.cond's capture restrictions.
    return torch.cond(x.sum() > 0, lambda t: t.sin(), lambda t: t.cos(), (x,))


def median_wall_ms(fn, fn_args, device: torch.device, warmup: int, repeats: int) -> float:
    for _ in range(warmup):
        fn(*fn_args)
    _sync(device)
    times = []
    for _ in range(repeats):
        _sync(device)
        start = time.perf_counter()
        fn(*fn_args)
        _sync(device)
        times.append((time.perf_counter() - start) * 1000)
    return statistics.median(times)


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--device", default="cpu")
    parser.add_argument("--backend", choices=("eager", "inductor"), default="eager")
    parser.add_argument("--repeats", type=int, default=20)
    parser.add_argument("--warmup", type=int, default=5)
    parser.add_argument("--include-cond", action="store_true",
                        help="also test structured torch.cond (availability/limits vary by PyTorch release)")
    parser.add_argument("--operator-backend", choices=("cuda", "triton", "both"), default="both",
                        help="custom-op GPU implementation; both use CPU references when --device cpu")
    args = parser.parse_args()
    if args.repeats <= 0 or args.warmup < 0:
        raise SystemExit("repeats must be positive and warmup nonnegative")
```

```
    device = torch.device(args.device)
    if device.type == "cuda" and not torch.cuda.is_available():
        raise SystemExit("CUDA is not available; use --device cpu for capture-only checks")
    print("PyTorch:", torch.__version__, "device:", device, "backend:", args.backend)
    if args.backend == "eager":
        print("Scope: Dynamo capture/result checks. This does NOT benchmark Inductor or prove GPU speedup.")
    torch.manual_seed(2027)
    positive = torch.ones(8192, device=device)
    negative = -positive
    with torch.no_grad():
        try:
            broken = torch.compile(bad_python_branch, backend=args.backend, fullgraph=True)
            broken(positive)
        except Exception as error:
            print("Expected fullgraph failure for data-dependent Python branch:", type(error).__name__)
            print(str(error).splitlines()[0][:250])
        else:
            print("This PyTorch/configuration captured the example; inspect its graph and guards rather than
assuming a break.")

        functions = [("where", tensor_where)]
        if args.include_cond:
            functions.append(("cond", tensor_cond))
        for name, fn in functions:
            compiled = torch.compile(fn, backend=args.backend, fullgraph=True)
            _sync(device)
            start = time.perf_counter()
            first = compiled(positive)
            _sync(device)
            first_call_ms = (time.perf_counter() - start) * 1000
            torch.testing.assert_close(first, bad_python_branch(positive))
            torch.testing.assert_close(compiled(negative), bad_python_branch(negative))
            eager_ms = median_wall_ms(fn, (positive,), device, args.warmup, args.repeats)
            compiled_ms = median_wall_ms(compiled, (positive,), device, args.warmup, args.repeats)
            print(f"{name}: first compiled call including capture/codegen/initialization = {first_call_ms:.3f}
ms")
            print(f"{name}: steady median wall time eager={eager_ms:.4f} ms, compiled={compiled_ms:.4f} ms")

        x = torch.randn(17, 33, device=device)
        bias = torch.randn(33, device=device)
        operators = []
        if args.operator_backend in ("cuda", "both"):
            operators.append(("CUDA wrapper", bias_relu))
        if args.operator_backend in ("triton", "both"):
            operators.append(("Triton wrapper", bias_relu_triton))
        for name, operation in operators:
            # First GPU use lazily builds/JITs the teaching implementation. Do it
            # before capture; compilation is not steady-state kernel latency.
            torch.testing.assert_close(operation(x, bias), torch.relu(x + bias))
            op_graph = torch.compile(lambda a, b: operation(a, b).square().sum(),
                                     backend=args.backend, fullgraph=True)
            torch.testing.assert_close(op_graph(x, bias), torch.relu(x + bias).square().sum())
            print(name + ": fullgraph capture and numerical agreement passed for this device/backend.")

    # Keep the registration check independent of the timing region.
    cpu_x = torch.tensor([[-2.0, 2.0, -3.0], [1.0, -4.0, 3.0]], dtype=torch.float64, requires_grad=True)
    cpu_bias = torch.tensor([0.1, 0.2, 0.3], dtype=torch.float64, requires_grad=True)
    for name, operation in operators:
        print(name, "opcheck (registration):", torch.library.opcheck(operation, (cpu_x, cpu_bias)))
        print(name, "gradcheck (mathematical derivative):", torch.autograd.gradcheck(operation, (cpu_x,
cpu_bias)))
```

第 11 题｜自定义 CUDA / Triton 算子：Fake、Autograd 与编译

完整源文件： gpu_prep/custom_ops.py。

第 11 题：把 CUDA 与 Triton bias+ReLU 接入 PyTorch 编译和训练

题目与交付的两条路径

本题固定一个易理解、容易验证的函数：

$$y_{m,n} = \max(x_{m,n} + b_n, 0).$$

在 `gpu_prep/custom_ops.py` 中注册了两个独立名字：

Python 入口	Dispatcher 名字	CUDA 张量的实现	CPU 张量的实现
`bias_relu(x,bias)`	`gpu_prep::bias_relu`	`gpu_prep.cuda_ops.bias_relu`	`torch.relu(x+bias)`
`bias_relu_triton(x,bias)`	`gpu_prep::bias_relu_triton`	`gpu_prep.triton_ops.bias_relu`	`torch.relu(x+bias)`

它们共用同一 metadata 检查、fake 规则与 backward，但调用不同 GPU 实现。GPU 分支都惰性导入；仅导入 `gpu_prep.custom_ops` 不会编译 C++/CUDA，也不要求 CPU 用户先安装一个可运行的 GPU 编译环境。

先明确接口合同

这里支持连续的 `x[M,N]` 和连续的 `bias[N]`；同 dtype、同 device；返回新分配的连续 `y[M,N]`，不修改输入，也不与输入共享输出存储。M 或 N 为零都允许，结果自然为空。

CPU 参考支持 FP16、BF16、FP32、FP64；本练习 GPU 接入层明确限制 FP32，以对齐 CUDA 基础实现。FP64 CPU 路径的一个用途是做数值梯度检查，不是用来代替 GPU 测试。数学 backward 的验证使用有限输入，在 ReLU 的零点采用导数为 0 的约定。

如果想接受非连续输入，有两种清楚的扩展：修改 kernel 按 stride 寻址；或在 wrapper 中先 `.contiguous()`，把复制成本纳入接口和性能测量。本实现选择主动报错，避免底层按连续地址访问而默默算错。

`mutates_args=()` 是对 dispatcher 的正式承诺：输入不会被原地修改。它不只是装饰器的格式要求。编译器、functionalization 和 autograd 会依赖这个信息推理程序行为。Custom Python Operators介绍了 schema、aliasing 与 mutation 合同。

为什么“一个 Python 函数能调用 CUDA”还不够?

普通 Python wrapper 能在 eager 中启动 kernel，只解决了“怎么执行”这一层。框架还需要知道：输出什么 shape/dtype、是否修改输入、没有真实数据时怎么推理、反向怎么计算、编译图中用什么算子节点表示。

`torch.library.custom_op` 给这个计算一个稳定的 dispatcher 身份。在编译器看来，它可以是一个已知输入输出合同的操作，而不是必须继续分析每一行 Python、JIT build 逻辑和底层外部调用。CUDA/C++ 接入的总体组件也可以在 PyTorch 官方 C++/CUDA custom op 教程中核对。

Fake kernel：不算答案，只给出正确的输出元数据

代码注册的 fake 实现只做 metadata 检查，并返回：

```
torch.empty(x.shape, dtype=x.dtype, device=x.device)
```

在 FakeTensor 模式下，这代表一个形状、类型、设备和布局正确的符号输出，不是申请真实 GPU 数据来执行融合运算。fake 函数不能用 `.item()` 读取数据，不能把输入拷到 CPU 检查元素，不能调用实际 CUDA/Triton kernel。

为什么不随便 `empty_like(x)`？在这个合同里输入已要求连续，因此通常也能匹配；但更一般的 custom op 可能总是返回连续输出，而 `empty_like` 可能沿用某种输入布局。fake 必须与真实输出的 stride、aliasing 等行为一致，而不只是 shape 看起来相同。本代码直接声明一个新连续张量，使合同更直观。Functional Custom Operators

训练接入：forward-only kernel 怎样有 backward?

底层 CUDA/Triton 练习明确只实现 forward。如果直接拿它们处理需要梯度的输入，会主动拒绝，避免静默丢梯度。custom op 的 GPU 分支在 `torch.no_grad()` 下调用 forward kernel，然后由注册的 autograd 公式负责整个操作的求导。

这不是把训练关掉：`no_grad` 只包住不打算让 PyTorch 逐行追踪的底层执行；外层 dispatcher 仍根据 `register_autograd` 提供的规则建立梯度关系。

设 `z=x+b`，上游梯度为 G。对有限输入并约定 ReLU 在 0 的导数为 0：

$$dX_{m,n} = G_{m,n}\,\mathbb{1}[z_{m,n} > 0],$$

$$dB_n = \sum_{m=0}^{M-1} dX_{m,n}.$$

代码的 setup_context 保存 x 和 bias；backward 用可被 PyTorch 理解的 tensor 操作重算符号 mask，再分别返回 x 与 bias 的梯度。bias 只有一维，但 forward 把它广播到了每一行，反向必须把所有行的贡献相加。M=0 时该和应为零，不能留下未初始化的梯度。

上面的数学公式用指示函数相乘表达导数；实际浮点代码使用 `torch.where(active, grad_output, torch.zeros_like(grad_output))`。原因是无效位置的上游梯度即使为 Inf 或 NaN，也应该像 PyTorch ReLU 一样返回零，而 `0*Inf` 或 `0*NaN` 会生成 NaN。针对 `x=[[-1,0,1]]`、零 bias、上游梯度 `[[Inf,NaN,1]]`，两条 custom op 都有回归测试，确认 x/bias 梯度为 `[0,0,1]`。

另一种实现可以保存输出或激活 mask，避免反向再做 x+b。选择时比较保存状态的显存、重算成本、mask 表示和实现复杂度。这里先用容易读懂的版本。官方 Adding Training and Other Registrations提供了 `setup_context` 与 `register_autograd` 的接口结构。

`opcheck` 与 `gradcheck` 分别证明什么?

检查	本包做了什么	能证明什么	不能代替什么
数值前向比较	与 `torch.relu(x+bias)` 比较	给定输入的数学结果和输出 metadata 一致	未覆盖输入范围、GPU 性能
backward 参考比较	比较 x/bias 的梯度	给定输入和上游梯度的求导结果一致	一般注册合同
`torch.library.opcheck`	检查 schema、autograd 注册、FakeTensor、AOT dispatch 等	框架注册使用符合相关规则	梯度公式的数学正确性
`torch.autograd.gradcheck`	在小 FP64 输入上做有限差分	被检查输入附近的解析梯度与数值梯度一致	GPU dtype/stream/内存错误
`torch.compile(fullgraph=True,backend='eager')`	捕获调用 custom op 的图并比较结果/梯度	该路径可被 Dynamo 捕获并正确执行	Inductor codegen、GPU 运行或加速

两个注册入口都跑了这些 CPU 检查。选择数值梯度输入时，刻意让 `x+bias` 远离 0：ReLU 在 0 不可导，中心差分与我们指定的导数 0 不一致，可能产生假失败。零点的行为通过另外一条离散断言检查，而不强迫有限差分在不可导点通过。

不能这样回答：“opcheck 全通过，所以我的 backward 肯定正确。”正确回答是：“opcheck 验证接入规则；我另外用 gradcheck 和参考梯度比较验证公式。”

CUDA 接入为什么必须处理 device 与 current stream?

Python wrapper 已确认两个输入在同一设备，但底层 launch 还必须切到输入设备对应的上下文，并使用 PyTorch 当前 stream。否则在用户切换设备或采用非默认 stream 时，kernel 可能在错误设备访问指针，或忽略与前后操作的执行依赖。

面试时可以回答：“我不会把 stream 0 写死。输入 tensor 的 device 决定设备 guard，PyTorch 当前 stream 决定 launch stream。kernel 保持异步；调试或 benchmark 的同步放在调用方合适的位置，而不是每个 forward 内无条件 `cudaDeviceSynchronize()`。”

跨 stream 使用资源还要考虑生产者/消费者依赖与内存生命周期。一个算子在默认 stream 的随机输入测试通过，不能证明并发情况下正确。PyTorch 的 CUDA semantics说明了 stream 顺序、跨 stream 同步和内存使用注意事项。

Triton 的 kernel launch 也应遵守当前 PyTorch CUDA 执行环境；自建 stream 并不会自动变快。两个路径都需要在 GPU 上用独立的非默认 stream 测试和实际 profiler 验证，CPU 注册测试覆盖不到这里。

custom op 与编译融合：边界本身也有成本

`custom_op` 在本基线中是一个对编译器不透明的调用。fake 让编译器知道结果的元数据，autograd 注册让它知道反向规则，但它并不会因此自动看见 CUDA/Triton kernel 内部每个算子。

因此有两种不同层次的融合：kernel 内部的 `x+bias+relu` 已经融合；它与外部残差加法、norm 或其他操作是否还能融合，是另一回事。如果把本来可被编译器完整看到的一小段纯 PyTorch 运算强行包成 opaque custom op，反而可能减少优化机会。

本练习为了练接入、调用外部 CUDA/Triton 实现，明确使用 custom op。真实项目若只是 `torch.relu(x+bias)`，通常先让编译器处理原始组合更简单。是否需要 custom op，应由外部 kernel、硬件能力或明确接口需求驱动。

Triton 的另一种较新接入方式

PyTorch 2.6 增加了 `torch.library.triton_op`。与本包使用的 opaque `custom_op` 相比，它允许编译器进入 Triton-backed 实现，并通过 `torch.library.wrap_triton` 表达 kernel 调用，同时继续提供框架注册能力。官方 Using User-Defined Triton Kernels with torch.compile列出了两种接入方式的差异。

本包以实际验证过的 PyTorch 2.5.1 为基线，所以可运行代码选择 `custom_op`，没有直接导入较新的 API。升级时可以单独试验 `triton_op` 路径，并重新检查 CPU fallback、fake/autograd 行为及目标版本限制。不能把 `triton_op` 文档中的可见性优点自动归给本包的 2.5 custom-op wrapper。

一个面试回答是：“如果只需要一个 Triton 函数在已支持的 compile 场景运行，直接集成可能更简单；如果还需要统一 dispatcher 身份、CPU fallback 和其他子系统行为，再选择适当的注册 API。基于版本和需求决定，不会只因为名字更新就替换。”

怎样在自己的 GPU 上进一步验收?

先运行底层 CUDA/Triton 的数值测试，确认 launch、尾部和 NaN 等约定。再运行本题 custom op 的 GPU 测试，使外层注册、底层实现和 backward 连起来。最后做实际 Inductor 的 forward/backward 数值和性能实验。

可以分别选择一个实现，避免把 CUDA extension 的首次编译与 Triton 首次 JIT 混在同一个性能比较中：

```
python -m examples.compile_demo --device cuda --backend inductor --operator-backend cuda
python -m examples.compile_demo --device cuda --backend inductor --operator-backend triton
```

脚本在 capture 实验之前先调用一次底层路径，触发该路径所需的初始化。首次构建 extension、Triton JIT、Dynamo/Inductor 编译和 kernel 稳态时间仍是不同指标。如果要比较纯 kernel 延迟，应使用包中专门的 benchmark，并统一是否包括 wrapper、分配与同步。

面试追问：可直接复述的回答

问：为什么 fake kernel 不能直接算一次真实结果?

答：“编译阶段可能只有符号形状和 fake tensor，没有实际数据。fake 的职责是描述输出元数据。读取元素会把它变成数据依赖，还可能启动不应该发生的 GPU 工作。我只用输入的 shape、dtype、device、stride 和 schema 来描述输出。”

问：为什么你不直接返回输入 x，当作 ReLU 的输出?

答：“这违反了我声明的 fresh functional 输出合同，也改变了 aliasing。如果要设计原地算子，就应明确声明修改哪个参数，并解决 autograd 和 functionalization 的对应规则。不能在 `mutates_args=()` 下偷偷原地写入。”

问：为什么 CPU 测试使用 double，CUDA 用 float?

答：“double 的 CPU fallback 用来检查数学梯度；基础 CUDA kernel 只实现了 float32，因此接口明确限制。我要分别证明注册和公式正确、GPU实现正确，再根据需要扩展 dtype。用 double CPU gradcheck 并不能证明 half/float 的 GPU 数值表现。”

问：注册了 autograd，就支持所有训练方式了吗?

答：“它支持我们给出的 VJP 和被测试的训练路径。AMP、vmap、forward-mode、二阶求导、export 或其他后端如果是需求，要分别确认和注册。这里 backward 由 PyTorch 表达式组成便于组合，但我不会据此默认所有子系统都已验证。”

问：bias 梯度为什么是 sum，不是 mean?

答：“链式法则要求把每个使用点对同一个参数的贡献累加。loss 是否取 mean 已经反映在上游梯度 G 中；在 backward 里再无条件取 mean 会重复缩放，产生错误。”

问：自定义 kernel 更快，整体为什么可能没更快?

答：“要检查它是否减少了实际热点时间，以及是否引入 contiguous 拷贝、额外分配、同步或打断周围融合。custom op 边界是实现能力，也可能带来优化限制。我会同时测底层 kernel、算子调用和实际模型路径。”

问：如果面试只让我接一个算子，最少需要交代什么?

答：“先说明 shape/device/dtype/stride 和是否修改输入；给出底层调用与正确 stream；补 fake 元数据；如果要求训练，再给 backward 和广播归约；最后分别用前向比较、梯度检查和 opcheck 验证。每项的验证范围说清楚。”

本题的完成标准

能够解释并运行两条注册入口；知道导入阶段与首次执行的区别；理解 fake 的 metadata 合同；推导 bias+ReLU 的 backward；区分 opcheck 和 gradcheck；解释 current stream、opaque boundary 和版本相关 API；对没有 GPU 的环境只报告实际完成的 CPU 验证。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/custom_ops.py

```
"""Exercise 11: one operator integrated with fake tensors and autograd.

Import registers metadata only: it does NOT import/compile the CUDA extension.
The CPU implementation makes registration and gradient checks possible on a
CPU-only machine. It is a reference path, not a GPU performance substitute.
```

```
"""

from __future__ import annotations

import torch
from torch import Tensor


def _validate(x: Tensor, bias: Tensor) -> None:
    if x.ndim != 2 or bias.ndim != 1:
        raise ValueError("bias_relu expects x[M,N] and bias[N]")
    if x.shape[1] != bias.shape[0]:
        raise ValueError("bias length must equal x.shape[1]")
    if x.device != bias.device:
        raise ValueError("x and bias must be on the same device")
    if x.dtype != bias.dtype:
        raise TypeError("x and bias must have the same dtype")
    if not x.is_contiguous() or not bias.is_contiguous():
        raise ValueError("this teaching operator requires contiguous inputs")
    if x.dtype not in (torch.float16, torch.bfloat16, torch.float32, torch.float64):
        raise TypeError("bias_relu supports floating tensors only")
    if x.device.type not in ("cpu", "cuda"):
        raise ValueError("bias_relu supports CPU and CUDA only")
    if x.device.type == "cuda" and x.dtype != torch.float32:
        raise TypeError("the CUDA teaching implementation supports float32 only")


@torch.library.custom_op("gpu_prep::bias_relu", mutates_args=())
def bias_relu(x: Tensor, bias: Tensor) -> Tensor:
    """Fresh contiguous y = relu(x + bias), without input mutation.

    CPU: fp16/bf16/fp32/fp64 reference. CUDA: fp32 custom kernel.
    Empty M or N is supported. Backward's derivative at exactly zero is zero.
    The backward contract and numerical tests use finite inputs.
    """
    _validate(x, bias)
    if x.device.type == "cuda":
        # Lazy import: CPU users do not need nvcc or the extension at import time.
        from gpu_prep.cuda_ops import bias_relu as cuda_bias_relu

        # The standalone CUDA exercise is explicitly forward-only; autograd is
        # supplied by this custom operator's registration below.
        with torch.no_grad():
            return cuda_bias_relu(x, bias)
    return torch.relu(x + bias)


@bias_relu.register_fake
def _bias_relu_fake(x: Tensor, bias: Tensor) -> Tensor:
    _validate(x, bias)
    # Match the real output's fresh, contiguous storage and metadata. Do not use
    # tensor values, .item(), .cpu(), or the actual CUDA kernel in a fake kernel.
    return torch.empty(x.shape, dtype=x.dtype, device=x.device)


def _setup_context(ctx, inputs, output) -> None:
    x, bias = inputs
    ctx.save_for_backward(x, bias)


def _backward(ctx, grad_output: Tensor) -> tuple[Tensor, Tensor]:
    x, bias = ctx.saved_tensors
    active = (x + bias) > 0
    # A branch is numerically different from multiplying by a 0/1 mask:
    # inactive ReLU entries must return zero even if upstream is inf or NaN.
    # Multiplication would evaluate 0 * inf / 0 * NaN and create NaN instead.
    grad_input = torch.where(active, grad_output, torch.zeros_like(grad_output))
```

```
    # Bias was broadcast over rows, so its VJP sums those rows, including M=0.
    grad_bias = grad_input.sum(dim=0)
    return grad_input, grad_bias


bias_relu.register_autograd(_backward, setup_context=_setup_context)


@torch.library.custom_op("gpu_prep::bias_relu_triton", mutates_args=())
def bias_relu_triton(x: Tensor, bias: Tensor) -> Tensor:
    """The same operator contract with a Triton GPU implementation.

    Uses the PyTorch >=2.5 custom_op interface, so its internals are opaque to
    the compiler. Newer torch.library.triton_op APIs are deliberately optional,
    not required by this package. CPU remains an ordinary reference fallback.
    """
    _validate(x, bias)
    if x.device.type == "cuda":
        from gpu_prep.triton_ops import bias_relu as triton_bias_relu

        with torch.no_grad():
            return triton_bias_relu(x, bias)
    return torch.relu(x + bias)


bias_relu_triton.register_fake(_bias_relu_fake)
bias_relu_triton.register_autograd(_backward, setup_context=_setup_context)


__all__ = ["bias_relu", "bias_relu_triton"]
```

第 12 题｜性能实验、诊断与项目答辩

完整源文件： gpu_prep/benchmark.py。

第 12 题：性能实验、诊断与项目答辩

题目与完整实现

选择前面的一个算子，对 PyTorch、CUDA 和 Triton 的实现做数值检查、稳态计时与瓶颈分析。提交硬件和软件版本、输入分布、shape、dtype、计算精度、测量范围、原始样本、优化前后结果及适用边界。不能只交一个 speedup 数字。

完整入口是 gpu_prep/benchmark.py。统一正确性演示是 examples/run_gpu.py。它们不在没有 GPU 时自动退回 CPU 并继续生成“GPU 性能”表。CPU 单元测试只检查模型计算、公式 reference 和计时统计的定义。

1. 先定义比较对象

“比较一个 kernel”可以指三种不同对象：

对象	计入什么	适合回答什么
单个 device kernel	该 kernel 的 GPU 执行；通常由 profiler 或预分配 launch 测量	线程映射、访存、指令改进是否有效
Resident-input 算子 wrapper	输入已在 GPU；包括 wrapper 发出的工作与输出分配	自定义算子 API 替换后能节省多少
应用端到端	实际请求中的调度、复制、模型计算、排队等选定阶段	用户可见延迟或吞吐是否改善

本项目统一脚本测第二类，同时报告 GPU event 和同步后的 host wall 时间。每个 wrapper 自己分配输出，PyTorch 基线也遵循同样的算子语义。没有把 Python 内层循环、编译器耗时和服务器排队拼到某个没有标注的“kernel latency”里。

每次 wrapper 调用中的 shape、dtype、device 等 metadata 检查也在该测量范围内；排除的是独立的数值正确性比较。FP32 matmul 明确关闭 TF32；FP16/BF16 matmul 明确关闭 PyTorch 的 reduced-precision reduction 开关，避免把不同累加策略混为同一种精度。环境 JSON 会记录这些策略。Attention 的 PyTorch 基线仍可由 SDPA 选择可用 fused backend，不承诺与教学实现拥有逐步相同的运算和舍入顺序。

物理转置的 PyTorch baseline 是 x.T.contiguous()。只测 x.T 通常只是视图/stride 变化，做的工作与写出转置结果的 CUDA kernel 不一样。Bias+ReLU 的 PyTorch eager baseline 包含其张量运算过程，融合 kernel 则可能减少中间张量；若又比较 torch.compile，需要把它列为独立实现并单独预热。

2. 为什么普通 Python 计时会错

GPU launch 通常异步。若写 start=time.perf_counter()、调用 kernel、立即读取 end，测到的主要可能是 CPU 提交时间，而 GPU 尚未完成。

正确的 host wall 测量需要在开始前同步清掉前序工作，在结束前同步等待本次工作完成。正确的 CUDA event 测量需要在被测工作所属的 stream 上 record start 和 end，并等待 end 完成后读取 elapsed_time。多 stream 程序必须建立相应依赖；单个 stream 的 end event 不会神奇地覆盖没有依赖的其他 stream 工作。

本脚本的函数合同是“所有工作在所选 device 的当前 stream 上发起”。CUDA 入口使用 PyTorch current stream，Triton 在当前 device/stream 下 launch。测试还包含非默认 stream 的情形。若你扩展为异步 H2D、计算与 D2H 多 stream pipeline，请同时修改计时边界。

3. 首次执行、预热与稳态

第一次调用可能触发 Python import、CUDA context 初始化、扩展编译、Triton JIT、torch.compile、autotune 或 allocator 初始化。这些对生产系统有时很重要，但不应混成一个没有说明的稳态 kernel 时间。

脚本先构建数据和 reference，再对所有后端分别做正确性检查，因此可能出现的 JIT 已经发生在计时前。随后 warmup，再做多个 repeat，每个 repeat 连续调用多次，将批量 elapsed time 除以调用次数。这样能降低短 kernel 的计时噪声。

这也带来一个明确限制：结果属于重复使用相同输入的 warm-cache 模式。工作集很小时，L2 复用可能显著。若实际线上每次访问不同数据，应设计轮换输入或真实 workload；不要只给脚本加一句“已经 warmup”就认为数据访问分布匹配。

4. 统计数字怎样解释

脚本保存每个 repeat 的平均每次耗时，给出 median、min 和“repeat 平均值的 p95”。它没有把这个 p95 冒称为线上单请求 p95，更不是在模型服务排队条件下测到的 tail latency。

例如 7 个 repeat，每个 100 次调用，得到的是 7 个平均耗时样本。它适合粗看稳定性，不能推断 700 个单次请求的完整分布。若需要单请求 p95/p99，应按真实请求边界采样足够多的请求，说明并发、到达过程、queueing、batching、warm/cold state。

报告 speedup 时写清分子分母：speedup = baseline_time / candidate_time。不要把“延迟减少 20%”与“吞吐增加 20%”等同。基线 10 ms，优化后 8 ms，延迟减少 20%，同样串行工作可得 1.25× 速度，也就是理想处理率增加 25%。

5. 可直接执行的实验矩阵

先小后大，每轮保留日志和 JSON。以下参数是建议的实验设计，不是已经跑出的性能数据。

```
# 非整除尺寸，先确认边界正确
python examples/run_gpu.py --backend all --op all

# launch/调度成本明显的小 vector 与较大 vector
python -m gpu_prep.benchmark --op vector_add --shape 1024 --output results/add_small.json
python -m gpu_prep.benchmark --op vector_add --shape 16777216 --output results/add_large.json

# 不同归约行宽，观察单行策略的限制
python -m gpu_prep.benchmark --op softmax --shape 1024,127 --output results/softmax_127.json
python -m gpu_prep.benchmark --op softmax --shape 1024,4097 --output results/softmax_4097.json

# 同一 shape 下比较 naive 与 tiled
python -m gpu_prep.benchmark --op transpose --shape 1023,769 --variant naive --output results/trans_naive.json
python -m gpu_prep.benchmark --op transpose --shape 1023,769 --variant tiled --output results/trans_tiled.json
python -m gpu_prep.benchmark --op matmul --shape 127,65,93 --variant naive --output results/gemm_naive.json
python -m gpu_prep.benchmark --op matmul --shape 127,65,93 --variant tiled --output results/gemm_tiled.json

# skinny 与 square 分开；大型 naive GEMM 可能很慢，先用 tiled
python -m gpu_prep.benchmark --op matmul --shape 1,4096,4096 --output results/gemm_skinny.json
python -m gpu_prep.benchmark --op matmul --shape 1024,1024,1024 --output results/gemm_square.json

# 严格 FP32 与低精度分别比较
python -m gpu_prep.benchmark --op matmul --shape 512,512,512 --dtype float16 --backends torch,triton --output
results/gemm_fp16.json

# 长度非整除的 noncausal attention；causal/decode 正确性另由 tests 覆盖
python -m gpu_prep.benchmark --op attention --shape 1,4,127,257,64 --output results/attention.json
```

执行前确认显存足够；尤其是 PyTorch float64 reference 会使用额外显存。脚本创建 reference 是为了正确性检查，不代表生产实现需要保留该 reference。只计时前向 wrapper，生成 reference 和复制输入不在计时范围。

6. Profiler 诊断顺序

第一步看应用时间线，确认目标算子在实际临界路径上的占比。第二步只捕获代表性 kernel，检查 grid/block、寄存器、shared memory、occupancy、DRAM/L2 吞吐与 stall。第三步提出一个可检验假设，修改一个主要因素。第四步重新跑同样的正确性与计时实验。

Nsight Systems 适合观察 CPU/GPU 时间线、launch gap、同步和重叠；Nsight Compute 适合深入单个 kernel 的硬件指标。性能分析工具可能重放 kernel 或显著改变运行过程，所以 profiler 下的总 wall time 不能直接当线上性能。先在普通运行里得到基准，再用 profiler 解释原因。

可作为起点的命令如下；不同版本支持的选项以已安装工具的 --help 为准：

```
nsys profile -o results/gemm_timeline python -m gpu_prep.benchmark --op matmul --shape 512,512,512 --backends
triton
ncu --set full -o results/gemm_kernel python examples/run_gpu.py --backend triton --op matmul
```

实际 profiling 时应缩小捕获范围并选择目标 kernel，避免让大量 reference/JIT/预热工作淹没报告。若 GPU counter 权限受限，先使用能获得的时间线和计时数据，并如实说明缺失；不要把“没有拿到计数器”解释成“带宽利用率为零”。

7. 面试追问与可直接练习的回答

问：你让 softmax 快了 2×，模型端到端能快多少?

“要看它在临界路径上的占比。假设原来只占 5%，且没有重叠关系变化和额外开销，Amdahl 模型给出的整体速度是 $1/(0.95+0.05/2)\approx1.02564\times$。对应总延迟减少 2.5%，处理率增加约 2.564%。如果 kernel 与其他工作重叠，就还要用真实时间线重算，不能直接相加各 kernel 的耗时。”

问：为什么你的 fused bias+ReLU 比两个 PyTorch 运算快?

“我的预期是减少一次中间结果写回和读取，且可能减少一次 launch。但我会按张量大小验证：小张量可能主要受 launch 影响，大张量可能受流量影响。若使用 torch.compile 作为基线，它也可能已经融合，所以我会把 eager 与 compiled baseline 分开报告。最终还要看融合后的寄存器压力是否抵消收益。”

问：你怎么证明一个优化有效，而不是噪声?

“先在相同输入语义、dtype、精度策略和版本下检查正确性，排除首次编译，再交替或打乱测量顺序，多次采样并保存原始数据。我会在目标 shape 分布上看稳定收益；若差异接近运行波动，增加有目的的重复或控制时钟/负载，再决定是否接受，而不只挑最漂亮的一次。”

问：为何 Triton tiled 版本比 naive 慢?

“Tiled 只描述组织方式，不保证适合所有 shape。小矩阵可能被 launch 成本支配，固定 tile 可能产生很多 masked work，layout conversion 和寄存器压力也可能增大。严格 FP32 路径未必使用 Tensor Core。我要检查具体 generated code 和 profiler，确认实际执行路径，再决定改 tile、dtype/精度合同还是调度。”

问：GPU event 时间为什么比 CPU wall 时间短?

“事件反映所选 stream 时间区间，host wall 还包含 Python 调度、CPU 工作和同步成本。不过不能机械地把二者之差解释成某一个固定的 launch overhead，因为异步提交、GPU idle gap 和流水重叠会影响二者。这个脚本把它们分开报告，避免混淆；需要更精细归因时用时间线。”

问：你是否考虑了编译和内存分配?

“我把第一次编译单独看，稳态测试中先完成 JIT 与预热。当前 wrapper benchmark 包含实现内部的输出分配；如果要分析纯 device kernel，我会改成预分配输入输出并直接 launch，明确这是另一个测量对象。生产端到端还应看模型加载、动态 shape 的重新编译、allocator 行为和服务生命周期。”

问：Baseline 应该选你自己的 naive 实现还是厂商库？

“两者回答不同问题。Naive baseline 能说明某个教学优化为何有效，但要判断生产价值还需对照成熟实现，如同语义的 PyTorch/cuBLAS/SDPA，并确保精度、layout 和功能一致。我会同时列出最初版本与强基线，避免只证明自己超过了一个刻意低效的起点。”

问：你做了一个只对少数 shape 很快的 kernel，怎么落地？

“先确认这些 shape 在真实分布里的频率和耗时占比，再设置清晰的 dispatch 条件，并保留其他 shape 的正确 fallback。测试覆盖条件边界，记录编译与缓存成本。只有加权端到端收益能够抵过维护复杂度和回归风险时，专用路径才有价值。”

8. 项目答辩模板：把自己的真实材料填进去

“我的场景是【训练/推理、prefill/decode、batch 和 shape 分布】。原始测量显示【明确的时间线或指标】，其中【目标工作】占【真实比例】。我据此提出【具体瓶颈假设】，实现了【具体改动】。我用【reference、边界 case、梯度检查】验证正确性，用【硬件、软件版本、计时范围】测性能。结果在【适用范围】为【实测数字】，但在【其他形状/条件】存在【限制】。下一步我会验证【一个仍未证实的假设】。”

如果某部分来自仿真、推算或模拟器，直接说清：“这里是模型估计/仿真结果，尚未在目标硬件验证。”如果 GPU 型号、驱动、功耗条件或真实日志缺失，不要补一个看似合理的数字。面试中能清楚说明证据的边界，比无法复现的大幅 speedup 更容易让对方判断你的工程方法。

9. 两小时模拟面试的建议分配

时间	练习内容	通过条件
0–15 分钟	Vector add 或 bias+ReLU	写出合法索引、尾块 mask、正确调用
15–35 分钟	Reduction 或 softmax	解释 identity、同步和稳定性，处理非整除
35–60 分钟	Transpose 或 GEMM	先正确基线，再推导 tile 的读写与同步
60–80 分钟	Linear/QKV 与 Norm	Shape、stride、统计轴、dtype 不出错
80–100 分钟	Attention 与 decode mask	能说明非方阵 causal 位置与 KV cache
100–120 分钟	Roofline、compile、实验答辩	给出模型假设与下一步可执行验证

首次练习可以边查边写；第二次去掉实现只看函数签名；第三次限时并口述每个关键选择。最终检查标准是你能解释并修改代码，而不是把所有源文件背下来。

本题核心实现：函数体全文

以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附录。

gpu_prep/benchmark.py

```
@dataclass
class Case:
    op: str
    shape: tuple[int, ...]
    inputs: tuple[torch.Tensor, ...]
    torch_fn: Callable[[], torch.Tensor]
    reference: torch.Tensor
    logical_bytes: int | None
    flops: int | None
    notes: str
```

```
@torch.inference_mode()
def build_case(op: str, shape: tuple[int, ...], dtype: torch.dtype, device: torch.device) -> Case:
    """Make data and a float64 numerical oracle outside every timing region."""
    def rand(*dims: int) -> torch.Tensor:
        return torch.randn(dims, dtype=dtype, device=device)

    size = torch.empty((), dtype=dtype).element_size()
    logical_bytes = None
    flops = None
    notes = "Finite random inputs; float64 oracle is computed outside timing."
    if op == "vector_add":
        _require_dims(shape, 1, op)
        x, y = rand(*shape), rand(*shape)
        args = (x, y)
        fn = lambda: x + y
        ref = (x.double() + y.double()).to(dtype)
        model = vector_add_workload(shape[0], element_bytes=size)
        logical_bytes, flops = model.ideal_bytes, model.flops
    elif op == "matmul":
        _require_dims(shape, 3, op)
        m, n, k = shape
        a, b = rand(m, k), rand(k, n)
        args = (a, b)
        fn = lambda: a @ b
        ref = (a.double() @ b.double()).to(dtype)
        model = gemm_workload(m, n, k, element_bytes=size)
        logical_bytes, flops = model.ideal_bytes, model.flops
        notes += " Shape order is M,N,K; beta=0; logical bytes assume ideal reuse."
    elif op == "attention":
        _require_dims(shape, 5, op)
        batch, heads, length, source, dim = shape
        q, k, v = rand(batch, heads, length, dim), rand(batch, heads, source, dim), rand(batch, heads, source,
dim)
        args = (q, k, v)
        fn = lambda: F.scaled_dot_product_attention(q, k, v, dropout_p=0.0, is_causal=False)
        scores = (q.double() @ k.double().transpose(-2, -1)) / math.sqrt(dim)
        ref = (scores.softmax(dim=-1) @ v.double()).to(dtype)
        flops = 4 * batch * heads * length * source * dim
        notes += " Noncausal, no dropout. FLOPs count two GEMMs only. PyTorch SDPA may choose a fused backend."
    else:
        _require_dims(shape, 2, op)
        m, n = shape
        x = rand(m, n)
        args = (x,)
        if op == "bias_relu":
            bias = rand(n)
            args = (x, bias)
            fn = lambda: torch.relu(x + bias)
            # The custom kernels add and apply ReLU in FP32 for low-precision inputs.
            ref = torch.relu(x.double() + bias.double()).to(dtype)
            logical_bytes = (2 * m * n + n) * size
            flops = m * n
            notes += " Bias traffic assumes ideal reuse; max comparisons are omitted from FLOPs."
        elif op == "row_sum":
            fn = lambda: x.sum(dim=-1)
            ref = x.double().sum(dim=-1).to(dtype)
            logical_bytes = (m * n + m) * size
            flops = m * max(n - 1, 0)
        elif op == "row_max":
            fn = lambda: x.amax(dim=-1)
            ref = x.double().amax(dim=-1).to(dtype)
            logical_bytes = (m * n + m) * size
        elif op == "softmax":
            fn = lambda: x.softmax(dim=-1)
            ref = x.double().softmax(dim=-1).to(dtype)
```

```
            logical_bytes = 2 * m * n * size
            notes += " Logical traffic is a one-read/one-write lower bound, not measured DRAM traffic."
        elif op == "transpose":
            fn = lambda: x.T.contiguous()
            ref = x.T.contiguous()
            logical_bytes = 2 * m * n * size
            notes += " Physical transpose; the reference includes contiguous(), not just a stride-changing view."
        elif op == "layer_norm":
            weight, bias = rand(n), rand(n)
            args = (x, weight, bias)
            fn = lambda: F.layer_norm(x, (n,), weight, bias, eps=1e-5)
            ref = F.layer_norm(x.double(), (n,), weight.double(), bias.double(), eps=1e-5).to(dtype)
        elif op == "rms_norm":
            weight = rand(n)
            args = (x, weight)
            fn = lambda: F.rms_norm(x, (n,), weight, eps=1e-5)
            xd = x.double()
            ref = (xd * torch.rsqrt(xd.square().mean(dim=-1, keepdim=True) + 1e-5) * weight.double()).to(dtype)
        else:
            raise ValueError(f"unknown op: {op}")
    return Case(op, shape, args, fn, ref, logical_bytes, flops, notes)


def make_callable(case: Case, backend: str, variant: str = "tiled") -> Callable[[], torch.Tensor]:
    if backend == "torch":
        return case.torch_fn
    if backend not in ("cuda", "triton"):
        raise ValueError(f"unknown backend: {backend}")
    module = importlib.import_module(f"gpu_prep.{backend}_ops")
    name = f"{case.op}_{variant}" if case.op in ("transpose", "matmul") else case.op
    operation = getattr(module, name)
    return lambda: operation(*case.inputs)


@torch.inference_mode()
def check_output(fn: Callable[[], torch.Tensor], reference: torch.Tensor) -> dict:
    out = fn()  # A first compilation or extension build can happen here.
    tolerance = {
        torch.float32: (3e-4, 3e-4),
        torch.float16: (8e-3, 8e-3),
        torch.bfloat16: (5e-2, 5e-2),
    }[reference.dtype]
    rtol, atol = tolerance
    torch.testing.assert_close(out, reference, rtol=rtol, atol=atol)
    error = (out.double() - reference.double()).abs()
    denominator = reference.double().abs().clamp_min(atol)
    return {
        "passed": True, "rtol": rtol, "atol": atol,
        "max_absolute_error": error.max().item() if error.numel() else 0.0,
        "max_relative_error_with_atol_floor": (error / denominator).max().item() if error.numel() else 0.0,
    }


@torch.inference_mode()
def measure_cuda(
    fn: Callable[[], torch.Tensor], device: torch.device,
    warmup: int = 25, iterations: int = 100, repeats: int = 7,
) -> dict:
    """Report GPU-event and synchronized host-wall averages separately.

    Contract: fn launches all its work on this device's current stream and returns
    a CUDA tensor. Cross-stream work needs explicit dependencies and a different
    measurement plan. Events may include GPU idle gaps caused by host dispatch.
    """
    if device.type != "cuda" or not torch.cuda.is_available():
        raise RuntimeError("A real NVIDIA CUDA device is required; CPU timings are not substituted.")
    if device.index is None:
```

```
        device = torch.device("cuda", torch.cuda.current_device())
    if min(warmup, iterations, repeats) <= 0:
        raise ValueError("warmup, iterations, and repeats must be positive")
    event_samples, wall_samples = [], []
    with torch.cuda.device(device):
        for _ in range(warmup):
            last = fn()
        if not isinstance(last, torch.Tensor) or not last.is_cuda or last.device != device:
            raise ValueError("fn must return a tensor on the selected CUDA device")
        torch.cuda.synchronize(device)
        # Create events before timing. Their initialization is forced once.
        start, end = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
        start.record()
        end.record()
        end.synchronize()
        for _ in range(repeats):
            start.record()
            for _ in range(iterations):
                last = fn()
            end.record()
            end.synchronize()
            event_samples.append(start.elapsed_time(end) / iterations)
        for _ in range(repeats):
            torch.cuda.synchronize(device)
            start_wall = time.perf_counter()
            for _ in range(iterations):
                last = fn()
            torch.cuda.synchronize(device)
            wall_samples.append((time.perf_counter() - start_wall) * 1000 / iterations)
    return {
        "gpu_event": summarize_samples(event_samples),
        "synchronized_host_wall": summarize_samples(wall_samples),
        "warmup_calls": warmup, "calls_per_repeat": iterations, "repeats": repeats,
        "measurement_scope": "Resident-input forward wrapper, including metadata checks, output allocation, and
launches; JIT and numerical correctness checks excluded.",
        "p95_scope": "Quantile over repeat averages, not p95 latency of individual requests.",
        "cache_policy": "Repeatedly reuse the same inputs; caches may be warm. No explicit cache flush.",
    }


def summarize_samples(samples_ms: list[float]) -> dict:
    """p95 is an interpolated quantile of repeat averages, not request-tail latency."""
    if not samples_ms or any(not math.isfinite(x) or x <= 0 for x in samples_ms):
        raise ValueError("latency samples must be nonempty, finite, and positive")
    ordered = sorted(samples_ms)
    position = 0.95 * (len(ordered) - 1)
    low, high = math.floor(position), math.ceil(position)
    p95 = ordered[low] + (position - low) * (ordered[high] - ordered[low])
    return {
        "median_ms": statistics.median(ordered), "min_ms": ordered[0],
        "p95_of_repeat_means_ms": p95, "samples_ms": samples_ms,
    }


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--op", choices=DEFAULT_SHAPES, default="matmul")
    parser.add_argument("--shape", type=parse_shape)
    parser.add_argument("--backends", default="torch,cuda,triton")
    parser.add_argument("--variant", choices=("naive", "tiled"), default="tiled")
    parser.add_argument("--dtype", choices=DTYPES, default="float32")
    parser.add_argument("--device", default="cuda:0")
    parser.add_argument("--warmup", type=int, default=25)
    parser.add_argument("--iterations", type=int, default=100)
    parser.add_argument("--repeats", type=int, default=7)
    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument("--output", type=Path)
```

```
    args = parser.parse_args()
    if not torch.cuda.is_available():
        parser.error("No NVIDIA CUDA device is available. Use CPU tests for formulas, not GPU timing.")
    device = torch.device(args.device)
    if device.type != "cuda":
        parser.error("--device must be CUDA")
    if device.index is None:
        device = torch.device("cuda", torch.cuda.current_device())
    backends = [name.strip() for name in args.backends.split(",")]
    if not backends or len(set(backends)) != len(backends) or any(x not in ("torch", "cuda", "triton") for x in
backends):
        parser.error("--backends must be distinct names chosen from torch,cuda,triton")
    if "cuda" in backends and args.dtype != "float32":
        parser.error("The CUDA C++ teaching backend supports float32; choose torch,triton for half/bfloat16.")
    if "triton" in backends and os.environ.get("TRITON_INTERPRET") == "1":
        parser.error("Unset TRITON_INTERPRET for GPU timing. Use tools/check_triton_interpreter.py in a separate
process for CPU math checks.")
    if min(args.warmup, args.iterations, args.repeats) <= 0:
        parser.error("warmup, iterations, and repeats must be positive")
    torch.manual_seed(args.seed)
    # Compare IEEE FP32 baselines; do not silently compare TF32 with FP32.
    torch.set_float32_matmul_precision("highest")
    torch.backends.cuda.matmul.allow_tf32 = False
    torch.backends.cuda.matmul.allow_fp16_reduced_precision_reduction = False
    torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction = False
    shape = args.shape or DEFAULT_SHAPES[args.op]
    with torch.cuda.device(device), torch.inference_mode():
        case = build_case(args.op, shape, DTYPES[args.dtype], device)
        functions = {name: make_callable(case, name, args.variant) for name in backends}
        checks = {name: check_output(fn, case.reference) for name, fn in functions.items()}
        # All backends are built and checked before timing any backend.
        order = list(backends)
        random.Random(args.seed).shuffle(order)
        results = {}
        for name in order:
            measured = measure_cuda(functions[name], device, args.warmup, args.iterations, args.repeats)
            seconds = measured["gpu_event"]["median_ms"] / 1000
            measured["correctness"] = checks[name]
            measured["logical_effective_GB_per_s"] = case.logical_bytes / seconds / 1e9 if case.logical_bytes is
not None else None
            measured["estimated_TFLOP_per_s"] = case.flops / seconds / 1e12 if case.flops is not None else None
            results[name] = measured
    report = {
        "environment": collect_environment(),
        "case": {"op": case.op, "shape": case.shape, "dtype": args.dtype, "variant": args.variant,
                 "logical_bytes": case.logical_bytes, "estimated_flops": case.flops, "notes": case.notes},
        "measurement_order": order, "seed": args.seed, "results": results,
        "traffic_warning": "Logical bytes are an algorithmic model, not Nsight-measured HBM traffic; cache reuse
can make effective bandwidth exceed HBM bandwidth.",
    }
    encoded = json.dumps(report, ensure_ascii=False, indent=2)
    if args.output is not None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(encoded + "\n", encoding="utf-8")
    print(encoded)
```

参考资料与来源边界

查阅日期：2026-10-10。下列链接用于核对 API 语义、数学背景和进一步学习。项目源码为针对这套练习编写的原创教学实现；没有复制某篇教程的完整 kernel 并声称是自己的性能成果。

CUDA 与性能分析

1. NVIDIA CUDA C++ Best Practices Guide：访存、共享内存、正确性与优化方法。
2. NVIDIA CUDA Programming Guide：线程协作、同步、执行与内存模型。
3. Nsight Compute Profiling Guide：roofline、内存层级与 kernel 指标。
4. Nsight Systems User Guide：CPU/GPU 时间线和并发观察。
5. PyTorch C++ Extension API：load、构建工具链与 JIT extension。
6. PyTorch CUDA semantics：设备、stream 与异步执行。

Triton 官方教程与 API

1. Vector Addition：program、arange、load/store mask。
2. Fused Softmax：融合归约与资源取舍。
3. Matrix Multiplication：矩阵 tile 与 dot。
4. Layer Normalization：统计量、前向与反向的组织。
5. Fused Attention：分块 attention 的更完整实现背景。
6. tl.dot：dtype、accumulator 与 input_precision。
7. tl.load：mask 与 other。
8. tl.trans：block 轴交换；不等同于手动声明 CUDA shared-memory tile。
9. Triton 项目与支持平台：安装和硬件支持说明。

官方最新文档可能描述比本次验证的 Triton 3.1.0 更多的参数和硬件路径。源码仅使用本次编译检查实际支持的功能，不因为新版文档列出了某种模式，就假定旧版也支持。

PyTorch 层、编译与注册

1. Linear：权重布局与输入末维约定。
2. LayerNorm：normalized_shape、population variance 与 affine。
3. RMSNorm：均方根归一化与 eps。
4. Scaled Dot Product Attention：boolean mask、dropout、非方阵 causal 对齐与 backend 选择。
5. Tensor Views：view、stride、contiguous 与复制。
6. torch.compile tutorial：图捕获与编译入门。
7. Common Graph Breaks：数据依赖和捕获问题。
8. torch.cond：结构化控制流及限制。
9. Custom C++ and CUDA Operators：fake、autograd、opcheck 与扩展接入。
10. User-Defined Triton Kernels with torch.compile：Triton kernel 接入与 triton_op 的版本相关差异。

11. Numerical Accuracy：归约顺序、TF32、低精度累加与不同实现的误差。
12. Gradcheck Mechanics：数值梯度检查的假设和用途。

本包把 opcheck 用于注册合同，把 double gradcheck 用于导数数学验证。CPU backend=eager 的 fullgraph 通过只证明相应图捕获和结果检查，不证明 GPU Inductor 性能。

Attention 数学原始论文

1. Online normalizer calculation for softmax — Milakov & Gimelshein：在线最大值/分母更新。
2. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness — Dao et al.：分块重排 attention 的 IO 视角。

“数学上等价”意味着在实数算术模型中等价，不意味着有限精度下逐 bit 一样。教材的 Triton attention 保留 FP32 probabilities 做第二个 dot，便于解释误差；它没有实现生产 FlashAttention 的所有 pipeline、反向或硬件专用优化。

岗位背景与进一步阅读

- Together AI: Inside the Together AI Kernels Team：岗位所处的 GPU kernel 与系统方向背景。
- CUDA GEMM Worklog — Simon Boehm：作者的逐步 GEMM 实验，可用于对照优化方法；本文不复用其硬件性能数字。
- GPU MODE Lectures：延伸课程与讲座。
- ThunderKittens：进一步研究 tile 级抽象与更复杂 kernel。

这些材料不能证明某公司必考某题。面试准备应以真实岗位职责、你的项目和收到的面试说明为主；本教材用公开技术知识构造练习和问答。

完整源码附录

下面逐文件给出工程源码、测试和构建工具的完整文本。文件之间存在 import/链接关系，建议解压工程后直接运行；本附录用于离线阅读、查找和逐行对照。

MANIFEST.in

```
recursive-include gpu_prep/csrc *.cu *.cpp *.h
recursive-include docs *.md *.svg
recursive-include examples *.py
recursive-include tests *.py
include README.md requirements.txt
```

examples/compile_demo.py

```
"""Exercises 10--11: graph breaks, tensor control flow and custom-op capture.

CPU capture smoke check:
  python -m examples.compile_demo --device cpu --backend eager
Actual Inductor experiment (requires an appropriate installed toolchain):
  python -m examples.compile_demo --device cuda --backend inductor

backend='eager' tests Dynamo graph capture and result agreement, not Inductor
code generation or GPU acceleration. Results are measured on YOUR machine.
"""

import argparse
import statistics
import time

import torch

from gpu_prep.custom_ops import bias_relu, bias_relu_triton


def bad_python_branch(x: torch.Tensor) -> torch.Tensor:
    # .item() transfers a data value to Python; the if depends on that value.
    if x.sum().item() > 0:
        return x.sin()
    return x.cos()


def tensor_where(x: torch.Tensor) -> torch.Tensor:
    # Semantically simple and easy to capture. Both sin/cos expressions are
    # evaluated before where selects their results; it is not lazy branching.
    return torch.where(x.sum() > 0, x.sin(), x.cos())


def tensor_cond(x: torch.Tensor) -> torch.Tensor:
    # Structured control flow: branch outputs need matching metadata and the
    # branches must meet torch.cond's capture restrictions.
    return torch.cond(x.sum() > 0, lambda t: t.sin(), lambda t: t.cos(), (x,))


def _sync(device: torch.device) -> None:
    if device.type == "cuda":
        torch.cuda.synchronize(device)
```

```
def median_wall_ms(fn, fn_args, device: torch.device, warmup: int, repeats: int) -> float:
    for _ in range(warmup):
        fn(*fn_args)
    _sync(device)
    times = []
    for _ in range(repeats):
        _sync(device)
        start = time.perf_counter()
        fn(*fn_args)
        _sync(device)
        times.append((time.perf_counter() - start) * 1000)
    return statistics.median(times)


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--device", default="cpu")
    parser.add_argument("--backend", choices=("eager", "inductor"), default="eager")
    parser.add_argument("--repeats", type=int, default=20)
    parser.add_argument("--warmup", type=int, default=5)
    parser.add_argument("--include-cond", action="store_true",
                        help="also test structured torch.cond (availability/limits vary by PyTorch release)")
    parser.add_argument("--operator-backend", choices=("cuda", "triton", "both"), default="both",
                        help="custom-op GPU implementation; both use CPU references when --device cpu")
    args = parser.parse_args()
    if args.repeats <= 0 or args.warmup < 0:
        raise SystemExit("repeats must be positive and warmup nonnegative")
    device = torch.device(args.device)
    if device.type == "cuda" and not torch.cuda.is_available():
        raise SystemExit("CUDA is not available; use --device cpu for capture-only checks")
    print("PyTorch:", torch.__version__, "device:", device, "backend:", args.backend)
    if args.backend == "eager":
        print("Scope: Dynamo capture/result checks. This does NOT benchmark Inductor or prove GPU speedup.")
    torch.manual_seed(2027)
    positive = torch.ones(8192, device=device)
    negative = -positive
    with torch.no_grad():
        try:
            broken = torch.compile(bad_python_branch, backend=args.backend, fullgraph=True)
            broken(positive)
        except Exception as error:
            print("Expected fullgraph failure for data-dependent Python branch:", type(error).__name__)
            print(str(error).splitlines()[0][:250])
        else:
            print("This PyTorch/configuration captured the example; inspect its graph and guards rather than
assuming a break.")

        functions = [("where", tensor_where)]
        if args.include_cond:
            functions.append(("cond", tensor_cond))
        for name, fn in functions:
            compiled = torch.compile(fn, backend=args.backend, fullgraph=True)
            _sync(device)
            start = time.perf_counter()
            first = compiled(positive)
            _sync(device)
            first_call_ms = (time.perf_counter() - start) * 1000
            torch.testing.assert_close(first, bad_python_branch(positive))
            torch.testing.assert_close(compiled(negative), bad_python_branch(negative))
            eager_ms = median_wall_ms(fn, (positive,), device, args.warmup, args.repeats)
            compiled_ms = median_wall_ms(compiled, (positive,), device, args.warmup, args.repeats)
            print(f"{name}: first compiled call including capture/codegen/initialization = {first_call_ms:.3f}
ms")
            print(f"{name}: steady median wall time eager={eager_ms:.4f} ms, compiled={compiled_ms:.4f} ms")

        x = torch.randn(17, 33, device=device)
```

```
        bias = torch.randn(33, device=device)
        operators = []
        if args.operator_backend in ("cuda", "both"):
            operators.append(("CUDA wrapper", bias_relu))
        if args.operator_backend in ("triton", "both"):
            operators.append(("Triton wrapper", bias_relu_triton))
        for name, operation in operators:
            # First GPU use lazily builds/JITs the teaching implementation. Do it
            # before capture; compilation is not steady-state kernel latency.
            torch.testing.assert_close(operation(x, bias), torch.relu(x + bias))
            op_graph = torch.compile(lambda a, b: operation(a, b).square().sum(),
                                     backend=args.backend, fullgraph=True)
            torch.testing.assert_close(op_graph(x, bias), torch.relu(x + bias).square().sum())
            print(name + ": fullgraph capture and numerical agreement passed for this device/backend.")

    # Keep the registration check independent of the timing region.
    cpu_x = torch.tensor([[-2.0, 2.0, -3.0], [1.0, -4.0, 3.0]], dtype=torch.float64, requires_grad=True)
    cpu_bias = torch.tensor([0.1, 0.2, 0.3], dtype=torch.float64, requires_grad=True)
    for name, operation in operators:
        print(name, "opcheck (registration):", torch.library.opcheck(operation, (cpu_x, cpu_bias)))
        print(name, "gradcheck (mathematical derivative):", torch.autograd.gradcheck(operation, (cpu_x,
cpu_bias)))


if __name__ == "__main__":
    main()
```

examples/run_gpu.py

```
"""Run a common correctness demonstration on the real GPU selected by the user."""
from __future__ import annotations

import argparse
import importlib
import os

import torch

from gpu_prep.benchmark import DEFAULT_SHAPES, build_case, check_output, make_callable


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--backend", choices=("cuda", "triton", "all"), default="all")
    parser.add_argument("--op", choices=(*DEFAULT_SHAPES, "all"), default="all")
    parser.add_argument("--device", default="cuda:0")
    args = parser.parse_args()
    if args.backend in ("triton", "all") and os.environ.get("TRITON_INTERPRET") == "1":
        parser.error("Unset TRITON_INTERPRET for the GPU demo; use tools/check_triton_interpreter.py separately.")
    if not torch.cuda.is_available():
        parser.error("This demonstration requires a real NVIDIA GPU. Run pytest on CPU for the PyTorch
exercises.")
    device = torch.device(args.device)
    if device.type != "cuda":
        parser.error("--device must be a CUDA device")
    torch.manual_seed(7)
    torch.set_float32_matmul_precision("highest")
    torch.backends.cuda.matmul.allow_tf32 = False
    names = ("cuda", "triton") if args.backend == "all" else (args.backend,)
    operations = tuple(DEFAULT_SHAPES) if args.op == "all" else (args.op,)
    with torch.cuda.device(device), torch.inference_mode():
        for op in operations:
            case = build_case(op, DEFAULT_SHAPES[op], torch.float32, device)
            variants = ("naive", "tiled") if op in ("transpose", "matmul") else ("tiled",)
```

```
            for backend in names:
                for variant in variants:
                    check = check_output(make_callable(case, backend, variant), case.reference)
                    label = f"{op}_{variant}" if op in ("transpose", "matmul") else op
                    print(f"PASS {backend:6s} {label:18s} shape={case.shape} max_abs=
{check['max_absolute_error']:.3g}")
        if args.op in ("all", "attention"):
            # A non-square causal decode example that a square-only test would miss.
            q, k, v = (torch.randn(shape, device=device) for shape in ((1, 2, 1, 32), (1, 2, 17, 32), (1, 2, 17,
32)))
            reference = torch.nn.functional.scaled_dot_product_attention(q, k, v, dropout_p=0.0, is_causal=False)
            for backend in names:
                module = importlib.import_module(f"gpu_prep.{backend}_ops")
                out = module.attention(q, k, v, causal=True, query_start=16)
                torch.testing.assert_close(out, reference, atol=3e-4, rtol=3e-4)
                print(f"PASS {backend:6s} causal decode: L=1, S=17, query_start=16")


if __name__ == "__main__":
    main()
```

examples/torch_layers_demo.py

```
"""Run with: python -m examples.torch_layers_demo --device cpu

Use --device cuda on a GPU machine. This demonstrates mathematical agreement,
not fused-kernel performance. It needs PyTorch only, not Triton or nvcc.
"""

import argparse

import torch
from torch.nn import functional as F

from gpu_prep.torch_layers import (
    KVCache,
    ManualLayerNorm,
    ManualLinear,
    ManualRMSNorm,
    MultiHeadSelfAttention,
    scaled_dot_product_attention,
    split_qkv_heads,
)


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--device", default="cpu")
    args = parser.parse_args()
    device = torch.device(args.device)
    if device.type == "cuda" and not torch.cuda.is_available():
        raise SystemExit("CUDA is not available. Choose --device cpu for the PyTorch reference exercises.")
    torch.manual_seed(2027)
    # float64 makes small-reference comparisons easy to inspect. GPU throughput
    # exercises should choose their target precision and tolerances separately.
    dtype = torch.float64
    x = torch.randn(2, 5, 8, device=device, dtype=dtype)
    linear = ManualLinear(8, 12, device=device, dtype=dtype)
    torch.testing.assert_close(linear(x), F.linear(x, linear.weight, linear.bias))
    print("ManualLinear: forward matches F.linear; parameters:", list(dict(linear.named_parameters())))

    packed = ManualLinear(8, 24, device=device, dtype=dtype)(x)
    q, k, v = split_qkv_heads(packed, 2)
    print("QKV:", tuple(packed.shape), "->", tuple(q.shape), tuple(k.shape), tuple(v.shape))
```

```
    print("Q stride / contiguous:", q.stride(), q.is_contiguous())

    for layer in (ManualLayerNorm(8, device=device, dtype=dtype),
                  ManualRMSNorm(8, device=device, dtype=dtype)):
        out = layer(x)
        if isinstance(layer, ManualLayerNorm):
            ref = F.layer_norm(x, (8,), layer.weight, layer.bias, eps=1e-5)
        else:
            ref = F.rms_norm(x, (8,), layer.weight, eps=1e-5)
        torch.testing.assert_close(out, ref, atol=1e-10, rtol=1e-10)
        print(type(layer).__name__, "reference match; output dtype:", out.dtype)

    causal_mask = torch.ones(5, 5, device=device, dtype=torch.bool).tril()
    own = scaled_dot_product_attention(q, k, v, is_causal=True)
    ref = F.scaled_dot_product_attention(q, k, v, attn_mask=causal_mask, dropout_p=0.0)
    torch.testing.assert_close(own, ref, atol=1e-10, rtol=1e-10)
    print("Manual attention: matches SDPA using an explicit causal mask")

    model = MultiHeadSelfAttention(8, 2, device=device, dtype=dtype).eval()
    cache = KVCache(2, 2, 5, 4, device=device, dtype=dtype)
    with torch.inference_mode():
        full = model(x)
        parts = [model.forward_step(x[:, :3], cache)]
        parts.extend(model.forward_step(x[:, t:t + 1], cache) for t in range(3, 5))
        incremental = torch.cat(parts, dim=1)
    torch.testing.assert_close(full, incremental, atol=1e-10, rtol=1e-10)
    print("KV cache: 3-token prefill + 2 decode steps match full causal output")
    print("Cache active length / capacity:", cache.length, "/", cache.max_seq_len)

    # Show the all-masked contract before any output projection bias.
    blocked = torch.zeros(5, 5, device=device, dtype=torch.bool)
    zero = scaled_dot_product_attention(q, k, v, attn_mask=blocked)
    assert torch.equal(zero, torch.zeros_like(zero))
    print("All-masked attention rows: zero output (before MHA output projection)")


if __name__ == "__main__":
    main()
```

gpu_prep/init.py

```
"""GPU interview exercises.

Backends are intentionally not imported here. Importing this package neither
compiles CUDA code nor requires Triton to be installed.
"""

__version__ = "1.0.0"
```

gpu_prep/benchmark.py

```
"""Correctness-first CUDA wrapper benchmarks, with honest measurement labels.

Run: python -m gpu_prep.benchmark --op matmul --shape 128,256,384
Shape for matmul means M,N,K. All inputs are already resident on the GPU.
The measured callable allocates its output. This is not a pure kernel-only
benchmark and does not time input copies, JIT compilation, or correctness checks.
"""
from __future__ import annotations

import argparse
```

```
from dataclasses import dataclass
import importlib
import json
import math
import os
from pathlib import Path
import random
import statistics
import time
from typing import Callable

import torch
import torch.nn.functional as F

from .check_env import collect_environment
from .roofline import gemm_workload, vector_add_workload


DTYPES = {"float32": torch.float32, "float16": torch.float16, "bfloat16": torch.bfloat16}
DEFAULT_SHAPES = {
    "vector_add": (1_048_576,), "bias_relu": (511, 769),
    "row_sum": (511, 769), "row_max": (511, 769), "softmax": (511, 769),
    "transpose": (511, 769), "matmul": (128, 256, 384),
    "layer_norm": (511, 769), "rms_norm": (511, 769),
    "attention": (1, 4, 127, 257, 64),
}


@dataclass
class Case:
    op: str
    shape: tuple[int, ...]
    inputs: tuple[torch.Tensor, ...]
    torch_fn: Callable[[], torch.Tensor]
    reference: torch.Tensor
    logical_bytes: int | None
    flops: int | None
    notes: str


def parse_shape(value: str) -> tuple[int, ...]:
    try:
        shape = tuple(int(s.strip()) for s in value.split(","))
    except ValueError as exc:
        raise argparse.ArgumentTypeError("shape must be comma-separated integers") from exc
    if not shape or any(n <= 0 for n in shape):
        raise argparse.ArgumentTypeError("benchmark dimensions must be positive; tests cover empty shapes")
    return shape


def _require_dims(shape: tuple[int, ...], count: int, op: str) -> None:
    if len(shape) != count or any(n <= 0 for n in shape):
        raise ValueError(f"{op} requires {count} positive dimensions; received {shape}")


@torch.inference_mode()
def build_case(op: str, shape: tuple[int, ...], dtype: torch.dtype, device: torch.device) -> Case:
    """Make data and a float64 numerical oracle outside every timing region."""
    def rand(*dims: int) -> torch.Tensor:
        return torch.randn(dims, dtype=dtype, device=device)

    size = torch.empty((), dtype=dtype).element_size()
    logical_bytes = None
    flops = None
    notes = "Finite random inputs; float64 oracle is computed outside timing."
    if op == "vector_add":
        _require_dims(shape, 1, op)
```

```
        x, y = rand(*shape), rand(*shape)
        args = (x, y)
        fn = lambda: x + y
        ref = (x.double() + y.double()).to(dtype)
        model = vector_add_workload(shape[0], element_bytes=size)
        logical_bytes, flops = model.ideal_bytes, model.flops
    elif op == "matmul":
        _require_dims(shape, 3, op)
        m, n, k = shape
        a, b = rand(m, k), rand(k, n)
        args = (a, b)
        fn = lambda: a @ b
        ref = (a.double() @ b.double()).to(dtype)
        model = gemm_workload(m, n, k, element_bytes=size)
        logical_bytes, flops = model.ideal_bytes, model.flops
        notes += " Shape order is M,N,K; beta=0; logical bytes assume ideal reuse."
    elif op == "attention":
        _require_dims(shape, 5, op)
        batch, heads, length, source, dim = shape
        q, k, v = rand(batch, heads, length, dim), rand(batch, heads, source, dim), rand(batch, heads, source,
dim)
        args = (q, k, v)
        fn = lambda: F.scaled_dot_product_attention(q, k, v, dropout_p=0.0, is_causal=False)
        scores = (q.double() @ k.double().transpose(-2, -1)) / math.sqrt(dim)
        ref = (scores.softmax(dim=-1) @ v.double()).to(dtype)
        flops = 4 * batch * heads * length * source * dim
        notes += " Noncausal, no dropout. FLOPs count two GEMMs only. PyTorch SDPA may choose a fused backend."
    else:
        _require_dims(shape, 2, op)
        m, n = shape
        x = rand(m, n)
        args = (x,)
        if op == "bias_relu":
            bias = rand(n)
            args = (x, bias)
            fn = lambda: torch.relu(x + bias)
            # The custom kernels add and apply ReLU in FP32 for low-precision inputs.
            ref = torch.relu(x.double() + bias.double()).to(dtype)
            logical_bytes = (2 * m * n + n) * size
            flops = m * n
            notes += " Bias traffic assumes ideal reuse; max comparisons are omitted from FLOPs."
        elif op == "row_sum":
            fn = lambda: x.sum(dim=-1)
            ref = x.double().sum(dim=-1).to(dtype)
            logical_bytes = (m * n + m) * size
            flops = m * max(n - 1, 0)
        elif op == "row_max":
            fn = lambda: x.amax(dim=-1)
            ref = x.double().amax(dim=-1).to(dtype)
            logical_bytes = (m * n + m) * size
        elif op == "softmax":
            fn = lambda: x.softmax(dim=-1)
            ref = x.double().softmax(dim=-1).to(dtype)
            logical_bytes = 2 * m * n * size
            notes += " Logical traffic is a one-read/one-write lower bound, not measured DRAM traffic."
        elif op == "transpose":
            fn = lambda: x.T.contiguous()
            ref = x.T.contiguous()
            logical_bytes = 2 * m * n * size
            notes += " Physical transpose; the reference includes contiguous(), not just a stride-changing view."
        elif op == "layer_norm":
            weight, bias = rand(n), rand(n)
            args = (x, weight, bias)
            fn = lambda: F.layer_norm(x, (n,), weight, bias, eps=1e-5)
            ref = F.layer_norm(x.double(), (n,), weight.double(), bias.double(), eps=1e-5).to(dtype)
        elif op == "rms_norm":
            weight = rand(n)
```

```
            args = (x, weight)
            fn = lambda: F.rms_norm(x, (n,), weight, eps=1e-5)
            xd = x.double()
            ref = (xd * torch.rsqrt(xd.square().mean(dim=-1, keepdim=True) + 1e-5) * weight.double()).to(dtype)
        else:
            raise ValueError(f"unknown op: {op}")
    return Case(op, shape, args, fn, ref, logical_bytes, flops, notes)


def make_callable(case: Case, backend: str, variant: str = "tiled") -> Callable[[], torch.Tensor]:
    if backend == "torch":
        return case.torch_fn
    if backend not in ("cuda", "triton"):
        raise ValueError(f"unknown backend: {backend}")
    module = importlib.import_module(f"gpu_prep.{backend}_ops")
    name = f"{case.op}_{variant}" if case.op in ("transpose", "matmul") else case.op
    operation = getattr(module, name)
    return lambda: operation(*case.inputs)


def summarize_samples(samples_ms: list[float]) -> dict:
    """p95 is an interpolated quantile of repeat averages, not request-tail latency."""
    if not samples_ms or any(not math.isfinite(x) or x <= 0 for x in samples_ms):
        raise ValueError("latency samples must be nonempty, finite, and positive")
    ordered = sorted(samples_ms)
    position = 0.95 * (len(ordered) - 1)
    low, high = math.floor(position), math.ceil(position)
    p95 = ordered[low] + (position - low) * (ordered[high] - ordered[low])
    return {
        "median_ms": statistics.median(ordered), "min_ms": ordered[0],
        "p95_of_repeat_means_ms": p95, "samples_ms": samples_ms,
    }


@torch.inference_mode()
def check_output(fn: Callable[[], torch.Tensor], reference: torch.Tensor) -> dict:
    out = fn()  # A first compilation or extension build can happen here.
    tolerance = {
        torch.float32: (3e-4, 3e-4),
        torch.float16: (8e-3, 8e-3),
        torch.bfloat16: (5e-2, 5e-2),
    }[reference.dtype]
    rtol, atol = tolerance
    torch.testing.assert_close(out, reference, rtol=rtol, atol=atol)
    error = (out.double() - reference.double()).abs()
    denominator = reference.double().abs().clamp_min(atol)
    return {
        "passed": True, "rtol": rtol, "atol": atol,
        "max_absolute_error": error.max().item() if error.numel() else 0.0,
        "max_relative_error_with_atol_floor": (error / denominator).max().item() if error.numel() else 0.0,
    }


@torch.inference_mode()
def measure_cuda(
    fn: Callable[[], torch.Tensor], device: torch.device,
    warmup: int = 25, iterations: int = 100, repeats: int = 7,
) -> dict:
    """Report GPU-event and synchronized host-wall averages separately.

    Contract: fn launches all its work on this device's current stream and returns
    a CUDA tensor. Cross-stream work needs explicit dependencies and a different
    measurement plan. Events may include GPU idle gaps caused by host dispatch.
    """
    if device.type != "cuda" or not torch.cuda.is_available():
        raise RuntimeError("A real NVIDIA CUDA device is required; CPU timings are not substituted.")
    if device.index is None:
```

```
        device = torch.device("cuda", torch.cuda.current_device())
    if min(warmup, iterations, repeats) <= 0:
        raise ValueError("warmup, iterations, and repeats must be positive")
    event_samples, wall_samples = [], []
    with torch.cuda.device(device):
        for _ in range(warmup):
            last = fn()
        if not isinstance(last, torch.Tensor) or not last.is_cuda or last.device != device:
            raise ValueError("fn must return a tensor on the selected CUDA device")
        torch.cuda.synchronize(device)
        # Create events before timing. Their initialization is forced once.
        start, end = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
        start.record()
        end.record()
        end.synchronize()
        for _ in range(repeats):
            start.record()
            for _ in range(iterations):
                last = fn()
            end.record()
            end.synchronize()
            event_samples.append(start.elapsed_time(end) / iterations)
        for _ in range(repeats):
            torch.cuda.synchronize(device)
            start_wall = time.perf_counter()
            for _ in range(iterations):
                last = fn()
            torch.cuda.synchronize(device)
            wall_samples.append((time.perf_counter() - start_wall) * 1000 / iterations)
    return {
        "gpu_event": summarize_samples(event_samples),
        "synchronized_host_wall": summarize_samples(wall_samples),
        "warmup_calls": warmup, "calls_per_repeat": iterations, "repeats": repeats,
        "measurement_scope": "Resident-input forward wrapper, including metadata checks, output allocation, and
launches; JIT and numerical correctness checks excluded.",
        "p95_scope": "Quantile over repeat averages, not p95 latency of individual requests.",
        "cache_policy": "Repeatedly reuse the same inputs; caches may be warm. No explicit cache flush.",
    }


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--op", choices=DEFAULT_SHAPES, default="matmul")
    parser.add_argument("--shape", type=parse_shape)
    parser.add_argument("--backends", default="torch,cuda,triton")
    parser.add_argument("--variant", choices=("naive", "tiled"), default="tiled")
    parser.add_argument("--dtype", choices=DTYPES, default="float32")
    parser.add_argument("--device", default="cuda:0")
    parser.add_argument("--warmup", type=int, default=25)
    parser.add_argument("--iterations", type=int, default=100)
    parser.add_argument("--repeats", type=int, default=7)
    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument("--output", type=Path)
    args = parser.parse_args()
    if not torch.cuda.is_available():
        parser.error("No NVIDIA CUDA device is available. Use CPU tests for formulas, not GPU timing.")
    device = torch.device(args.device)
    if device.type != "cuda":
        parser.error("--device must be CUDA")
    if device.index is None:
        device = torch.device("cuda", torch.cuda.current_device())
    backends = [name.strip() for name in args.backends.split(",")]
    if not backends or len(set(backends)) != len(backends) or any(x not in ("torch", "cuda", "triton") for x in
backends):
        parser.error("--backends must be distinct names chosen from torch,cuda,triton")
    if "cuda" in backends and args.dtype != "float32":
        parser.error("The CUDA C++ teaching backend supports float32; choose torch,triton for half/bfloat16.")
```

```
    if "triton" in backends and os.environ.get("TRITON_INTERPRET") == "1":
        parser.error("Unset TRITON_INTERPRET for GPU timing. Use tools/check_triton_interpreter.py in a separate
process for CPU math checks.")
    if min(args.warmup, args.iterations, args.repeats) <= 0:
        parser.error("warmup, iterations, and repeats must be positive")
    torch.manual_seed(args.seed)
    # Compare IEEE FP32 baselines; do not silently compare TF32 with FP32.
    torch.set_float32_matmul_precision("highest")
    torch.backends.cuda.matmul.allow_tf32 = False
    torch.backends.cuda.matmul.allow_fp16_reduced_precision_reduction = False
    torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction = False
    shape = args.shape or DEFAULT_SHAPES[args.op]
    with torch.cuda.device(device), torch.inference_mode():
        case = build_case(args.op, shape, DTYPES[args.dtype], device)
        functions = {name: make_callable(case, name, args.variant) for name in backends}
        checks = {name: check_output(fn, case.reference) for name, fn in functions.items()}
        # All backends are built and checked before timing any backend.
        order = list(backends)
        random.Random(args.seed).shuffle(order)
        results = {}
        for name in order:
            measured = measure_cuda(functions[name], device, args.warmup, args.iterations, args.repeats)
            seconds = measured["gpu_event"]["median_ms"] / 1000
            measured["correctness"] = checks[name]
            measured["logical_effective_GB_per_s"] = case.logical_bytes / seconds / 1e9 if case.logical_bytes is
not None else None
            measured["estimated_TFLOP_per_s"] = case.flops / seconds / 1e12 if case.flops is not None else None
            results[name] = measured
    report = {
        "environment": collect_environment(),
        "case": {"op": case.op, "shape": case.shape, "dtype": args.dtype, "variant": args.variant,
                 "logical_bytes": case.logical_bytes, "estimated_flops": case.flops, "notes": case.notes},
        "measurement_order": order, "seed": args.seed, "results": results,
        "traffic_warning": "Logical bytes are an algorithmic model, not Nsight-measured HBM traffic; cache reuse
can make effective bandwidth exceed HBM bandwidth.",
    }
    encoded = json.dumps(report, ensure_ascii=False, indent=2)
    if args.output is not None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(encoded + "\n", encoding="utf-8")
    print(encoded)


if __name__ == "__main__":
    main()
```

gpu_prep/check_env.py

```
"""Print environment information without compiling or reserving a GPU."""
from __future__ import annotations

import argparse
import importlib.metadata
import json
import os
import platform
import shutil
import sys


def package_version(name: str) -> str | None:
    try:
        return importlib.metadata.version(name)
    except importlib.metadata.PackageNotFoundError:
```

```
        return None


def collect_environment() -> dict:
    result = {
        "python": sys.version.split()[0],
        "platform": platform.platform(),
        "torch": package_version("torch"),
        "triton": package_version("triton"),
        "pytest": package_version("pytest"),
        "nvcc": shutil.which("nvcc"),
        "nvidia_smi": shutil.which("nvidia-smi"),
        "cuda_available": False,
        "triton_interpret": os.environ.get("TRITON_INTERPRET", "0"),
        "devices": [],
    }
    if result["torch"] is None:
        return result
    import torch
    result["torch_cuda_runtime"] = torch.version.cuda
    result["cuda_available"] = torch.cuda.is_available()
    result["matmul_float32_precision"] = torch.get_float32_matmul_precision()
    result["matmul_allow_tf32"] = torch.backends.cuda.matmul.allow_tf32
    result["matmul_allow_fp16_reduced_precision_reduction"] =
torch.backends.cuda.matmul.allow_fp16_reduced_precision_reduction
    result["matmul_allow_bf16_reduced_precision_reduction"] =
torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction
    if torch.cuda.is_available():
        for i in range(torch.cuda.device_count()):
            properties = torch.cuda.get_device_properties(i)
            result["devices"].append({
                "index": i,
                "name": properties.name,
                "compute_capability": [properties.major, properties.minor],
                "total_memory_bytes": properties.total_memory,
                "multiprocessor_count": properties.multi_processor_count,
            })
    return result


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--require-gpu", action="store_true")
    args = parser.parse_args()
    info = collect_environment()
    print(json.dumps(info, ensure_ascii=False, indent=2))
    if args.require_gpu and not info["cuda_available"]:
        raise SystemExit("This command requires an NVIDIA CUDA device; no CPU timing is substituted.")


if __name__ == "__main__":
    main()
```

gpu_prep/csrc/bindings.cpp

```
#include <torch/extension.h>

#include "ops.h"

namespace py = pybind11;

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
  m.doc() = "Original CUDA FP32 forward-only interview teaching kernels";
  m.def("vector_add", &gpu_prep::vector_add_cuda, py::arg("x"), py::arg("y"));
```

```
  m.def("bias_relu", &gpu_prep::bias_relu_cuda, py::arg("x"), py::arg("bias"));
  m.def("row_sum", &gpu_prep::row_sum_cuda, py::arg("x"));
  m.def("row_max", &gpu_prep::row_max_cuda, py::arg("x"));
  m.def("softmax", &gpu_prep::softmax_cuda, py::arg("x"));
  m.def("transpose_naive", &gpu_prep::transpose_naive_cuda, py::arg("x"));
  m.def("transpose_tiled", &gpu_prep::transpose_tiled_cuda, py::arg("x"));
  m.def("matmul_naive", &gpu_prep::matmul_naive_cuda, py::arg("a"), py::arg("b"));
  m.def("matmul_tiled", &gpu_prep::matmul_tiled_cuda, py::arg("a"), py::arg("b"));
  m.def("layer_norm", &gpu_prep::layer_norm_cuda, py::arg("x"),
        py::arg("weight"), py::arg("bias"), py::arg("eps") = 1e-5);
  m.def("rms_norm", &gpu_prep::rms_norm_cuda, py::arg("x"),
        py::arg("weight"), py::arg("eps") = 1e-5);
  m.def("attention", &gpu_prep::attention_cuda, py::arg("q"), py::arg("k"),
        py::arg("v"), py::arg("causal") = false,
        py::arg("query_start") = 0);
}
```

gpu_prep/csrc/kernels.cu

```
#include "ops.h"

#include <c10/core/GradMode.h>
#include <c10/cuda/CUDAException.h>
#include <c10/cuda/CUDAGuard.h>
#include <c10/cuda/CUDAStream.h>
#include <cuda_runtime.h>
#include <math_constants.h>

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>

namespace gpu_prep {
namespace {

constexpr int kThreads = 256;       // reduction 要求为 2 的幂。
constexpr int kGridCap = 65535;     // 用 grid-stride loop 覆盖更大的输入。
constexpr int kTransposeTile = 32;
constexpr int kTransposeRows = 8;
constexpr int kMatmulTile = 16;
constexpr int64_t kMaxAttentionKeys = 4096;

// 使用除法和余数求 ceil，避免 (n + d - 1) 的整数溢出。
int64_t ceil_div(int64_t n, int64_t d) { return n / d + (n % d != 0); }

int blocks_for(int64_t work, int64_t work_per_block = 1) {
  return static_cast<int>(
      std::min<int64_t>(ceil_div(work, work_per_block), kGridCap));
}

int64_t checked_product(int64_t a, int64_t b, const char* what) {
  TORCH_CHECK(a >= 0 && b >= 0, what, ": negative dimension");
  TORCH_CHECK(a == 0 || b <= std::numeric_limits<int64_t>::max() / a,
              what, ": number of elements overflows int64");
  return a * b;
}

void check_tensor(const at::Tensor& x, const char* name) {
  TORCH_CHECK(x.is_cuda(), name, " must be a CUDA tensor");
  TORCH_CHECK(x.scalar_type() == at::kFloat, name, " must be torch.float32");
  TORCH_CHECK(x.is_contiguous(), name, " must be contiguous");
  TORCH_CHECK(!c10::GradMode::is_enabled() || !x.requires_grad(), name,
              ": these teaching kernels are forward-only; use torch.no_grad() "
```

```
              "for inference, or implement/register a backward formula");
}

void check_same_device(const at::Tensor& a, const at::Tensor& b,
                       const char* name) {
  TORCH_CHECK(a.device() == b.device(), name, " must be on ", a.device());
}

void check_matrix(const at::Tensor& x, const char* name) {
  check_tensor(x, name);
  TORCH_CHECK(x.dim() == 2, name, " must have shape [M, N]");
}

float check_eps(double eps) {
  const float e = static_cast<float>(eps);
  TORCH_CHECK(std::isfinite(eps) && eps > 0.0 && std::isfinite(e) && e > 0.0f,
              "eps must be finite, positive, and representable in float32");
  return e;
}

void check_norm_weight(const at::Tensor& x, const at::Tensor& weight) {
  check_tensor(x, "x");
  TORCH_CHECK(x.dim() >= 1, "x must have at least one dimension");
  TORCH_CHECK(x.size(-1) > 0, "normalization width must be positive");
  check_tensor(weight, "weight");
  check_same_device(x, weight, "weight");
  TORCH_CHECK(weight.dim() == 1 && weight.numel() == x.size(-1),
              "weight must have shape [x.size(-1)]");
}

// 所有线程都必须调用。本例用完整 shared-memory 树，便于观察同步。
// 最后一个 barrier 保证每个线程都读到了结果，scratch 才能被下次复用。
template <bool IsMax>
__device__ float block_reduce(float value, float* scratch) {
  const int tid = threadIdx.x;
  scratch[tid] = value;
  __syncthreads();
  for (int stride = kThreads / 2; stride > 0; stride /= 2) {
    if (tid < stride) {
      scratch[tid] = IsMax ? fmaxf(scratch[tid], scratch[tid + stride])
                           : scratch[tid] + scratch[tid + stride];
    }
    __syncthreads();
  }
  const float result = scratch[0];
  __syncthreads();
  return result;
}

__global__ void vector_add_kernel(const float* x, const float* y, float* out,
                                  int64_t n) {
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < n; i += step) {
    out[i] = x[i] + y[i];
  }
}

__global__ void bias_relu_kernel(const float* x, const float* bias, float* out,
                                 int64_t numel, int64_t width) {
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < numel; i += step) {
    const float z = x[i] + bias[i % width];
    // 正常有限值下等价于 ReLU；z 是 NaN 时本分支仍保留 NaN。
    out[i] = z < 0.0f ? 0.0f : z;
  }
```

```
}

template <bool IsMax>
__global__ void row_reduce_kernel(const float* x, float* out,
                                  int64_t rows, int64_t width) {
  __shared__ float scratch[kThreads];
  for (int64_t row = blockIdx.x; row < rows; row += gridDim.x) {
    float local = IsMax ? -CUDART_INF_F : 0.0f;
    // 相邻线程读相邻列；width 不需要是 2 的幂。
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x) {
      const float value = x[row * width + col];
      local = IsMax ? fmaxf(local, value) : local + value;
    }
    const float reduced = block_reduce<IsMax>(local, scratch);
    if (threadIdx.x == 0) out[row] = reduced;
  }
}

__global__ void softmax_kernel(const float* x, float* out,
                               int64_t rows, int64_t width) {
  __shared__ float scratch[kThreads];
  for (int64_t row = blockIdx.x; row < rows; row += gridDim.x) {
    const int64_t base = row * width;
    float local_max = -CUDART_INF_F;
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
      local_max = fmaxf(local_max, x[base + col]);
    const float row_max = block_reduce<true>(local_max, scratch);

    float local_sum = 0.0f;
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
      local_sum += expf(x[base + col] - row_max);
    const float sum = block_reduce<false>(local_sum, scratch);

    // 为了让实现适用于任意 width，不把整行放进 shared memory。
    // 代价是再次读取 x 和再次计算 exp；优化版可在寄存器/共享内存中复用。
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
      out[base + col] = expf(x[base + col] - row_max) / sum;
  }
}

__global__ void transpose_naive_kernel(const float* x, float* out,
                                       int64_t rows, int64_t cols) {
  const int64_t n = rows * cols;
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < n; i += step) {
    const int64_t row = i / cols;
    const int64_t col = i % cols;
    out[col * rows + row] = x[i];
  }
}

__global__ void transpose_tiled_kernel(const float* x, float* out,
                                       int64_t rows, int64_t cols,
                                       int64_t tile_cols, int64_t tiles) {
  // 针对经典 FP32 32-bank 模型加一列 padding。
  __shared__ float tile[kTransposeTile][kTransposeTile + 1];
  for (int64_t tile_id = blockIdx.x; tile_id < tiles; tile_id += gridDim.x) {
    const int64_t tile_row = tile_id / tile_cols;
    const int64_t tile_col = tile_id % tile_cols;
    const int64_t in_col = tile_col * kTransposeTile + threadIdx.x;
    for (int j = 0; j < kTransposeTile; j += kTransposeRows) {
      const int r = threadIdx.y + j;
      const int64_t in_row = tile_row * kTransposeTile + r;
      tile[r][threadIdx.x] = (in_row < rows && in_col < cols)
                             ? x[in_row * cols + in_col] : 0.0f;
    }
```

```
    __syncthreads();  // 写 tile 的线程与读转置 tile 的线程不同。

    const int64_t out_col = tile_row * kTransposeTile + threadIdx.x;
    for (int j = 0; j < kTransposeTile; j += kTransposeRows) {
      const int r = threadIdx.y + j;
      const int64_t out_row = tile_col * kTransposeTile + r;
      if (out_row < cols && out_col < rows)
        out[out_row * rows + out_col] = tile[threadIdx.x][r];
    }
    __syncthreads();  // 防止下一 tile 提前覆盖仍在被读取的 shared memory。
  }
}

__global__ void matmul_naive_kernel(const float* a, const float* b, float* out,
                                    int64_t m, int64_t n, int64_t k) {
  const int64_t total = m * n;
  const int64_t first = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
  const int64_t step = static_cast<int64_t>(gridDim.x) * blockDim.x;
  for (int64_t i = first; i < total; i += step) {
    const int64_t row = i / n;
    const int64_t col = i % n;
    float acc = 0.0f;
    for (int64_t kk = 0; kk < k; ++kk)
      acc = fmaf(a[row * k + kk], b[kk * n + col], acc);
    out[i] = acc;
  }
}

__global__ void matmul_tiled_kernel(const float* a, const float* b, float* out,
                                    int64_t m, int64_t n, int64_t k,
                                    int64_t tile_cols, int64_t tiles) {
  __shared__ float as[kMatmulTile][kMatmulTile];
  __shared__ float bs[kMatmulTile][kMatmulTile];
  const int tx = threadIdx.x;
  const int ty = threadIdx.y;
  const int64_t stages = k / kMatmulTile + (k % kMatmulTile != 0);

  for (int64_t tile_id = blockIdx.x; tile_id < tiles; tile_id += gridDim.x) {
    const int64_t row = (tile_id / tile_cols) * kMatmulTile + ty;
    const int64_t col = (tile_id % tile_cols) * kMatmulTile + tx;
    float acc = 0.0f;
    for (int64_t stage = 0; stage < stages; ++stage) {
      const int64_t a_col = stage * kMatmulTile + tx;
      const int64_t b_row = stage * kMatmulTile + ty;
      // 尾块显式填零；不能让越界线程提前退出而跳过同步。
      as[ty][tx] = (row < m && a_col < k) ? a[row * k + a_col] : 0.0f;
      bs[ty][tx] = (b_row < k && col < n) ? b[b_row * n + col] : 0.0f;
      __syncthreads();
      #pragma unroll
      for (int kk = 0; kk < kMatmulTile; ++kk)
        acc = fmaf(as[ty][kk], bs[kk][tx], acc);
      __syncthreads();  // 所有乘加完成后，才允许覆盖 as/bs。
    }
    if (row < m && col < n) out[row * n + col] = acc;
  }
}

template <bool IsRms>
__global__ void norm_kernel(const float* x, const float* weight,
                            const float* bias, float* out,
                            int64_t rows, int64_t width, float eps) {
  __shared__ float scratch[kThreads];
  for (int64_t row = blockIdx.x; row < rows; row += gridDim.x) {
    const int64_t base = row * width;
    float mean = 0.0f;
    if constexpr (!IsRms) {
      float local_sum = 0.0f;
```

```
      for (int64_t col = threadIdx.x; col < width; col += blockDim.x)
        local_sum += x[base + col];
      mean = block_reduce<false>(local_sum, scratch) / static_cast<float>(width);
    }
    float local_sq = 0.0f;
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x) {
      const float centered = x[base + col] - mean;
      local_sq = fmaf(centered, centered, local_sq);
    }
    const float variance = block_reduce<false>(local_sq, scratch)
                           / static_cast<float>(width);
    const float inv_std = rsqrtf(variance + eps);
    for (int64_t col = threadIdx.x; col < width; col += blockDim.x) {
      const float normalized = (x[base + col] - mean) * inv_std;
      const float affine = normalized * weight[col];
      if constexpr (IsRms) out[base + col] = affine;
      else out[base + col] = affine + bias[col];
    }
  }
}

// 教学版 dense attention：每个 CTA 负责一个 query，shared memory 保存 S 个分数。
// 这不是 FlashAttention；并未做 QK/PV 矩阵分块或 online softmax。
__global__ void attention_kernel(const float* q, const float* k, const float* v,
                                 float* out, int64_t query_rows,
                                 int64_t query_len, int64_t key_len,
                                 int64_t head_dim, bool causal,
                                 int64_t query_start) {
  extern __shared__ float scores[];
  __shared__ float scratch[kThreads];
  const float scale = rsqrtf(static_cast<float>(head_dim));
  for (int64_t qr = blockIdx.x; qr < query_rows; qr += gridDim.x) {
    const int64_t bh = qr / query_len;
    const int64_t qi = qr % query_len;
    const int64_t q_base = qr * head_dim;
    const int64_t kv_base = bh * key_len * head_dim;
    float local_max = -CUDART_INF_F;
    for (int64_t key = threadIdx.x; key < key_len; key += blockDim.x) {
      float score = -CUDART_INF_F;
      if (!causal || key <= query_start + qi) {
        float dot = 0.0f;
        for (int64_t d = 0; d < head_dim; ++d)
          dot = fmaf(q[q_base + d], k[kv_base + key * head_dim + d], dot);
        score = dot * scale;
      }
      scores[key] = score;
      local_max = fmaxf(local_max, score);
    }
    const float maximum = block_reduce<true>(local_max, scratch);

    float local_sum = 0.0f;
    for (int64_t key = threadIdx.x; key < key_len; key += blockDim.x) {
      const float value = expf(scores[key] - maximum);
      scores[key] = value;
      local_sum += value;
    }
    const float inv_sum = 1.0f / block_reduce<false>(local_sum, scratch);

    for (int64_t d = threadIdx.x; d < head_dim; d += blockDim.x) {
      float acc = 0.0f;
      for (int64_t key = 0; key < key_len; ++key)
        acc = fmaf(scores[key] * inv_sum,
                   v[kv_base + key * head_dim + d], acc);
      out[q_base + d] = acc;
    }
    __syncthreads();  // 下一 query 不能覆盖其他线程仍在读取的 scores。
  }
```

```
}

at::Tensor transpose_impl(const at::Tensor& x, bool tiled) {
  check_matrix(x, "x");
  const c10::cuda::CUDAGuard guard(x.device());
  const int64_t m = x.size(0), n = x.size(1);
  auto out = at::empty({n, m}, x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  if (tiled) {
    const int64_t tile_cols = ceil_div(n, kTransposeTile);
    const int64_t tiles = checked_product(ceil_div(m, kTransposeTile),
                                          tile_cols, "transpose tiles");
    transpose_tiled_kernel<<<blocks_for(tiles),
                             dim3(kTransposeTile, kTransposeRows), 0, stream>>>(
        x.data_ptr<float>(), out.data_ptr<float>(), m, n, tile_cols, tiles);
  } else {
    transpose_naive_kernel<<<blocks_for(x.numel(), kThreads), kThreads, 0, stream>>>(
        x.data_ptr<float>(), out.data_ptr<float>(), m, n);
  }
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor matmul_impl(const at::Tensor& a, const at::Tensor& b, bool tiled) {
  check_matrix(a, "a");
  check_matrix(b, "b");
  check_same_device(a, b, "b");
  TORCH_CHECK(a.size(1) == b.size(0), "matmul inner dimensions must match");
  const c10::cuda::CUDAGuard guard(a.device());
  const int64_t m = a.size(0), k = a.size(1), n = b.size(1);
  const int64_t total = checked_product(m, n, "matmul output");
  // 空的求和有单位元 0；不能把未初始化的 empty 当作 K=0 的答案。
  if (k == 0) return at::zeros({m, n}, a.options().requires_grad(false));
  auto out = at::empty({m, n}, a.options().requires_grad(false));
  if (total == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(a.get_device()).stream();
  if (tiled) {
    const int64_t tile_cols = ceil_div(n, kMatmulTile);
    const int64_t tiles = checked_product(ceil_div(m, kMatmulTile),
                                          tile_cols, "matmul tiles");
    matmul_tiled_kernel<<<blocks_for(tiles), dim3(kMatmulTile, kMatmulTile),
                          0, stream>>>(a.data_ptr<float>(), b.data_ptr<float>(),
                                       out.data_ptr<float>(), m, n, k,
                                       tile_cols, tiles);
  } else {
    matmul_naive_kernel<<<blocks_for(total, kThreads), kThreads, 0, stream>>>(
        a.data_ptr<float>(), b.data_ptr<float>(), out.data_ptr<float>(), m, n, k);
  }
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

template <bool IsMax>
at::Tensor row_reduce_impl(const at::Tensor& x) {
  check_matrix(x, "x");
  TORCH_CHECK(x.size(1) > 0, "reduction width N must be positive");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty({x.size(0)}, x.options().requires_grad(false));
  if (x.size(0) == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  row_reduce_kernel<IsMax><<<blocks_for(x.size(0)), kThreads, 0, stream>>>(
      x.data_ptr<float>(), out.data_ptr<float>(), x.size(0), x.size(1));
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}
```

```
} // namespace

at::Tensor vector_add_cuda(const at::Tensor& x, const at::Tensor& y) {
  check_tensor(x, "x");
  check_tensor(y, "y");
  check_same_device(x, y, "y");
  TORCH_CHECK(x.sizes() == y.sizes(), "x and y must have identical shapes");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  vector_add_kernel<<<blocks_for(x.numel(), kThreads), kThreads, 0, stream>>>(
      x.data_ptr<float>(), y.data_ptr<float>(), out.data_ptr<float>(), x.numel());
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor bias_relu_cuda(const at::Tensor& x, const at::Tensor& bias) {
  check_matrix(x, "x");
  check_tensor(bias, "bias");
  check_same_device(x, bias, "bias");
  TORCH_CHECK(bias.dim() == 1 && bias.numel() == x.size(1),
              "bias must have shape [N] for x[M, N]");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;  // 也避免在 width=0 时取模。
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  bias_relu_kernel<<<blocks_for(x.numel(), kThreads), kThreads, 0, stream>>>(
      x.data_ptr<float>(), bias.data_ptr<float>(), out.data_ptr<float>(),
      x.numel(), x.size(1));
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor row_sum_cuda(const at::Tensor& x) { return row_reduce_impl<false>(x); }
at::Tensor row_max_cuda(const at::Tensor& x) { return row_reduce_impl<true>(x); }

at::Tensor softmax_cuda(const at::Tensor& x) {
  check_matrix(x, "x");
  TORCH_CHECK(x.size(1) > 0, "softmax width N must be positive");
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.size(0) == 0) return out;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  softmax_kernel<<<blocks_for(x.size(0)), kThreads, 0, stream>>>(
      x.data_ptr<float>(), out.data_ptr<float>(), x.size(0), x.size(1));
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor transpose_naive_cuda(const at::Tensor& x) { return transpose_impl(x, false); }
at::Tensor transpose_tiled_cuda(const at::Tensor& x) { return transpose_impl(x, true); }
at::Tensor matmul_naive_cuda(const at::Tensor& a, const at::Tensor& b) {
  return matmul_impl(a, b, false);
}
at::Tensor matmul_tiled_cuda(const at::Tensor& a, const at::Tensor& b) {
  return matmul_impl(a, b, true);
}

at::Tensor layer_norm_cuda(const at::Tensor& x, const at::Tensor& weight,
                           const at::Tensor& bias, double eps) {
  check_norm_weight(x, weight);
  check_tensor(bias, "bias");
  check_same_device(x, bias, "bias");
  TORCH_CHECK(bias.dim() == 1 && bias.numel() == x.size(-1),
              "bias must have shape [x.size(-1)]");
  const float e = check_eps(eps);
```

```
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const int64_t width = x.size(-1), rows = x.numel() / width;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  norm_kernel<false><<<blocks_for(rows), kThreads, 0, stream>>>(
      x.data_ptr<float>(), weight.data_ptr<float>(), bias.data_ptr<float>(),
      out.data_ptr<float>(), rows, width, e);
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor rms_norm_cuda(const at::Tensor& x, const at::Tensor& weight, double eps) {
  check_norm_weight(x, weight);
  const float e = check_eps(eps);
  const c10::cuda::CUDAGuard guard(x.device());
  auto out = at::empty(x.sizes(), x.options().requires_grad(false));
  if (x.numel() == 0) return out;
  const int64_t width = x.size(-1), rows = x.numel() / width;
  const auto stream = c10::cuda::getCurrentCUDAStream(x.get_device()).stream();
  norm_kernel<true><<<blocks_for(rows), kThreads, 0, stream>>>(
      x.data_ptr<float>(), weight.data_ptr<float>(), nullptr, out.data_ptr<float>(),
      rows, width, e);
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

at::Tensor attention_cuda(const at::Tensor& q, const at::Tensor& k,
                          const at::Tensor& v, bool causal, int64_t query_start) {
  check_tensor(q, "q");
  check_tensor(k, "k");
  check_tensor(v, "v");
  check_same_device(q, k, "k");
  check_same_device(q, v, "v");
  TORCH_CHECK(q.dim() == 4 && k.dim() == 4 && v.dim() == 4,
              "attention expects q[B,H,L,D], k/v[B,H,S,D]");
  TORCH_CHECK(q.size(0) == k.size(0) && q.size(1) == k.size(1)
                  && q.size(3) == k.size(3) && k.sizes() == v.sizes(),
              "q/k/v batch, head count and head dimension must agree; k/v shapes match");
  TORCH_CHECK(q.size(3) > 0, "head dimension D must be positive");
  TORCH_CHECK(k.size(2) > 0 && k.size(2) <= kMaxAttentionKeys,
              "teaching attention requires 1 <= key length S <= 4096");
  TORCH_CHECK(query_start >= 0, "query_start must be nonnegative");
  TORCH_CHECK(q.size(2) == 0 ||
                  query_start <= std::numeric_limits<int64_t>::max() - (q.size(2) - 1),
              "query_start + query index overflows int64");
  const c10::cuda::CUDAGuard guard(q.device());
  auto out = at::empty(q.sizes(), q.options().requires_grad(false));
  if (q.numel() == 0) return out;
  const int64_t query_rows = q.numel() / q.size(3);
  const auto stream = c10::cuda::getCurrentCUDAStream(q.get_device()).stream();
  const size_t shared_bytes = static_cast<size_t>(k.size(2)) * sizeof(float);
  attention_kernel<<<blocks_for(query_rows), kThreads, shared_bytes, stream>>>(
      q.data_ptr<float>(), k.data_ptr<float>(), v.data_ptr<float>(),
      out.data_ptr<float>(), query_rows, q.size(2), k.size(2), q.size(3),
      causal, query_start);
  C10_CUDA_KERNEL_LAUNCH_CHECK();
  return out;
}

}  // namespace gpu_prep
```

gpu_prep/csrc/ops.h

```h
#pragma once

#include <ATen/ATen.h>
#include <cstdint>

// 原始教学实现：连续 CUDA FP32 张量；仅前向，不注册 autograd。
namespace gpu_prep {

at::Tensor vector_add_cuda(const at::Tensor& x, const at::Tensor& y);
at::Tensor bias_relu_cuda(const at::Tensor& x, const at::Tensor& bias);
at::Tensor row_sum_cuda(const at::Tensor& x);
at::Tensor row_max_cuda(const at::Tensor& x);
at::Tensor softmax_cuda(const at::Tensor& x);
at::Tensor transpose_naive_cuda(const at::Tensor& x);
at::Tensor transpose_tiled_cuda(const at::Tensor& x);
at::Tensor matmul_naive_cuda(const at::Tensor& a, const at::Tensor& b);
at::Tensor matmul_tiled_cuda(const at::Tensor& a, const at::Tensor& b);
at::Tensor layer_norm_cuda(const at::Tensor& x, const at::Tensor& weight,
                           const at::Tensor& bias, double eps);
at::Tensor rms_norm_cuda(const at::Tensor& x, const at::Tensor& weight,
                         double eps);
at::Tensor attention_cuda(const at::Tensor& q, const at::Tensor& k,
                          const at::Tensor& v, bool causal,
                          int64_t query_start);

}  // namespace gpu_prep
```

gpu_prep/cuda_ops.py

```python
"""原始 CUDA C++ 教学 kernels 的延迟编译入口。

约定：CUDA、torch.float32、contiguous、仅前向；第一次调用时编译，import 不编译。
需要带 CUDA 支持的 PyTorch、匹配的 CUDA toolkit/nvcc、C++ 编译器、Ninja。
"""

from __future__ import annotations

from functools import lru_cache
import os
from pathlib import Path
from types import ModuleType

import torch


@lru_cache(maxsize=1)
def load_extension() -> ModuleType:
    """显式预编译入口；不要把首次编译耗时计入 kernel benchmark。"""
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA teaching kernels require an available NVIDIA GPU")
    # 留到实际调用再导入：CPU 用户阅读/导入本包时不会触发构建。
    from torch.utils.cpp_extension import CUDA_HOME, load

    if CUDA_HOME is None:
        raise RuntimeError(
            "CUDA toolkit/nvcc not found. A PyTorch CUDA wheel alone does not "
            "supply the CUDA extension compiler; configure CUDA_HOME."
        )
    source_dir = Path(__file__).resolve().parent / "csrc"
    return load(
```

```
        name="together_gpu_prep_cuda_v1",
        sources=[str(source_dir / "bindings.cpp"), str(source_dir / "kernels.cu")],
        extra_include_paths=[str(source_dir)],
        extra_cflags=["-O3", "-std=c++17"],
        # 不使用 --use_fast_math；保留便于参考比较的 FP32 数学路径。
        extra_cuda_cflags=["-O3", "-std=c++17", "-lineinfo"],
        with_cuda=True,
        verbose=os.environ.get("GPU_PREP_VERBOSE_BUILD", "0") == "1",
    )


def vector_add(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """同形状张量逐元素相加；支持零元素，输出形状与 x 相同。"""
    return load_extension().vector_add(x, y)


def bias_relu(x: torch.Tensor, bias: torch.Tensor) -> torch.Tensor:
    """x[M,N] + bias[N] 后做 ReLU；支持 M=0 或 N=0。"""
    return load_extension().bias_relu(x, bias)


def row_sum(x: torch.Tensor) -> torch.Tensor:
    """x[M,N] -> [M]，沿列求和；要求 N>0，M 可以为 0。"""
    return load_extension().row_sum(x)


def row_max(x: torch.Tensor) -> torch.Tensor:
    """x[M,N] -> [M]，沿列取 max；要求 N>0，有限值输入。"""
    return load_extension().row_max(x)


def softmax(x: torch.Tensor) -> torch.Tensor:
    """数值稳定的行 softmax；要求 x[M,N]、N>0、有限值输入。"""
    return load_extension().softmax(x)


def transpose_naive(x: torch.Tensor) -> torch.Tensor:
    """朴素物理转置 x[M,N] -> contiguous [N,M]，不是只改 stride。"""
    return load_extension().transpose_naive(x)


def transpose_tiled(x: torch.Tensor) -> torch.Tensor:
    """32x32 shared tile + FP32 padding 的物理转置。"""
    return load_extension().transpose_tiled(x)


def matmul_naive(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """a[M,K] @ b[K,N]；每线程一个输出，支持 K=0（全零输出）。"""
    return load_extension().matmul_naive(a, b)


def matmul_tiled(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """16x16 shared-memory tiled FP32 SIMT GEMM；没有 Tensor Core 路径。"""
    return load_extension().matmul_tiled(a, b)


def layer_norm(
    x: torch.Tensor, weight: torch.Tensor, bias: torch.Tensor, eps: float = 1e-5
) -> torch.Tensor:
    """沿最后一维做 LayerNorm，weight/bias 均为 [D]，方差使用分母 D。"""
    return load_extension().layer_norm(x, weight, bias, eps)


def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5) -> torch.Tensor:
    """沿最后一维做 RMSNorm；没有 mean subtraction 或 bias。"""
    return load_extension().rms_norm(x, weight, eps)
```

```
def attention(
    q: torch.Tensor,
    k: torch.Tensor,
    v: torch.Tensor,
    causal: bool = False,
    query_start: int = 0,
) -> torch.Tensor:
    """Dense attention 教学基线。

    q[B,H,L,D]，k/v[B,H,S,D]，1<=S<=4096，D>0。
    causal=True 时，第 i 个 query 只看 key_position <= query_start+i。
    单 token decode 通常传 query_start=S-1；默认 0 是从位置 0 开始的 query。
    不支持 dropout、padding mask、GQA、backward；不是 FlashAttention。
    """
    return load_extension().attention(q, k, v, causal, query_start)
```

gpu_prep/custom_ops.py

```
"""Exercise 11: one operator integrated with fake tensors and autograd.

Import registers metadata only: it does NOT import/compile the CUDA extension.
The CPU implementation makes registration and gradient checks possible on a
CPU-only machine. It is a reference path, not a GPU performance substitute.
"""

from __future__ import annotations

import torch
from torch import Tensor


def _validate(x: Tensor, bias: Tensor) -> None:
    if x.ndim != 2 or bias.ndim != 1:
        raise ValueError("bias_relu expects x[M,N] and bias[N]")
    if x.shape[1] != bias.shape[0]:
        raise ValueError("bias length must equal x.shape[1]")
    if x.device != bias.device:
        raise ValueError("x and bias must be on the same device")
    if x.dtype != bias.dtype:
        raise TypeError("x and bias must have the same dtype")
    if not x.is_contiguous() or not bias.is_contiguous():
        raise ValueError("this teaching operator requires contiguous inputs")
    if x.dtype not in (torch.float16, torch.bfloat16, torch.float32, torch.float64):
        raise TypeError("bias_relu supports floating tensors only")
    if x.device.type not in ("cpu", "cuda"):
        raise ValueError("bias_relu supports CPU and CUDA only")
    if x.device.type == "cuda" and x.dtype != torch.float32:
        raise TypeError("the CUDA teaching implementation supports float32 only")


@torch.library.custom_op("gpu_prep::bias_relu", mutates_args=())
def bias_relu(x: Tensor, bias: Tensor) -> Tensor:
    """Fresh contiguous y = relu(x + bias), without input mutation.

    CPU: fp16/bf16/fp32/fp64 reference. CUDA: fp32 custom kernel.
    Empty M or N is supported. Backward's derivative at exactly zero is zero.
    The backward contract and numerical tests use finite inputs.
    """
    _validate(x, bias)
    if x.device.type == "cuda":
        # Lazy import: CPU users do not need nvcc or the extension at import time.
        from gpu_prep.cuda_ops import bias_relu as cuda_bias_relu
```

```
        # The standalone CUDA exercise is explicitly forward-only; autograd is
        # supplied by this custom operator's registration below.
        with torch.no_grad():
            return cuda_bias_relu(x, bias)
    return torch.relu(x + bias)


@bias_relu.register_fake
def _bias_relu_fake(x: Tensor, bias: Tensor) -> Tensor:
    _validate(x, bias)
    # Match the real output's fresh, contiguous storage and metadata. Do not use
    # tensor values, .item(), .cpu(), or the actual CUDA kernel in a fake kernel.
    return torch.empty(x.shape, dtype=x.dtype, device=x.device)


def _setup_context(ctx, inputs, output) -> None:
    x, bias = inputs
    ctx.save_for_backward(x, bias)


def _backward(ctx, grad_output: Tensor) -> tuple[Tensor, Tensor]:
    x, bias = ctx.saved_tensors
    active = (x + bias) > 0
    # A branch is numerically different from multiplying by a 0/1 mask:
    # inactive ReLU entries must return zero even if upstream is inf or NaN.
    # Multiplication would evaluate 0 * inf / 0 * NaN and create NaN instead.
    grad_input = torch.where(active, grad_output, torch.zeros_like(grad_output))
    # Bias was broadcast over rows, so its VJP sums those rows, including M=0.
    grad_bias = grad_input.sum(dim=0)
    return grad_input, grad_bias


bias_relu.register_autograd(_backward, setup_context=_setup_context)


@torch.library.custom_op("gpu_prep::bias_relu_triton", mutates_args=())
def bias_relu_triton(x: Tensor, bias: Tensor) -> Tensor:
    """The same operator contract with a Triton GPU implementation.

    Uses the PyTorch >=2.5 custom_op interface, so its internals are opaque to
    the compiler. Newer torch.library.triton_op APIs are deliberately optional,
    not required by this package. CPU remains an ordinary reference fallback.
    """
    _validate(x, bias)
    if x.device.type == "cuda":
        from gpu_prep.triton_ops import bias_relu as triton_bias_relu

        with torch.no_grad():
            return triton_bias_relu(x, bias)
    return torch.relu(x + bias)


bias_relu_triton.register_fake(_bias_relu_fake)
bias_relu_triton.register_autograd(_backward, setup_context=_setup_context)


__all__ = ["bias_relu", "bias_relu_triton"]
```

gpu_prep/roofline.py

```
"""Exercise 6: an explicit, ideal-traffic roofline model.

All throughput inputs use decimal SI units: 1 TFLOP/s = 1e12 FLOP/s,
1 TB/s = 1e12 byte/s. Memory *capacity* may be displayed in GiB elsewhere.
```

```
These functions make mathematical estimates; they do not measure a GPU.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass
import argparse
import json
import math


@dataclass(frozen=True)
class Workload:
    name: str
    flops: int
    ideal_bytes: int
    assumptions: str

    @property
    def arithmetic_intensity(self) -> float:
        return self.flops / self.ideal_bytes if self.ideal_bytes else 0.0


def _dims(*values: int) -> None:
    if any(not isinstance(v, int) or isinstance(v, bool) or v < 0 for v in values):
        raise ValueError("Dimensions must be non-negative integers.")


def _element_bytes(value: int) -> None:
    if not isinstance(value, int) or isinstance(value, bool) or value <= 0:
        raise ValueError("element_bytes must be a positive integer.")


def vector_add_workload(n: int, element_bytes: int = 4) -> Workload:
    _dims(n)
    _element_bytes(element_bytes)
    return Workload(
        "vector_add", n, 3 * n * element_bytes,
        "Read each of two inputs once, write one output once; ignore caching and launch latency.",
    )


def gemm_workload(m: int, n: int, k: int, element_bytes: int = 4) -> Workload:
    _dims(m, n, k)
    _element_bytes(element_bytes)
    # If the output is empty, a well-written wrapper skips the operation.
    if m == 0 or n == 0:
        return Workload("gemm", 0, 0, "Empty output: skip the launch.")
    return Workload(
        "gemm", 2 * m * n * k, element_bytes * (m * k + k * n + m * n),
        "Conventional 2MNK FLOP count, beta=0; load each A/B element once, store each C once. "
        "This ideal reuse is not guaranteed by a real kernel.",
    )


def roofline_estimate(workload: Workload, peak_tflops: float, bandwidth_tbps: float) -> dict:
    if not math.isfinite(peak_tflops) or peak_tflops <= 0:
        raise ValueError("peak_tflops must be finite and positive.")
    if not math.isfinite(bandwidth_tbps) or bandwidth_tbps <= 0:
        raise ValueError("bandwidth_tbps must be finite and positive.")
    peak = peak_tflops * 1e12
    bandwidth = bandwidth_tbps * 1e12
    compute_seconds = workload.flops / peak
    memory_seconds = workload.ideal_bytes / bandwidth
    lower_bound = max(compute_seconds, memory_seconds)
    ceiling = workload.flops / lower_bound / 1e12 if lower_bound else 0.0
    return {
        **asdict(workload),
```

```
        "arithmetic_intensity_flop_per_byte": workload.arithmetic_intensity,
        "ridge_point_flop_per_byte": peak / bandwidth,
        "hypothetical_peak_tflops": peak_tflops,
        "hypothetical_bandwidth_tbps": bandwidth_tbps,
        "ideal_compute_time_ms": compute_seconds * 1e3,
        "ideal_memory_time_ms": memory_seconds * 1e3,
        "ideal_time_lower_bound_ms": lower_bound * 1e3,
        "roofline_tflops": ceiling,
        "model_limiting_resource": (
            "empty" if workload.ideal_bytes == 0 and workload.flops == 0
            else "memory" if memory_seconds >= compute_seconds else "compute"
        ),
        "measured": False,
    }


def amdahl_speedup(fraction: float, local_speedup: float) -> float:
    """Serial critical-path model, with no new overhead and fixed workload."""
    if not 0.0 <= fraction <= 1.0:
        raise ValueError("fraction must lie in [0, 1].")
    if not math.isfinite(local_speedup) or local_speedup <= 0:
        raise ValueError("local_speedup must be finite and positive.")
    return 1.0 / ((1.0 - fraction) + fraction / local_speedup)


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--peak-tflops", type=float, default=100.0,
                        help="Hypothetical peak for the SAME datatype/compute path.")
    parser.add_argument("--bandwidth-tbps", type=float, default=2.0)
    args = parser.parse_args()
    workloads = [
        vector_add_workload(1_000_000),
        gemm_workload(1, 4096, 4096),
        gemm_workload(1024, 1024, 1024),
        gemm_workload(127, 65, 93),
    ]
    print(json.dumps({
        "kind": "mathematical_estimates_not_measurements",
        "examples": [
            roofline_estimate(w, args.peak_tflops, args.bandwidth_tbps)
            for w in workloads
        ],
        "amdahl_example": {
            "original_time_fraction": 0.05,
            "local_speedup": 2.0,
            "end_to_end_speedup": amdahl_speedup(0.05, 2.0),
            "note": "Speedup factor and percent latency reduction are different quantities.",
        },
    }, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

gpu_prep/torch_layers.py

```
"""Readable PyTorch implementations for interview exercises 7--9.

These are correctness-oriented teaching implementations, not fused production
kernels.  Half and bfloat16 statistics/attention math use float32; float64 is
preserved so numerical gradient checks remain meaningful.
"""
```

```python
from __future__ import annotations

import math
from collections.abc import Sequence

import torch
from torch import Tensor, nn
from torch.nn import functional as F


_FLOAT_DTYPES = (torch.float16, torch.bfloat16, torch.float32, torch.float64)

# Round-to-nearest-even float32 boundaries, computed as ordinary host doubles.
# Half the smallest subnormal rounds to zero; the overflow midpoint rounds to
# infinity. Strict inequalities are equivalent to a finite, positive float32
# conversion, without a device Tensor, .item(), or nontraceable struct calls.
_F32_ZERO_ROUNDING_BOUNDARY = 2.0 ** -150
_F32_INFINITY_ROUNDING_BOUNDARY = 2.0 ** 128 - 2.0 ** 103


def _check_float(x: Tensor, name: str) -> None:
    if not isinstance(x, Tensor) or x.dtype not in _FLOAT_DTYPES:
        raise TypeError(f"{name} must be a floating Tensor (fp16/bf16/fp32/fp64)")


def _accumulator(x: Tensor) -> Tensor:
    return x.float() if x.dtype in (torch.float16, torch.bfloat16) else x


def _check_accumulator_eps(eps: float, dtype: torch.dtype) -> None:
    if not math.isfinite(eps) or eps <= 0:
        raise ValueError("eps must be finite and positive")
    if dtype == torch.float32 and not (
        _F32_ZERO_ROUNDING_BOUNDARY < eps < _F32_INFINITY_ROUNDING_BOUNDARY
    ):
        raise ValueError("eps must remain finite and positive when represented as float32")


def _normalized_shape(shape: int | Sequence[int]) -> tuple[int, ...]:
    result = (shape,) if isinstance(shape, int) else tuple(shape)
    if not result or any(not isinstance(d, int) or d <= 0 for d in result):
        raise ValueError("normalized_shape must contain positive integers")
    return result


def _check_normalized_input(x: Tensor, shape: tuple[int, ...]) -> None:
    _check_float(x, "x")
    if x.ndim < len(shape) or tuple(x.shape[-len(shape) :]) != shape:
        raise ValueError(f"expected input trailing shape {shape}, got {tuple(x.shape)}")


class ManualLinear(nn.Module):
    """y = x @ weight.T + bias, with genuinely registered Parameters.

    Input shape is (..., in_features); arbitrary leading dimensions, including
    empty ones, are supported.  Feature dimensions themselves must be positive.
    """

    def __init__(self, in_features: int, out_features: int, bias: bool = True,
                 device=None, dtype=None) -> None:
        super().__init__()
        if in_features <= 0 or out_features <= 0:
            raise ValueError("in_features and out_features must be positive")
        self.in_features = in_features
        self.out_features = out_features
        factory = {"device": device, "dtype": dtype}
        self.weight = nn.Parameter(torch.empty(out_features, in_features, **factory))
```

```
        if bias:
            self.bias = nn.Parameter(torch.empty(out_features, **factory))
        else:
            self.register_parameter("bias", None)
        self.reset_parameters()

    def reset_parameters(self) -> None:
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        if self.bias is not None:
            bound = 1 / math.sqrt(self.in_features)
            nn.init.uniform_(self.bias, -bound, bound)

    def forward(self, x: Tensor) -> Tensor:
        _check_float(x, "x")
        if x.ndim == 0 or x.shape[-1] != self.in_features:
            raise ValueError(f"last input dimension must be {self.in_features}")
        out = x @ self.weight.transpose(0, 1)
        return out if self.bias is None else out + self.bias


def split_qkv_heads(qkv: Tensor, num_heads: int) -> tuple[Tensor, Tensor, Tensor]:
    """[B,T,3*D] -> three [B,H,T,D/H] tensors.

    Projection output layout is [all Q features, all K features, all V features]
    for each token. Returned tensors are views when the input layout permits;
    no contiguity promise is made after permute.
    """
    _check_float(qkv, "qkv")
    if qkv.ndim != 3 or num_heads <= 0:
        raise ValueError("qkv must be [B,T,3*D] and num_heads must be positive")
    if qkv.shape[-1] == 0 or qkv.shape[-1] % (3 * num_heads) != 0:
        raise ValueError("qkv last dimension must be a positive multiple of 3*num_heads")
    batch, length, packed_dim = qkv.shape
    head_dim = packed_dim // (3 * num_heads)
    # [B,T,3,H,d] -> [3,B,H,T,d]; unbind selects Q, K and V.
    packed = qkv.reshape(batch, length, 3, num_heads, head_dim)
    q, k, v = packed.permute(2, 0, 3, 1, 4).unbind(0)
    return q, k, v


class ManualLayerNorm(nn.Module):
    """LayerNorm with population variance, explicit epsilon and autograd.

    Statistics reduce all normalized_shape axes. Empty leading dimensions are
    supported; an empty reduction domain is rejected at construction time.
    """

    def __init__(self, normalized_shape: int | Sequence[int], eps: float = 1e-5,
                 elementwise_affine: bool = True, bias: bool = True,
                 device=None, dtype=None) -> None:
        super().__init__()
        self.normalized_shape = _normalized_shape(normalized_shape)
        if not math.isfinite(eps) or eps <= 0:
            raise ValueError("eps must be finite and positive")
        self.eps = eps
        if elementwise_affine:
            self.weight = nn.Parameter(torch.ones(self.normalized_shape, device=device, dtype=dtype))
            if bias:
                self.bias = nn.Parameter(torch.zeros(self.normalized_shape, device=device, dtype=dtype))
            else:
                self.register_parameter("bias", None)
        else:
            self.register_parameter("weight", None)
            self.register_parameter("bias", None)

    def forward(self, x: Tensor) -> Tensor:
        _check_normalized_input(x, self.normalized_shape)
```

```
        acc = _accumulator(x)
        _check_accumulator_eps(self.eps, acc.dtype)
        axes = tuple(range(acc.ndim - len(self.normalized_shape), acc.ndim))
        mean = acc.mean(dim=axes, keepdim=True)
        centered = acc - mean
        # mean of squared deviations is correction=0, not sample variance.
        variance = centered.square().mean(dim=axes, keepdim=True)
        out = centered * torch.rsqrt(variance + self.eps)
        if self.weight is not None:
            out = out * self.weight.to(dtype=acc.dtype)
        if self.bias is not None:
            out = out + self.bias.to(dtype=acc.dtype)
        return out.to(dtype=x.dtype)


class ManualRMSNorm(nn.Module):
    """RMSNorm without mean subtraction; epsilon is deliberately explicit."""

    def __init__(self, normalized_shape: int | Sequence[int], eps: float = 1e-5,
                 elementwise_affine: bool = True, device=None, dtype=None) -> None:
        super().__init__()
        self.normalized_shape = _normalized_shape(normalized_shape)
        if not math.isfinite(eps) or eps <= 0:
            raise ValueError("eps must be finite and positive")
        self.eps = eps
        if elementwise_affine:
            self.weight = nn.Parameter(torch.ones(self.normalized_shape, device=device, dtype=dtype))
        else:
            self.register_parameter("weight", None)

    def forward(self, x: Tensor) -> Tensor:
        _check_normalized_input(x, self.normalized_shape)
        acc = _accumulator(x)
        _check_accumulator_eps(self.eps, acc.dtype)
        axes = tuple(range(acc.ndim - len(self.normalized_shape), acc.ndim))
        out = acc * torch.rsqrt(acc.square().mean(dim=axes, keepdim=True) + self.eps)
        if self.weight is not None:
            out = out * self.weight.to(dtype=acc.dtype)
        return out.to(dtype=x.dtype)


def scaled_dot_product_attention(
    q: Tensor,
    k: Tensor,
    v: Tensor,
    attn_mask: Tensor | None = None,
    is_causal: bool = False,
    query_start: int = 0,
    dropout_p: float = 0.0,
    training: bool = False,
    return_weights: bool = False,
) -> Tensor | tuple[Tensor, Tensor]:
    """Explicit, differentiable attention with safe all-masked rows.

    q: [B,H,L,d]; k: [B,H,S,d]; v: [B,H,S,dv]. Batch/head broadcasting and GQA
    are intentionally not implicit. All three inputs share dtype and device.

    A BOOLEAN mask is broadcastable to [B,H,L,S], with True = allowed. For a
    [B,S] key padding mask, explicitly pass mask[:,None,None,:]. Causality means
    key_position <= query_start + local_query_position. Keys start at position 0.
    All-masked rows, and the S=0 case, produce zero pre-projection outputs with
    finite zero gradients for finite inputs/logits. The explicit S=0 convention
    differs from the restricted CUDA/Triton teaching kernels in this package.

    Attention probabilities (if requested) use the accumulator dtype. Returned
    output uses q.dtype. Optional dropout is applied only when training=True.
    """
```

```
    for name, tensor in (("q", q), ("k", k), ("v", v)):
        _check_float(tensor, name)
        if tensor.ndim != 4:
            raise ValueError(f"{name} must have four dimensions [B,H,T,d]")
    if q.device != k.device or q.device != v.device:
        raise ValueError("q, k and v must be on the same device")
    if q.dtype != k.dtype or q.dtype != v.dtype:
        raise TypeError("q, k and v must have the same dtype")
    if q.shape[:2] != k.shape[:2] or q.shape[:2] != v.shape[:2]:
        raise ValueError("q, k and v must have matching batch and head dimensions")
    if q.shape[-1] <= 0 or v.shape[-1] <= 0 or q.shape[-1] != k.shape[-1]:
        raise ValueError("q/k head dimensions must match and head dimensions must be positive")
    if k.shape[-2] != v.shape[-2]:
        raise ValueError("k and v sequence lengths must match")
    if not isinstance(query_start, int) or query_start < 0:
        raise ValueError("query_start must be a nonnegative integer")
    if not is_causal and query_start != 0:
        raise ValueError("query_start is meaningful only with is_causal=True")
    if not 0.0 <= dropout_p < 1.0:
        raise ValueError("dropout_p must lie in [0,1)")
    if attn_mask is not None:
        if attn_mask.dtype != torch.bool:
            raise TypeError("attn_mask must be boolean, with True meaning allowed")
        if attn_mask.device != q.device:
            raise ValueError("attn_mask must be on the same device as q")
    q_acc, k_acc, v_acc = _accumulator(q), _accumulator(k), _accumulator(v)
    # Explicit float32 operands alone are not sufficient inside an outer AMP
    # region: matmul can be autocast back to half/bfloat16. Disable autocast for
    # these reference contractions so our advertised accumulator policy holds.
    with torch.autocast(device_type=q.device.type, enabled=False):
        scores = (q_acc @ k_acc.transpose(-2, -1)) * (q.shape[-1] ** -0.5)
    query_len, key_len = q.shape[-2], k.shape[-2]
    allowed = torch.ones((query_len, key_len), dtype=torch.bool, device=q.device)
    if is_causal:
        query_positions = query_start + torch.arange(query_len, device=q.device)
        key_positions = torch.arange(key_len, device=q.device)
        allowed = key_positions[None, :] <= query_positions[:, None]
    if attn_mask is not None:
        try:
            user_allowed = torch.broadcast_to(attn_mask, scores.shape)
        except RuntimeError as error:
            raise ValueError("attn_mask is not broadcastable to [B,H,L,S]") from error
        allowed = allowed & user_allowed
    if key_len == 0:
        # Matrix products on the empty contraction domain give zero outputs and
        # preserve the gradient connection to all three (possibly empty) inputs.
        with torch.autocast(device_type=q.device.type, enabled=False):
            out = (scores @ v_acc).to(dtype=q.dtype)
        return (out, scores) if return_weights else out
    masked_scores = scores.masked_fill(~allowed, float("-inf"))
    has_allowed_key = allowed.any(dim=-1, keepdim=True)
    # Never run softmax on an all -inf row. Replacing NaNs after softmax is too
    # late: its backward can still contain NaN. Make the input safe beforehand.
    safe_scores = torch.where(has_allowed_key, masked_scores, torch.zeros_like(masked_scores))
    row_max = safe_scores.amax(dim=-1, keepdim=True)
    exponentials = torch.exp(safe_scores - row_max)
    exponentials = torch.where(allowed, exponentials, torch.zeros_like(exponentials))
    # A finite nonempty row has an exponent equal to 1. An empty masked row has
    # denominator 0; clamping to 1 gives the explicitly chosen all-zero result.
    probabilities = exponentials / exponentials.sum(dim=-1, keepdim=True).clamp_min(1.0)
    if training and dropout_p:
        probabilities = F.dropout(probabilities, p=dropout_p, training=True)
    with torch.autocast(device_type=q.device.type, enabled=False):
        out = (probabilities @ v_acc).to(dtype=q.dtype)
    return (out, probabilities) if return_weights else out
```

```python
class KVCache:
    """Preallocated, contiguous K/V storage for one attention layer and batch.

    This teaching cache has a single shared sequence length for the whole batch.
    It is inference-only, append-only, and not thread-safe. No paging, eviction,
    RoPE, prefix sharing or per-request ragged lengths are hidden inside it.
    Allocate one cache per layer and independent active batch/session.
    """

    def __init__(self, batch_size: int, num_heads: int, max_seq_len: int,
                 head_dim: int, device, dtype: torch.dtype) -> None:
        dimensions = (batch_size, num_heads, max_seq_len, head_dim)
        if any(not isinstance(d, int) for d in dimensions):
            raise TypeError("cache dimensions must be integers")
        if batch_size < 0 or max_seq_len < 0 or num_heads <= 0 or head_dim <= 0:
            raise ValueError("invalid cache dimensions")
        if dtype not in _FLOAT_DTYPES:
            raise TypeError("cache dtype must be floating point")
        shape = (batch_size, num_heads, max_seq_len, head_dim)
        self.key = torch.empty(shape, device=device, dtype=dtype)
        self.value = torch.empty(shape, device=device, dtype=dtype)
        self.length = 0
        self.batch_size = batch_size
        self.num_heads = num_heads
        self.max_seq_len = max_seq_len
        self.head_dim = head_dim
        self.device = self.key.device
        self.dtype = dtype

    def reset(self) -> None:
        # Old storage is not zeroed; get() and append() expose only the active
        # prefix. External code must not treat unused capacity as valid tokens.
        self.length = 0

    def get(self) -> tuple[Tensor, Tensor]:
        return self.key[:, :, : self.length], self.value[:, :, : self.length]

    def append(self, key: Tensor, value: Tensor) -> tuple[Tensor, Tensor]:
        if torch.is_grad_enabled():
            raise RuntimeError("KVCache is inference-only; use torch.no_grad() or torch.inference_mode()")
        if key.ndim != 4 or value.shape != key.shape:
            raise ValueError("new key/value must have matching [B,H,T,d] shapes")
        if key.shape[:2] != (self.batch_size, self.num_heads) or key.shape[-1] != self.head_dim:
            raise ValueError("new key/value shape does not match this cache")
        if key.device != self.device or value.device != self.device:
            raise ValueError("new key/value device does not match this cache")
        if key.dtype != self.dtype or value.dtype != self.dtype:
            raise TypeError("new key/value dtype does not match this cache")
        end = self.length + key.shape[-2]
        if end > self.max_seq_len:
            raise ValueError(f"cache capacity exceeded: {end} > {self.max_seq_len}")
        # All metadata/capacity checks happen before either mutation.
        self.key[:, :, self.length : end].copy_(key)
        self.value[:, :, self.length : end].copy_(value)
        self.length = end
        return self.get()


class MultiHeadSelfAttention(nn.Module):
    """Packed QKV + manual attention + output projection, batch-first."""

    def __init__(self, embed_dim: int, num_heads: int, bias: bool = True,
                 dropout: float = 0.0, device=None, dtype=None) -> None:
        super().__init__()
        if embed_dim <= 0 or num_heads <= 0 or embed_dim % num_heads:
            raise ValueError("positive embed_dim must be divisible by positive num_heads")
        if not 0.0 <= dropout < 1.0:
```

```
            raise ValueError("dropout must lie in [0,1)")
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.dropout = dropout
        self.qkv = ManualLinear(embed_dim, 3 * embed_dim, bias=bias, device=device, dtype=dtype)
        self.out_proj = ManualLinear(embed_dim, embed_dim, bias=bias, device=device, dtype=dtype)

    def _project(self, x: Tensor) -> tuple[Tensor, Tensor, Tensor]:
        if x.ndim != 3 or x.shape[-1] != self.embed_dim:
            raise ValueError(f"x must be [B,T,{self.embed_dim}]")
        return split_qkv_heads(self.qkv(x), self.num_heads)

    def _merge(self, x: Tensor) -> Tensor:
        batch, _, length, _ = x.shape
        return self.out_proj(x.transpose(1, 2).reshape(batch, length, self.embed_dim))

    def forward(self, x: Tensor, attn_mask: Tensor | None = None,
                is_causal: bool = True, return_weights: bool = False
                ) -> Tensor | tuple[Tensor, Tensor]:
        q, k, v = self._project(x)
        attended, weights = scaled_dot_product_attention(
            q, k, v, attn_mask=attn_mask, is_causal=is_causal,
            dropout_p=self.dropout, training=self.training, return_weights=True,
        )
        out = self._merge(attended)
        # An all-masked row is zero BEFORE out_proj; a nonzero output bias may
        # make this module's final row nonzero. This is intentional and explicit.
        return (out, weights) if return_weights else out

    def forward_step(self, x: Tensor, cache: KVCache) -> Tensor:
        """Append a prefill chunk or new decode token(s), then attend to cache.

        Requires eval() AND a no_grad/inference_mode context. When a chunk starts
        after p cached tokens, query i is at absolute position p+i, not i.
        """
        if self.training or torch.is_grad_enabled():
            raise RuntimeError("forward_step requires model.eval() and torch.no_grad()/inference_mode()")
        if not isinstance(cache, KVCache):
            raise TypeError("cache must be a KVCache")
        if cache.num_heads != self.num_heads or cache.head_dim != self.head_dim:
            raise ValueError("cache head layout does not match this attention module")
        q, new_k, new_v = self._project(x)
        query_start = cache.length
        all_k, all_v = cache.append(new_k, new_v)
        attended = scaled_dot_product_attention(
            q, all_k, all_v, is_causal=True, query_start=query_start,
            dropout_p=0.0, training=False,
        )
        return self._merge(attended)
```

gpu_prep/triton_ops.py

```
"""Readable forward-only Triton kernels for GPU interview preparation.

Contract
--------
* NVIDIA CUDA, compute capability >= 8.0, contiguous tensors only.
* float32 / float16 / bfloat16; all floating operands share dtype and device.
* Reductions, statistics, GEMM accumulators, and attention states use float32.
* Floating outputs retain the input dtype. Row width is capped at 8192 for
  one-program-per-row reduction, softmax, and normalization exercises.
* These are forward kernels, not differentiable PyTorch operators. Calls with
  requires_grad inputs are rejected while grad mode is enabled. Use no_grad()
```

```
  for inference, or supply a separately implemented autograd rule.
* This module does not auto-copy noncontiguous inputs or scan their values.
  Unless otherwise documented, numerical examples assume finite inputs and no
  intermediate overflow. No NaN/Inf propagation compatibility is promised.

The code is intentionally original and compact enough to explain at a board.
Fixed tiles make it easier to study; they are not claimed to be optimal.
"""

import math
from numbers import Real
import struct

import torch
import triton
import triton.language as tl


MAX_ROW_WIDTH = 8192
MAX_TENSOR_ELEMENTS = 2**31 - 1  # Valid element offsets fit i32; tail accesses are masked.
SUPPORTED_DTYPES = (torch.float32, torch.float16, torch.bfloat16)
ATTENTION_HEAD_DIMS = (16, 32, 64, 128)

__all__ = [
    "vector_add", "bias_relu", "row_sum", "row_max", "softmax",
    "transpose_naive", "transpose_tiled", "matmul_naive", "matmul_tiled",
    "layer_norm", "rms_norm", "attention",
]


def _check_tensor(x: torch.Tensor, name: str, ndim: int | None = None) -> None:
    if not isinstance(x, torch.Tensor):
        raise TypeError(f"{name} must be a torch.Tensor")
    if ndim is not None and x.ndim != ndim:
        raise ValueError(f"{name} must have {ndim} dimensions; got {x.ndim}")
    if not x.is_cuda or torch.version.hip is not None:
        raise ValueError(f"{name} must be on an NVIDIA CUDA device")
    if x.dtype not in SUPPORTED_DTYPES:
        raise TypeError(f"{name} must have dtype float32, float16, or bfloat16")
    if not x.is_contiguous():
        raise ValueError(f"{name} must be contiguous; call contiguous() explicitly")
    if x.numel() > MAX_TENSOR_ELEMENTS:
        raise ValueError(f"{name} exceeds this exercise's i32 indexing limit")
    if torch.is_grad_enabled() and x.requires_grad:
        raise ValueError("forward-only kernel: detach inputs or use torch.no_grad()")
    if torch.cuda.get_device_capability(x.device)[0] < 8:
        raise ValueError("these Triton exercises require NVIDIA compute capability >= 8.0")


def _check_peer(x: torch.Tensor, y: torch.Tensor, name: str, ndim: int | None = None) -> None:
    _check_tensor(y, name, ndim)
    if x.device != y.device or x.dtype != y.dtype:
        raise ValueError(f"{name} must have the same device and dtype as the first input")


def _row_shape(x: torch.Tensor) -> tuple[int, int]:
    _check_tensor(x, "x", ndim=2)
    m, n = x.shape
    if n == 0 or n > MAX_ROW_WIDTH:
        raise ValueError(f"row width must be in [1, {MAX_ROW_WIDTH}]")
    return m, n


def _row_launch(n: int) -> tuple[int, int]:
    block = triton.next_power_of_2(n)
    return block, 4 if block <= 2048 else 8
```

```
def _check_eps(eps: float) -> float:
    if isinstance(eps, bool) or not isinstance(eps, Real):
        raise TypeError("eps must be a positive finite real number")
    eps = float(eps)
    if not math.isfinite(eps) or eps <= 0:
        raise ValueError("eps must be positive and finite")
    # Python floats have a wider exponent range than the FP32 kernel constant.
    # Reject values that disappear to zero or overflow when converted, without
    # allocating a tensor or synchronizing/reading any device data.
    try:
        eps_fp32 = struct.unpack(">f", struct.pack(">f", eps))[0]
    except OverflowError as exc:
        raise ValueError("eps must remain positive and finite when represented as float32") from exc
    if not math.isfinite(eps_fp32) or eps_fp32 <= 0:
        raise ValueError("eps must remain positive and finite when represented as float32")
    return eps_fp32


@triton.jit
def _vector_add_kernel(X, Y, Z, N: tl.constexpr, BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    valid = idx < N
    x = tl.load(X + idx, valid, other=0).to(tl.float32)
    y = tl.load(Y + idx, valid, other=0).to(tl.float32)
    tl.store(Z + idx, x + y, valid)


def vector_add(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """z = x + y for contiguous one-dimensional tensors of identical shape."""
    _check_tensor(x, "x", ndim=1)
    _check_peer(x, y, "y", ndim=1)
    if x.shape != y.shape:
        raise ValueError("x and y must have identical shapes")
    out = torch.empty_like(x)
    if x.numel() == 0:
        return out
    with torch.cuda.device(x.device):
        _vector_add_kernel[(triton.cdiv(x.numel(), 256),)](
            x, y, out, x.numel(), BLOCK=256, num_warps=4,
        )
    return out


@triton.jit
def _bias_relu_kernel(X, Bias, Y, TOTAL: tl.constexpr, N: tl.constexpr, BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    valid = idx < TOTAL
    x = tl.load(X + idx, valid, other=0).to(tl.float32)
    b = tl.load(Bias + idx % N, valid, other=0).to(tl.float32)
    tl.store(Y + idx, tl.maximum(x + b, 0.0), valid)


def bias_relu(x: torch.Tensor, bias: torch.Tensor) -> torch.Tensor:
    """y[m,n] = max(x[m,n] + bias[n], 0); no materialized broadcast tensor."""
    _check_tensor(x, "x", ndim=2)
    _check_peer(x, bias, "bias", ndim=1)
    if bias.shape[0] != x.shape[1]:
        raise ValueError("bias must have shape [x.shape[1]]")
    out = torch.empty_like(x)
    if x.numel() == 0:  # Also prevents taking a remainder by N=0 inside the kernel.
        return out
    with torch.cuda.device(x.device):
        _bias_relu_kernel[(triton.cdiv(x.numel(), 256),)](
            x, bias, out, x.numel(), x.shape[1], BLOCK=256, num_warps=4,
        )
    return out
```

```python
@triton.jit
def _row_reduce_kernel(X, Y, N: tl.constexpr, IS_MAX: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    if IS_MAX:
        values = tl.load(X + row * N + col, col < N, other=-float("inf")).to(tl.float32)
        result = tl.max(values, axis=0)
    else:
        values = tl.load(X + row * N + col, col < N, other=0).to(tl.float32)
        result = tl.sum(values, axis=0)
    tl.store(Y + row, result)


def _row_reduce(x: torch.Tensor, is_max: bool) -> torch.Tensor:
    m, n = _row_shape(x)
    out = torch.empty((m,), dtype=x.dtype, device=x.device)
    if m == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        _row_reduce_kernel[(m,)](x, out, n, is_max, BLOCK=block, num_warps=warps)
    return out


def row_sum(x: torch.Tensor) -> torch.Tensor:
    """Sum each row in float32 and cast the result back to x.dtype; N must be > 0."""
    return _row_reduce(x, False)


def row_max(x: torch.Tensor) -> torch.Tensor:
    """Maximum of each nonempty row; output dtype equals x.dtype."""
    return _row_reduce(x, True)


@triton.jit
def _softmax_kernel(X, Y, N: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    values = tl.load(X + row * N + col, col < N, other=-float("inf")).to(tl.float32)
    shifted = values - tl.max(values, axis=0)
    numerator = tl.exp(shifted)
    result = numerator / tl.sum(numerator, axis=0)
    tl.store(Y + row * N + col, result, col < N)


def softmax(x: torch.Tensor) -> torch.Tensor:
    """Stable row softmax for finite inputs; 1 <= N <= MAX_ROW_WIDTH.

    This standalone operation has no mask argument. A row consisting only of
    -inf is outside its finite-input contract. attention() separately defines
    its boolean-mask all-masked-row behavior as a zero output.
    """
    m, n = _row_shape(x)
    out = torch.empty_like(x)
    if m == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        _softmax_kernel[(m,)](x, out, n, BLOCK=block, num_warps=warps)
    return out


@triton.jit
def _transpose_naive_kernel(X, Y, M: tl.constexpr, N: tl.constexpr, BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
```

```
    row, col = idx // N, idx % N
    value = tl.load(X + idx, idx < M * N, other=0)
    tl.store(Y + col * M + row, value, idx < M * N)


@triton.jit
def _transpose_tiled_kernel(X, Y, M: tl.constexpr, N: tl.constexpr, TILE: tl.constexpr):
    pid = tl.program_id(0)
    tiles_n = tl.cdiv(N, TILE)
    row = (pid // tiles_n) * TILE + tl.arange(0, TILE)
    col = (pid % tiles_n) * TILE + tl.arange(0, TILE)
    values = tl.load(
        X + row[:, None] * N + col[None, :],
        (row[:, None] < M) & (col[None, :] < N), other=0,
    )
    # Express the output tile in [output-row, output-column] order.
    # Triton, rather than Python code, chooses lane layout and any conversions.
    tl.store(
        Y + col[:, None] * M + row[None, :], tl.trans(values),
        (col[:, None] < N) & (row[None, :] < M),
    )


def _transpose(x: torch.Tensor, tiled: bool) -> torch.Tensor:
    _check_tensor(x, "x", ndim=2)
    m, n = x.shape
    out = torch.empty((n, m), dtype=x.dtype, device=x.device)
    if x.numel() == 0:
        return out
    with torch.cuda.device(x.device):
        if tiled:
            _transpose_tiled_kernel[(triton.cdiv(m, 32) * triton.cdiv(n, 32),)](
                x, out, m, n, TILE=32, num_warps=4,
            )
        else:
            _transpose_naive_kernel[(triton.cdiv(x.numel(), 256),)](
                x, out, m, n, BLOCK=256, num_warps=4,
            )
    return out


def transpose_naive(x: torch.Tensor) -> torch.Tensor:
    """Materialize x.T using flattened input indices; educational strided stores."""
    return _transpose(x, False)


def transpose_tiled(x: torch.Tensor) -> torch.Tensor:
    """Materialize x.T with 2D tiles; this is not an explicit CUDA shared-memory tile."""
    return _transpose(x, True)


@triton.jit
def _matmul_naive_kernel(A, B, C, M: tl.constexpr, N: tl.constexpr, K: tl.constexpr,
                         BLOCK: tl.constexpr):
    idx = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    row, col = idx // N, idx % N
    valid = idx < M * N
    acc = tl.full((BLOCK,), 0, tl.float32)
    # A sequential K loop makes the repeated global loads easy to see.
    for k in range(0, K):
        a = tl.load(A + row * K + k, valid, other=0).to(tl.float32)
        b = tl.load(B + k * N + col, valid, other=0).to(tl.float32)
        acc = tl.fma(a, b, acc)
    tl.store(C + idx, acc, valid)


@triton.jit
```

```
def _matmul_tiled_kernel(A, B, C, M: tl.constexpr, N: tl.constexpr, K: tl.constexpr,
                         BM: tl.constexpr, BN: tl.constexpr, BK: tl.constexpr):
    pid = tl.program_id(0)
    tiles_n = tl.cdiv(N, BN)
    row = (pid // tiles_n) * BM + tl.arange(0, BM)
    col = (pid % tiles_n) * BN + tl.arange(0, BN)
    inner = tl.arange(0, BK)
    acc = tl.full((BM, BN), 0, tl.float32)
    for tile in range(0, tl.cdiv(K, BK)):
        k = tile * BK + inner
        a = tl.load(A + row[:, None] * K + k[None, :],
                    (row[:, None] < M) & (k[None, :] < K), other=0)
        b = tl.load(B + k[:, None] * N + col[None, :],
                    (k[:, None] < K) & (col[None, :] < N), other=0)
        # For f32 inputs, do not silently compare TF32 math with a strict f32 baseline.
        acc = tl.dot(a, b, acc, input_precision="ieee")
    tl.store(C + row[:, None] * N + col[None, :], acc,
             (row[:, None] < M) & (col[None, :] < N))


def _matmul(a: torch.Tensor, b: torch.Tensor, tiled: bool) -> torch.Tensor:
    _check_tensor(a, "a", ndim=2)
    _check_peer(a, b, "b", ndim=2)
    m, k = a.shape
    kb, n = b.shape
    if k != kb:
        raise ValueError("a.shape[1] must equal b.shape[0]")
    if m * n > MAX_TENSOR_ELEMENTS:
        raise ValueError("output exceeds this exercise's i32 indexing limit")
    out = torch.empty((m, n), dtype=a.dtype, device=a.device)
    if m == 0 or n == 0:
        return out
    if k == 0:
        return out.zero_()
    with torch.cuda.device(a.device):
        if tiled:
            _matmul_tiled_kernel[(triton.cdiv(m, 32) * triton.cdiv(n, 32),)](
                a, b, out, m, n, k, BM=32, BN=32, BK=32,
                num_warps=4, num_stages=2,
            )
        else:
            _matmul_naive_kernel[(triton.cdiv(m * n, 128),)](
                a, b, out, m, n, k, BLOCK=128, num_warps=4,
            )
    return out


def matmul_naive(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """A[M,K] @ B[K,N]; sequential K loop per output, FP32 accumulation."""
    return _matmul(a, b, False)


def matmul_tiled(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    """A[M,K] @ B[K,N]; fixed 32x32x32 tiles, FP32 accumulation, f32 IEEE math."""
    return _matmul(a, b, True)


@triton.jit
def _norm_kernel(X, W, Bias, Y, N: tl.constexpr, EPS: tl.constexpr,
                 IS_RMS: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    valid = col < N
    x = tl.load(X + row * N + col, valid, other=0).to(tl.float32)
    w = tl.load(W + col, valid, other=0).to(tl.float32)
    if IS_RMS:
        second_moment = tl.sum(x * x, axis=0) / N
```

```
        normalized = x * tl.rsqrt(second_moment + EPS)
        out = normalized * w
    else:
        mean = tl.sum(x, axis=0) / N
        # Padding must contribute zero, not (0 - mean)^2, to the variance.
        centered = tl.where(valid, x - mean, 0.0)
        variance = tl.sum(centered * centered, axis=0) / N
        normalized = centered * tl.rsqrt(variance + EPS)
        b = tl.load(Bias + col, valid, other=0).to(tl.float32)
        out = normalized * w + b
    tl.store(Y + row * N + col, out, valid)


def _norm_shape(x: torch.Tensor, weight: torch.Tensor, eps: float) -> tuple[int, int, float]:
    _check_tensor(x, "x")
    if x.ndim < 1:
        raise ValueError("x must have at least one dimension")
    n = x.shape[-1]
    if n == 0 or n > MAX_ROW_WIDTH:
        raise ValueError(f"normalized dimension must be in [1, {MAX_ROW_WIDTH}]")
    _check_peer(x, weight, "weight", ndim=1)
    if weight.shape[0] != n:
        raise ValueError("weight must have shape [x.shape[-1]]")
    return x.numel() // n, n, _check_eps(eps)


def layer_norm(x: torch.Tensor, weight: torch.Tensor, bias: torch.Tensor,
               eps: float = 1e-5) -> torch.Tensor:
    """Last-dimension LayerNorm; population variance (divide by N), affine=True."""
    rows, n, eps = _norm_shape(x, weight, eps)
    _check_peer(x, bias, "bias", ndim=1)
    if bias.shape != weight.shape:
        raise ValueError("bias must have the same shape as weight")
    out = torch.empty_like(x)
    if rows == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        _norm_kernel[(rows,)](
            x, weight, bias, out, n, eps, False, BLOCK=block, num_warps=warps,
        )
    return out


def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5) -> torch.Tensor:
    """Last-dimension RMSNorm with learnable weight, no bias, explicit eps."""
    rows, n, eps = _norm_shape(x, weight, eps)
    out = torch.empty_like(x)
    if rows == 0:
        return out
    block, warps = _row_launch(n)
    with torch.cuda.device(x.device):
        # Bias is unused in the IS_RMS specialization, so weight is a harmless placeholder.
        _norm_kernel[(rows,)](
            x, weight, weight, out, n, eps, True, BLOCK=block, num_warps=warps,
        )
    return out


@triton.jit
def _attention_kernel(Q, K, V, Mask, O, L: tl.constexpr, S: tl.constexpr,
                      D: tl.constexpr, SCALE: tl.constexpr,
                      CAUSAL: tl.constexpr, QUERY_START: tl.constexpr,
                      HAS_MASK: tl.constexpr, MASK_BH_STRIDE: tl.constexpr,
                      BM: tl.constexpr, BN: tl.constexpr):
    pid = tl.program_id(0)
    q_blocks = tl.cdiv(L, BM)
```

```
    q_block = pid % q_blocks
    bh = pid // q_blocks
    r = q_block * BM + tl.arange(0, BM)
    d = tl.arange(0, D)
    q = tl.load(Q + bh * L * D + r[:, None] * D + d[None, :],
                r[:, None] < L, other=0)

    # Three online-softmax states for each query: maximum, denominator, numerator.
    running_max = tl.full((BM,), -float("inf"), tl.float32)
    denominator = tl.full((BM,), 0.0, tl.float32)
    numerator = tl.full((BM, D), 0.0, tl.float32)

    for tile in range(0, tl.cdiv(S, BN)):
        c = tile * BN + tl.arange(0, BN)
        k = tl.load(K + bh * S * D + c[None, :] * D + d[:, None],
                    c[None, :] < S, other=0)
        v = tl.load(V + bh * S * D + c[:, None] * D + d[None, :],
                    c[:, None] < S, other=0).to(tl.float32)
        scores = tl.dot(q, k, input_precision="ieee") * SCALE
        visible = (r[:, None] < L) & (c[None, :] < S)
        if CAUSAL:
            visible = visible & (c[None, :] <= QUERY_START + r[:, None])
        if HAS_MASK:
            keep = tl.load(Mask + bh * MASK_BH_STRIDE + r[:, None] * S + c[None, :],
                           (r[:, None] < L) & (c[None, :] < S), other=0)
            visible = visible & keep
        scores = tl.where(visible, scores, -float("inf"))
        new_max = tl.maximum(running_max, tl.max(scores, axis=1))

        # Before any key has been visible, new_max is -inf. Replace it only in
        # exponent calculations so we never evaluate -inf - (-inf).
        safe_max = tl.where(new_max == -float("inf"), 0.0, new_max)
        old_scale = tl.exp(running_max - safe_max)
        probabilities = tl.exp(scores - safe_max[:, None])
        denominator = denominator * old_scale + tl.sum(probabilities, axis=1)
        numerator = numerator * old_scale[:, None]
        # Keep the probabilities in FP32. This deliberately sacrifices some
        # Tensor Core throughput to avoid a hidden FP16/BF16 probability cast.
        numerator = tl.dot(probabilities, v, numerator, input_precision="ieee")
        running_max = new_max

    safe_denominator = tl.where(denominator > 0.0, denominator, 1.0)
    output = numerator / safe_denominator[:, None]
    tl.store(O + bh * L * D + r[:, None] * D + d[None, :], output, r[:, None] < L)


def attention(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor,
              causal: bool = False, query_start: int = 0, *,
              attn_mask: torch.Tensor | None = None) -> torch.Tensor:
    """Tiled online-softmax attention forward, without materializing [L,S].

    q: [B,H,L,D], k/v: [B,H,S,D], S>0, D in {16,32,64,128}.
    Keys have positions 0..S-1. For causal=True, query i has absolute position
    query_start+i and sees key j exactly when j <= query_start+i.
    query_start is explicit: decode with a length-S cache usually uses S-1.

    Optional bool attn_mask is [L,S] shared across B/H or exactly [B,H,L,S].
    True means visible; it is ANDed with the causal mask. All-masked rows return
    zeros, by this exercise's explicit convention. No additive mask, dropout,
    GQA, custom scale, backward, or arbitrary-stride inputs are implemented.
    """
    _check_tensor(q, "q", ndim=4)
    _check_peer(q, k, "k", ndim=4)
    _check_peer(q, v, "v", ndim=4)
    b, h, length, d = q.shape
    if k.shape != v.shape or k.shape[:2] != (b, h) or k.shape[-1] != d:
        raise ValueError("expected q[B,H,L,D] and matching k/v[B,H,S,D]")
```

```
    s = k.shape[2]
    if s <= 0:
        raise ValueError("attention key/value length S must be positive")
    if d not in ATTENTION_HEAD_DIMS:
        raise ValueError(f"head dimension D must be one of {ATTENTION_HEAD_DIMS}")
    if not isinstance(causal, bool):
        raise TypeError("causal must be bool")
    if isinstance(query_start, bool) or not isinstance(query_start, int):
        raise TypeError("query_start must be a nonnegative Python int")
    if query_start < 0 or query_start + length > MAX_TENSOR_ELEMENTS:
        raise ValueError("query positions must be nonnegative and fit i32")

    mask_stride = 0
    if attn_mask is not None:
        if not isinstance(attn_mask, torch.Tensor):
            raise TypeError("attn_mask must be a bool torch.Tensor")
        if attn_mask.device != q.device or attn_mask.dtype != torch.bool:
            raise ValueError("attn_mask must be bool and on q.device")
        if not attn_mask.is_contiguous():
            raise ValueError("attn_mask must be contiguous")
        if attn_mask.numel() > MAX_TENSOR_ELEMENTS:
            raise ValueError("attn_mask exceeds this exercise's i32 indexing limit")
        if attn_mask.shape == (length, s):
            mask_stride = 0
        elif attn_mask.shape == (b, h, length, s):
            mask_stride = length * s
        else:
            raise ValueError("attn_mask must have shape [L,S] or exactly [B,H,L,S]")

    out = torch.empty_like(q)
    if q.numel() == 0:
        return out
    mask_pointer = q if attn_mask is None else attn_mask
    with torch.cuda.device(q.device):
        _attention_kernel[(triton.cdiv(length, 16) * b * h,)](
            q, k, v, mask_pointer, out, length, s, d, 1.0 / math.sqrt(d),
            causal, query_start, attn_mask is not None, mask_stride,
            BM=16, BN=32, num_warps=4, num_stages=1,
        )
    return out
```

pyproject.toml

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "together-gpu-interview-prep"
version = "1.0.0"
description = "Twelve GPU programming interview exercises: CUDA, Triton, PyTorch, and detailed Chinese
explanations"
requires-python = ">=3.10"
dependencies = ["torch>=2.5"]

[project.optional-dependencies]
cuda = ["ninja>=1.11"]
triton = ["triton>=3.1; platform_system=='Linux'"]
test = ["pytest>=8", "numpy>=1.26"]
docs = ["Markdown>=3.7", "Pygments>=2.18", "matplotlib>=3.8", "pymdown-extensions>=10"]

[tool.setuptools.packages.find]
include = ["gpu_prep*"]
```

```
[tool.setuptools.package-data]
gpu_prep = ["csrc/*.cu", "csrc/*.cpp", "csrc/*.h"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra"
markers = [
  "gpu: requires an NVIDIA GPU",
  "cuda_ext: requires an NVIDIA GPU and a CUDA toolkit to build the extension",
  "triton: requires an NVIDIA GPU and Triton",
]
```

requirements.txt

```
# Install the PyTorch build appropriate for your GPU before this file.
# The CPU reference implementation can also be studied without a GPU.
torch>=2.5
pytest>=8
numpy>=1.26
# Optional GPU backend: use the Triton version compatible with your PyTorch.
# triton>=3.1
```

tests/test_benchmark_contract.py

```
"""Tests for benchmark accounting; these do not simulate GPU performance."""
import pytest
import torch

from gpu_prep.benchmark import build_case, parse_shape, summarize_samples


def test_summary_is_repeat_mean_quantile():
    result = summarize_samples([1.0, 2.0, 4.0])
    assert result["median_ms"] == 2.0
    assert result["p95_of_repeat_means_ms"] == pytest.approx(3.8)
    assert result["samples_ms"] == [1.0, 2.0, 4.0]


@pytest.mark.parametrize("op,shape", [
    ("vector_add", (13,)), ("bias_relu", (3, 7)), ("row_sum", (3, 7)),
    ("row_max", (3, 7)), ("softmax", (3, 7)), ("transpose", (3, 7)),
    ("matmul", (3, 7, 5)), ("layer_norm", (3, 7)), ("rms_norm", (3, 7)),
    ("attention", (1, 2, 3, 7, 4)),
])
def test_reference_and_torch_baseline_describe_same_operation(op, shape):
    # Verifies shapes, oracle equations, and timed baseline semantics on CPU.
    # It does not exercise CUDA-event measurement.
    case = build_case(op, shape, torch.float32, torch.device("cpu"))
    torch.testing.assert_close(case.torch_fn(), case.reference, atol=1e-5, rtol=1e-5)


def test_physical_transpose_reference():
    case = build_case("transpose", (3, 7), torch.float32, torch.device("cpu"))
    assert case.torch_fn().is_contiguous()
    assert case.logical_bytes == 2 * 3 * 7 * 4


def test_shape_order_is_m_n_k():
    case = build_case("matmul", parse_shape("3,7,5"), torch.float32, torch.device("cpu"))
```

```
    assert tuple(case.inputs[0].shape) == (3, 5)
    assert tuple(case.inputs[1].shape) == (5, 7)
    assert case.flops == 2 * 3 * 7 * 5
```

tests/test_cuda_ops.py

```
"""CUDA teaching kernels: correctness, contract and stream tests.

These tests are skipped without an NVIDIA GPU. A skip is NOT GPU validation.
Run from the repository root: pytest -q tests/test_cuda_ops.py
"""

from __future__ import annotations

import math

import pytest

torch = pytest.importorskip("torch")
from gpu_prep import cuda_ops

pytestmark = pytest.mark.skipif(
    not torch.cuda.is_available(), reason="NVIDIA GPU unavailable; CUDA tests not executed"
)


@pytest.fixture(scope="module", autouse=True)
def compiled_extension():
    # GPU 存在但构建失败时应报错；不能把编译错误伪装成测试跳过。
    cuda_ops.load_extension()
    torch.manual_seed(2027)


def assert_close(actual, expected, *, atol=3e-5, rtol=3e-5):
    assert actual.shape == expected.shape
    assert actual.dtype == torch.float32
    assert actual.device == expected.device
    assert actual.is_contiguous()
    torch.testing.assert_close(actual, expected, atol=atol, rtol=rtol)


@pytest.mark.parametrize("n", [0, 1, 31, 32, 33, 255, 256, 257, 1_000_003])
def test_vector_add_tail(n):
    x = torch.randn(n, device="cuda")
    y = torch.randn_like(x)
    assert_close(cuda_ops.vector_add(x, y), x + y, atol=0, rtol=0)


@pytest.mark.parametrize("shape", [(), (3, 7), (0, 4), (2, 0, 3)])
def test_vector_add_preserves_shape(shape):
    x = torch.randn(shape, device="cuda")
    y = torch.randn_like(x)
    assert_close(cuda_ops.vector_add(x, y), x + y, atol=0, rtol=0)


@pytest.mark.parametrize("m,n", [(5, 257), (1, 1), (0, 13), (4, 0)])
def test_bias_relu(m, n):
    x = torch.randn(m, n, device="cuda")
    bias = torch.randn(n, device="cuda")
    assert_close(cuda_ops.bias_relu(x, bias), torch.relu(x + bias), atol=0, rtol=0)


def test_bias_relu_preserves_nan():
    x = torch.tensor([[float("nan"), -2.0, 3.0]], device="cuda")
```

```
    b = torch.zeros(3, device="cuda")
    torch.testing.assert_close(
        cuda_ops.bias_relu(x, b), torch.relu(x + b), equal_nan=True, atol=0, rtol=0
    )


@pytest.mark.parametrize("m,n", [(7, 1), (5, 33), (3, 513), (2, 1001), (2, 8193), (0, 31)])
def test_row_reductions(m, n):
    x = torch.randn(m, n, device="cuda")
    assert_close(cuda_ops.row_sum(x), x.double().sum(dim=-1).float(), atol=6e-5, rtol=4e-5)
    assert_close(cuda_ops.row_max(x), x.max(dim=-1).values, atol=0, rtol=0)


def test_row_max_negative_values():
    x = -torch.rand(4, 513, device="cuda") - 0.25
    assert_close(cuda_ops.row_max(x), x.max(dim=-1).values, atol=0, rtol=0)


@pytest.mark.parametrize("m,n", [(7, 1), (5, 33), (3, 513), (2, 1001), (2, 8193), (0, 31)])
def test_softmax(m, n):
    x = torch.randn(m, n, device="cuda") * 7
    actual = cuda_ops.softmax(x)
    expected = torch.softmax(x.double(), dim=-1).float()
    assert_close(actual, expected, atol=3e-6, rtol=3e-5)
    if m:
        torch.testing.assert_close(actual.sum(-1), torch.ones(m, device="cuda"), atol=3e-6, rtol=3e-6)


def test_softmax_extreme_finite_scores():
    x = torch.tensor(
        [[10000.0, 10001.0, 9999.0], [-10000.0, -10001.0, -9999.0], [8.0, 8.0, 8.0]],
        device="cuda",
    )
    actual = cuda_ops.softmax(x)
    assert torch.isfinite(actual).all()
    assert_close(actual, torch.softmax(x.double(), dim=-1).float(), atol=1e-6, rtol=1e-6)


@pytest.mark.parametrize("op", [cuda_ops.transpose_naive, cuda_ops.transpose_tiled])
@pytest.mark.parametrize("m,n", [(1, 1), (3, 65), (31, 33), (1003, 769), (0, 13), (7, 0)])
def test_transpose(op, m, n):
    x = torch.randn(m, n, device="cuda")
    assert_close(op(x), x.t().contiguous(), atol=0, rtol=0)


@pytest.mark.parametrize("op", [cuda_ops.matmul_naive, cuda_ops.matmul_tiled])
@pytest.mark.parametrize("m,n,k", [(1, 1, 1), (7, 5, 3), (127, 65, 93), (16, 16, 16),
                                   (3, 5, 0), (0, 5, 7), (3, 0, 7), (0, 0, 0)])
def test_matmul(op, m, n, k):
    a = torch.randn(m, k, device="cuda")
    b = torch.randn(k, n, device="cuda")
    # FP64 reference 避免受 PyTorch TF32/FP32 matmul 全局设置影响。
    expected = (a.double() @ b.double()).float()
    assert_close(op(a, b), expected, atol=2e-4, rtol=5e-5)


@pytest.mark.parametrize("shape", [(1,), (7,), (2, 3, 1), (2, 3, 33), (3, 513), (0, 4, 33)])
@pytest.mark.parametrize("eps", [1e-5, 1e-3])
def test_norms(shape, eps):
    x = torch.randn(shape, device="cuda")
    w = torch.randn(shape[-1], device="cuda")
    b = torch.randn(shape[-1], device="cuda")
    xd = x.double()
    mean = xd.mean(dim=-1, keepdim=True)
    var = ((xd - mean) ** 2).mean(dim=-1, keepdim=True)
    layer_reference = ((xd - mean) * torch.rsqrt(var + eps) * w.double() + b.double()).float()
    rms_reference = (xd * torch.rsqrt((xd * xd).mean(dim=-1, keepdim=True) + eps) * w.double()).float()
```

```
    assert_close(cuda_ops.layer_norm(x, w, b, eps), layer_reference, atol=4e-5, rtol=4e-5)
    assert_close(cuda_ops.rms_norm(x, w, eps), rms_reference, atol=4e-5, rtol=4e-5)


def test_layer_norm_large_mean_small_variance():
    # 比较 two-pass 中心方差与高精度 reference；避免 E[x²]-E[x]² 的消减问题。
    x = 1000.0 + torch.randn(3, 513, device="cuda")
    w = torch.ones(513, device="cuda")
    b = torch.zeros(513, device="cuda")
    xd = x.double()
    center = xd - xd.mean(-1, keepdim=True)
    expected = (center * torch.rsqrt((center ** 2).mean(-1, keepdim=True) + 1e-5)).float()
    assert_close(cuda_ops.layer_norm(x, w, b), expected, atol=3e-4, rtol=2e-4)


def attention_oracle(q, k, v, causal, query_start):
    # 显式 absolute-position mask，避免非方阵 is_causal 的对齐语义歧义。
    scores = (q.double() @ k.double().transpose(-1, -2)) / math.sqrt(q.shape[-1])
    if causal:
        query_positions = torch.arange(q.shape[-2], device=q.device) + query_start
        key_positions = torch.arange(k.shape[-2], device=q.device)
        visible = key_positions[None, :] <= query_positions[:, None]
        scores = scores.masked_fill(~visible, float("-inf"))
    return (torch.softmax(scores, dim=-1) @ v.double()).float()


@pytest.mark.parametrize(
    "b,h,l,s,d,causal,offset",
    [(2, 3, 7, 11, 17, False, 0), (1, 2, 33, 33, 32, True, 0),
     (2, 1, 1, 65, 16, True, 64), (1, 1, 3, 8, 7, True, 5),
     (1, 1, 0, 8, 7, False, 0), (0, 2, 3, 8, 7, False, 0),
     (1, 1, 1, 1, 1, True, 0), (1, 1, 1, 4096, 8, False, 0)],
)
def test_attention(b, h, l, s, d, causal, offset):
    q = torch.randn(b, h, l, d, device="cuda")
    k = torch.randn(b, h, s, d, device="cuda")
    v = torch.randn_like(k)
    expected = attention_oracle(q, k, v, causal, offset)
    assert_close(cuda_ops.attention(q, k, v, causal, offset), expected, atol=4e-5, rtol=4e-5)


def test_attention_decode_matches_full_last_query():
    q = torch.randn(1, 2, 17, 13, device="cuda")
    k = torch.randn_like(q)
    v = torch.randn_like(q)
    full = cuda_ops.attention(q, k, v, causal=True)
    decode = cuda_ops.attention(q[:, :, -1:, :].contiguous(), k, v,
                                causal=True, query_start=16)
    assert_close(decode, full[:, :, -1:, :].contiguous(), atol=2e-6, rtol=2e-6)


def test_rejects_wrong_contract():
    x = torch.randn(3, 7, device="cuda")
    with pytest.raises(RuntimeError, match="float32"):
        cuda_ops.vector_add(x.half(), x.half())
    with pytest.raises(RuntimeError, match="contiguous"):
        cuda_ops.transpose_tiled(x.t())
    with pytest.raises(RuntimeError, match="identical shapes"):
        cuda_ops.vector_add(x, x[:1].contiguous())
    with pytest.raises(RuntimeError, match="CUDA tensor"):
        cuda_ops.vector_add(torch.ones(7), torch.ones(7))
    with pytest.raises(RuntimeError, match="bias must have shape"):
        cuda_ops.bias_relu(x, torch.ones(3, device="cuda"))
    with pytest.raises(RuntimeError, match="inner dimensions"):
        cuda_ops.matmul_tiled(x, x)
    for op in [cuda_ops.row_sum, cuda_ops.row_max, cuda_ops.softmax]:
        with pytest.raises(RuntimeError, match="positive"):
```

```
            op(torch.empty(3, 0, device="cuda"))
    with pytest.raises(RuntimeError, match="normalization width"):
        cuda_ops.rms_norm(torch.empty(2, 0, device="cuda"), torch.empty(0, device="cuda"))
    for eps in [0.0, -1.0, float("inf"), float("nan"), 1e-300]:
        with pytest.raises(RuntimeError, match="eps must"):
            cuda_ops.rms_norm(x, torch.ones(7, device="cuda"), eps)


def test_attention_rejects_invalid_dimensions():
    q = torch.randn(1, 1, 1, 8, device="cuda")
    with pytest.raises(RuntimeError, match="1 <= key length"):
        empty = torch.empty(1, 1, 0, 8, device="cuda")
        cuda_ops.attention(q, empty, empty)
    with pytest.raises(RuntimeError, match="1 <= key length"):
        too_long = torch.empty(1, 1, 4097, 8, device="cuda")
        cuda_ops.attention(q, too_long, too_long)
    with pytest.raises(RuntimeError, match="query_start must"):
        cuda_ops.attention(q, q, q, causal=True, query_start=-1)


def test_forward_only_guard_and_no_grad_escape():
    x = torch.randn(7, device="cuda", requires_grad=True)
    with pytest.raises(RuntimeError, match="forward-only"):
        cuda_ops.vector_add(x, x)
    with torch.no_grad():
        out = cuda_ops.vector_add(x, x)
    assert not out.requires_grad
    torch.testing.assert_close(out, 2 * x.detach())


@pytest.mark.skipif(torch.cuda.device_count() < 2, reason="two CUDA devices required")
def test_device_guard_and_device_mismatch():
    x = torch.randn(257, device="cuda:1")
    y = torch.randn_like(x)
    with torch.cuda.device(0):
        out = cuda_ops.vector_add(x, y)
        assert torch.cuda.current_device() == 0
        assert out.device.index == 1
        assert_close(out, x + y, atol=0, rtol=0)
        other = torch.randn(257, device="cuda:0")
        with pytest.raises(RuntimeError, match="must be on"):
            cuda_ops.vector_add(x, other)


def test_respects_non_default_stream():
    # 先让输入在 default stream 上有确定的初始值。
    x = torch.zeros(4097, device="cuda")
    y = torch.zeros_like(x)
    torch.cuda.synchronize()
    stream = torch.cuda.Stream()
    with torch.cuda.stream(stream):
        # 若可用，延迟 producer，增强错误使用 default stream 时的检测能力。
        # 这是测试辅助；kernel 和用户 API 均不依赖私有 _sleep。
        if hasattr(torch.cuda, "_sleep"):
            torch.cuda._sleep(5_000_000)
        x.fill_(3.0)
        y.fill_(5.0)
        out = cuda_ops.vector_add(x, y)
        row = cuda_ops.row_sum(out.view(1, -1))
        done = torch.cuda.Event()
        done.record()
    done.synchronize()
    assert_close(out, torch.full_like(x, 8.0), atol=0, rtol=0)
    assert_close(row, torch.full((1,), 4097.0 * 8.0, device="cuda"), atol=0, rtol=0)
```

tests/test_custom_ops.py

```python
"""Registration, mathematical gradient and graph-capture checks for exercise 11."""

import pytest
import torch

from gpu_prep.custom_ops import bias_relu, bias_relu_triton


@pytest.fixture(params=[bias_relu, bias_relu_triton], ids=["cuda_registration", "triton_registration"])
def operation(request):
    return request.param


@pytest.mark.parametrize("shape", [(3, 5), (0, 5), (3, 0)])
def test_custom_op_forward_and_backward_match_reference(shape, operation):
    torch.manual_seed(42)
    x = torch.randn(shape, dtype=torch.float64, requires_grad=True)
    bias = torch.randn(shape[1], dtype=torch.float64, requires_grad=True)
    xr = x.detach().clone().requires_grad_()
    br = bias.detach().clone().requires_grad_()
    actual = operation(x, bias)
    expected = torch.relu(xr + br)
    torch.testing.assert_close(actual, expected)
    upstream = torch.randn_like(actual)
    actual.backward(upstream)
    expected.backward(upstream)
    torch.testing.assert_close(x.grad, xr.grad)
    torch.testing.assert_close(bias.grad, br.grad)
    assert actual.is_contiguous()


def test_relu_at_zero_uses_zero_derivative(operation):
    x = torch.tensor([[-1.0, 0.0, 2.0]], dtype=torch.float64, requires_grad=True)
    bias = torch.zeros(3, dtype=torch.float64, requires_grad=True)
    operation(x, bias).sum().backward()
    torch.testing.assert_close(x.grad, torch.tensor([[0.0, 0.0, 1.0]], dtype=torch.float64))
    torch.testing.assert_close(bias.grad, torch.tensor([0.0, 0.0, 1.0], dtype=torch.float64))


def test_inactive_relu_discards_nonfinite_upstream_gradient(operation):
    x = torch.tensor([[-1.0, 0.0, 1.0]], dtype=torch.float64, requires_grad=True)
    bias = torch.zeros(3, dtype=torch.float64, requires_grad=True)
    xr = x.detach().clone().requires_grad_()
    br = bias.detach().clone().requires_grad_()
    upstream = torch.tensor([[float("inf"), float("nan"), 1.0]], dtype=torch.float64)
    actual = torch.autograd.grad(operation(x, bias), (x, bias), grad_outputs=upstream)
    expected = torch.autograd.grad(torch.relu(xr + br), (xr, br), grad_outputs=upstream)
    for gradient, reference in zip(actual, expected):
        assert torch.isfinite(gradient).all()
        torch.testing.assert_close(gradient, reference, atol=0, rtol=0)
    torch.testing.assert_close(actual[0], torch.tensor([[0.0, 0.0, 1.0]], dtype=torch.float64))
    torch.testing.assert_close(actual[1], torch.tensor([0.0, 0.0, 1.0], dtype=torch.float64))


def test_custom_op_gradcheck_away_from_relu_kink(operation):
    x = torch.tensor([[-2.0, 2.0, -3.0], [1.0, -4.0, 3.0]], dtype=torch.float64, requires_grad=True)
    bias = torch.tensor([0.1, 0.2, 0.3], dtype=torch.float64, requires_grad=True)
    assert torch.autograd.gradcheck(operation, (x, bias))


@pytest.mark.parametrize("shape", [(2, 3), (0, 3), (2, 0)])
def test_custom_op_registration_contract(shape, operation):
    torch.manual_seed(7)
    x = torch.randn(shape, dtype=torch.float64, requires_grad=True)
```

```
    bias = torch.randn(shape[1], dtype=torch.float64, requires_grad=True)
    results = torch.library.opcheck(operation, (x, bias))
    assert all(result == "SUCCESS" for result in results.values())


def test_fake_metadata_without_running_cpu_or_cuda_data_kernel(operation):
    # FakeTensorMode is a diagnostic utility under torch._subclasses; production
    # fake registration uses the public register_fake API in custom_ops.py.
    from torch._subclasses.fake_tensor import FakeTensorMode

    with FakeTensorMode():
        x = torch.empty(2, 3, dtype=torch.float64)
        bias = torch.empty(3, dtype=torch.float64)
        output = operation(x, bias)
        assert output.shape == (2, 3)
        assert output.stride() == (3, 1)
        assert output.dtype == x.dtype
        assert output.device == x.device


def test_fullgraph_capture_with_eager_backend_and_backward(operation):
    # backend=eager verifies Dynamo capture; it does not prove Inductor codegen,
    # AOT training compilation or GPU speedup. Those are separate experiments.
    fn = lambda x, b: operation(x, b).square().sum()
    compiled = torch.compile(fn, backend="eager", fullgraph=True)
    x = torch.tensor([[2.0, -2.0, 1.0]], dtype=torch.float64, requires_grad=True)
    b = torch.tensor([0.3, 0.2, -0.1], dtype=torch.float64, requires_grad=True)
    xr = x.detach().clone().requires_grad_()
    br = b.detach().clone().requires_grad_()
    actual = compiled(x, b)
    expected = fn(xr, br)
    torch.testing.assert_close(actual, expected)
    actual.backward()
    expected.backward()
    torch.testing.assert_close(x.grad, xr.grad)
    torch.testing.assert_close(b.grad, br.grad)


def test_invalid_contract_is_rejected(operation):
    with pytest.raises(ValueError, match="length"):
        operation(torch.randn(2, 3), torch.randn(4))
    with pytest.raises(ValueError, match="contiguous"):
        operation(torch.randn(3, 2).t(), torch.randn(3))
    with pytest.raises(TypeError, match="same dtype"):
        operation(torch.randn(2, 3), torch.randn(3, dtype=torch.float64))
    with pytest.raises(ValueError, match="expects"):
        operation(torch.randn(3), torch.randn(3))


@pytest.mark.skipif(not torch.cuda.is_available(), reason="requires CUDA runtime, GPU and nvcc extension build")
def test_cuda_custom_op_forward_and_backward(operation):
    torch.manual_seed(11)
    x = torch.randn(17, 33, device="cuda", dtype=torch.float32, requires_grad=True)
    bias = torch.randn(33, device="cuda", dtype=torch.float32, requires_grad=True)
    xr = x.detach().clone().requires_grad_()
    br = bias.detach().clone().requires_grad_()
    actual = operation(x, bias)
    expected = torch.relu(xr + br)
    torch.testing.assert_close(actual, expected)
    upstream = torch.randn_like(actual)
    actual.backward(upstream)
    expected.backward(upstream)
    torch.testing.assert_close(x.grad, xr.grad)
    torch.testing.assert_close(bias.grad, br.grad)
    results = torch.library.opcheck(operation, (x, bias))
    assert all(result == "SUCCESS" for result in results.values())
```

tests/test_roofline.py

```python
import math
import pytest
from gpu_prep.roofline import (
    amdahl_speedup, gemm_workload, roofline_estimate, vector_add_workload,
)


def test_vector_add_units_and_roofline():
    result = roofline_estimate(vector_add_workload(1_000_000), 100.0, 2.0)
    assert result["arithmetic_intensity_flop_per_byte"] == pytest.approx(1 / 12)
    assert result["ridge_point_flop_per_byte"] == pytest.approx(50)
    assert result["ideal_time_lower_bound_ms"] == pytest.approx(0.006)
    assert result["roofline_tflops"] == pytest.approx(1 / 6)


def test_gemm_changes_regime_with_shape():
    small_m = roofline_estimate(gemm_workload(1, 4096, 4096), 100, 2)
    square = roofline_estimate(gemm_workload(1024, 1024, 1024), 100, 2)
    assert small_m["model_limiting_resource"] == "memory"
    assert square["model_limiting_resource"] == "compute"
    assert square["arithmetic_intensity_flop_per_byte"] == pytest.approx(1024 / 6)


def test_empty_work_and_zero_k_have_distinct_traffic():
    assert roofline_estimate(gemm_workload(0, 3, 4), 100, 2)["ideal_time_lower_bound_ms"] == 0
    zero_k = gemm_workload(2, 3, 0)
    assert zero_k.flops == 0
    assert zero_k.ideal_bytes == 24


def test_amdahl_limits_and_latency_vs_throughput():
    assert amdahl_speedup(0, 2) == 1
    assert amdahl_speedup(1, 2) == 2
    speedup = amdahl_speedup(0.05, 2)
    assert speedup == pytest.approx(1 / 0.975)
    assert 1 - 1 / speedup == pytest.approx(0.025)


@pytest.mark.parametrize("bad", [-1, 1.5, True])
def test_dimensions_are_explicit(bad):
    with pytest.raises(ValueError):
        gemm_workload(bad, 3, 4)


@pytest.mark.parametrize("bad", [0, -1, math.inf, math.nan])
def test_invalid_roofline_hardware(bad):
    with pytest.raises(ValueError):
        roofline_estimate(vector_add_workload(10), bad, 2)
```

tests/test_torch_layers.py

```python
"""CPU-executable mathematical and integration checks for exercises 7--9.

GPU kernel correctness/performance is deliberately not inferred from these tests.
"""

import pytest
import torch
from torch.nn import functional as F
```

```python
from gpu_prep.torch_layers import (
    KVCache,
    ManualLayerNorm,
    ManualLinear,
    ManualRMSNorm,
    MultiHeadSelfAttention,
    scaled_dot_product_attention,
    split_qkv_heads,
)


@pytest.fixture(autouse=True)
def _seed():
    torch.manual_seed(314159)


@pytest.mark.parametrize("shape", [(4,), (3, 4), (2, 3, 4), (0, 3, 4)])
@pytest.mark.parametrize("use_bias", [False, True])
def test_linear_matches_reference_forward_and_backward(shape, use_bias):
    layer = ManualLinear(4, 5, bias=use_bias, dtype=torch.float64)
    x = torch.randn(shape, dtype=torch.float64, requires_grad=True)
    xr = x.detach().clone().requires_grad_()
    wr = layer.weight.detach().clone().requires_grad_()
    br = layer.bias.detach().clone().requires_grad_() if use_bias else None
    actual = layer(x)
    expected = F.linear(xr, wr, br)
    torch.testing.assert_close(actual, expected, atol=1e-12, rtol=1e-12)
    upstream = torch.randn_like(actual)
    actual.backward(upstream)
    expected.backward(upstream)
    torch.testing.assert_close(x.grad, xr.grad)
    torch.testing.assert_close(layer.weight.grad, wr.grad)
    if use_bias:
        torch.testing.assert_close(layer.bias.grad, br.grad)
    assert "weight" in dict(layer.named_parameters())


def test_qkv_split_preserves_feature_order_and_supports_noncontiguous_input():
    # Use visibly different numbers in each Q/K/V segment; a wrong permutation
    # can otherwise look plausible when tested only by shape.
    source = torch.arange(2 * 4 * 24, dtype=torch.float64).reshape(2, 4, 24)
    packed = source[:, ::2, :]
    assert not packed.is_contiguous()
    q, k, v = split_qkv_heads(packed, num_heads=2)
    assert q.shape == (2, 2, 2, 4)
    for b in range(2):
        for h in range(2):
            for t in range(2):
                torch.testing.assert_close(q[b, h, t], packed[b, t, h * 4 : h * 4 + 4])
                torch.testing.assert_close(k[b, h, t], packed[b, t, 8 + h * 4 : 12 + h * 4])
                torch.testing.assert_close(v[b, h, t], packed[b, t, 16 + h * 4 : 20 + h * 4])


@pytest.mark.parametrize("shape,normalized", [((4,), 4), ((2, 3, 4), 4), ((2, 3, 4), (3, 4)), ((0, 3, 4), 4)])
@pytest.mark.parametrize("norm_class", [ManualLayerNorm, ManualRMSNorm])
def test_norm_matches_pytorch_forward_and_backward(shape, normalized, norm_class):
    layer = norm_class(normalized, eps=1e-5, dtype=torch.float64)
    with torch.no_grad():
        layer.weight.uniform_(0.5, 1.5)
        if isinstance(layer, ManualLayerNorm):
            layer.bias.normal_()
    x = torch.randn(shape, dtype=torch.float64, requires_grad=True)
    xr = x.detach().clone().requires_grad_()
    wr = layer.weight.detach().clone().requires_grad_()
    normalized_tuple = (normalized,) if isinstance(normalized, int) else normalized
    if norm_class is ManualLayerNorm:
        br = layer.bias.detach().clone().requires_grad_()
```

```
        reference = F.layer_norm(xr, normalized_tuple, wr, br, eps=1e-5)
    else:
        reference = F.rms_norm(xr, normalized_tuple, wr, eps=1e-5)
    actual = layer(x)
    torch.testing.assert_close(actual, reference, atol=1e-11, rtol=1e-11)
    upstream = torch.randn_like(actual)
    actual.backward(upstream)
    reference.backward(upstream)
    torch.testing.assert_close(x.grad, xr.grad, atol=1e-10, rtol=1e-10)
    torch.testing.assert_close(layer.weight.grad, wr.grad, atol=1e-10, rtol=1e-10)
    if norm_class is ManualLayerNorm:
        torch.testing.assert_close(layer.bias.grad, br.grad, atol=1e-10, rtol=1e-10)


def test_layernorm_statistics_constant_and_single_feature():
    layer = ManualLayerNorm(4, elementwise_affine=False, dtype=torch.float64)
    x = torch.tensor([[1.0, 2.0, 4.0, 8.0]], dtype=torch.float64)
    y = layer(x)
    torch.testing.assert_close(y.mean(-1), torch.zeros(1, dtype=x.dtype), atol=1e-14, rtol=0)
    original_variance = x.var(-1, correction=0)
    expected_variance = original_variance / (original_variance + layer.eps)
    torch.testing.assert_close(y.var(-1, correction=0), expected_variance)
    constant = torch.full((2, 4), 7.0, dtype=torch.float64, requires_grad=True)
    out = layer(constant)
    assert torch.equal(out, torch.zeros_like(out))
    out.sum().backward()
    assert torch.isfinite(constant.grad).all()
    singleton = ManualLayerNorm(1, elementwise_affine=False)(torch.tensor([[3.0], [-2.0]]))
    assert torch.equal(singleton, torch.zeros_like(singleton))


@pytest.mark.parametrize("dtype", [torch.float16, torch.bfloat16])
@pytest.mark.parametrize("norm_class", [ManualLayerNorm, ManualRMSNorm])
def test_low_precision_norm_accumulates_in_float32(dtype, norm_class):
    layer = norm_class(7, eps=1e-5)  # fp32 parameters with a low-precision input
    x = (torch.randn(2, 3, 7) * 5).to(dtype)
    actual = layer(x)
    if norm_class is ManualLayerNorm:
        expected = F.layer_norm(x.float(), (7,), layer.weight, layer.bias, eps=1e-5).to(dtype)
    else:
        expected = F.rms_norm(x.float(), (7,), layer.weight, eps=1e-5).to(dtype)
    assert actual.dtype == dtype
    torch.testing.assert_close(actual, expected, atol=2e-3 if dtype == torch.float16 else 2e-2, rtol=2e-3 if dtype
== torch.float16 else 2e-2)


@pytest.mark.parametrize("dtype", [torch.float16, torch.bfloat16, torch.float32])
@pytest.mark.parametrize("norm_class", [ManualLayerNorm, ManualRMSNorm])
@pytest.mark.parametrize("eps", [1e-300, 1e300])
def test_norm_rejects_epsilon_that_rounds_to_zero_or_inf_in_accumulator(dtype, norm_class, eps):
    layer = norm_class(3, eps=eps, dtype=dtype)
    with pytest.raises(ValueError, match="represented as float32"):
        layer(torch.zeros(2, 3, dtype=dtype))


@pytest.mark.parametrize("norm_class", [ManualLayerNorm, ManualRMSNorm])
def test_float64_norm_keeps_valid_tiny_epsilon(norm_class):
    layer = norm_class(3, eps=1e-300, dtype=torch.float64)
    output = layer(torch.zeros(2, 3, dtype=torch.float64))
    assert output.dtype == torch.float64
    assert torch.equal(output, torch.zeros_like(output))


@pytest.mark.parametrize("norm_class", [ManualLayerNorm, ManualRMSNorm])
def test_float32_norm_accepts_positive_representable_subnormal_epsilon(norm_class):
    # The requirement is representability, not eps >= torch.finfo(float32).tiny.
    # Check with the default denormal handling used by this CPU environment.
```

```
    layer = norm_class(3, eps=1e-40, dtype=torch.float32)
    output = layer(torch.zeros(2, 3, dtype=torch.float32))
    assert torch.equal(output, torch.zeros_like(output))


@pytest.mark.parametrize("norm_class", [ManualLayerNorm, ManualRMSNorm])
def test_norm_float64_gradcheck_including_parameters(norm_class):
    layer = norm_class(3, dtype=torch.float64)
    x = torch.randn(2, 3, dtype=torch.float64, requires_grad=True)
    weight = torch.randn(3, dtype=torch.float64, requires_grad=True)
    if norm_class is ManualLayerNorm:
        bias = torch.randn(3, dtype=torch.float64, requires_grad=True)
        fn = lambda a, w, b: torch.func.functional_call(layer, {"weight": w, "bias": b}, (a,))
        assert torch.autograd.gradcheck(fn, (x, weight, bias))
    else:
        fn = lambda a, w: torch.func.functional_call(layer, {"weight": w}, (a,))
        assert torch.autograd.gradcheck(fn, (x, weight))


def test_attention_matches_sdpa_with_explicit_padding_and_causal_mask():
    q = torch.randn(2, 2, 3, 4, dtype=torch.float64, requires_grad=True)
    k = torch.randn(2, 2, 5, 4, dtype=torch.float64, requires_grad=True)
    v = torch.randn(2, 2, 5, 6, dtype=torch.float64, requires_grad=True)
    # Queries have absolute positions 2,3,4. Every row has at least one key.
    causal = torch.arange(5)[None, :] <= (torch.arange(3) + 2)[:, None]
    padding = torch.tensor([[True, True, True, False, False], [True, True, True, True, True]])
    combined = causal & padding[:, None, None, :]
    actual = scaled_dot_product_attention(q, k, v, attn_mask=padding[:, None, None, :], is_causal=True,
query_start=2)
    qr, kr, vr = (t.detach().clone().requires_grad_() for t in (q, k, v))
    expected = F.scaled_dot_product_attention(qr, kr, vr, attn_mask=combined, dropout_p=0.0, is_causal=False)
    torch.testing.assert_close(actual, expected, atol=1e-11, rtol=1e-11)
    upstream = torch.randn_like(actual)
    actual.backward(upstream)
    expected.backward(upstream)
    for own, ref in ((q, qr), (k, kr), (v, vr)):
        torch.testing.assert_close(own.grad, ref.grad, atol=1e-10, rtol=1e-10)


def test_non_square_causality_has_correct_absolute_offset():
    q = torch.zeros(1, 1, 2, 1, dtype=torch.float64)
    k = torch.zeros(1, 1, 4, 1, dtype=torch.float64)
    v = torch.tensor([1.0, 3.0, 7.0, 9.0], dtype=torch.float64).reshape(1, 1, 4, 1)
    out = scaled_dot_product_attention(q, k, v, is_causal=True, query_start=2)
    expected = torch.tensor([11.0 / 3.0, 5.0], dtype=torch.float64).reshape(1, 1, 2, 1)
    torch.testing.assert_close(out, expected)


def test_all_masked_attention_has_zero_output_and_finite_zero_gradients():
    q = torch.randn(1, 2, 3, 4, dtype=torch.float64, requires_grad=True)
    k = torch.randn(1, 2, 5, 4, dtype=torch.float64, requires_grad=True)
    v = torch.randn(1, 2, 5, 2, dtype=torch.float64, requires_grad=True)
    out, weights = scaled_dot_product_attention(q, k, v, attn_mask=torch.zeros(3, 5, dtype=torch.bool),
return_weights=True)
    assert torch.equal(out, torch.zeros_like(out))
    assert torch.equal(weights, torch.zeros_like(weights))
    out.sum().backward()
    for tensor in (q, k, v):
        assert torch.isfinite(tensor.grad).all()
        assert torch.equal(tensor.grad, torch.zeros_like(tensor.grad))


def test_attention_gradcheck_with_partially_and_fully_masked_rows():
    args = tuple(torch.randn(*shape, dtype=torch.float64, requires_grad=True) for shape in
                 ((1, 1, 2, 2), (1, 1, 3, 2), (1, 1, 3, 2)))
    mask = torch.tensor([[True, False, True], [False, False, False]])
    fn = lambda a, b, c: scaled_dot_product_attention(a, b, c, attn_mask=mask)
```

```
    assert torch.autograd.gradcheck(fn, args)


def test_outer_autocast_does_not_downgrade_attention_accumulators():
    q = torch.randn(1, 2, 3, 4).bfloat16()
    k = torch.randn(1, 2, 5, 4).bfloat16()
    v = torch.randn(1, 2, 5, 6).bfloat16()
    with torch.autocast("cpu", dtype=torch.bfloat16):
        out, probabilities = scaled_dot_product_attention(q, k, v, return_weights=True)
    assert probabilities.dtype == torch.float32
    assert out.dtype == torch.bfloat16
    reference = F.scaled_dot_product_attention(q.float(), k.float(), v.float(), dropout_p=0.0)
    torch.testing.assert_close(out, reference.bfloat16(), atol=1e-2, rtol=1e-2)


@pytest.mark.parametrize("length,key_length", [(0, 3), (2, 0), (0, 0)])
def test_empty_attention_preserves_shapes_and_autograd(length, key_length):
    q = torch.randn(1, 2, length, 4, dtype=torch.float64, requires_grad=True)
    k = torch.randn(1, 2, key_length, 4, dtype=torch.float64, requires_grad=True)
    v = torch.randn(1, 2, key_length, 6, dtype=torch.float64, requires_grad=True)
    out = scaled_dot_product_attention(q, k, v)
    assert out.shape == (1, 2, length, 6)
    out.sum().backward()
    for tensor in (q, k, v):
        assert tensor.grad is not None
        assert torch.equal(tensor.grad, torch.zeros_like(tensor.grad))


def test_mha_matches_reference_and_does_not_leak_future_tokens():
    model = MultiHeadSelfAttention(8, 2, dropout=0.0, dtype=torch.float64).eval()
    x = torch.randn(2, 5, 8, dtype=torch.float64)
    output = model(x)
    q, k, v = split_qkv_heads(F.linear(x, model.qkv.weight, model.qkv.bias), 2)
    mask = torch.ones(5, 5, dtype=torch.bool).tril()
    attended = F.scaled_dot_product_attention(q, k, v, attn_mask=mask, dropout_p=0.0)
    expected = F.linear(attended.transpose(1, 2).reshape(2, 5, 8), model.out_proj.weight, model.out_proj.bias)
    torch.testing.assert_close(output, expected, atol=1e-11, rtol=1e-11)
    modified = x.clone()
    modified[:, 3:] = torch.randn_like(modified[:, 3:]) * 20
    torch.testing.assert_close(model(modified)[:, :3], output[:, :3], atol=1e-11, rtol=1e-11)


def test_eval_disables_attention_dropout():
    model = MultiHeadSelfAttention(8, 2, dropout=0.8).eval()
    x = torch.randn(2, 4, 8)
    first = model(x)
    second = model(x)
    torch.testing.assert_close(first, second, atol=0, rtol=0)


def test_cache_prefill_and_decode_match_full_causal_attention():
    model = MultiHeadSelfAttention(8, 2, dtype=torch.float64).eval()
    x = torch.randn(2, 7, 8, dtype=torch.float64)
    cache = KVCache(2, 2, 7, 4, device="cpu", dtype=torch.float64)
    initial_storage = cache.key.data_ptr()
    with torch.inference_mode():
        expected = model(x)
        pieces = [model.forward_step(x[:, :3], cache)]
        pieces.append(model.forward_step(x[:, 3:4], cache))
        pieces.append(model.forward_step(x[:, 4:6], cache))  # non-square causal chunk
        pieces.append(model.forward_step(x[:, 6:7], cache))
        actual = torch.cat(pieces, dim=1)
    torch.testing.assert_close(actual, expected, atol=1e-11, rtol=1e-11)
    assert cache.length == 7
    assert cache.key.data_ptr() == initial_storage
    cache.reset()
    assert cache.length == 0
```

```
    assert cache.get()[0].shape[-2] == 0


def test_cache_rejects_overflow_without_changing_active_prefix():
    cache = KVCache(1, 2, 3, 4, device="cpu", dtype=torch.float32)
    with torch.inference_mode():
        block = torch.randn(1, 2, 2, 4)
        cache.append(block, block)
        saved_key = cache.get()[0].clone()
        with pytest.raises(ValueError, match="capacity"):
            cache.append(block, block)
        assert cache.length == 2
        torch.testing.assert_close(cache.get()[0], saved_key)


def test_cache_is_explicitly_inference_only():
    cache = KVCache(1, 2, 3, 4, device="cpu", dtype=torch.float32)
    model = MultiHeadSelfAttention(8, 2)
    x = torch.randn(1, 1, 8)
    with torch.no_grad(), pytest.raises(RuntimeError, match="eval"):
        model.forward_step(x, cache)
    model.eval()
    with pytest.raises(RuntimeError, match="no_grad"):
        model.forward_step(x, cache)


def test_invalid_shapes_and_masks_are_reported():
    with pytest.raises(ValueError):
        ManualLinear(0, 4)
    with pytest.raises(ValueError):
        ManualLinear(4, 3)(torch.randn(2, 5))
    with pytest.raises(ValueError):
        ManualLayerNorm(0)
    with pytest.raises(ValueError):
        ManualLayerNorm(4)(torch.randn(2, 3))
    with pytest.raises(ValueError):
        MultiHeadSelfAttention(7, 2)
    with pytest.raises(ValueError):
        split_qkv_heads(torch.randn(1, 2, 17), 2)
    q = torch.randn(1, 1, 3, 4)
    with pytest.raises(TypeError, match="boolean"):
        scaled_dot_product_attention(q, q, q, attn_mask=torch.ones(3, 3))
    with pytest.raises(ValueError, match="broadcastable"):
        scaled_dot_product_attention(q, q, q, attn_mask=torch.ones(2, 3, dtype=torch.bool))
    with pytest.raises(ValueError, match="query_start"):
        scaled_dot_product_attention(q, q, q, query_start=2)
    with pytest.raises(TypeError, match="same dtype"):
        scaled_dot_product_attention(q, q.double(), q)
```

tests/test_triton_host.py

```
"""CPU-only checks for Triton wrapper metadata; these do not execute GPU kernels."""

import math
import struct

import pytest

pytest.importorskip("torch")
pytest.importorskip("triton")

from gpu_prep.triton_ops import _check_eps
```

```
@pytest.mark.parametrize("eps", [1e-5, 1.0, 1e-30, 1e-45, 3e38])
def test_eps_is_positive_finite_after_float32_conversion(eps):
    result = _check_eps(eps)
    assert result == struct.unpack(">f", struct.pack(">f", eps))[0]
    assert math.isfinite(result) and result > 0


@pytest.mark.parametrize("eps", [0.0, -1.0, float("nan"), float("inf"), -float("inf"), 1e-300, 1e300, 1e-46])
def test_eps_rejects_invalid_python_or_float32_values(eps):
    with pytest.raises(ValueError, match="eps"):
        _check_eps(eps)


@pytest.mark.parametrize("eps", [True, False, "1e-5", None])
def test_eps_rejects_nonreal_metadata(eps):
    with pytest.raises(TypeError, match="eps"):
        _check_eps(eps)
```

tests/test_triton_ops.py

```
"""Correctness tests requiring a real NVIDIA GPU; absent GPU/Triton => skip.

These tests intentionally include awkward shapes and explicit reference math.
They test correctness, not speed. Their existence is not evidence that a GPU
run has happened in the environment in which this package was authored.
"""

import importlib.util
import math

import pytest

try:
    import torch
except ModuleNotFoundError as exc:
    if exc.name != "torch":
        raise  # An installed but broken dependency is not an absent-package skip.
    torch = None

if torch is None:
    SKIP_REASON = "PyTorch is not installed"
elif importlib.util.find_spec("triton") is None:
    SKIP_REASON = "Triton is not installed"
elif not torch.cuda.is_available() or torch.version.hip is not None:
    SKIP_REASON = "Triton exercises require an NVIDIA CUDA GPU"
elif torch.cuda.get_device_capability()[0] < 8:
    SKIP_REASON = "Triton exercises target NVIDIA compute capability >= 8.0"
else:
    SKIP_REASON = ""

# Keep collecting tests even without a GPU so running this file alone exits
# normally with explicit skips, instead of pytest's exit 5 (nothing collected).
pytestmark = pytest.mark.skipif(bool(SKIP_REASON), reason=SKIP_REASON)
if not SKIP_REASON:
    from gpu_prep import triton_ops as ops

DTYPES = (torch.float32, torch.float16, torch.bfloat16) if torch is not None else ("torch_missing",)


@pytest.fixture(autouse=True)
def _seed():
    torch.manual_seed(2027)
```

```python
def _randn(shape, dtype):
    return torch.randn(shape, dtype=dtype, device="cuda")


def _assert_close(actual, reference, *, kind="normal"):
    assert actual.shape == reference.shape
    assert actual.dtype == reference.dtype
    assert actual.device == reference.device
    if actual.dtype == torch.float32:
        atol, rtol = (3e-4, 5e-5) if kind == "sum" else (4e-5, 2e-4)
    elif actual.dtype == torch.float16:
        atol, rtol = (1e-2, 3e-3)
    else:
        atol, rtol = (3e-2, 2e-2)
    torch.testing.assert_close(actual, reference, atol=atol, rtol=rtol)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("n", [0, 1, 31, 32, 33, 255, 256, 257, 1_000_003])
def test_vector_add(dtype, n):
    x, y = _randn((n,), dtype), _randn((n,), dtype)
    out = ops.vector_add(x, y)
    reference = (x.float() + y.float()).to(dtype)
    torch.testing.assert_close(out, reference, atol=0, rtol=0)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("shape", [(0, 7), (3, 0), (1, 1), (3, 33), (5, 513)])
def test_bias_relu(dtype, shape):
    x, bias = _randn(shape, dtype), _randn((shape[1],), dtype)
    reference = (x.float() + bias.float()).clamp_min(0).to(dtype)
    torch.testing.assert_close(ops.bias_relu(x, bias), reference, atol=0, rtol=0)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("shape", [(0, 7), (3, 1), (5, 33), (3, 513), (7, 1001), (2, 8192)])
def test_row_reductions(dtype, shape):
    x = _randn(shape, dtype)
    # Double precision separates reference arithmetic from the kernel's reduction order.
    _assert_close(ops.row_sum(x), x.double().sum(dim=-1).to(dtype), kind="sum")
    torch.testing.assert_close(ops.row_max(x), x.amax(dim=-1), atol=0, rtol=0)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("shape", [(0, 7), (3, 1), (5, 33), (3, 513), (7, 1001), (2, 8192)])
def test_softmax(dtype, shape):
    x = _randn(shape, dtype)
    reference = torch.softmax(x.double(), dim=-1).to(dtype)
    actual = ops.softmax(x)
    _assert_close(actual, reference)
    if actual.numel():
        torch.testing.assert_close(
            actual.float().sum(dim=-1), torch.ones(shape[0], device="cuda"),
            atol=1e-2 if dtype == torch.bfloat16 else 1e-3, rtol=0,
        )


@pytest.mark.parametrize("dtype", DTYPES)
def test_softmax_stability(dtype):
    x = torch.tensor(
        [[1000, 1001, -1000], [17, 17, 17], [-1000, -1001, -1002]],
        dtype=dtype, device="cuda",
    )
    actual = ops.softmax(x)
    assert torch.isfinite(actual).all()
    _assert_close(actual, torch.softmax(x.double(), dim=-1).to(dtype))
```

```
@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("shape", [(0, 23), (3, 0), (1, 1), (31, 33), (1003, 769)])
@pytest.mark.parametrize("name", ["transpose_naive", "transpose_tiled"])
def test_transpose(dtype, shape, name):
    x = _randn(shape, dtype)
    actual = getattr(ops, name)(x)
    assert actual.is_contiguous()
    torch.testing.assert_close(actual, x.T.contiguous(), atol=0, rtol=0)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("m,k,n", [
    (0, 3, 7), (7, 3, 0), (5, 0, 9), (1, 1, 1),
    (7, 19, 3), (31, 33, 17), (127, 93, 65),
])
@pytest.mark.parametrize("name", ["matmul_naive", "matmul_tiled"])
def test_matmul(dtype, m, k, n, name):
    a, b = _randn((m, k), dtype), _randn((k, n), dtype)
    reference = (a.double() @ b.double()).to(dtype)
    actual = getattr(ops, name)(a, b)
    _assert_close(actual, reference)
    if k == 0:
        assert torch.count_nonzero(actual) == 0


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("shape", [(1,), (33,), (0, 7), (2, 3, 1), (2, 3, 33), (3, 1001), (2, 8192)])
def test_norms(dtype, shape):
    x = _randn(shape, dtype)
    weight, bias = _randn((shape[-1],), dtype), _randn((shape[-1],), dtype)
    eps = 1e-5
    xf, wf, bf = x.double(), weight.double(), bias.double()
    mean = xf.mean(dim=-1, keepdim=True)
    variance = ((xf - mean) ** 2).mean(dim=-1, keepdim=True)
    layer_reference = ((xf - mean) * torch.rsqrt(variance + eps) * wf + bf).to(dtype)
    rms_reference = (xf * torch.rsqrt((xf * xf).mean(dim=-1, keepdim=True) + eps) * wf).to(dtype)
    _assert_close(ops.layer_norm(x, weight, bias, eps), layer_reference)
    _assert_close(ops.rms_norm(x, weight, eps), rms_reference)


@pytest.mark.parametrize("dtype", DTYPES)
def test_norm_constant_row_and_padding(dtype):
    # A constant row must stay well behaved when variance is zero.
    x = torch.full((3, 33), 7.0, dtype=dtype, device="cuda")
    w, b = _randn((33,), dtype), _randn((33,), dtype)
    actual = ops.layer_norm(x, w, b)
    torch.testing.assert_close(actual, b.expand_as(x), atol=2e-3, rtol=2e-3)


def test_norm_padding_does_not_pollute_variance():
    # With a large nonzero mean, mistakenly counting padded (0-mean)^2 values
    # would dominate the variance and almost erase the normalized signal.
    x = _randn((3, 33), torch.float32) + 100.0
    w = torch.ones(33, device="cuda")
    b = torch.zeros(33, device="cuda")
    xf = x.double()
    centered = xf - xf.mean(dim=-1, keepdim=True)
    reference = (centered * torch.rsqrt((centered * centered).mean(dim=-1, keepdim=True) + 1e-5)).float()
    torch.testing.assert_close(ops.layer_norm(x, w, b), reference, atol=1e-4, rtol=1e-4)


def _attention_reference(q, k, v, *, causal=False, query_start=0, attn_mask=None):
    """Explicit double-precision reference with the same all-masked-row policy."""
    _, _, length, d = q.shape
    s = k.shape[2]
    scores = (q.double() @ k.double().transpose(-2, -1)) / math.sqrt(d)
```

```
    visible = torch.ones_like(scores, dtype=torch.bool)
    if causal:
        query_position = query_start + torch.arange(length, device=q.device)
        key_position = torch.arange(s, device=q.device)
        visible = visible & (key_position[None, :] <= query_position[:, None])
    if attn_mask is not None:
        visible = visible & attn_mask
    scores = scores.masked_fill(~visible, -float("inf"))
    maximum = scores.amax(dim=-1, keepdim=True)
    safe_maximum = torch.where(torch.isfinite(maximum), maximum, torch.zeros_like(maximum))
    numerator = torch.exp(scores - safe_maximum)
    denominator = numerator.sum(dim=-1, keepdim=True)
    safe_denominator = torch.where(denominator > 0, denominator, torch.ones_like(denominator))
    probabilities = numerator / safe_denominator
    return (probabilities @ v.double()).to(q.dtype)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("b,h,length,s,d", [
    (1, 2, 1, 13, 16), (2, 2, 17, 37, 32),
    (1, 1, 33, 17, 64), (1, 2, 7, 65, 128),
])
@pytest.mark.parametrize("causal,query_start", [(False, 0), (True, 0), (True, 23)])
def test_attention(dtype, b, h, length, s, d, causal, query_start):
    q = _randn((b, h, length, d), dtype)
    k, v = _randn((b, h, s, d), dtype), _randn((b, h, s, d), dtype)
    actual = ops.attention(q, k, v, causal=causal, query_start=query_start)
    reference = _attention_reference(q, k, v, causal=causal, query_start=query_start)
    _assert_close(actual, reference)


@pytest.mark.parametrize("dtype", DTYPES)
@pytest.mark.parametrize("batched_mask", [False, True])
def test_attention_masked_rows_and_empty_early_tiles(dtype, batched_mask):
    b, h, length, s, d = 2, 2, 19, 65, 32
    q = _randn((b, h, length, d), dtype)
    k, v = _randn((b, h, s, d), dtype), _randn((b, h, s, d), dtype)
    mask = torch.ones((length, s), dtype=torch.bool, device="cuda")
    mask[0, :] = False             # Empty row for every key tile.
    mask[1, :] = False
    mask[1, 40] = True             # First tile empty; only a later key becomes visible.
    mask[2, 32:] = False           # Valid early tile, then entirely empty later tiles.
    if batched_mask:
        mask = mask.expand(b, h, length, s).clone()
        mask[1, 1, 3, :] = False   # A different empty row in just one B/H slice.
    actual = ops.attention(q, k, v, attn_mask=mask)
    reference = _attention_reference(q, k, v, attn_mask=mask)
    assert torch.isfinite(actual).all()
    assert torch.count_nonzero(actual[:, :, 0]) == 0
    _assert_close(actual[:, :, 1], v[:, :, 40])
    _assert_close(actual, reference)


def test_attention_causal_and_explicit_mask_are_combined():
    q = _randn((1, 2, 17, 32), torch.float32)
    k, v = _randn((1, 2, 37, 32), torch.float32), _randn((1, 2, 37, 32), torch.float32)
    mask = torch.rand((17, 37), device="cuda") > 0.4
    mask[0] = False
    actual = ops.attention(q, k, v, causal=True, query_start=20, attn_mask=mask)
    reference = _attention_reference(q, k, v, causal=True, query_start=20, attn_mask=mask)
    _assert_close(actual, reference)


def test_decode_offset_matches_last_row_of_full_causal_attention():
    q, k, v = [_randn((1, 2, 37, 32), torch.float32) for _ in range(3)]
    full = ops.attention(q, k, v, causal=True)
    decode = ops.attention(q[:, :, -1:].contiguous(), k, v, causal=True, query_start=36)
```

```
    _assert_close(decode, full[:, :, -1:])


@pytest.mark.parametrize("b,h,length", [(0, 2, 17), (1, 0, 17), (1, 2, 0)])
def test_attention_empty_output(b, h, length):
    q = _randn((b, h, length, 32), torch.float32)
    k, v = [_randn((b, h, 7, 32), torch.float32) for _ in range(2)]
    actual = ops.attention(q, k, v)
    assert actual.shape == q.shape and actual.numel() == 0


@pytest.mark.parametrize("name", ["row_sum", "row_max", "softmax"])
@pytest.mark.parametrize("n", [0, 8193])
def test_reduction_width_validation(name, n):
    with pytest.raises(ValueError, match="row width"):
        getattr(ops, name)(_randn((3, n), torch.float32))


def test_reject_noncontiguous_inputs_and_bad_shapes():
    x = _randn((4, 6), torch.float32)
    with pytest.raises(ValueError, match="contiguous"):
        ops.row_sum(x.T)
    with pytest.raises(ValueError, match="identical shapes"):
        ops.vector_add(x.flatten(), x.flatten()[:3])
    with pytest.raises(ValueError, match="bias must have shape"):
        ops.bias_relu(x, _randn((5,), x.dtype))
    with pytest.raises(ValueError, match="a.shape"):
        ops.matmul_tiled(x, _randn((5, 2), x.dtype))
    with pytest.raises(ValueError, match="same device and dtype"):
        ops.vector_add(x.flatten(), x.flatten().half())


def test_forward_only_and_no_grad_inference():
    x = _randn((17,), torch.float32).requires_grad_()
    y = _randn((17,), torch.float32)
    with pytest.raises(ValueError, match="forward-only"):
        ops.vector_add(x, y)
    with torch.no_grad():
        actual = ops.vector_add(x, y)
    assert not actual.requires_grad
    torch.testing.assert_close(actual, x.detach() + y, atol=0, rtol=0)


@pytest.mark.parametrize("eps", [0.0, -1.0, float("nan"), float("inf")])
def test_invalid_norm_eps(eps):
    x, w = _randn((3, 7), torch.float32), _randn((7,), torch.float32)
    with pytest.raises(ValueError, match="eps"):
        ops.rms_norm(x, w, eps)


def test_invalid_norm_width():
    x, w = _randn((3, 0), torch.float32), _randn((0,), torch.float32)
    with pytest.raises(ValueError, match="normalized dimension"):
        ops.rms_norm(x, w)


def test_attention_input_validation():
    q = _randn((1, 2, 3, 32), torch.float32)
    k, v = [_randn((1, 2, 7, 32), torch.float32) for _ in range(2)]
    with pytest.raises(ValueError, match="positions"):
        ops.attention(q, k, v, query_start=-1)
    with pytest.raises(TypeError, match="Python int"):
        ops.attention(q, k, v, query_start=True)
    with pytest.raises(ValueError, match="length S"):
        ops.attention(q, k[:, :, :0].contiguous(), v[:, :, :0].contiguous())
    with pytest.raises(ValueError, match="head dimension"):
        ops.attention(q[..., :24].contiguous(), k[..., :24].contiguous(), v[..., :24].contiguous())
```

```
    with pytest.raises(ValueError, match="bool"):
        ops.attention(q, k, v, attn_mask=torch.ones(3, 7, device="cuda"))
    with pytest.raises(ValueError, match="shape"):
        ops.attention(q, k, v, attn_mask=torch.ones(1, 1, 3, 7, dtype=torch.bool, device="cuda"))


def test_respects_nondefault_stream():
    x = torch.zeros(1001, device="cuda")
    y = torch.zeros_like(x)
    torch.cuda.synchronize()
    stream = torch.cuda.Stream()
    with torch.cuda.stream(stream):
        if hasattr(torch.cuda, "_sleep"):
            torch.cuda._sleep(1_000_000)  # Delay producer to expose a wrong default-stream launch.
        x.fill_(3.0)
        y.fill_(5.0)
        actual = ops.vector_add(x, y)
    stream.synchronize()
    torch.cuda.synchronize()
    torch.testing.assert_close(actual, torch.full_like(actual, 8.0), atol=0, rtol=0)


@pytest.mark.skipif(torch is None or torch.cuda.device_count() < 2, reason="requires two CUDA devices")
def test_nondefault_device_and_current_device_restore():
    original_device = torch.cuda.current_device()
    other_device = 1 if original_device == 0 else 0
    if torch.cuda.get_device_capability(other_device)[0] < 8:
        pytest.skip("other device is older than the supported Triton target")
    x = torch.randn(257, device=f"cuda:{other_device}")
    y = torch.randn_like(x)
    actual = ops.vector_add(x, y)
    assert torch.cuda.current_device() == original_device
    torch.testing.assert_close(actual, x + y, atol=0, rtol=0)
```

tools/build_book.py

```
"""Build a self-contained Chinese textbook from chapters and exact source files."""
from __future__ import annotations

import argparse
import ast
import base64
import html
import json
import os
from pathlib import Path
import re
import shutil
import subprocess

import markdown
from pygments.formatters import HtmlFormatter


ROOT = Path(__file__).resolve().parents[1]
BOOK_NAME = "GPU_Interview_12_Exercises_ZH"
FENCE = chr(96) * 3
CHAPTERS = [
    ("01_cuda.zh.md", "CUDA C++"),
    ("02_triton.zh.md", "Triton"),
    ("03_torch_layers.zh.md", "PyTorch"),
    ("04_compile_custom_ops.zh.md", "编译与算子注册"),
    ("05_roofline_benchmark.zh.md", "模型与实验"),
]
```

```
TITLES = {
    1: "Vector Add 与 Bias + ReLU 融合",
    2: "Row Sum 与 Row Max 归约",
    3: "数值稳定的行 Softmax",
    4: "Naive 与 Tiled 物理转置",
    5: "Naive 与 Tiled GEMM",
    6: "Roofline：工作量、流量与瓶颈模型",
    7: "Linear、QKV、Shape 与 Stride",
    8: "LayerNorm 与 RMSNorm：前向、反向与 GPU 实现",
    9: "Attention、MHA、Causal Mask 与 KV Cache",
    10: "torch.compile：Graph Break、Guard 与稳态性能",
    11: "自定义 CUDA / Triton 算子：Fake、Autograd 与编译",
    12: "性能实验、诊断与项目答辩",
}
CODE_MAP = {
    1: {
        "gpu_prep/csrc/kernels.cu": ["vector_add_kernel", "bias_relu_kernel", "vector_add_cuda",
"bias_relu_cuda"],
        "gpu_prep/triton_ops.py": ["_vector_add_kernel", "vector_add", "_bias_relu_kernel", "bias_relu"],
    },
    2: {
        "gpu_prep/csrc/kernels.cu": ["block_reduce", "row_reduce_kernel", "row_reduce_impl", "row_sum_cuda",
"row_max_cuda"],
        "gpu_prep/triton_ops.py": ["_row_reduce_kernel", "_row_reduce", "row_sum", "row_max"],
    },
    3: {
        "gpu_prep/csrc/kernels.cu": ["softmax_kernel", "softmax_cuda"],
        "gpu_prep/triton_ops.py": ["_softmax_kernel", "softmax"],
    },
    4: {
        "gpu_prep/csrc/kernels.cu": ["transpose_naive_kernel", "transpose_tiled_kernel", "transpose_impl",
"transpose_naive_cuda", "transpose_tiled_cuda"],
        "gpu_prep/triton_ops.py": ["_transpose_naive_kernel", "_transpose_tiled_kernel", "_transpose",
"transpose_naive", "transpose_tiled"],
    },
    5: {
        "gpu_prep/csrc/kernels.cu": ["matmul_naive_kernel", "matmul_tiled_kernel", "matmul_impl",
"matmul_naive_cuda", "matmul_tiled_cuda"],
        "gpu_prep/triton_ops.py": ["_matmul_naive_kernel", "_matmul_tiled_kernel", "_matmul", "matmul_naive",
"matmul_tiled"],
    },
    6: {"gpu_prep/roofline.py": ["Workload", "vector_add_workload", "gemm_workload", "roofline_estimate",
"amdahl_speedup"]},
    7: {"gpu_prep/torch_layers.py": ["ManualLinear", "split_qkv_heads"]},
    8: {
        "gpu_prep/torch_layers.py": ["ManualLayerNorm", "ManualRMSNorm"],
        "gpu_prep/csrc/kernels.cu": ["norm_kernel", "layer_norm_cuda", "rms_norm_cuda"],
        "gpu_prep/triton_ops.py": ["_norm_kernel", "_norm_shape", "layer_norm", "rms_norm"],
    },
    9: {
        "gpu_prep/torch_layers.py": ["scaled_dot_product_attention", "KVCache", "MultiHeadSelfAttention"],
        "gpu_prep/csrc/kernels.cu": ["attention_kernel", "attention_cuda"],
        "gpu_prep/triton_ops.py": ["_attention_kernel", "attention"],
    },
    10: {"examples/compile_demo.py": ["bad_python_branch", "tensor_where", "tensor_cond", "median_wall_ms",
"main"]},
    11: {"gpu_prep/custom_ops.py": None},
    12: {"gpu_prep/benchmark.py": ["Case", "build_case", "make_callable", "check_output", "measure_cuda",
"summarize_samples", "main"]},
}


def source_anchor(relative: str) -> str:
    return "source-" + re.sub(r"[^a-z0-9]+", "-", relative.lower()).strip("-")


def shift_headings(text: str, levels: int) -> str:
```

```
    lines, fenced = [], False
    for line in text.splitlines():
        if line.lstrip().startswith((FENCE, "~~~")):
            fenced = not fenced
        match = re.match(r"^(#{1,6})\s+(.*)$", line)
        if match and not fenced:
            line = "#" * min(6, len(match[1]) + levels) + " " + match[2]
        lines.append(line)
    return "\n".join(lines)


def extract_python(relative: str, names: list[str] | None) -> str:
    text = (ROOT / relative).read_text(encoding="utf-8")
    if names is None:
        return text
    nodes = {node.name: node for node in ast.parse(text).body
             if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef))}
    lines = text.splitlines()
    result = []
    for name in names:
        node = nodes[name]
        first = min([node.lineno] + [d.lineno for d in node.decorator_list]) - 1
        result.append("\n".join(lines[first:node.end_lineno]))
    return "\n\n\n".join(result) + "\n"


def extract_cuda(relative: str, names: list[str]) -> str:
    text = (ROOT / relative).read_text(encoding="utf-8")
    results = []
    for name in names:
        # Kernel source uses brace-balanced functions without raw-string literals.
        pattern = r"(?m)^(?:template\s*<[^\n]+>\n)?(?:__global__ void|__device__ float|at::Tensor)\s+" +
re.escape(name) + r"\s*\("
        match = re.search(pattern, text)
        if match is None:
            raise ValueError(f"Could not locate {relative}:{name}")
        opening = text.index("{", match.start())
        depth, position = 1, opening + 1
        # Strip comments/strings only for brace counting while preserving offsets.
        masked = list(text[opening + 1:])
        tail = text[opening + 1:]
        lexical = r'//[^\n]*|/\*.*?\*/|"(?:\\.|[^"\\])*"|\'(?:\\.|[^\'\\])*\''
        for token in re.finditer(lexical, tail, flags=re.S):
            for i in range(token.start(), token.end()):
                masked[i] = " "
        for character in masked:
            if character == "{":
                depth += 1
            elif character == "}":
                depth -= 1
            position += 1
            if depth == 0:
                break
        if depth:
            raise ValueError(f"Unbalanced braces: {name}")
        results.append(text[match.start():position])
    return "\n\n".join(results) + "\n"


def code_block(text: str, language: str) -> str:
    fence = FENCE
    while fence in text:
        fence += chr(96)
    return fence + language + "\n" + text.rstrip() + "\n" + fence + "\n"


def all_source_files() -> list[Path]:
```

```
    paths = []
    for folder in ("gpu_prep", "examples", "tests", "tools"):
        for path in (ROOT / folder).rglob("*"):
            if path.is_file() and path.suffix in (".py", ".cu", ".cpp", ".h", ".cjs") and "__pycache__" not in
path.parts:
                paths.append(path)
    for name in ("pyproject.toml", "requirements.txt", "MANIFEST.in"):
        paths.append(ROOT / name)
    return sorted(paths, key=lambda p: p.relative_to(ROOT).as_posix())


def assemble_markdown() -> tuple[str, dict]:
    sections = {number: [] for number in TITLES}
    prefaces = []
    for filename, label in CHAPTERS:
        content = (ROOT / "docs" / "chapters" / filename).read_text(encoding="utf-8")
        first_marker = content.find("<!-- exercise:")
        prefix = content[:first_marker].strip() if first_marker >= 0 else ""
        if prefix:
            prefaces.append(shift_headings(prefix, 1))
        for number, body in re.findall(
            r"<!-- exercise:(\d+):start -->(.*?)<!-- exercise:\1:end -->", content, flags=re.S
        ):
            sections[int(number)].append((label, body.strip()))
    if any(not content for content in sections.values()):
        raise ValueError("Every exercise must have an explanation.")
    parts = [
        "# GPU Programming 面试: 12 题完整代码与中文答案\n",
        "CUDA C++ · Triton · PyTorch · 数学推导 · 优化问答 · 测试与实验\n",
        "**阅读说明: ** 每道题先给讲解和追问答案，再列出本题核心函数的完整代码。"
        "章节中的代码由工程源码自动提取，没有省略函数体；公共依赖、完整模块、测试与工具在文末逐文件全文收录。"
        "直接运行请使用代码包中的工程，避免只复制一段函数后遗漏 import、常量或公共校验函数。\n",
        "[TOC]\n",
        shift_headings((ROOT / "docs" / "chapters" / "00_start.zh.md").read_text(encoding="utf-8"), 1),
    ]
    if (ROOT / "VALIDATION.md").exists():
        parts.append(shift_headings((ROOT / "VALIDATION.md").read_text(encoding="utf-8"), 1))
    parts.append("## 各后端共享的工程约定\n\n" + "\n\n".join(prefaces))
    function_count = 0
    for number, title in TITLES.items():
        parts.append(f'<a id="exercise-{number}"></a>\n\n## 第 {number} 题 | {title}\n')
        links = [f'[{path}](#{source_anchor(path)})' for path in CODE_MAP[number]]
        parts.append("**完整源文件: ** " + "; ".join(links) + "。\n")
        for label, body in sections[number]:
            parts.append(shift_headings(body, 1))
        if number == 6:
            svg = ROOT / "docs" / "assets" / "roofline_model.svg"
            if svg.exists():
                image_data = base64.b64encode(svg.read_bytes()).decode()
                parts.append('### Roofline 模型图\n\n'
                             '下图使用假想的 100 TFLOP/s、2 TB/s; 圆点表示数学屋顶上的位置，不是性能测量。\n\n'
                             f'![Roofline 模型，所有点均为计算上界而非 GPU 实测](
{image_data})\n')
        parts.append("### 本题核心实现: 函数体全文\n\n"
                     "以下内容直接来自随包源码。import、常量、共用的检查/归约辅助函数及所有绑定，见上面的完整源文件链接和文末源码附
录。\n")
        for relative, names in CODE_MAP[number].items():
            parts.append(f"#### {relative}\n")
            if relative.endswith(".cu"):
                extracted = extract_cuda(relative, names)
                language = "cuda"
            else:
                extracted = extract_python(relative, names)
                language = "python"
            function_count += len(names) if names else 1
            parts.append(code_block(extracted, language))
    parts.append(shift_headings((ROOT / "docs" / "SOURCES.md").read_text(encoding="utf-8"), 1))
```

```
    paths = all_source_files()
    parts.append("## 完整源码附录\n\n"
                 "下面逐文件给出工程源码、测试和构建工具的完整文本。文件之间存在 import/链接关系，"
                 "建议解压工程后直接运行；本附录用于离线阅读、查找和逐行对照。\n")
    source_lines = 0
    for path in paths:
        relative = path.relative_to(ROOT).as_posix()
        language = {".py": "python", ".cu": "cuda", ".cpp": "cpp", ".h": "cpp", ".cjs": "javascript", ".toml":
"toml"}.get(path.suffix, "text")
        text = path.read_text(encoding="utf-8")
        source_lines += len(text.splitlines())
        parts.append(f'<a id="{source_anchor(relative)}"></a>\n\n### {relative}\n')
        parts.append(code_block(text, language))
    stats = {
        "exercises": len(TITLES), "source_files_embedded_in_full": len(paths),
        "source_lines_embedded_in_full": source_lines, "chapter_code_sections": function_count,
        "explanation_blocks": {str(k): len(v) for k, v in sections.items()},
    }
    return "\n\n".join(parts).rstrip() + "\n", stats


def render_math(body: str, katex_dir: Path) -> tuple[str, int]:
    pattern = re.compile(
        r'<(?P<tag>span|div) class="arithmatex">\s*<(?:span|div) class="MathJax_Preview">.*?</(?:span|div)>'
        r'\s*<script type="math/tex(?P<display>; mode=display)?">(?P<tex>.*?)</script>\s*</(?P=tag)>',
        flags=re.S,
    )
    matches = list(pattern.finditer(body))
    data = [{"tex": html.unescape(m["tex"]), "display": bool(m["display"])} for m in matches]
    node = os.environ.get("CODEX_PRIMARY_RUNTIME_NODE") or shutil.which("node")
    if not node:
        raise RuntimeError("Node is required to typeset formulas at build time.")
    run = subprocess.run([node, str(ROOT / "tools" / "render_math.cjs"), str(katex_dir.resolve())],
                         input=json.dumps(data), text=True, capture_output=True, check=True)
    rendered = json.loads(run.stdout)
    failures = [{"tex": data[i]["tex"], "error": r["error"]} for i, r in enumerate(rendered) if r["error"]]
    if failures:
        raise RuntimeError("Math rendering failed: " + json.dumps(failures, ensure_ascii=False))
    iterator = iter(rendered)
    body = pattern.sub(lambda _: next(iterator)["html"], body)
    if '<script type="math/tex' in body:
        raise RuntimeError("An unrendered formula remains in the HTML.")
    return body, len(rendered)


def embedded_katex_css(katex_dir: Path) -> str:
    css = (katex_dir / "dist" / "katex.min.css").read_text(encoding="utf-8")
    def replace_src(match):
        urls = re.findall(r"url\(([^)]+)\)", match[0])
        name = next((u.strip("\"'") for u in urls if ".woff2" in u), None)
        if not name:
            return match[0]
        encoded = base64.b64encode((katex_dir / "dist" / name).read_bytes()).decode()
        return 'src:url(data:font/woff2; base64,' + encoded + ') format("woff2");'
    return re.sub(r"src:[^;]+;", replace_src, css)


def embedded_cjk_css(font_dir: Path, text: str) -> tuple[str, int]:
    """Keep required original Unicode-range font subsets; no font CDN at read time."""
    css = (font_dir / "400.css").read_text(encoding="utf-8")
    used = {ord(character) for character in text if ord(character) > 127}
    blocks = []
    for block in re.findall(r"@font-face\s*\{[^}]+\}", css):
        range_match = re.search(r"unicode-range:\s*([^;]+);", block)
        if range_match:
            intervals = [(int(low, 16), int(high or low, 16)) for low, high in
                         re.findall(r"U\+([0-9a-fA-F]+)(?:-([0-9a-fA-F]+))?", range_match[1])]
```

```
            if not any(low <= point <= high for point in used for low, high in intervals):
                continue
        font_path = re.search(r"url\(([^)]+\.woff2)\)", block)
        if not font_path:
            continue
        data = base64.b64encode((font_dir / font_path[1].strip("\"'")).read_bytes()).decode()
        block = re.sub(r"src:[^;]+;", "src:url(data:font/woff2; base64," + data + ") format('woff2');", block)
        blocks.append(block)
    return "\n".join(blocks), len(blocks)


STYLE = r"""
:root {--ink:#172e41;--muted:#536979;--line:#d9e3ea;--accent:#006e61;--paper:#ffffff;--back:#f4f7f9}
*{box-sizing:border-box}html{scroll-behavior:smooth;scroll-padding-top:24px}
body{margin:0;background:var(--back);color:var(--ink);font:16px/1.9 "Noto Sans SC",system-ui,-apple-
system,"Microsoft YaHei",sans-serif}
.layout{display:grid;grid-template-columns:280px minmax(0,1fr);max-width:1560px;margin:auto}
nav{position:sticky;top:0;height:100vh;overflow:auto;padding:28px 24px;border-right:1px solid var(--line);font-
size:13px;line-height:1.65}
.brand{font-size:18px;font-weight:750;color:var(--accent);margin-bottom:8px}
.navhint{color:var(--muted);font-size:12px;margin:0 0 20px}
nav ul{padding-left:15px}nav>div>ul{padding-left:0;list-style:none}nav li{margin:7px 0}nav a{color:var(--
muted);text-decoration:none}nav a:hover{color:var(--accent)}
main{min-width:0;padding:44px 5.2vw 90px;background:var(--paper)}
article{max-width:1100px;margin:auto}
h1,h2,h3,h4,h5{line-height:1.4;letter-spacing:.005em}
h1{font-size:34px;margin:0 0 18px}h2{font-size:26px;margin:65px 0 24px;border-top:2px solid var(--line);padding-
top:30px}
h3{font-size:21px;margin:35px 0 18px}h4{font-size:18px;margin:26px 0 14px;color:#1a655d}
p{margin:14px 0}a{color:var(--accent);text-underline-offset:3px;overflow-wrap:anywhere}
strong{font-weight:720}li{margin:8px 0}blockquote{border-left:4px solid var(--
accent);background:#eef7f4;margin:22px 0;padding:4px 20px}
table{border-collapse:collapse;width:100%;font-size:14px;line-height:1.6;margin:24px
0;display:block;overflow:auto}
th{background:#eaf1f5;text-align:left;white-space:normal}td,th{border:1px solid var(--line);padding:12px
14px;vertical-align:top;min-width:90px}
tr:nth-child(even) td{background:#f8fafb}
code{font-family:"KaTeX_Typewriter","SFMono-Regular",Consolas,"Liberation Mono","Noto Sans SC",monospace;font-
size:.88em;background:#edf2f6;padding:2px 5px;border-radius:4px;overflow-wrap:anywhere}
pre{padding:23px 22px;margin:0;overflow:auto;font-family:"KaTeX_Typewriter","SFMono-Regular",Consolas,"Liberation
Mono","Noto Sans SC",monospace;font-size:13px;line-height:1.65;tab-size:4}
pre code{background:none;padding:0;font-size:inherit;overflow-wrap:normal}
.codehilite{position:relative;background:#f7f9fc;border:1px solid #dce5ed;border-radius:9px;margin:22px 0}
.copy{position:sticky;float:right;right:8px;top:8px;margin:8px 8px -34px 0;border:1px solid #cedbe4;border-
radius:5px;background:#fff;color:#426073;padding:4px 9px;cursor:pointer;font-family:"Noto Sans SC",sans-
serif;font-size:11px;z-index:1}
.copy:hover{background:#e8f3ee}
img{max-width:100%;height:auto}.katex-display{overflow-x:auto;overflow-y:hidden;padding:10px 0}
.katex{font-size:1.06em}.katex .base{white-space:nowrap}
.tagline{font-size:12px;letter-spacing:.12em;color:var(--accent);font-weight:750;margin-bottom:20px}
.footer{margin-top:70px;padding-top:20px;border-top:1px solid var(--line);font-size:12px;color:var(--muted)}
@media(max-width:1000px){.layout{grid-template-columns:220px minmax(0,1fr)}nav{padding:22px 15px}main{padding:35px
28px 70px}h1{font-size:29px}}
@media(max-width:760px){.layout{display:block}nav{position:relative;height:auto;max-height:300px;border-bottom:1px
solid var(--line);border-right:0}main{padding:30px 19px 60px}h1{font-size:26px}h2{font-size:23px}pre{font-
size:12px;padding:20px 15px}table{font-size:13px}}
@media print{nav,.copy{display:none}.layout{display:block}main{padding:0}body{background:white;font-
size:10pt}h2{break-before:page}h3,h4{break-after:avoid}pre{white-space:pre-wrap;overflow-wrap:anywhere;font-
size:8pt}table{display:table;font-size:8pt}a{color:inherit}img{max-height:18cm}.codehilite{break-
inside:auto}.katex-display{overflow:visible}}
"""


SCRIPT = r"""
document.querySelectorAll('.codehilite').forEach(function(block) {
  var button=document.createElement('button');
  button.className='copy'; button.textContent='复制代码'; button.type='button';
```

```
  button.addEventListener('click',async function(){
    var text=block.querySelector('pre').innerText;
    try {
      if(navigator.clipboard && window.isSecureContext) {
        await navigator.clipboard.writeText(text);
      } else {
        var field=document.createElement('textarea');
        field.value=text; field.style.position='fixed'; field.style.opacity='0';
        document.body.appendChild(field); field.select();
        if(!document.execCommand('copy')) throw new Error('copy unavailable');
        field.remove();
      }
      button.textContent='已复制';
    } catch(e) {button.textContent='请选中代码复制';}
    setTimeout(function(){button.textContent='复制代码';},1800);
  });
  block.prepend(button);
});
"""


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--katex-dir", type=Path, required=True)
    parser.add_argument("--cjk-font-dir", type=Path, required=True)
    args = parser.parse_args()
    content, stats = assemble_markdown()
    markdown_path = ROOT / (BOOK_NAME + ".md")
    markdown_path.write_text(content, encoding="utf-8")
    converter = markdown.Markdown(
        extensions=["tables", "fenced_code", "codehilite", "toc", "pymdownx.arithmatex"],
        extension_configs={
            "toc": {"toc_depth": "2-3", "permalink": False},
            "codehilite": {"guess_lang": False, "css_class": "codehilite"},
            "pymdownx.arithmatex": {"generic": False},
        },
    )
    body = converter.convert(content)
    # The TOC is kept in the sidebar; do not duplicate it before the main text.
    toc = converter.toc
    body = body.replace(toc, "", 1)
    body, formula_count = render_math(body, args.katex_dir)
    cjk_css, font_subsets = embedded_cjk_css(args.cjk_font_dir, content)
    css = cjk_css + STYLE + HtmlFormatter(style="friendly").get_style_defs(".codehilite") +
embedded_katex_css(args.katex_dir)
    document = (
        '<!doctype html>\n<html lang="zh-CN"><head><meta charset="utf-8">'
        '<meta name="viewport" content="width=device-width, initial-scale=1">'
        '<title>GPU Programming 面试 · 12 题完整代码与中文答案</title>'
        '<style>' + css + '</style></head><body><div class="layout">'
        '<nav aria-label="教材目录"><div class="brand">GPU Programming · 12 题</div>'
        '<p class="navhint">完整源码与答案 · 离线阅读<br>点击目录跳转; Ctrl / ⌘ + F 搜索全文。</p>' + toc + '</nav>'
        '<main><article><div class="tagline">CUDA · TRITON · PYTORCH · FULL SOURCE</div>' + body +
        '<div class="footer">原创教学材料。性能模型与实际测量分别标注; 请参照验证记录。数学排版使用 KaTeX (MIT), 中文字体使用
Noto Sans SC (OFL)。</div>'
        '</article></main></div><script>' + SCRIPT + '</script></body></html>\n'
    )
    html_path = ROOT / (BOOK_NAME + ".html")
    html_path.write_text(document, encoding="utf-8")
    license_dir = ROOT / "licenses"
    license_dir.mkdir(exist_ok=True)
    shutil.copyfile(args.katex_dir / "LICENSE", license_dir / "KaTeX_LICENSE.txt")
    shutil.copyfile(args.cjk_font_dir / "LICENSE", license_dir / "Noto_Sans_SC_OFL.txt")
    stats.update({
        "display_and_inline_formulas_rendered": formula_count,
        "markdown_characters": len(content), "markdown_bytes": markdown_path.stat().st_size,
        "html_bytes": html_path.stat().st_size, "code_blocks": body.count('<div class="codehilite">'),
```

```
        "embedded_cjk_font_subsets": font_subsets,
    })
    (ROOT / "results").mkdir(exist_ok=True)
    (ROOT / "results" / "book_build.json").write_text(json.dumps(stats, ensure_ascii=False, indent=2) + "\n",
encoding="utf-8")
    print(json.dumps(stats, ensure_ascii=False, indent=2))
    print(html_path)
    print(markdown_path)


if __name__ == "__main__":
    main()
```

tools/check_triton_interpreter.py

```
"""Optional CPU math validation using Triton's own interpreter, never a fallback.

Run in a fresh process, before importing Triton:
    TRITON_INTERPRET=1 python tools/check_triton_interpreter.py

For the validated older Triton 3.1 interpreter, use a compatible NumPy 1.26.x
environment. Do not interpret this report as GPU numerical, race, or performance
validation. Public CUDA-only wrappers are deliberately bypassed here and remain
unchanged. No substitute emulator is implemented by this script.
"""

import argparse
from datetime import datetime, timezone
import json
import math
import os
from pathlib import Path
import platform
import sys

if os.environ.get("TRITON_INTERPRET") != "1":
    raise SystemExit("Start a separate process with TRITON_INTERPRET=1; this tool never selects a fallback
automatically.")

import numpy as np
import torch
import triton
import triton.language as tl

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from gpu_prep import triton_ops as ops


@triton.jit
def _memory_probe(X, Y, N: tl.constexpr):
    idx = tl.arange(0, 16)
    value = tl.load(X + idx, idx < N, other=0)
    tl.store(Y + idx, value + 7.0, idx < N)


def attention_reference(q, k, v, causal, query_start, mask):
    scores = (q.double() @ k.double().transpose(-2, -1)) / math.sqrt(q.shape[-1])
    length, source = q.shape[2], k.shape[2]
    visible = torch.ones_like(scores, dtype=torch.bool)
    if causal:
        rows = torch.arange(length) + query_start
        cols = torch.arange(source)
        visible &= cols[None, :] <= rows[:, None]
    if mask is not None:
```

```
        visible &= mask
    scores = scores.masked_fill(~visible, -float("inf"))
    maximum = scores.amax(-1, keepdim=True)
    maximum = torch.where(torch.isfinite(maximum), maximum, 0.0)
    numerator = (scores - maximum).exp()
    denominator = numerator.sum(-1, keepdim=True)
    denominator = torch.where(denominator > 0, denominator, 1.0)
    return ((numerator / denominator) @ v.double()).float()


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output", type=Path, default=Path("results/triton_interpreter.json"))
    args = parser.parse_args()
    report = {
        "created_utc": datetime.now(timezone.utc).isoformat(),
        "scope": "CPU execution of Triton's own interpreter for FP32 indexing, masks, and high-level arithmetic.",
        "not_validated": [
            "GPU numerical execution", "GPU lane/layout lowering", "thread/warp synchronization and races",
            "register/shared-memory resource limits", "GPU performance", "FP16/BF16 execution",
        ],
        "public_cuda_wrappers_changed": False,
        "environment": {
            "python": platform.python_version(), "torch": torch.__version__,
            "triton": triton.__version__, "numpy": np.__version__,
            "TRITON_INTERPRET": os.environ.get("TRITON_INTERPRET"),
        },
        "tests": [],
    }

    def save():
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")

    # Detect an incompatible interpreter/numpy memory bridge before attributing
    # incorrect results to the exercise kernels. This probe is independent of them.
    x, out = torch.arange(7, dtype=torch.float32), torch.full((7,), -999.0)
    try:
        _memory_probe[(1,)](x, out, 7)
        torch.testing.assert_close(out, x + 7, atol=0, rtol=0)
        report["interpreter_memory_preflight"] = "passed"
    except Exception as exc:
        report["status"] = "blocked_interpreter_memory_preflight"
        report["interpreter_memory_preflight"] = {
            "error": f"{type(exc).__name__}: {exc}", "actual": out.tolist(), "expected": (x + 7).tolist(),
        }
        save()
        print("BLOCKED: Triton's own minimal CPU load/store probe failed; no exercise result is claimed.")
        return 2

    torch.manual_seed(2027)

    def check(name, kernel, grid, call_args, call_kwargs, actual, expected, atol=5e-5, rtol=3e-4):
        record = {"name": name, "shape": list(actual.shape), "dtype": "float32", "atol": atol, "rtol": rtol}
        try:
            kernel[grid](*call_args, **call_kwargs)
            torch.testing.assert_close(actual, expected, atol=atol, rtol=rtol)
            record["status"] = "passed"
            record["max_absolute_error"] = (actual.double() - expected.double()).abs().max().item() if
actual.numel() else 0.0
            print("PASS", name)
        except Exception as exc:
            record["status"] = "failed"
            record["error"] = f"{type(exc).__name__}: {exc}"
            print("FAIL", name, type(exc).__name__)
        report["tests"].append(record)
```

```
    for n in (1, 257, 1001):
        x, y = torch.randn(n), torch.randn(n)
        out = torch.full_like(x, float("nan"))
        check(f"vector_add_n{n}", ops._vector_add_kernel, (triton.cdiv(n, 256),),
              (x, y, out, n), {"BLOCK": 256}, out, x + y, atol=0, rtol=0)

    x, bias = torch.randn(3, 33), torch.randn(33)
    out = torch.full_like(x, float("nan"))
    check("bias_relu_tail", ops._bias_relu_kernel, (triton.cdiv(x.numel(), 256),),
          (x, bias, out, x.numel(), 33), {"BLOCK": 256}, out, (x + bias).relu(), atol=0, rtol=0)

    for n in (1, 33, 513):
        x = torch.randn(3, n)
        block = triton.next_power_of_2(n)
        for is_max in (False, True):
            out = torch.full((3,), float("nan"))
            reference = x.amax(-1) if is_max else x.double().sum(-1).float()
            check(f"row_{'max' if is_max else 'sum'}_n{n}", ops._row_reduce_kernel, (3,),
                  (x, out, n, is_max), {"BLOCK": block}, out, reference)
        out = torch.full_like(x, float("nan"))
        check(f"softmax_n{n}", ops._softmax_kernel, (3,), (x, out, n),
              {"BLOCK": block}, out, x.double().softmax(-1).float())

    x = torch.tensor([[1000., 1001., -1000.], [17., 17., 17.]])
    out = torch.full_like(x, float("nan"))
    check("softmax_large_logits", ops._softmax_kernel, (2,), (x, out, 3),
          {"BLOCK": 4}, out, x.double().softmax(-1).float())

    for m, n in ((31, 33), (65, 17)):
        x = torch.randn(m, n)
        for tiled in (False, True):
            out = torch.full((n, m), float("nan"))
            kernel = ops._transpose_tiled_kernel if tiled else ops._transpose_naive_kernel
            grid = (triton.cdiv(m, 32) * triton.cdiv(n, 32),) if tiled else (triton.cdiv(m * n, 256),)
            config = {"TILE": 32} if tiled else {"BLOCK": 256}
            check(f"transpose_{'tiled' if tiled else 'naive'}_{m}x{n}", kernel, grid,
                  (x, out, m, n), config, out, x.T.contiguous(), atol=0, rtol=0)

    for m, n, k in ((7, 3, 19), (31, 17, 33), (5, 9, 0)):
        a, b = torch.randn(m, k), torch.randn(k, n)
        reference = (a.double() @ b.double()).float()
        for tiled in (False, True):
            out = torch.full((m, n), float("nan"))
            kernel = ops._matmul_tiled_kernel if tiled else ops._matmul_naive_kernel
            grid = (triton.cdiv(m, 32) * triton.cdiv(n, 32),) if tiled else (triton.cdiv(m * n, 128),)
            config = {"BM": 32, "BN": 32, "BK": 32} if tiled else {"BLOCK": 128}
            check(f"matmul_{'tiled' if tiled else 'naive'}_{m}x{n}x{k}", kernel, grid,
                  (a, b, out, m, n, k), config, out, reference)

    for shifted in (False, True):
        x = torch.randn(3, 33) + (100.0 if shifted else 0.0)
        weight, bias = torch.randn(33), torch.randn(33)
        xd, wd, bd = x.double(), weight.double(), bias.double()
        for is_rms in (False, True):
            out = torch.full_like(x, float("nan"))
            if is_rms:
                reference = (xd * torch.rsqrt((xd * xd).mean(-1, keepdim=True) + 1e-5) * wd).float()
            else:
                centered = xd - xd.mean(-1, keepdim=True)
                reference = (centered * torch.rsqrt((centered * centered).mean(-1, keepdim=True) + 1e-5) * wd +
bd).float()
            check(f"{'rms' if is_rms else 'layer'}_norm_shifted{shifted}", ops._norm_kernel, (3,),
                  (x, weight, bias, out, 33, 1e-5, is_rms), {"BLOCK": 64}, out, reference, atol=1e-4)

    attention_cases = [
        ("noncausal_tail_d16", 17, 37, 16, False, 0, None),
        ("causal_tail_d32", 19, 37, 32, True, 0, None),
```

```
        ("decode_offset_d64", 1, 37, 64, True, 36, None),
        ("offset_d128", 7, 65, 128, True, 23, None),
    ]
    mask = torch.ones(19, 65, dtype=torch.bool)
    mask[0] = False
    mask[1] = False
    mask[1, 40] = True
    mask[2, 32:] = False
    attention_cases.append(("shared_mask_empty_rows_and_tiles", 19, 65, 32, False, 0, mask))
    batched = mask.expand(2, 2, 19, 65).clone()
    batched[1, 1, 3] = False
    attention_cases.append(("batched_mask_with_causal_offset", 19, 65, 32, True, 50, batched))
    for name, length, source, dim, causal, query_start, mask in attention_cases:
        q = torch.randn(2, 2, length, dim)
        k, v = torch.randn(2, 2, source, dim), torch.randn(2, 2, source, dim)
        out = torch.full_like(q, float("nan"))
        mask_stride = length * source if mask is not None and mask.ndim == 4 else 0
        mask_pointer = q if mask is None else mask
        reference = attention_reference(q, k, v, causal, query_start, mask)
        check(f"attention_{name}", ops._attention_kernel, (triton.cdiv(length, 16) * 4,),
              (q, k, v, mask_pointer, out, length, source, dim, 1.0 / math.sqrt(dim),
               causal, query_start, mask is not None, mask_stride),
              {"BM": 16, "BN": 32}, out, reference)

    passed = sum(test["status"] == "passed" for test in report["tests"])
    report["passed"] = passed
    report["failed"] = len(report["tests"]) - passed
    report["status"] = "passed_interpreter_math_only" if report["failed"] == 0 else "failed_interpreter_checks"
    save()
    print(f"{passed}/{len(report['tests'])} interpreter math checks passed; no GPU execution or performance was
measured.")
    return 0 if report["failed"] == 0 else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

tools/compile_triton_offline.py

```
"""Compile every teaching Triton kernel for an explicit target, without a GPU.

This is compiler validation, not a numerical, race, or performance test.
The ASTSource API here is validated against Triton 3.1.0 and is not a stable
cross-version public compatibility promise. Run GPU tests on your actual stack.

Usage from an installed checkout:
    python tools/compile_triton_offline.py --arch 80 --output results/triton_offline_compile.json
"""
from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path

import triton
from triton.backends.compiler import GPUTarget
from triton.compiler.compiler import ASTSource

from gpu_prep import triton_ops as ops


def configurations():
    for dtype in ("fp32", "fp16", "bf16"):
```

```
        ptr = "*" + dtype
        def entry(name, pointers, constants, warps=4, stages=2):
            return {
                "kernel": name, "dtype": dtype,
                "signature": {arg: ptr for arg in pointers.split()},
                "constants": constants,
                "options": {"num_warps": warps, "num_stages": stages},
            }
        yield entry("_vector_add_kernel", "X Y Z", {"N": 1003, "BLOCK": 256})
        yield entry("_bias_relu_kernel", "X Bias Y", {"TOTAL": 3 * 769, "N": 769, "BLOCK": 256})
        for is_max in (False, True):
            yield entry("_row_reduce_kernel", "X Y", {"N": 769, "IS_MAX": is_max, "BLOCK": 1024})
        for n, block, warps in ((769, 1024, 4), (8192, 8192, 8)):
            yield entry("_softmax_kernel", "X Y", {"N": n, "BLOCK": block}, warps)
        yield entry("_transpose_naive_kernel", "X Y", {"M": 127, "N": 65, "BLOCK": 256})
        yield entry("_transpose_tiled_kernel", "X Y", {"M": 127, "N": 65, "TILE": 32})
        yield entry("_matmul_naive_kernel", "A B C", {"M": 17, "N": 33, "K": 19, "BLOCK": 128})
        yield entry("_matmul_tiled_kernel", "A B C", {"M": 127, "N": 65, "K": 93, "BM": 32, "BN": 32, "BK": 32})
        for rms in (False, True):
            yield entry("_norm_kernel", "X W Bias Y", {"N": 769, "EPS": 1e-5, "IS_RMS": rms, "BLOCK": 1024})
        for dim, causal, mask, length, source, start in (
            (16, False, False, 19, 37, 0),
            (32, True, False, 1, 65, 64),
            (64, True, True, 19, 37, 0),
            (128, False, True, 19, 37, 0),
        ):
            config = entry("_attention_kernel", "Q K V Mask O", {
                "L": length, "S": source, "D": dim, "SCALE": dim ** -0.5,
                "CAUSAL": causal, "QUERY_START": start, "HAS_MASK": mask,
                "MASK_BH_STRIDE": length * source if mask else 0,
                "BM": 16, "BN": 32,
            }, stages=1)
            if mask:
                config["signature"]["Mask"] = "*i1"
            yield config


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--arch", type=int, default=80)
    parser.add_argument("--output", type=Path, default=Path("results/triton_offline_compile.json"))
    args = parser.parse_args()
    target = GPUTarget("cuda", args.arch, 32)
    results = []
    for index, config in enumerate(configurations(), 1):
        record = dict(config)
        try:
            compiled = triton.compile(
                ASTSource(getattr(ops, config["kernel"]), signature=config["signature"],
constants=config["constants"]),
                target=target, options=config["options"],
            )
            ptx, cubin = compiled.asm["ptx"], compiled.asm["cubin"]
            record.update({
                "status": "compiled", "ptx_characters": len(ptx), "cubin_bytes": len(cubin),
                "ptx_sha256": hashlib.sha256(ptx.encode()).hexdigest(),
                "cubin_sha256": hashlib.sha256(cubin).hexdigest(),
            })
            print(f"{index:02d} COMPILED {config['dtype']:4s} {config['kernel']}", flush=True)
        except Exception as exc:
            record.update({"status": "failed", "error": f"{type(exc).__name__}: {exc}"})
            print(f"{index:02d} FAILED {config['dtype']:4s} {config['kernel']}: {exc}", flush=True)
        results.append(record)
    report = {
        "triton_version": triton.__version__, "target": {"backend": "cuda", "arch": args.arch, "warp_size": 32},
        "scope": "Offline AST -> PTX -> cubin compilation only. No GPU numerical execution, race checks, or
timing.",
```

```
        "successes": sum(r["status"] == "compiled" for r in results),
        "failures": sum(r["status"] != "compiled" for r in results), "results": results,
    }
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(f"Result: {report['successes']} compiled, {report['failures']} failed; {args.output}")
    raise SystemExit(1 if report["failures"] else 0)


if __name__ == "__main__":
    main()
```

tools/make_roofline_figure.py

```
"""Generate a scientific figure of an explicitly hypothetical roofline model."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from gpu_prep.roofline import gemm_workload, vector_add_workload


def main():
    root = Path(__file__).resolve().parents[1]
    target = root / "docs" / "assets"
    target.mkdir(parents=True, exist_ok=True)
    intensity = np.logspace(-2, 3, 600)
    peak, bandwidth = 100.0, 2.0
    ceiling = np.minimum(peak, bandwidth * intensity)
    with plt.rc_context({
        "font.family": "DejaVu Sans", "font.size": 10,
        "axes.spines.top": False, "axes.spines.right": False,
        "svg.fonttype": "none",
    }):
        fig, ax = plt.subplots(figsize=(10.4, 5.1), constrained_layout=True)
        fig.patch.set_facecolor("#f7fafc")
        ax.set_facecolor("#f7fafc")
        ax.loglog(intensity, ceiling, color="#0b7466", linewidth=3, label="Model performance ceiling")
        ax.axvline(peak / bandwidth, color="#687785", linewidth=1.2, linestyle="--")
        cases = [
            ("Vector add, FP32", vector_add_workload(1_000_000), (15, 13)),
            ("GEMM 1 x 4096 x 4096, FP32", gemm_workload(1, 4096, 4096), (18, -23)),
            ("GEMM 1024 cubed, FP32", gemm_workload(1024, 1024, 1024), (-153, -33)),
        ]
        for label, workload, offset in cases:
            x = workload.arithmetic_intensity
            y = min(peak, bandwidth * x)
            ax.scatter([x], [y], color="#e49b32", edgecolors="#56320a", s=64, zorder=5)
            ax.annotate(label, (x, y), xytext=offset, textcoords="offset points", fontsize=9)
        ax.text(peak / bandwidth * 1.12, 0.034, "Ridge = 50 FLOP/byte", fontsize=9, color="#56616d")
        ax.set(xlabel="Arithmetic intensity (FLOP/byte; ideal logical traffic)",
               ylabel="Model ceiling (TFLOP/s)", xlim=(0.01, 1000), ylim=(0.02, 240))
        ax.grid(True, which="major", alpha=0.16)
        ax.set_title("Hypothetical GPU: 100 TFLOP/s and 2 TB/s\nAll points lie on a mathematical roof; none are
measured kernel results.",
                     loc="left", fontsize=13, pad=17)
        fig.savefig(target / "roofline_model.svg")
        fig.savefig(target / "roofline_model.png", dpi=160)
        plt.close(fig)
    print(target / "roofline_model.svg")
```

```
if __name__ == "__main__":
    main()
```

tools/render_math.cjs

```
// Render formulas at build time. Reading the output needs no math CDN or JS.
const fs = require("fs");
const path = require("path");
const katex = require(path.resolve(process.argv[2]));
const input = JSON.parse(fs.readFileSync(0, "utf8"));
const results = input.map((item) => {
  try {
    return {
      html: katex.renderToString(item.tex, {
        displayMode: item.display, throwOnError: true,
        strict: "ignore", trust: false, output: "htmlAndMathml",
      }),
      error: null,
    };
  } catch (error) {
    return {html: "", error: String(error)};
  }
});
process.stdout.write(JSON.stringify(results));
```

原创教学材料。性能模型与实际测量分别标注；请参照验证记录。数学排版使用 KaTeX（MIT），中文字体使用 Noto Sans SC（OFL）。